第1章　Python语言概述

https://www.bilibili.com/video/BV1oW4y1q7qv/?spm_id_from=333.337.search-card.all.click&vd_source=66b7318d84f8ecd0c0696e474c789b86

- There are two ways of constructing a software design: one way is to make it so simple that there are obviously no deficiencies; the other is to make it so complicated that there are no obvious deficiencies.
- 有两种方式构建软件设计：一种是把软件做得很简单以至于很明显没有缺陷；另一种是把它做得很复杂以至于没有明显的缺陷。

——C. A. R. Hoare

- Success in life is a matter not so much of talent and opportunity as of concentration and perseverance.
- 获得人生的成功所需要的专注与坚持不懈多过天赋与机遇。

——C.W.Wendte

1.1 Python是这样一种语言

1.1.1 Python语言的起源



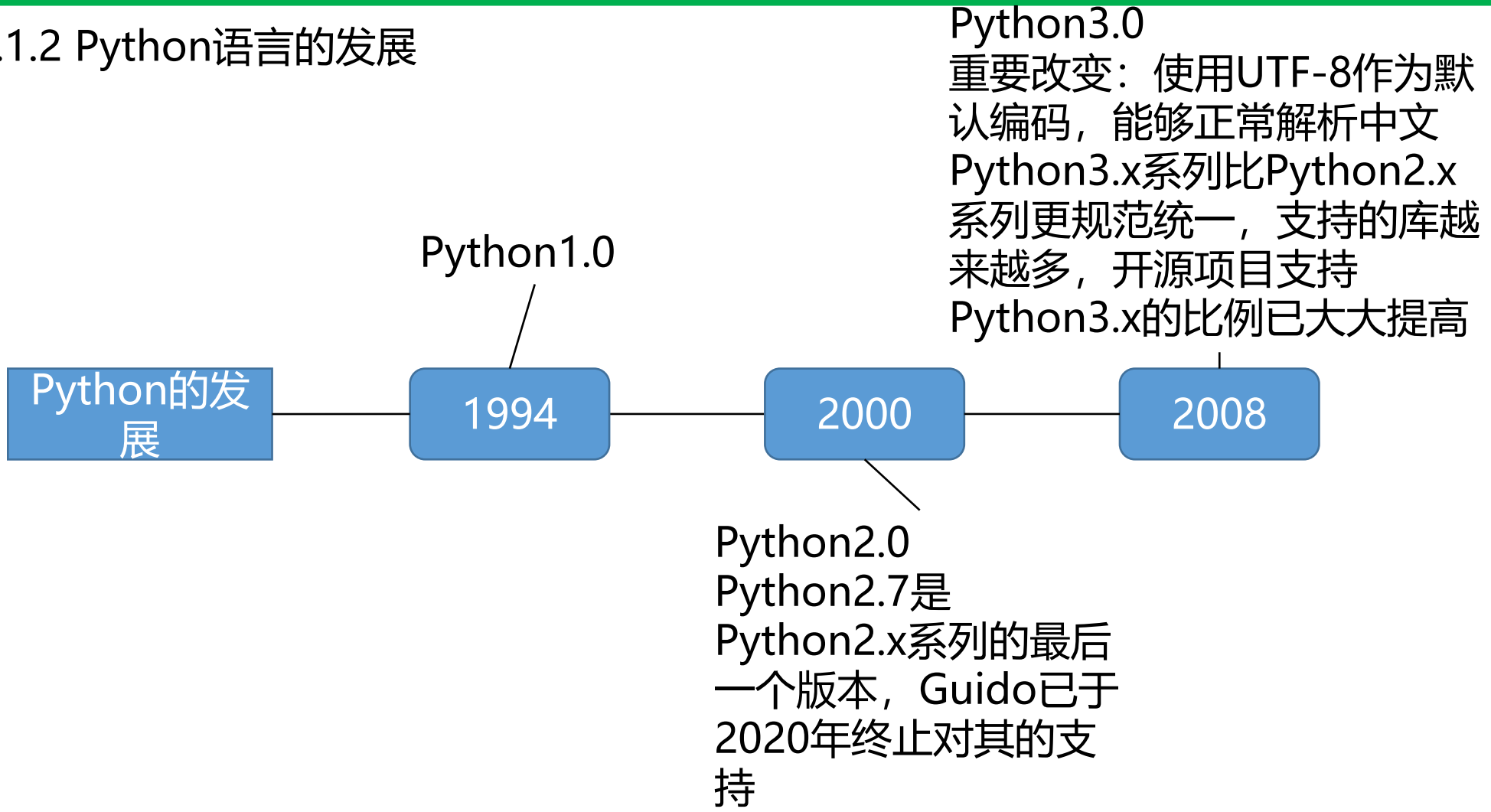
创始人：荷兰人Guido van Rossum(人称：龟叔)

创始过程：1989年的圣诞节，Guido为了打发时间，决定在ABC语言的基础上开发一款新型脚本解释程序。由于Guido非常喜欢一部名为《Monty Python’s Flying Circus》的英国肥皂剧，于是这门新语言便有了自己的名字——Python。

Python简介：1991年，Python的第一个公开版本发行。Python的源代码和解释器Cpython遵循通用公开许可证协议，其语法简洁清晰，强制用空白符作为语句缩进是其特色之一。Python具有丰富和强大的库，常被称为“胶水”语言，能够轻松地把其他语言制作的各种模块联结在一起。

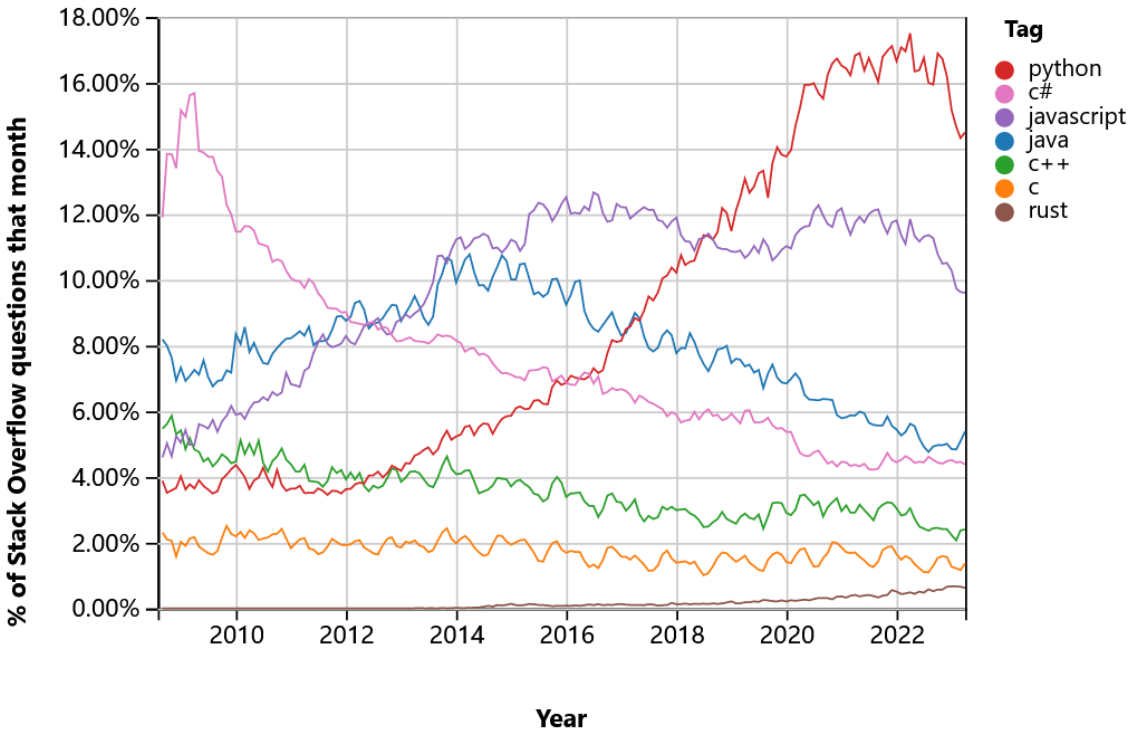
1.1 Python是这样一种语言

1.1.2 Python语言的发展



1.1 Python是这样一种语言

人生苦短，我用 Python





1.1 Python是这样一种语言

1.1.3 Python语言的特点：

1 简单、易学、免费、开源: Python简单、易学。我们可以自由发布其复制版本,阅读、修改其源代码,将其(部分)用于新软件中。
2 解释型: Python是边解释边执行的,Python解释器会将源代码转换为中间字节码形式,然后将其解释为机器语言并执行。
3 可移植: Python解释器已被移植到许多平台上,Python程序无须经过修改就可以在多个平台上运行。
4 代码规范: Python所采用的强制缩进的方式,使得其代码具有极佳的可读性。
5 面向对象: 与C++和Java等相比,Python以强大而简单的方式实现了面向对象编程。
6 胶水语言: 标准版本的Python调用C语言,并可以借助C语言的接口驱动调用所有编程语言。
7 丰富的库: Python的标准库种类繁多,可以帮助我们处理各种工作,我们不需要安装就可以直接使用这些库。
8 动态类型: Python不会检查数据类型,在声明变量时不需要指定数据类型。

1.1 Python是这样一种语言

1.1.4 Python语言的应用领域



Python现已成为Web开发的主流语言之一，其有丰富的Web开发框架，如Django、Flask、Weppy、Zope2、Tornado、CubicWeb及Web2py等。其中，Python+Django框架最为流行，其应用范围广、开发速度快、学习门槛低，可以让程序员轻松地开发和管理复杂的Web程序。

1.1 Python是这样一种语言

1.1.4 Python语言的应用领域

2、网络安全

自2013年“棱镜门”事件后，网络安全问题越发引起世界各国的重视。Python作为高级编程语言的一种，编程简单、易学及易用，且有强大的第三方编程模块的支持，越来越受网络安全维护人员的青睐。一些常用的网络安全工具都是基于Python语言来编写的，如Scapy、Pcapy及Sulley等。

1.1 Python是这样一种语言

1.1.4 Python语言的应用领域

3、网络爬虫

在编写网络爬虫程序的众多语言中，Python起到了举足轻重的作用，其中Scrapy爬虫框架应用得非常广泛，可以应用在数据挖掘、信息处理或存储历史数据中。此外，Python还提供了多个网络爬虫框架以满足不同的应用需求，如PySpider、Crawley、Newspaper、Beautiful Soup、Grab及Cola等。

1.1 Python是这样一种语言

1.1.4 Python语言的应用领域

4、游戏开发

Python在游戏开发方面也有很多应用，如专门为游戏开发而设计的模块pygame，能够使开发者快速开发出自己的游戏。相比于Lua 或C++，Python比Lua具有更高阶的抽象能力，可以用更少的代码描述游戏的业务逻辑。例如，用C++开发游戏，有时必须写一些扩展，从而增加游戏的代码量。

1.1 Python是这样一种语言

1.1.4 Python语言的应用领域

5、数据分析

Python拥有丰富的第三方库，如Pandas、NumPy、Matplotlib及SciPy等，可以让程序编写人员完成各种数据分析需求。

1.1 Python是这样一种语言

1.1.4 Python语言的应用领域

6、人工智能

Python在人工智能、神经网络、深度学习方面，有强大而丰富的库。Python是面向对象的动态语言，且适用于科学计算，这就使得Python在人工智能方面备受青睐。目前，世界优秀的人工智能学习框架，如Google的TensorFlow、Facebook的PyTorch及开源社区的神经网络库Keras等均是用Python来实现的。

1.1 Python是这样一种语言

使用Python编写的著名应用

- Youtube - 视频社交网站
- Reddit - 社交分享网站
- Dropbox - 文件分享服务
- 豆瓣网 - 图书、唱片、电影等文化产品的资料数据库网站
- Django - 鼓励快速开发的Web应用框架
- Pylons - Web应用框架
- Zope - 应用服务器
- Plone - 内容管理系统
- Instagram - 是一款免费提供在线图片及视频分享的社交应用软件，使用Django作为后台
- TurboGears - 另一个Web应用快速开发框架
- Twisted (http://twistedmatrix.com) - Python的网络应用程序框架
- Fabric (http://fabfile.org/) - 用于管理成百上千台Linux主机的程序库
- Python Wikipedia Robot Framework - MediaWiki的机器人程式
- MoinMoinWiki (http://moinmo.in/) - Python写成的Wiki程序
- Trac (http://trac.edgewall.org/) - 使用Python编写的BUG管理系统
- Mailman (http://www.gnu.org/s/mailman) - 使用Python编写的邮件列表软件

1.1 Python是这样一种语言

核心物理领域专用库

- **经典力学**

SciPy (scipy.integrate.solve_ivp)

用途：解算常微分方程（ODE）的初值问题。

物理应用：数值模拟行星运动、弹簧振子、双摆等混沌系统。

- **电动力学与电磁学**

PyCharge

用途：计算移动点电荷产生的电磁场并进行模拟。

EMpy (Electromagnetic Python)

用途：计算电磁问题的解，如光纤模式、衍射。

1.1 Python是这样一种语言

- **量子力学**

QuTiP (Quantum Toolbox in Python)

用途：量子物理的终极工具包。用于模拟量子系统的动力学、能级、开放系统等。

ProjectQ

用途：量子计算模拟框架，可以编写和模拟量子算法。

- **热力学与统计物理**

NumPy / SciPy

用途：进行蒙特卡洛模拟、计算积分和统计量。

物理应用：模拟伊辛模型（Ising Model）、计算配分函数。

1.1 Python是这样一种语言

- **天文学与天体物理**

Astropy

用途：天文学家的“瑞士军刀”。提供天文学常用常数、单位转换、坐标转换、时间处理、光谱分析等功能。

物理应用：处理 FITS 格式的天文图像数据、计算宇宙学距离、坐标转换。

REBOUND

用途：高性能的 N 体模拟软件包。

物理应用：模拟太阳系演化、行星形成、恒星团动力学。

1.1 Python是这样一种语言

- **实验物理与数据分析**

Pandas

用途：处理结构化表格数据（类似 Excel），功能无比强大。

物理应用：整理、清洗、分析和可视化实验测量数据。例如，处理一系列在不同温度、压力下测量的数据点。

SciKit-Learn

用途：机器学习工具库。

物理应用：从实验数据中挖掘规律、分类信号、回归预测。例如，区分粒子碰撞事件、预测材料性质。

Uncertainties

用途：误差传播自动计算库。物理实验必备！

1.2 Python版本之争

- Python目前存在2.x和3.x两个系列的版本，互相之间不兼容。
- Python 2.x系列于2020年全面放弃维护和更新。
- 本课程将使用Python 3.6.0及以上

1.3 开发环境的安装与使用

- **默认编程环境：IDLE(Integrated Development and Learning Environment)**

- 在掌握了Python的环境配置后，为了能更高效地进行代码开发，建议读者选择集成开发环境（Integrated Development Environment，IDE）。常用的Python IDE有PyCharm、Anaconda(Jupter和Spyder)、Sublime Text、VS code等，**本课程选择的IDE是Spyder**。

1.3 开发环境的安装与使用

1.3.1 在Windows系统中安装Python默认环境IDLE的具体安装步骤如下：

（1）访问Python官网https://www.python.org/downloads/，进入Python的下载页面，如右图所示。单击矩形框中的链接地址，即可进入Windows系统的Python下载页面。Python同样为Windows提供了多个版本，用户可以根据自身的需求，选择适合自己需要的版本。本书以Python 3.6.3版本(其他更高版本是一样的)为例，演示Python的安装过程。



Python下载页面

1.3 开发环境的安装与使用

1.3.1 在Windows系统中安装Python默认环境IDLE的具体安装步骤如下：

（2）双击下载好的.exe文件，进入Python 3.6.3的安装界面，如下左图所示。建议在安装的过程中选中“Add Python 3.6 to PATH”复选框，如下右图所示。这样，Python路径就会自动添加到系统的环境变量中，无须再手动为Python配置环境变量。



Python安装方式选择

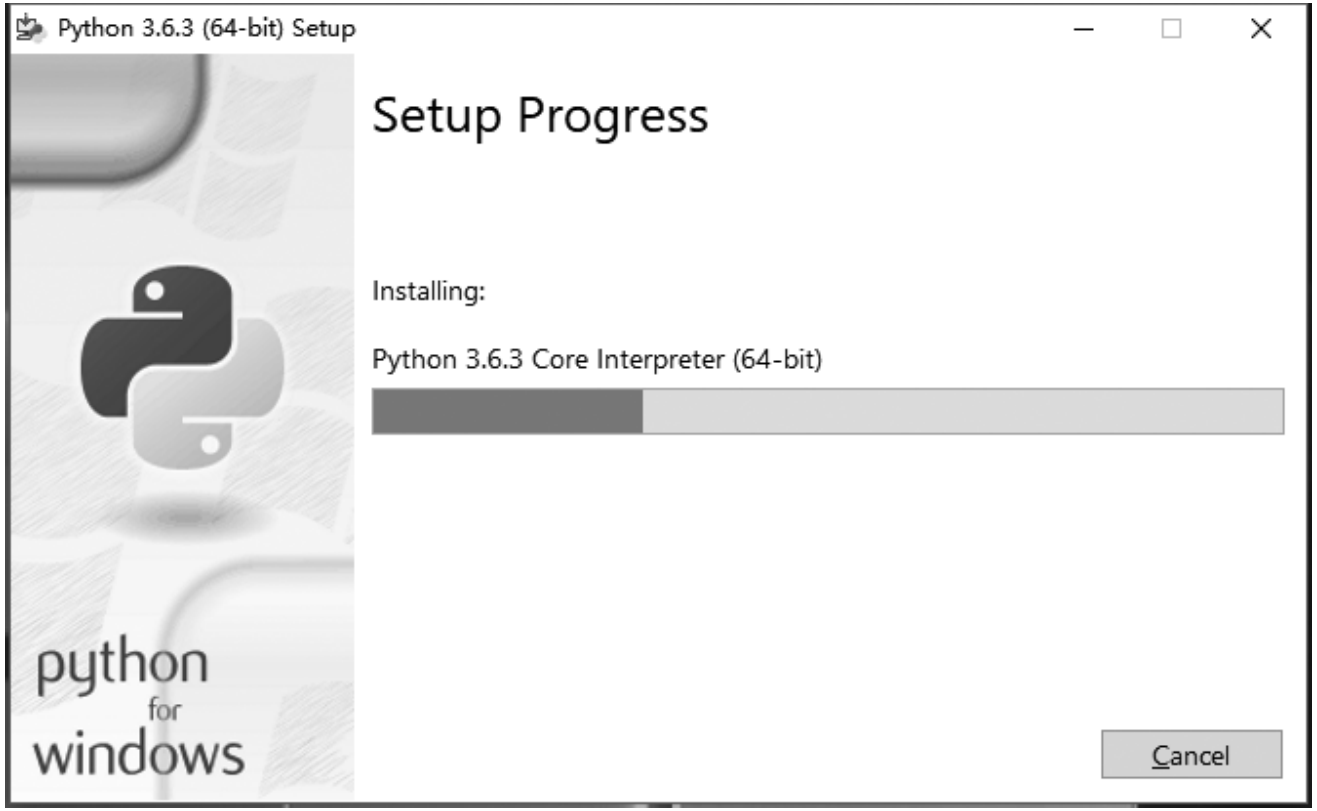


选择添加Python路径

1.3 开发环境的安装与使用

1.3.1 在Windows系统中安装Python默认环境IDLE的具体安装步骤如下：

（3）选择第1种安装方式Install Now，安装界面如下图所示。若选择第2种安装方式Customize installation，则可以选择安装路径及可选项等内容。

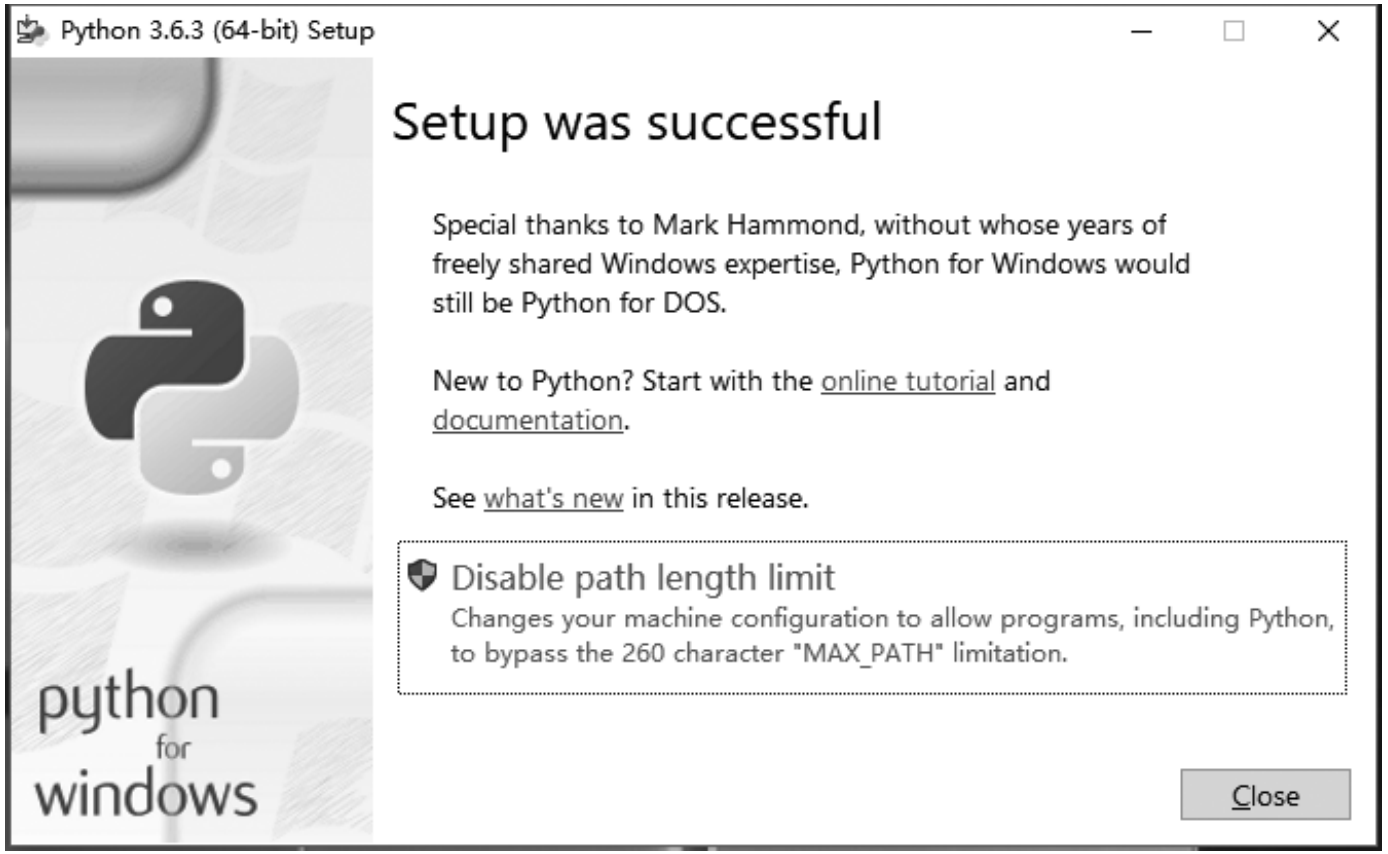


安装过程界面

1.3 开发环境的安装与使用

1.3.1 在Windows系统中安装Python默认环境IDLE的具体安装步骤如下：

（4）安装成功界面



安装成功界面

1.3 开发环境的安装与使用

1.3.1 IDLE的使用：

- 使用交互式解释环境

```
>>>print('Hello World')
Hello World
```

- 使用源文件

编辑器：使用Python自带的IDLE（语法高亮，自动缩进，自动补全）

```
#Filename: helloworld.py
print('Hello World')
```

Run → Run Module

注意：Python 是大小写敏感的

确保：每一行的开始字符前没有空格或者制表符

1.3 开发环境的安装与使用

1.3.2 在Windows系统中安装Python集成开发环境Spyder的具体安装步骤如下：

https://www.spyder-ide.org/

1.访问 Spyder 官网
2.下载 Windows 独立安装包
3.双击安装文件按向导完成安装



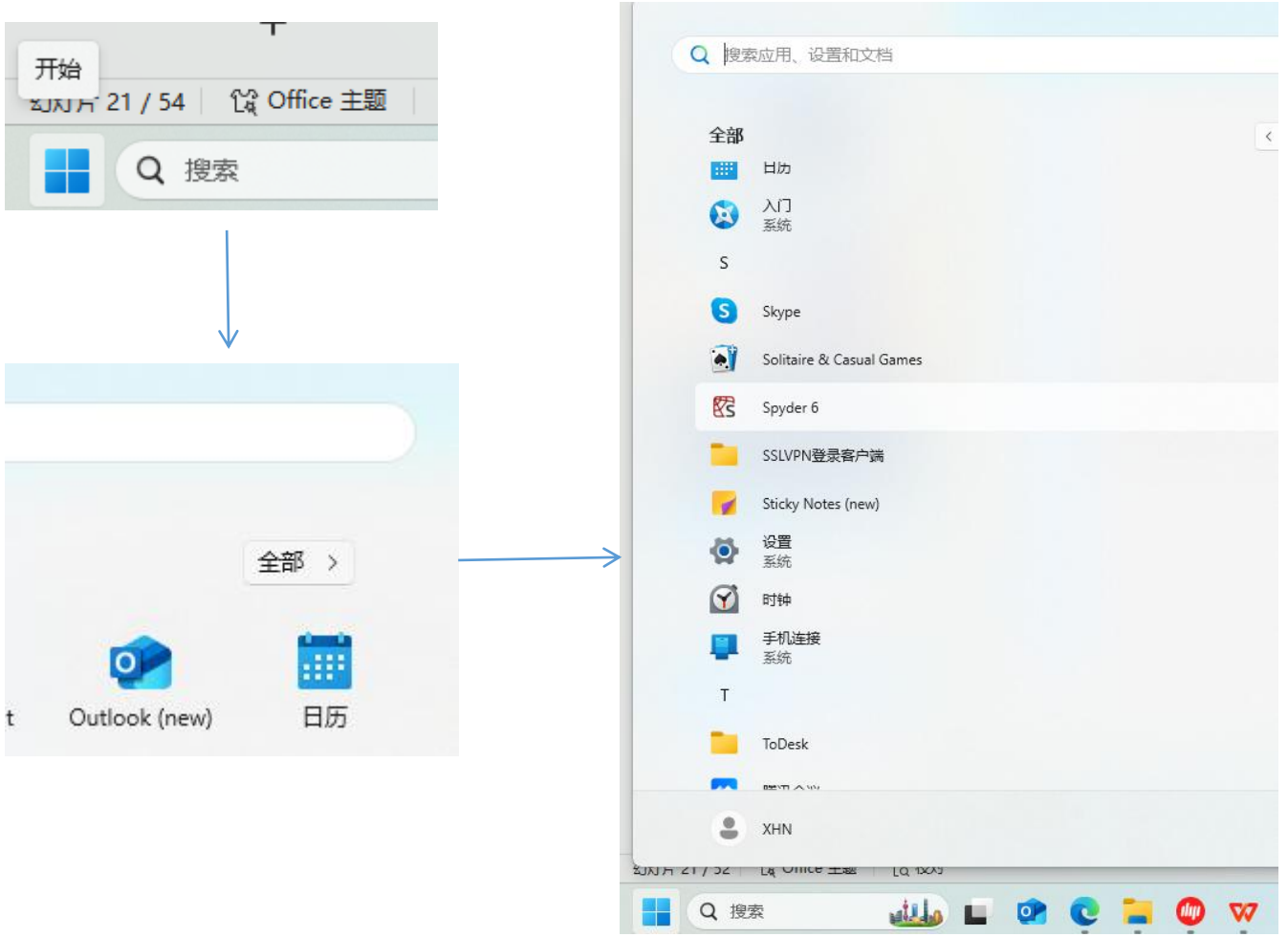
•操作系统: Windows 7+
•内存: 至少 4GB RAM（推荐 8GB）
•磁盘空间: 至少 1GB 可用空间

1.3 开发环境的安装与使用

1.3.3 Spyder的简单使用：

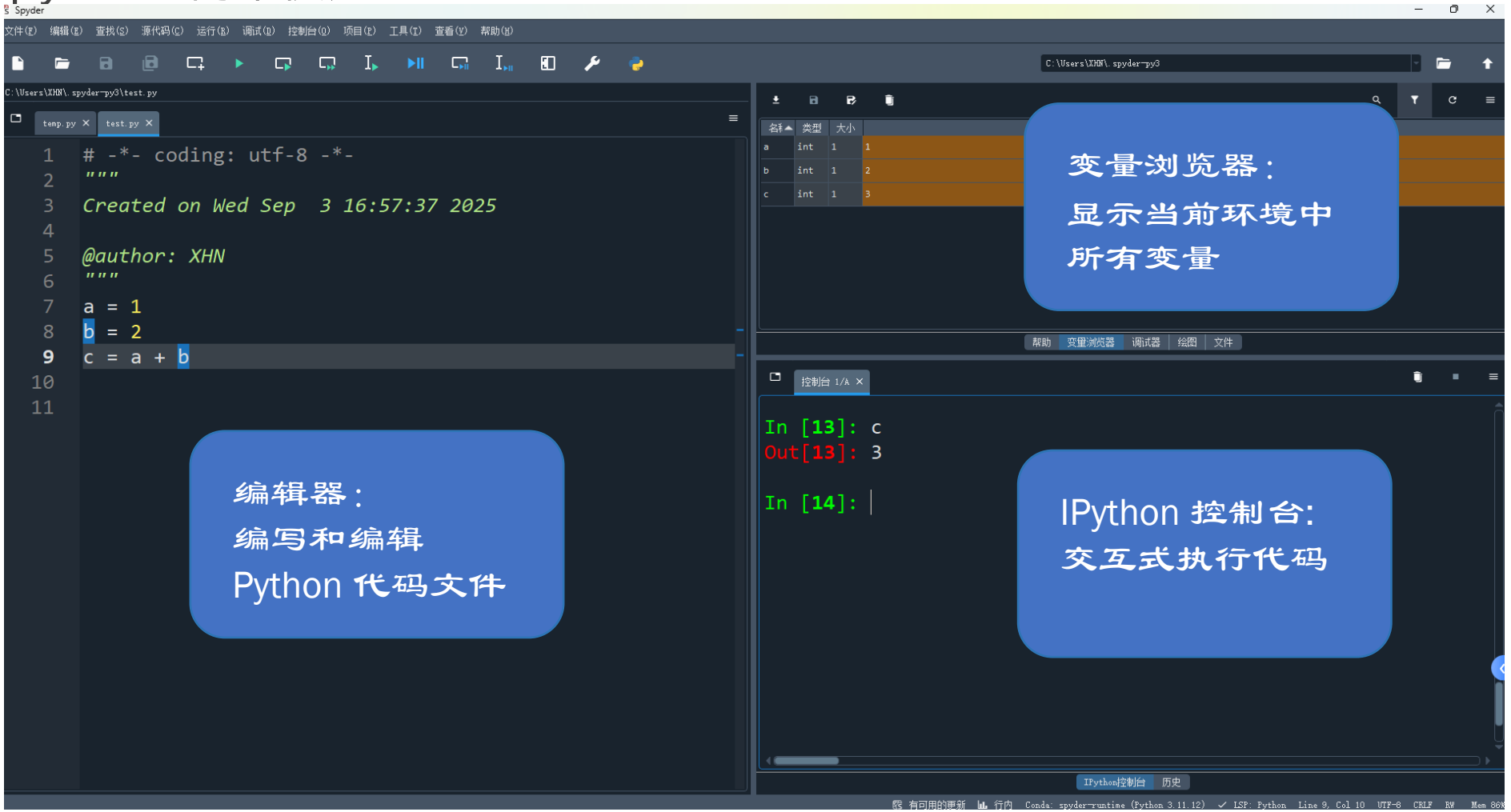
1. 打开Spyder

点击电脑左下角的开始图标，
然后点击右上角的全部，
下拉找到Spyder，
点击图标即可打开Spyder界面。



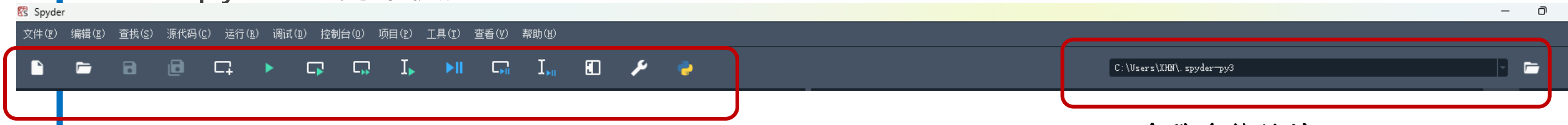
1.3 开发环境的安装与使用

1.3.3 Spyder的简单使用：



1.3 开发环境的安装与使用

1.3.3 Spyder的简单使用：



文件存放地址

一般使用这些功能键即可。

鼠标放到相应图标位置，会有功能显示。

1. 点击左1图标，新建一个test.py文档，保存位置会显示在右上角。
2. 在文档中简单输入如下代码，并点击保存。

```
a = 1
b = 2
c = a + b
```

3. 按运行文件按钮(左6)，会在变量浏览器(右上窗口)中看到三个变量a,b,c的名称、类型、大小、值等变量具体信息，并在右下Ipython控制台输入c，Out显示3。
4. 点击左5，可以新建一个单元格，输入: print(c)，点击左7，运行单元格，即可在Ipython控制台得到输出结果为：Hello。
5. 任选一个单元格，可点击右5图标，可以对此单元格进行调试。右6图标可以调试整个test.py文件。

1.4 Python编程规范与代码优化建议

（1）缩进

✓python程序是依靠代码块的缩进来体现代码之间的逻辑关系的，缩进结束就表示一个代码块结束了。

✓需要缩进的代码块：类定义、函数定义、选择结构、循环结构、with块。

✓行尾的冒号表示缩进的开始；同一个级别的代码块的缩进量必须相同。

✓一般而言，以4个空格为基本缩进单位。

```
with open(fn) as fp:
    for line in csv.reader(fp):
        if line:
            print(*line)
```

1.4 Python编程规范与代码优化建议

✓

```
if True:
    print("如果为真，输出：")
    print(True)
else:
    print("否则，输出：")
    print(False)
```

✗

```
if True:
    print("如果为真，输出：")
    print(True)
else:
 print("否则，输出：")
print(False)
```

IndentationError: unindent does not match any outer indentation level #缩进异常

1.4 Python编程规范与代码优化建议

（2）每个import语句只导入一个模块，最好按标准库、扩展库、自定义库的顺序依次导入。

```
import csv
import random
import datetime
import pandas as pd
import matplotlib.pyplot as plt
```

1.4 Python编程规范与代码优化建议

（3）最好在每个类、函数定义和一段完整的功能代码之后增加一个空行，在运算符两侧各增加一个空格，逗号后面增加一个空格。

```
    sockDianming.close()
    sockDianming = None

# 开始点名
def buttonStartDianmingClick():
    # 如果还没有注册，拒绝运行
    if int_zhuce.get() == 0:
        tkinter.messagebox.showerror('很抱歉', '请联系作者进行软件注册！')
        return
    if int_zuoye.get() == 1:
        tkinter.messagebox.showerror('很抱歉', '现在正在收作业')
        return
    if int_canDianming.get() == 1:
        tkinter.messagebox.showerror('很抱歉', '现在正在点名')
        return
    tkinter.messagebox.showinfo('恭喜', '设置成功，现在开始点名')
    #开始点名
    int_canDianming.set(1)
    global tDianming_id
    t = threading.Thread(target=thread_Dianming)
    t.start()
    tDianming_id = t.ident
buttonStartDianming = tkinter.Button(root, text='开始点名', command=buttonStartD
buttonStartDianming.place(x=20, y=60, height=30, width=100)

def buttonStopDianmingClick():
    # 如果还没有注册，拒绝运行
    if int_zhuce.get() == 0:
        tkinter.messagebox.showerror('很抱歉', '请联系作者进行软件注册！')
```

空行

有空格

没有空格

空行

1.4 Python编程规范与代码优化建议

（4）尽量不要写过长的语句。如果语句确实太长，最好使用续行符“\”，或者使用圆括号将多行代码括起来表示是一条语句(推荐！)。（建议一行最多79个字符）

```
x = 1 + 2 + 3\
    + 4 + 5\
    + 6

y = (1 + 2 + 3
     + 4 + 5
     + 6)
```

1.4 Python编程规范与代码优化建议

（5）虽然Python运算符有明确的优先级，但对于复杂的表达式建议在适当的位置使用括号使得各种运算的隶属关系和顺序更加明确、清晰。

#没有括号的写法

```
result = 2 + 3 * 4 ** 2 / 8 - 1
print(result)  # 输出：7.0
```

#有括号的清晰写法：

```
result = 2+(3*(4**2))/8-1
print(result)   # 输出：7.0
```

1.4 Python编程规范与代码优化建议

（6）注释

✓ 以符号#开始，表示本行#之后的内容为注释。

```
#在这里单行注释
print("单行注释") #或在这里
```

✓ 包含在一对三引号'''...'''或"""..."""之间且不属于任何语句的内容将被解释器认为是注释。

```
'''
多行注释
三对单引号注释
'''
print("多行注释")
```

```
"""
多行注释
三对双引号注释
"""
print("多行注释")
```

1.4 Python编程规范与代码优化建议

**（7）在开发速度和运行速度之间尽量取得最佳平衡。

✓内置对象运行速度最快，标准库对象次之，用C或Fortran编写的扩展库速度也比较快，而纯Python的扩展库往往速度慢一些。

✓在开发项目时，应优先使用Python内置对象，其次考虑使用Python标准库提供的对象，最后考虑使用第三方扩展库。

1.4 Python编程规范与代码优化建议

**（8）根据运算特点选择最合适的数据类型来提高程序的运行效率。

✓如果定义一些数据只是用来频繁遍历并且关心顺序，最好优先考虑元组。

✓如果需要频繁地测试一个元素是否存在于一个序列中并且不关心其顺序，尽量采用集合。

✓列表和元组的in操作的时间复杂度是线性的，而对于集合和字典却是常数级的，与问题规模几乎无关。

1.4 Python编程规范与代码优化建议

**（9）充分利用关系运算符以及逻辑运算符and和or的惰性求值特点，合理组织条件表达式中多个条件的先后顺序，减少不必要的计算。

**（10）充分利用生成器对象或类似迭代对象的惰性计算特点，尽量避免将其转换为列表、元组等类型，这样可以减少对内存的占用，降低空间复杂度。

1.4 Python编程规范与代码优化建议

更详细的编程规范：

https://www.python.org/dev/peps/pep-0008/

PEP 8 -- Style Guide for Python Code

PEP:	8
Title:	Style Guide for Python Code
Author:	Guido van Rossum <guido at python.org>, Barry Warsaw <barry at python.org>, Nick Coghlan <ncoghlan at gmail.com>
Status:	Active
Type:	Process
Created:	05-Jul-2001
Post-History:	05-Jul-2001, 01-Aug-2013

Practice

- 缩进的作用及形式?

✓ 用缩进表示语句所属的代码块,缩进结束表示代码块的结束。
✓ 行尾的冒号表示缩进的开始，下一行缩进四个空格。

- 两种为Python程序添加注释的方法?

✓ #和独立的一对三引号

1.4 Python编程规范与代码优化建议

```
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

1.4 Python编程规范与代码优化建议

- Python之禅 by Tim Peters
- 优美胜于丑陋（Python 以编写优美的代码为目标）
- 明了胜于晦涩（优美的代码应当是明了的，命名规范，风格相似）
- 简洁胜于复杂（优美的代码应当是简洁的，不要有复杂的内部实现）
- 复杂胜于凌乱（如果复杂不可避免，那代码间也不能有难懂的关系，要保持接口简洁）
- 扁平胜于嵌套（优美的代码应当是扁平的，不能有太多的嵌套）
- 间隔胜于紧凑（优美的代码有适当的间隔，不要奢望一行代码解决问题）
- 可读性很重要（优美的代码是可读的）
- 即便假借特例的实用性之名，也不可违背这些规则（这些规则至高无上）
- 不要包容所有错误，除非你确定需要这样做（精准地捕获异常，不写 except:pass 风格的代码）
- 当存在多种可能，不要尝试去猜测而是尽量找一种，最好是唯一一种明显的解决方案（如果不确定，就用穷举法）
- 虽然这并不容易，因为你不是 Python 之父（这里的 Dutch 是指 Guido ）
- 做也许好过不做，但不假思索就动手还不如不做（动手之前要细思量）
- 如果你无法向人描述你的方案，那肯定不是一个好方案；反之亦然（方案测评标准）
- 命名空间是一种绝妙的理念，我们应当多加利用（倡导与号召）

翻译：赖勇浩

https://blog.csdn.net/gzlaiyonghao/article/details/2151918

1.5 安装扩展库的几种方法

Python中的库分为3类：

- **内置标准库** Python的官方库，可以直接导入程序
- **第三方扩展库** 非官方制作发布的库，需要用户安装后才能使用
- **自定义库** 用户自行编写的库，对功能性代码块进行复用

- pip在线安装（命令提示符环境，切换至Python安装目录中的scripts文件夹执行pip.exe）
- pip离线安装：https://www.lfd.uci.edu/~gohlke/pythonlibs/ (需要手动解决依赖问题)
- 如果机器上安装了多个Python开发环境，那么在一个环境下安装的扩展库无法在另一个环境下使用，需要分别安装

1.5 安装扩展库的几种方法

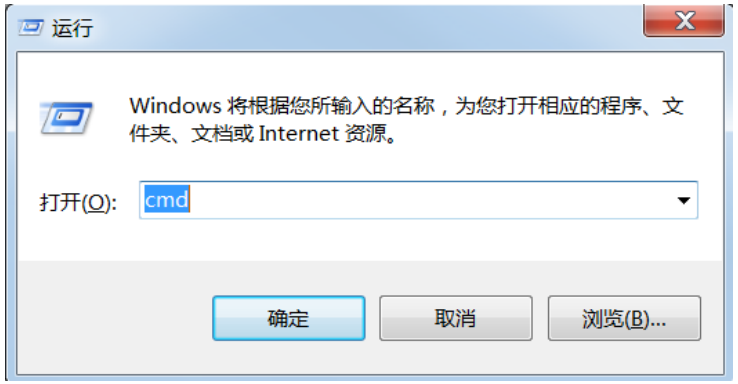
pip命令示例	说明
pip download SomePackage[==version]	下载扩展库的指定版本，不安装
pip freeze [> requirements.txt]	以requirements的格式列出已安装模块
pip list	列出当前已安装的所有模块
pip install SomePackage[==version]	在线安装SomePackage模块的指定版本
pip install SomePackage.whl	通过whl文件离线安装扩展库
pip install package1 package2 ...	依次（在线）安装package1、package2等扩展模块
pip install -r requirements.txt	安装requirements.txt文件中指定的扩展库
pip install --upgrade SomePackage	升级SomePackage模块
pip uninstall SomePackage[==version]	卸载SomePackage模块的指定版本

把SomePackage替换为实际要安装或卸载的扩展库名

1.5 安装扩展库的几种方法

1.IDLE环境在线安装：

Windows-Key + R



然后点：确定



如果python安装在C盘根目录下，直接安装即可

如果python安装在D盘根目录下，需要切换文件夹到D盘，

输入：

```
d:
cd D:\Python\Python310   # 根据你的实际安装路径修改
```

尝试安装numpy库：

```
C:\Users\XHN>pip install numpy
Collecting numpy
  Using cached numpy-1.19.5-cp36-cp36m-win_amd64.whl (13.2 MB)
Installing collected packages: numpy
Successfully installed numpy-1.19.5
```

```
C:\Users\XHN>pip list
Package              Version
-------------------- -----------
beautifulsoup4       4.12.3
certifi              2025.4.26
charset-normalizer   2.0.12
cycler               0.11.0
idna                 3.10
jieba                0.42.1
kiwisolver           1.3.1
matplotlib           3.3.4
mpmath               1.3.0
numpy                1.19.5
```

1.5 安装扩展库的几种方法

1.IDLE环境在线安装：

尝试安装numpy库：

安装：pip install numpy

```
C:\Users\XHN>pip install numpy
Collecting numpy
  Using cached numpy-1.19.5-cp36-cp36m-win_amd64.whl (13.2 MB)
Installing collected packages: numpy
Successfully installed numpy-1.19.5
```

安装成功：Succeesfully installed numpy...

查看：pip list

```
C:\Users\XHN>pip list
Package            Version
------------------ -----------
beautifulsoup4     4.12.3
certifi            2025.4.26
charset-normalizer 2.0.12
cycler             0.11.0
idna               3.10
jieba              0.42.1
kiwisolver         1.3.1
matplotlib         3.3.4
mpmath             1.3.0
numpy              1.19.5
```

1.5 安装扩展库的几种方法

2.Spyder环境在线安装：(最好为每个环境单独安装)

尝试安装numpy库：
(在控制台窗口直接输入命令即可，只是需要操作命令前加‘！’)

安装：!pip install numpy

```
控制台 1/A ×

In [2]: !pip install numpy
Collecting numpy
  Using cached numpy-1.19.5-cp36-cp36m-win_amd64.whl (13.2
MB)
Installing collected packages: numpy
Successfully installed numpy-1.19.5
```

安装成功：Succeesfully installed numpy...

查看：!pip list

```
控制台 1/A ×

In [3]: !pip list
Package            Version
------------------ -----------
beautifulsoup4     4.12.3
certifi            2025.4.26
charset-normalizer 2.0.12
cycler             0.11.0
idna               3.10
jieba              0.42.1
kiwisolver         1.3.1
matplotlib         3.3.4
mpmath             1.3.0
numpy              1.19.5
```

1.5 安装扩展库的几种方法

3.离线安装：(常用清华镜像)

IDLE环境：在命令提示符中输入

pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple

Spyder环境：在控制台中输入

!pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple

1.6 标准库与扩展库对象的导入与使用

- Python启动时仅加载了基本模块，在需要时再显式地导入标准库和第三方扩展库（需正确安装），这样可以减小程序运行的压力，并且具有很强的可扩展性。
- Python标准库在线文档：https://docs.python.org/3/library/

数学计算：

```
math       # 数学函数
random     # 随机数生成
statistics # 统计计算
```

日期和时间：

```
datetime # 日期时间处理
         #（最常用）
time     # 时间相关功能
```

文件和系统处理：

```
os         # 文件和目录操作
sys        # 系统参数和函数
pathlib    # 现代路径操作
```

1.6 标准库与扩展库对象的导入与使用

导入模块有三种方法

- **import 模块名 [as 别名]**
- **from 模块名 import 对象名 [as 别名]**
- **from 模块名 import *** #导入全部对象

1.6.1 import 模块名 [as 别名]

```
>>> import math                #导入标准库math，最常用方式
>>> help(math)                 #输出库的帮助内容
```

```
>>> import math
>>> help(math)
Help on built-in module math:

NAME
    math

DESCRIPTION
    This module is always available.  It provides access to the
    mathematical functions defined by the C standard.

FUNCTIONS
    acos(...)
        acos(x)

        Return the arc cosine (measured in radians) of x.

    acosh(...)
        acosh(x)

        Return the inverse hyperbolic cosine of x.
```

网上中文版帮助文档
https://docs.python.org/zh-cn/3.8/library/math.html

1.6.1 import 模块名 [as 别名]

>>> help(math.sin)　　#输出库中函数的帮助内容，注意sin无括号

```
>>> help(math.sin)
Help on built-in function sin in module math:

sin(...)
    sin(x)

    Return the sine of x (measured in radians).
```

1.6.1 import 模块名 [as 别名]

```
>>> import math                    #导入标准库math，最常用方式
>>> math.sin(0.5)                 #求0.5（单位是弧度）的正弦
0.479425538604203

>>> import random                  #导入标准库random
>>> n = random.random()           #获得[0,1) 内的随机小数

>>> import os.path as path         #导入标准库os.path，并设置别名为path
>>> path.isfile(r'C:\windows\notepad.exe')
True

>>> import numpy as np            #导入扩展库numpy，并设置别名为np
>>> a = np.array((1,2,3,4))       #通过模块的别名来访问其中的对象
>>> a
array([1, 2, 3, 4])
```

- 一点.表示成员访问符
- 函数名(参数) 表示函数调用，一对()是必需的

1.6.2 from 模块名 import 对象名[as 别名]

```
>>> from math import sin,cos        #只导入模块中的指定对象，访问速度略快
>>> sin(3)
0.1411200080598672
>>> from math import sin as f    #给导入的对象起个别名
>>> f(3)
0.1411200080598672
>>>from math import log as _log, exp as _exp, pi as _pi, e as _e, ceil as _ceil
```

1.6.3 from 模块名 import *

```
>>> from math import *        #导入标准库math中所有对象
>>> sin(3)                    #求正弦值
0.1411200080598672
>>> gcd(36, 18)               #最大公约数
18
>>> pi                        #常数π
3.141592653589793
>>> e                         #常数e
2.718281828459045
>>> log2(8)                   #计算以2为底的对数值
3.0
>>> log10(100)                #计算以10为底的对数值
2.0
>>> radians(180)              #把角度转换为弧度
3.141592653589793
```

在当前交互式解释器环境中运行源代码文件，如ch01.py

```
from ch01 import *
```

Practice

```
>>> ceil(1.2)
>>> math.ceil(1.2)
>>> mt.ceil(1.2)
```

为正确调用math模块中的ceil()函数，对上面三个命令，应分别使用什么语句来导入math模块？

from math import *
import math
import math as mt

习题(只提问不上交)

习　题

1.1　Python 安装扩展库常用的工具是_____和 conda，其中后者需要安装 Python 集成开发环境 Anaconda3 之后才可以使用。

1.2　Python 程序文件的扩展名主要有_____和_____两种，其中后者常用于 GUI 程序。

1.3　多选题：下面属于 Python 语言特点的有（　　）。

A. 开源　　B. 免费　　C. 跨平台　　D. 解释执行

1.4　多选题：下面导入标准库对象的语句中正确的有（　　）。

A. from math import sin

B. from random import random

C. from math import *

D. import *

1.5　多选题：下面描述中符合 Python 语言的有（　　）。

A. 跨平台　　B. 开源　　C. 解释执行　　D. 支持函数式编程

1.6　多选题：下面描述中符合 Python 语言的有（　　）。

A. 跨平台　　B. 开源　　C. 免费　　D. 扩展库丰富

习题(只提问不上交)

1.9 多选题：下面（ ）是正确的 Python 标准库对象导入方式。

A. import math.sin　　B. from math import sin

C. import math.*　　D. from math import *

1.10 判断对错：安装 Python 扩展库时只能使用 pip 工具联网在线安装，如果安装不成功就没有别的办法了。

1.11 判断对错：不管计算机上安装了多少个 Python 的版本，每个扩展库只需要安装一次，就可以在所有 Python 版本的开发环境中使用了。( )

习题(只提问不上交)

1.16 判断对错：如果只需要 math 模块中的 sin() 函数，建议使用 from math import sin 来导入，不推荐使用 import math 导入整个模块然后再使用 math.sin() 的方式进行调用。

1.17 判断对错：内置对象在程序中任何位置都可以直接使用，不需要导入任何内置模块、标准库或扩展库。

1.18 判断对错：Python 官方安装包没有包含任何扩展库，只有内置对象、内置模块和标准库，这些是 Python 自带的，不需要导入就可以直接使用。

1.19 判断对错：在编写 Python 程序时，对代码进行缩进只是为了好看，不缩进也不影响程序的正常执行。

1.20 判断对错：在编写程序时，不管一条语句有多长，都必须写在同一行中，不能换行。

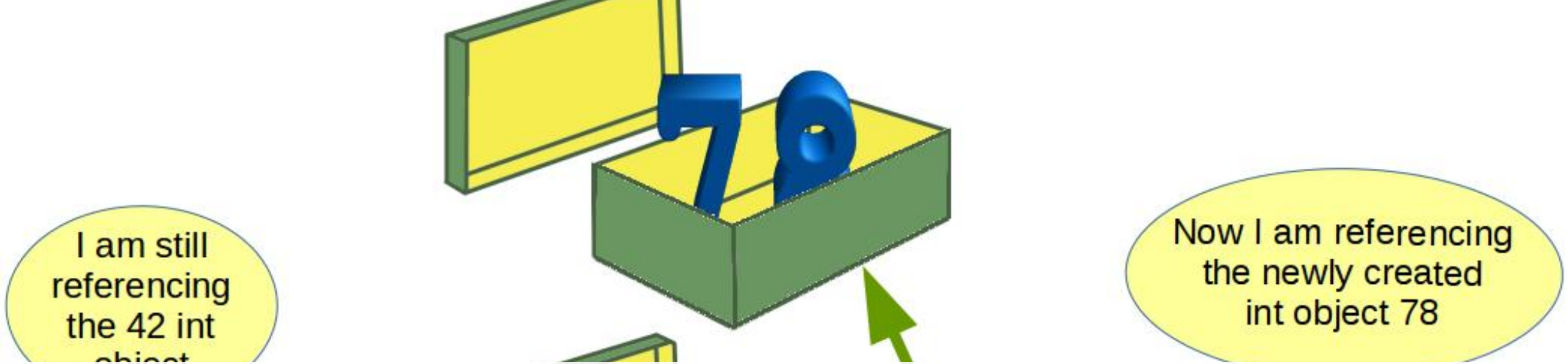
作业(上交)

1. 使用pip install安装requests包，然后写下pip list完整的输出

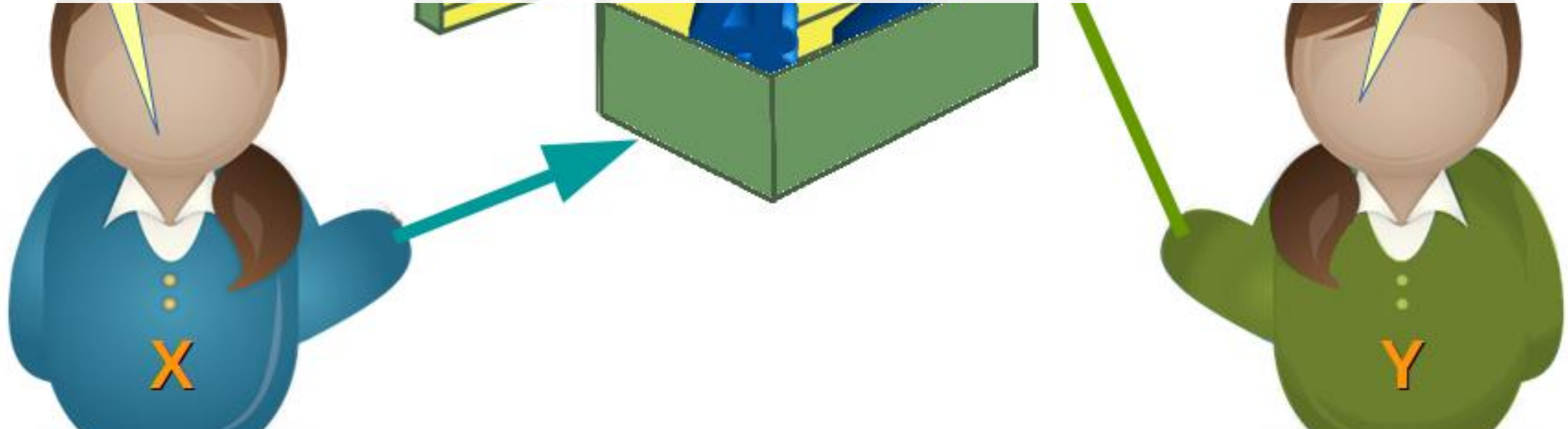
2. 设三角形三条边长分别为a,b,c，计算边长c所对的角(theta)的度数
(单位为度，不是弧度)
help(math)，**表示幂运算，其他算术运算符同C语言

3.

4.



第2章内置对象、运算符与表达式



第2章 万丈高楼平地起: 运算符、表达式与内置对象

- 2.1 Python常用内置对象
  - 2.1.1 常量与变量
  - 2.1.2 数字
  - 2.1.3 字符串与字节串
  - 2.1.4 列表、元组、字典、集合
  - 其他可迭代对象
  - 布尔型

2.1 Python常用内置对象

- 对象是Python语言中最基本的概念，在Python中处理的一切都是对象。

- Python完美的支持面向对象编程。一个抽象的例子：

➢ 抽象出一个类(class)：矩阵(matrix)

$$a_matrix = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

➢ a_matrix是matrix的一个对象，它由数据和行为构成

➢ a_matrix的数据：矩阵元素的值、矩阵的行数和列数、....，使用a_matrix.shape等访问这些数据

➢ a_matrix的行为：矩阵的转置、删除矩阵的一行、....，使用a_matrix.transpose()等执行这些行为

2.1 Python常用内置对象

- Python中有许多内置对象可供编程者使用，内置对象可直接使用，如数字、字符串、列表等。
- 非内置对象需要导入模块才能使用，如numpy中的array对象。

2.1 Python常用内置对象

常用内置对象

对象类型	类型名称	示例	简要说明
数字	**int** **float** **complex**	1234 3.14, 1.3e5 3+4j	数字大小没有限制，内置支持复数及其运算
字符串	**str**	'swfu', "I'm student", '''Python ''', r'abc', R'bcd'	使用单引号、双引号、三引号作为定界符，以字母r或R引导的表示原始字符串
字节串	**bytes**	b'hello world'	以字母b引导，可以使用单引号、双引号、三引号作为定界符
列表	**list**	[1, 2, 3] ['a', 'b', ['c', 2]]	所有元素放在一对方括号中，元素之间使用逗号分隔，其中的元素可以是任意类型
字典	**dict**	{1:'food' ,2:'taste', 3:'import'}	所有元素放在一对大括号中，元素之间使用逗号分隔，**元素形式为“键:值”**
元组	**tuple**	(2, -5, 6) (3,)	**不可变**，所有元素放在一对圆括号中，元素之间使用逗号分隔，**如果元组中只有一个元素的话，后面的逗号不能省略**
集合	**set** frozenset	{'a', 'b', 'c'}	所有元素放在一对大括号中，元素之间使用逗号分隔，**元素不允许重复**;另外，set是可变的，而frozenset是不可变的

2.1 Python常用内置对象

续表

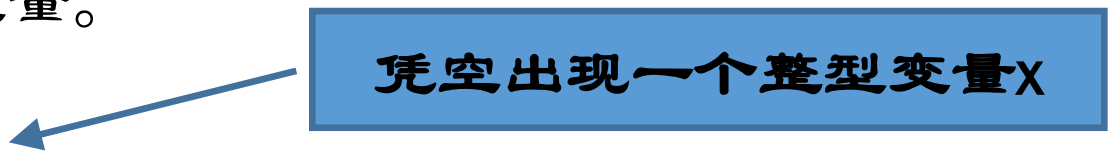
对象类型	类型名称	示例	简要说明
布尔型	**bool**	True, False	逻辑值，关系运算符、成员测试运算符、同一性测试运算符组成的表达式的值一般为True或False
空类型	NoneType	None	空值
异常	Exception ValueError TypeError		Python内置大量异常类，分别对应不同类型的异常
文件		f = open('data.dat', 'rb')	open是Python内置函数，使用指定的模式打开文件，返回文件对象
其他可迭代对象		生成器对象、range对象、zip对象、enumerate对象、map对象、filter对象等等	具有**惰性求值**的特点，除range对象之外，其他对象中的元素只能看一次
编程单元		函数（使用def定义） 类（使用class定义） 模块（类型为module）	类和函数都属于**可调用对象**，模块用来集中存放函数、类、常量或其他对象

第2章 万丈高楼平地起: 运算符、表达式与内置对象

- 2.1 Python常用内置对象
  - **2.1.1 常量与变量**
  - 2.1.2 数字
  - 2.1.3 字符串与字节串
  - 2.1.4 列表、元组、字典、集合
  - 其他可迭代对象
  - 布尔型

2.1.1 常量与变量

- 在Python中，**不需要事先声明变量名及其类型，直接赋值**即可创建各种类型的对象变量。




```
>>> x = 3
```

```
>>> x = 'Hello world.'
```



- Python中对象自带类型，变量x只是对象的一个标签，找到这个标签就可以使用数据。

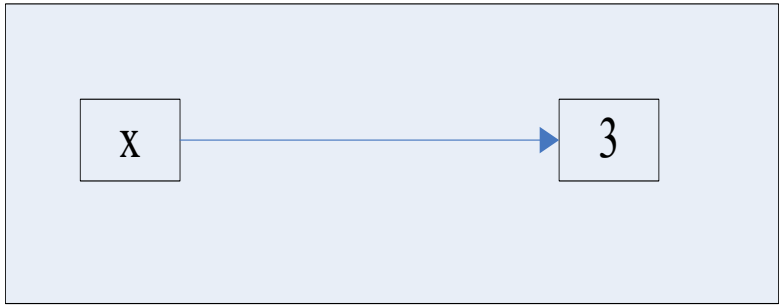
C语言中变量带类型，是容器。

python: x = 3 #给3这个值贴一个标签x
C: x = 3 #给x这个盒子里装入3

2.1.1 常量与变量

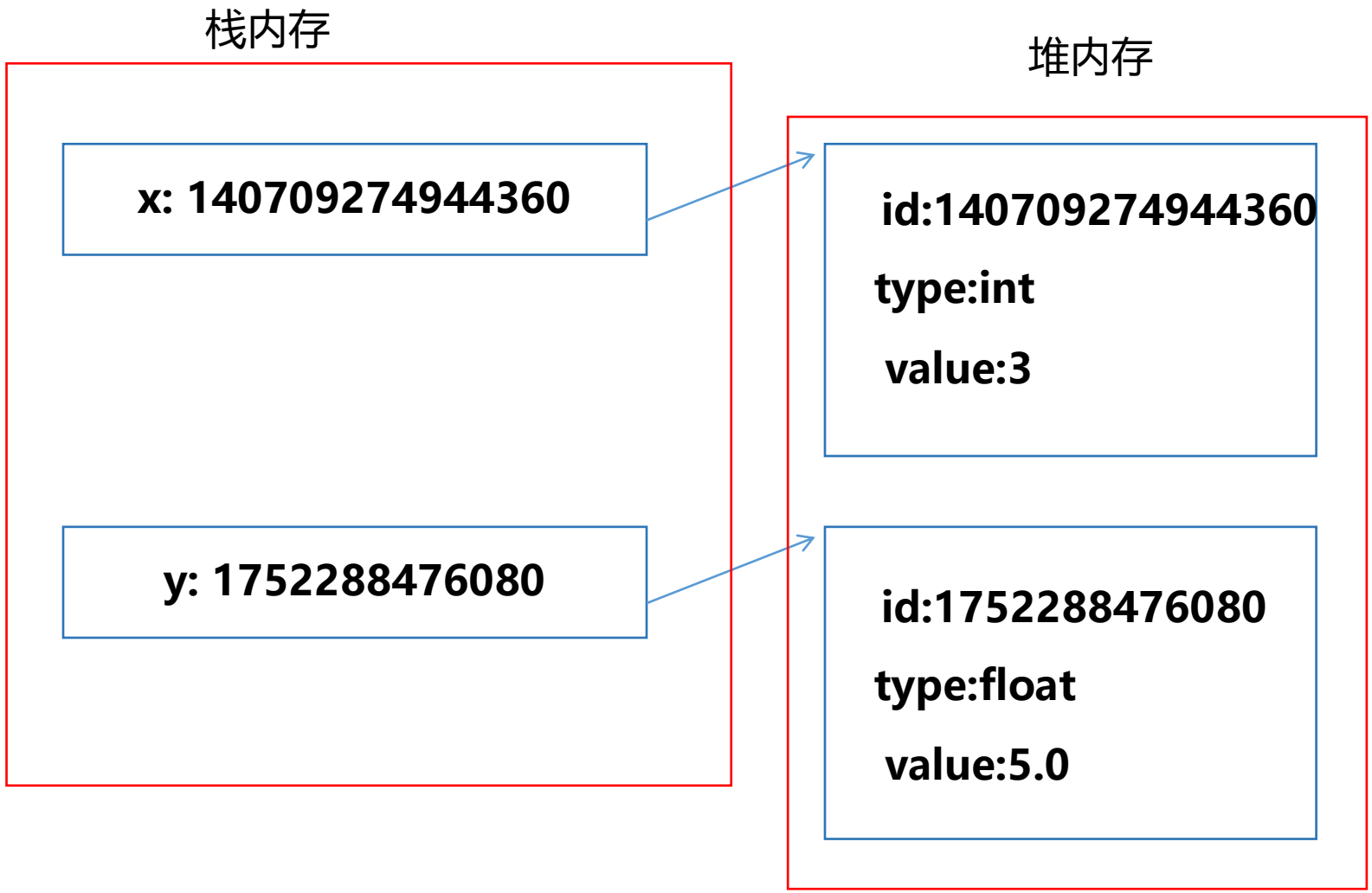
- 赋值语句的执行过程是：首先把等号右侧表达式的值计算出来，然后在内存中寻找一个位置把值存放进去，最后创建变量并**指向**这个内存地址。

```
>>> x = 3
```



- Python采用的是**基于值的内存管理方式**：Python中的变量并不直接存储值，而是存储了值的内存地址或者引用，因此变量类型随时可以改变（**动态类型**）。

2.1.1 常量与变量


栈内存
堆内存
x: 140709274944360
y: 1752288476080
id:140709274944360
type:int
value:3
id:1752288476080
type:float
value:5.0

2.1.1 常量与变量

- 在Python中，允许多个变量指向同一个值，例如：

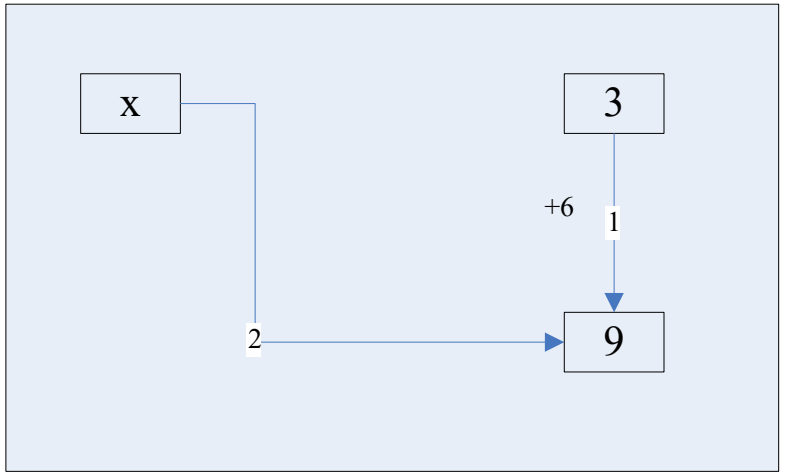
```
>>> x = 3
>>> id(x)
1786684560
>>> y = x
>>> id(y)
1786684560
```

id()函数用来查看对象的内存地址

```
id(obj, /)
    Return the identity of an object.
```

- 接着上面的代码再继续执行下面的代码：

```
>>> x = x+6
>>> id(x)
1786684752
```



2.1.1 常量与变量

❖Python属于**强类型编程语言**，Python解释器会根据所赋的值来自动**推断**变量类型，可以使用内置函数type()返回变量类型。

```
>>> x = 3
>>>type(x)
<class 'int'>
>>> x = 'Hello world.'
>>> type(x)
<class 'str'>
>>> type(type(x))
<class 'type'>
```

#查看变量类型

```
class type(object)
|  type(object) -> the object's type
```

```
>>> isinstance(x, int)
False
>>> isinstance(x, str)
True
```

#测试对象是否是某个类型的实例

```
isinstance(obj, class_or_tuple, /)
    Return whether an object is an instance of a class or of a
subclass thereof.
```

查内置函数: dir(__builtins__)

https://docs.python.org/zh-cn/3/library/functions.html

2.1.1 常量与变量

- Python语言规定，变量名由字母、数字和下划线组成，且不允许以数字开头。在使用变量名时应注意以下几点：

✓ 变量名**必须**以字母或下划线开头，但以下划线开头的变量在Python中有特殊含义；

✓ 变量名中**不能**有空格以及标点符号；

✓ **不能**使用关键字作变量名；

✓ 变量名对英文字母的**大小写敏感**，例如student和Student是不同的变量。

✓ **不建议**使用系统内置的模块名、类型名或函数名以及已导入的模块名及其成员名作变量名，**这将会改变其类型和含义**；

```
>>> id(3)
1667343520
>>> id = '3724....'
>>> id(3)
TypeError: 'str' object is not callable
>>> id = __builtins__.id
#可恢复id的内置函数属性
>>> id(3)
```

合法变量名　　student_1、addNumber、_num

不合法变量名　　3number、3 num、2_student、class、math

2.1.1 常量与变量

常见的变量名命名方式有以下两种。

- **下划线命名法。**用下划线分割小写字母段或者大写字母段，例如my_name、my_age、GlOBAL_NAME等。
- **驼峰式命名法，包括小驼峰法和大驼峰法。**其中小驼峰法是指首字母小写，其他单词的首字母大写，例如myName、myAge、myStudentCount等。大驼峰法又称帕斯卡命名法，是指首字母大写的多个单词，例如MyName、MyAge、MyStudentCount等。

第2章 万丈高楼平地起: 运算符、表达式与内置对象

- 2.1 Python常用内置对象
  - 2.1.1 常量与变量
  - **2.1.2 数字**
  - 2.1.3 字符串与字节串
  - 2.1.4 列表、元组、字典、集合
  - 其他可迭代对象
  - 布尔型

2.1.2 数字

- Python支持任意大的整数
- 由于精度的问题，对于实数运算可能会有一定的误差，应尽量避免在实数之间直接进行相等性测试，而是应该以二者之差的绝对值是否足够小作为两个实数是否相等的依据。
- 在数字的算术运算表达式求值时会进行隐式的类型转换，如果存在复数则都变成复数，如果没有复数但是有实数就都变成实数

2.1.2 数字

```
>>> 9999 ** 99                    #这里**是幂乘运算符


>>> 0.4 - 0.1                     #实数相减，结果稍微有点偏差
0.30000000000000004
>>> 0.4 - 0.1 == 0.3              #应尽量避免直接比较两个实数是否相等
False
>>> abs(0.4-0.1 - 0.3) < 1e-6     #这里1e-6表示10的-6次方
True
```

```
>>> type(99)
<class 'int'>

>>> type(0.4)
<class 'float'>
```

2.1.2 数字

```
#浮点数的精度：约16位有效数字

>>> x = 1.0
>>> y = 1.0 + 2.0e-17
>>> y - x > 1.0e-17
False

>>> x = 1.0
>>> y = 1.0 + 2.0e-16
>>> y - x > 1.0e-16
True
```

2.1.2 数字

- Python内置支持复数类型及其运算，并且形式与数学上的复数完全一致。

```
>>> x = 3 + 4j                  #使用j或J表示复数虚部
>>> y = 5 + 6j

>>> x * y                       #支持复数之间的加、减、乘、除以及幂乘等运算
(-9+38j)

>>> abs(x)                      #内置函数abs()可用来计算复数的模
5.0

>>> x.imag                      #虚部
4.0
>>> x.real                      #实部
3.0

>>> x.conjugate()               #共轭复数
(3-4j)
```

```
>>> type(4j)
<class 'complex'>
>>> type(4J)
<class 'complex'>

imag和real为对象x的属性(attribute)
conjugate()为对象x的方法(method)
```

2.1.2 数字

- Python内置支持复数类型及其运算，并且形式与数学上的复数完全一致。

```
>>> x = 3 + 4j                  #使用j或J表示复数虚部

>>> x ** 3                      #复数幂运算
(-117+44j)

>>> x ** (1/2)
(2+1j)

>>> import math
>>> math.log(x)    #不支持
TypeError: must be real number, not complex

>>> import numpy as np
>>> np.log(x)
np.complex128(1.6094379124341003+0.9272952180016122j)
```

2.1.2 数字

- Python标准库fractions中的Fraction对象支持分数及其运算

```
>>> from fractions import Fraction

>>> x = Fraction(3, 5)     #创建分数对象
>>> y = Fraction(3, 7)
>>> x
Fraction(3, 5)

>>> x.numerator            #查看分子
3
>>> x.denominator          #查看分母
5
>>> x + y                  #支持分数之间的四则运算，自动进行通分
Fraction(36, 35)
>>> x ** 2                 #支持幂运算
Fraction(9, 25)

>>> str(x)                 #类型转换
'3/5'
>>> float(x)               #类型转换
0.6
```

```
>>> import fractions
>>> dir(fractions)
```

2.1.2 数字

- 标准库fractions中提供的Decimal类实现了更高精度实数的运算。

```
>>> from fractions import Decimal
>>> 1 / 9                    #内置的实数类型
0.1111111111111111

>>> Decimal(1/9)             #高精度实数，但是不准确
Decimal('0.111111111111111104943205418749130330979824066162109375')

>>> Decimal(1)/Decimal(9)  #高精度，且准确
Decimal('0.111111111111111111111111111111')
```

2.1.2 数字

- 标准库random可产生各种随机数

```
>>> import random

>>> random.randrange(0, 10)
5   #[0, 10)之间的随机整数

>>> random.random()
0.9124818648228413  #[0, 1)之间的随机浮点数

>>> random.uniform(-1, 1)
-0.10770901677038269  #连续均匀分布

>>> sum(random.random() for i in range(10000))/10000
0.504904980528766    #[0,1)之间的随机数的平均值

>>> random.randint(0,10)
6   #[0, 10]之间的随机整数 #包括右端数
```

```
>>> import random
>>> dir(random)
```

https://docs.python.org/zh-cn/3/library/random.html

Practice

- Python“基于值的内存管理方式”的含义？

✓ Python中的变量并不直接存储值，而是存储了值的内存地址或者引用；变量类型随时可以改变

- Python内置的三种数字类型（type）是什么？

✓ int, float, complex

- 为变量x赋值x = 3+4j，x.real和x.imag的类型(type)是什么？

✓ float

第2章 万丈高楼平地起: 运算符、表达式与内置对象

- 2.1 Python常用内置对象
  - 2.1.1 常量与变量
  - 2.1.2 数字
  - **2.1.3 字符串与字节串**
  - 2.1.4 列表、元组、字典、集合
  - 其他可迭代对象
  - 布尔型

2.1.3 字符串与字节串

- 在Python中，单个字符也是字符串。使用单引号、双引号、三单引号、三双引号作为定界符（delimiter）来表示字符串，并且不同的定界符之间可以互相嵌套。
- Python 3.x全面支持中文，中文和英文字母都作为一个字符对待，甚至可以使用中文作为变量名。

2.1.3 字符串与字节串

```
>>> x = 'Hello world.'                    #使用单引号作为定界符
>>> x = "Python is a great language."    #使用双引号作为定界符
>>> x = '''Tom said, "Let's go."'''      #不同定界符之间可以互相嵌套
>>> print(x)
Tom said, "Let's go. “

>>> x = 'good ' + 'morning'              #连接字符串
>>> x
'good morning'
>>> x = 'good ''morning'                 #连接字符串，仅适用于字符串常量
>>> x
'good morning'
>>> x = 'good '
>>> x = x'morning'                       #不适用于字符串变量
SyntaxError: invalid syntax
>>> x = x + 'morning'                    #字符串变量之间的连接可以使用加号
>>> x
'goodmorning'
```

2.1.3 字符串与字节串

```
>>> x = 'good morning'
>>> y = 'good'
>>> x - y    #不支持
TypeError: unsupported operand type(s) for -: 'str' and 'str'

>>> x * 2
'good morninggood morning'


>>>   '0'*100    #可以很容易得到100个0的字符串
'0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000'


>>> x ** 2
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

2.1.3 字符串与字节串

- 对str类型的字符串调用其encode()**方法**进行编码得到bytes字节串，对bytes字节串调用其decode()**方法**并指定**正确的编码格式**则得到str字符串。

```
>>> a_bytes = '工大'.encode('utf-8')
>>> a_bytes
b'\xe5\xb7\xa5\xe5\xa4\xa7'
#\x代表16进制
>>> type(a_bytes)
<class 'bytes'>
>>> a_bytes.decode('utf-8')
'工大‘
>>> '工大'.encode().decode()
'工大‘ #‘utf-8’是默认，可不写
>>> len('工大')
2
>>> len(a_bytes)
```

```
len(obj, /)
    Return the number of items in a container.
```

第2章 万丈高楼平地起: 运算符、表达式与内置对象

- 2.1 Python常用内置对象
  - 2.1.1 常量与变量
  - 2.1.2 数字
  - 2.1.3 字符串与字节串
  - **2.1.4 列表、元组、字典、集合**
  - 其他可迭代对象
  - 布尔型

2.1.4 列表、元组、字典、集合

列表：打了激素的数组

```
>>> x_list = [1, 2, ‘工大’]                #创建列表对象
```

元组：轻量级列表

```
>>> x_tuple = (1,0.2,'a')                  #创建元组对象
```

字典：反映对应关系的函数映射

```
>>> x_dict = {'a':97, 'b':98, 'c':99}     #创建字典对象
```

集合：元素之间不允许重复

```
>>> x_set = {1, 2, 3}                     #创建集合对象
```

2.1.4 列表、元组、字典、集合

	列表(list)	元组(tuple)	字典(dict)	集合(set)
类型名称	list	tuple	dict	set
定界符	方括号[]	圆括号()	大括号{}	大括号{}
是否可变	是	否	是	是
是否有序	是	是	否	否
是否支持下标	是（使用序号作为下标）	是（使用序号作为下标）	是（使用“键”作为下标）	否
元素分隔符	逗号	逗号	逗号	逗号
对元素形式的要求	无	无	键:值	必须可哈希
对元素值的要求	无	无	“键”必须**可哈希（不可改变，例：数字、字符串、元组）**	必须可哈希
元素是否可重复	是	是	“键”不允许重复，“值”可以重复	否
元素查找速度	非常慢	很慢	非常快	非常快
新增和删除元素速度	尾部操作快 其他位置慢	不允许	快	快

2.1.4 列表、元组、字典、集合

■对象“不可变”(可哈希)的含义?

- python为每个对象分配一块内存空间，“不可变”意味着一旦生成该对象，为分配的内存空间填好数据，就不可以对这块内存空间做任何改变。
- 分配给元组的内存空间实际存储的是其他对象的地址；元组不可变只意味着不能再让元组中的元素指向新的对象，也不能再增加和减少元组中的元素，但可对元组中的对象做原地操作。

■不可变对象：int, float, complex, bool, **str**, bytes, **tuple**

■可变对象：list, dict, set

2.1.4 列表、元组、字典、集合

```
>>> x_list = [1, 2, 3]                          #创建列表对象
>>> x_tuple = (1, 0.2, 3)                       #创建元组对象
>>> x_dict = {'a':97, 'b':98, 'c':99}           #创建字典对象
>>> x_set = {1, 2, 3}                           #创建集合对象
>>> print(x_list[1])                            #使用下标访问指定位置的元素
2
>>> print(x_tuple[1])                           #元组也支持使用序号作为下标
2
>>> print(x_dict['a'])                          #字典对象的下标是“键”
97
>>> 3 in x_set                                  #成员测试
True
```

```
>>> a = (1,2,[1,2])
>>> a[2]
[1, 2]
>>> a[2][0] = 3      #元组不可变，但可对元组的元素做原地操作
>>> a
(1, 2, [3, 2])
```

第2章　万丈高楼平地起：运算符、表达式与内置对象

- 2.1 Python常用内置对象
  - 2.1.1 常量与变量
  - 2.1.2 数字
  - 2.1.3 字符串与字节串
  - 2.1.4 列表、元组、字典、集合
  - **其他可迭代对象**
  - **布尔型**
  - **日期与时间**

其他可迭代对象(iterator)

■可迭代对象支持成员测试运算：item in x或可以遍历的对象

- generator(生成器)对象
- range对象（内置函数range的返回值）
- zip对象（内置函数zip的返回值）
- enumerate对象（内置函数enumerate的返回值）
- reversed对象（内置函数reversed的返回值）
- （了解）map对象（内置函数map的返回值）
- （了解）filter对象（内置函数filter的返回值）

布尔型

```
>>> type(True)
<class 'bool'>

>>> bool(0)          #0、空的序列和None对应于False
False
>>> bool([])
False
>>> bool('')
False
>>> bool(None)
False

>>> bool(1)
True
>>> bool('a')
True
```

bool(x)	返回与x等价的布尔值 True或False

日期与时间

- 标准库time的time()返回当前时间，以秒为单位，时间零点为1970-1-1 8:00
- time()函数可用于测量程序的运行时间

```
import time   #dir(time)

# 获取当前时间戳（秒数，从1970年1月1日开始）
start_time = time.time()

# 让程序暂停3秒
time.sleep(3)

# 获取3秒后的时间戳
end_time = time.time()

print("开始时间:", start_time)
print("结束时间:", end_time)
print("实际耗时:", end_time - start_time, "秒")
```

```
开始时间: 1700000000.123456
结束时间: 1700000003.126789
实际耗时: 3.003333 秒
```

日期与时间

■ 标准库datetime提供了datetime类，以面向对象编程方式处理时间

```
>>> from datetime import datetime  #从内置datetime模块导入datetime类
                                    #import datetime;dir(datetime)

>>> t1 = datetime(year=2019, month=5, day=17) #使用命名参数创建datetime对象
>>> t2 = datetime(year=2019, month=5, day=21) #可用type(t1)测试datetime是类
>>> t1 < t2
True
```

■ 创建datetime对象的一般格式是

```
datetime(year, month, day, hour=0, minute=0, second=0,
microsecond=0,tzinfo=None)

>>> t = datetime.strptime('2019-5-17', '%Y-%m-%d')
>>> t
datetime.datetime(2019, 5, 17, 0, 0)
#将一个格式化的日期字符串，解析并转换成一个 datetime 对象
```

https://docs.python.org/zh-cn/3/library/datetime.html

日期与时间

```
>>> from datetime import datetime #从Python内置的datetime模块中导入datetime类
>>> t = datetime.now()   #调用datetime类的now()方法，创建了一个datetime对象
datetime.datetime(2025, 9, 20, 18, 1, 6, 858877)
>>> t.ctime()   #current time #ctime()方法将datetime对象转换为易读的字符串格式
'Sat Sep 20 18:03:16 2025'
>>> t.isoformat()  #datetime模块中的ISO 8601标准格式
'2025-09-20T18:08:36.464998'

>>> t1 = datetime(2010, 5, 17, hour=8)
>>> t2 = datetime(2020, 2, 8, hour=10)
>>> dt = t2 - t1
>>> dt
datetime.timedelta(3554, 7200)
>>> dt.days
3554
>>> dt.seconds
7200
```

Practice

- [1,2],{1,2},(1,2),{1:2}的类型(type)分别是什么?

✓ list,set,tuple,dict

- 元组和列表最大的不同点？集合和列表最大的不同点?

✓ 列表可变，元组不可变

✓ 列表中元素可重复，集合中的元素不允许重复

- a = (1,[2,1],[1,2])，a[2][1]的值?

✓ 2

习题

2.3 表达式 3**2 + 5 的值为_____。

2.30 多选题：下面属于可哈希对象的有（ ）。

A. 345 B. '0b10001' C.[1, 2, 3] D.{3}

2.32 单选题：下面不是合法变量名的有（ ）。

A. age B. name C. 3_name D. height

2.33 单选题：下面不是合法变量名的有（ ）。

A. width B. area C. for D. door_number

2.37 多选题：下面属于 Python 内置对象的有（ ）。

A. str B. list C. dict D. set

习题

2.39 多选题：下面属于合法变量名的有（ ）。

A. def B. while C. age D. name

2.40 多选题：下面属于合法变量名的有（ ）。

A. 3name B. name_3 C. 姓名 D. _name

2.42 多选题：下面属于合法数字的有（ ）。

A. 1_234_567 B. 1e8 C. 9.8 D. .3

2.58 判断对错：3+4j 不是合法的 Python 表达式。

2.63 判断对错：4j 是合法 Python 数字类型。

2.65 判断对错：在 Python 程序中可以使用 for 作为变量名。

2.66 判断对错：在 Python 程序中可以使用 while 作为变量名。

休息一下：丹尼斯·里奇(C语言之父)小传（自述）



我于1941年9月9日出生在纽约州布朗克斯维尔（Bronxville），后来在哈佛大学读了本科并进一步深造，我的本科专业是物理学，研究生阶段学的是应用数学。我的博士论文（1968年）是关于函数的子递归层次（subrecursive hierarchies）。本科阶段的学习让我明白，以自己的才智还不足以成为一名物理学者，而往计算机方向发展却相当不错。研究生阶段的经历又让我清醒，自己的才智也不足以让我成为算法理论方面的专家。我自己更喜欢过程式语言，而不是函数式语言。

我在1967年加入了贝尔实验室，算是步父亲的后尘，我的父亲Alistair E. Ritchie就在贝尔实验室工作了很长时间。我为BCPL语言写一个编译器。然后，我帮助Ken Thompson创建了Unix操作系统。

在Unix开发的早期，我对Thompson的B语言进行了改造，添加了数据类型和新语法，由此创造了C语言。C是Unix可移植性的技术基础，后来在许多其他环境下也被广泛采用。从手持设备到超级计算机，各类大小的电脑的许多应用开发和系统开发，都用到了C。这门语言后来出现了统一的美国标准和国际标准，并且Stroustrup在此基础上发明了后续的C++。

我获得过的奖励如下：1974年ACM的杰出论文奖（系统和语言方面）；1982年IEEE的Emmanuel Piore奖；1983年贝尔实验室会士荣誉称号；1983年ACM图灵奖；1983年ACM软件系统奖；1989年NEC的C&C基金奖；1990年IEEE的汉明奖。
我在1988年被选为美国工程院院士。1999年4月获得了美国国家技术奖。这些荣誉都是和Ken Thompson一起获得的。附注：像我这样靠Ken的裙带关系获得好处的人还真不少。但除了他的夫人Bonnie T.，我是少有的几个真的看见过他穿大衣（甚至打上黑领带）的人，还见过不止一次。

第2章　万丈高楼平地起：运算符、表达式与内置对象

- 2.2 Python运算符与表达式
  - 2.2.1 算术运算符
  - 2.2.2 关系运算符
  - 2.2.3 成员测试运算符in与同一性测试运算符is
  - 2.2.4 位运算符与集合运算符
  - 2.2.5 逻辑运算符
  - 2.2.6 矩阵乘法运算符@
  - 2.2.7 补充说明

2.2 Python运算符与表达式

- 运算符优先级遵循的规则为：算术运算符优先级最高，其次是位运算符、成员测试运算符、关系运算符、逻辑运算符等，算术运算符遵循“先乘除，后加减”的基本运算原则。
- 虽然Python运算符有一套严格的优先级规则，但是强烈建议在编写复杂表达式时使用**圆括号**来明确说明其中的逻辑来提高代码可读性。

eg. c/2/a/b 写为 c/(2*a*b)更清晰

2.2 Python运算符与表达式

运算符	功能说明
+	算术加法，**列表、元组、字符串合并与连接**，正号
-	算术减法，**集合差集**，相反数
*	算术乘法，**序列重复**
/	真除法
//	**求整商**，但如果操作数中有实数的话，结果为实数形式的整数
%	求余数，字符串格式化
**	**幂运算**
<、<=、>、>=、==、!=	（值）大小比较，**集合的包含关系比较**
or	**逻辑或**
and	**逻辑与**
not	**逻辑非**
in	**成员测试**
is	**对象同一性测试，即测试是否为同一个对象(内存地址是否相同)**
|、^、&、<<、>>、~	位或、位异或、位与、左移位、右移位、位求反
&、|、^	**集合交集、并集、对称差集**
@	**矩阵相乘运算符**

第2章　万丈高楼平地起：运算符、表达式与内置对象

■ 2.2 Python运算符与表达式

- □ **2.2.1 算术运算符**
- □ 2.2.2 关系运算符
- □ 2.2.3 成员测试运算符in与同一性测试运算符is
- □ 2.2.4 位运算符与集合运算符
- □ 2.2.5 逻辑运算符
- □ 2.2.6 矩阵乘法运算符@
- □ 2.2.7 补充说明

2.2.1 算术运算符

运算符	功能说明
+	算术加法，**列表、元组、字符串合并与连接**，正号
-	算术减法，**集合差集**，相反数
*	算术乘法，**序列重复**
/	真除法
//	**求整商**，但如果操作数中有实数的话，结果为实数形式的整数
%	求余数，字符串格式化
**	**幂运算**

2.2.1 算术运算符

（1）+运算符除了用于算术加法以外，还可以用于**列表、元组、字符串**的连接。

```
>>> [1, 2, 3] + [4, 5, 6]          #连接两个列表
[1, 2, 3, 4, 5, 6]
>>> (1, 2, 3) + (4,)               #连接两个元组
(1, 2, 3, 4)
>>> 'abcd' + '1234'                #连接两个字符串
'abcd1234'
>>>{1,2}  + {3,4}                  #报错，应{1,2}|{3,4}
>>>{1:'a',2:'b'} + {3:'c',4:'d'}       #报错，python3.9后可{1:'a',2:'b'}|{3:'c',4:'d'}

>>> True + 3                       #Python内部把True当作1处理
4
>>> False + 3                      #把False当作0处理
3
```

演示：

```
>>> a = (4,)
>>> type(a)
>>> a = (4)
>>> type(a)
>>> a = [4,]
>>> type(a)
>>> a = [4]
>>> type(a)
>>> a = ()
>>> type(a)
```

2.2.1 算术运算符

（2）*运算符除了表示算术乘法，还可用于列表、元组、字符串这几个序列类型与**整数**的乘法，表示序列元素的重复，生成新的序列对象。

```
>>> [1, 2, 3] * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]

>>> 'abc' * 3
'abcabcabc‘
```

演示：

```
>>> [0] * 20
>>> '*' * 50
>>> '=' * 50
>>> '*=' * 50
```

■ 字典和集合不支持与整数的相乘，因为其中的元素是不允许重复的。

■（了解）运算符*执行的是浅复制（只复制引用）

```
>>> b = [[1,2],3] * 2
>>> b
[[1, 2], 3, [1, 2], 3]
>>> b[0][0] = 0
>>> b
[[0, 2], 3, [0, 2], 3]
```

```
>>> [id(i) for i in b]
[2635893955776, 140711837336424,
2635893955776, 140711837336424]
```

2.2.1 算术运算符

（3）运算符/和//在Python中分别表示算术除法和算术求整商（floor division）。

```
>>> type(4/2)
<class 'float'>
```

```
>>> 3 / 2                    #数学意义上的除法
1.5

>>> 15 // 4                  #向下取整
3

>>> -15//4                   #(-15)//4
-4
```

2.2.1 算术运算符

（4）%运算符用于求余数，（了解）还可以用于字符串格式化，但是这种用法并不推荐。

```
>>> a,b = -7,3
>>> (a//b)*b + a%b == a
True
```

```
>>> 789 % 23                    #余数
7
```

```
>>> '%d' % (65)      #%d 表示将一个值格式化为十进制整数 # %字符串格式化操作符
'65'                  #(65)：这是要插入到格式字符串中的值
```

2.2.1 算术运算符

（5）运算符**表示幂乘：

```
>>> 3 ** 2                    #3的2次方，等价于pow(3, 2)
9
>>> 9 ** 0.5                  #9的0.5次方，平方根
3.0
>>> (-9) **  0.5              #可以计算负数的平方根
(1.8369701987210297e-16+3j)

>>> (1 + 2j) ** 0.5           #可以计算复数的幂次
(1.272019649514069+0.7861513777574233j)

>>> (1 + 2j) ** (3+4j)
(0.12900959407446697+0.03392409290517014j)
```

Practice

- [[1,2], 3]*2结果是什么？*运算是一种浅复制，浅复制的含义是什么?

✓ [[1,2], 3, [1,2], 3]。

✓ 第二个[1,2]与第一个[1,2]是同一个列表，具有相同的内存地址

习题

2.14 表达式 13 // 4 的值为_____。

2.15 表达式 -13 // 4 的值为_____。

2.16 表达式 -10 % 4 的值为_____。

2.17 表达式 10 % -3 的值为_____。

2.23 单选题：表达式 [1, 2, 3] * 2 的值为（　　）。

A. [1, 2, 3, 1, 2, 3]　　B. [2, 4, 6]

C. [1, 1, 2, 2, 3, 3]　　D. 无法计算

2.24 单选题：表达式 68 // -7 的值为（　　）。

A. -10　　B. 10　　C. 9　　D. -9

习题

2.34 单选题：已知 x = [1, 2] 和 y = [3, 4]，那么 x+y 的结果是（

A. 3　　B. 7　　C. [4, 6]　　D. [1, 2, 3, 4]

2.38 单选题：执行语句 d, _ = divmod(36, 12) 之后，变量 d 的值为（

A. 36　　B. 12　　C. 3　　D. 语法错误无法执

2.43 单选题：已知 x = [1, 2, 3]，那么 x*3 的值为（　　）。

A. 6　　B. 18

C. [3, 6, 9]　　D. [1, 2, 3, 1, 2, 3, 1, 2, 3]

2.56 判断对错：加法运算符可以连接两个字典得到新字典。

2.57 判断对错：加法运算符可以连接两个列表得到新列表。

第2章 万丈高楼平地起: 运算符、表达式与内置对象

- 2.2 Python运算符与表达式
  - 2.2.1 算术运算符
  - **2.2.2 关系运算符**
  - 2.2.3 成员测试运算符in与同一性测试运算符is
  - 2.2.4 位运算符与集合运算符
  - 2.2.5 逻辑运算符
  - 2.2.6 矩阵乘法运算符@
  - 2.2.7 补充说明

2.2.2 关系运算符

运算符	功能说明
<、<=、>、>=、==、 !=	（值）大小比较，集合的包含关系比较

2.2.2 关系运算符

- Python关系运算符最大的特点是可以连用。关系运算符要求操作数之间必须可比较大小。

```
>>> 1 < 3 < 5
True
>>> 3 < 5 > 2
True
>>> 1 > 6 < 8
False
```

■连用的关系运算符具有**惰性求值**或者逻辑短路的特点

```
>>> 1 > 6 < math.sqrt(9)   #左边False，整体就False
False
>>> 1 < 6 < math.sqrt(9)        #还没有导入math模块，抛出异常
NameError: name 'math' is not defined
>>> import math
>>> 1 < 6 < math.sqrt(9)
False
```

2.2.2 关系运算符

```
>>> 'Hello' > 'world'              #比较字符串大小 #ord('H'):72, ord('w'):119
False
>>> [1, 2, 3] < [1, 2, 4]          #比较列表大小
True
>>> 'Hello' > 3                    #字符串和数字不能比较
TypeError: unorderable types: str() > int()
```

■ 集合的包含关系

```
>>> {1, 2, 3} < {1, 2, 3, 4}       #测试是否子集
True
>>> {1, 2, 3} == {3, 2, 1}         #测试两个集合是否相等
True
>>> {1, 2, 4} > {1, 2, 3}          #集合之间的包含测试
False
```

第2章　万丈高楼平地起：运算符、表达式与内置对象

■ 2.2 Python运算符与表达式

- □ 2.2.1 算术运算符
- □ 2.2.2 关系运算符
- □ **2.2.3 成员测试运算符in与同一性测试运算符is**
- □ 2.2.4 位运算符与集合运算符
- □ 2.2.5 逻辑运算符
- □ 2.2.6 矩阵乘法运算符@
- □ 2.2.7 补充说明

2.2.3 成员测试运算符in与同一性测试运算符is

运算符	功能说明
in	**成员测试**
is	**对象同一性测试，即测试是否为同一个对象(内存地址是否相同)**

2.2.3 成员测试运算符in与同一性测试运算符is

- 成员测试运算符in用于成员测试，即测试一个对象是否为另一个对象的元素。

```
>>> 3 in [1, 2, 3]                #测试3是否存在于列表[1, 2, 3]中
True

>>> 'abc' in 'abcdefg'            #子字符串测试
True

>>> for item in [1,2,3]:          #循环，成员遍历
        print(item)
```

2.2.3 成员测试运算符in与同一性测试运算符is

- 同一性测试运算符is用来测试两个对象是否是同一个(具有相同的内存地址)，如果是则返回True，否则返回False。

```
>>> x = [1, 2, 3]
>>> y = [1, 2, 3]
>>> x is y      #x和y指向两个不同的列表
False
>>> x == y
True

>>> y = x
>>> x is y
True
```

is运算符在实际编程中最常见的用法：

```
if x is None:
    ...
if x is not None:
    ...
```

- 2.2 Python运算符与表达式

- 2.2.1 算术运算符
- 2.2.2 关系运算符
- 2.2.3 成员测试运算符in与同一性测试运算符is
- **2.2.4 位运算符与集合运算符**
- 2.2.5 逻辑运算符
- 2.2.6 矩阵乘法运算符@
- 2.2.7 补充说明

2.2.4 位运算符与集合运算符

运算符	描述
&（位与）	1&1=1、1&0=0&1=0&0=0
|（位或）	1|1=1|0=0|1=1、0|0=0
^（位异或）	1^1=0^0=0、1^0=0^1=1
<<（位左移）	左移位时右侧补0，每左移一位相当于乘以2
>>（位右移）	右移位时左侧补0，每右移一位相当于整除以2
~（位取反）	~1=0，~0=1
&（交集）	
|（并集）	
^（对称差集）	

2.2.4 位运算符与集合运算符

- 位运算符只能用于整数，其内部执行过程为：首先将整数转换为二进制数，然后右对齐，必要的时候左侧补0，按位进行运算，最后再把计算结果转换为十进制数字返回。

2.2.4 位运算符与集合运算符

1．按位与运算符“&”

1&1=1、1&0=0&1=0&0=0

十进制数	二进制数							
60=	0	0	1	1	1	1	0	0
13=	0	0	0	0	1	1	0	1
60&13=	0	0	0	0	1	1	0	0

其运算结果为60&13=12，二进制数为0000 1100。

二进制位	2^5	2^4	2^3	2^2	2^1	2^0
权重值	32	16	8	4	2	1
60的二进制	1	1	1	1	0	0

总和：32 *1+ 16*1 + 8*1 + 4*1 + 2*0 + 1*0 = 60

2.2.4 位运算符与集合运算符

2．按位或运算符“|”

1|1=1|0=0|1=1、0|0=0

十进制数	二进制数							
60=	0	0	1	1	1	1	0	0
13=	0	0	0	0	1	1	0	1
60|13=	0	0	1	1	1	1	0	1

其运算结果为60|13=61，二进制数为0011 1101。

2.2.4 位运算符与集合运算符

3．按位异或运算符“^”

1^1=0^0=0、1^0=0^1=1

十进制数	二进制数							
60=	0	0	1	1	1	1	0	0
13=	0	0	0	0	1	1	0	1
60^13=	0	0	1	1	0	0	0	1

其运算结果为60^13=49，二进制数为0011 0001。

2.2.4 位运算符与集合运算符

4．按位左移运算符“<<”

十进制数	二进制数							
60=	0	0	1	1	1	1	0	0
60<<2	1	1	1	1	0	0	0	0

其运算结果为60<<2=240，二进制数为1111 0000。

每左移一位相当于乘以2: 240=60*2*2

2.2.4 位运算符与集合运算符

5．按位右移运算符“>>”

十进制数	二进制数							
60=	0	0	1	1	1	1	0	0
60>>2	0	0	0	0	1	1	1	1

其运算结果为60>>2=15，二进制数为0000 1111。

每右移一位相当于除以2: 15=60/2/2

2.2.4 位运算符与集合运算符

6．按位求反运算符“~”

~1=0，~0=1

十进制数	二进制数							
60=	0	0	1	1	1	1	0	0
~60=	1	1	0	0	0	0	1	1
13=	0	0	0	0	1	1	0	1
~13=	1	1	1	1	0	0	1	0

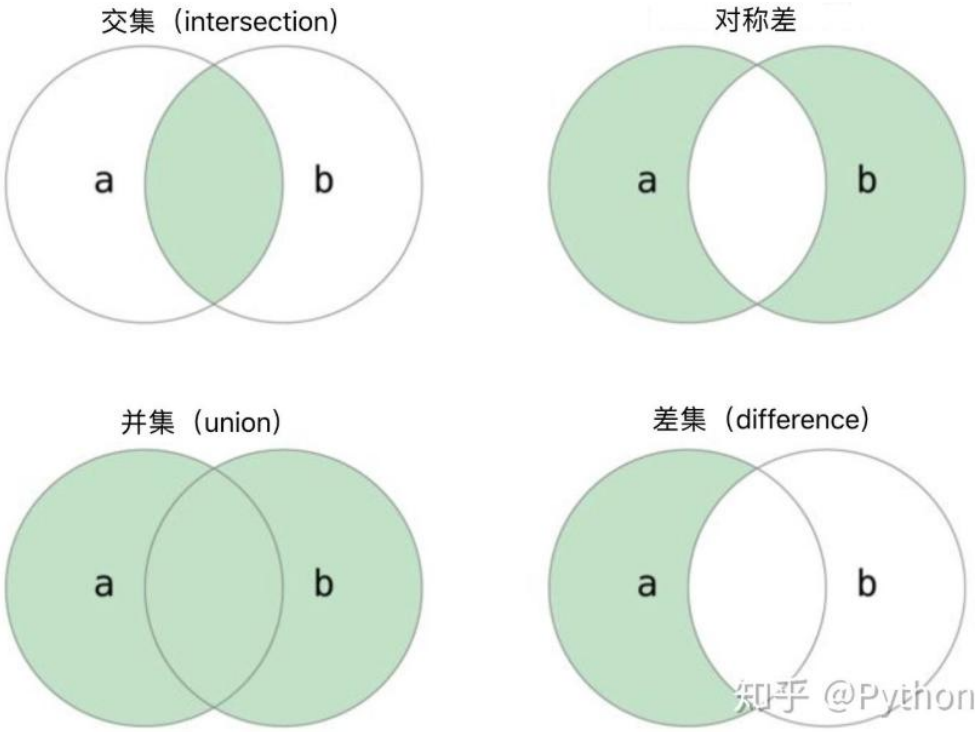
其运算结果为~60 = -61，~13 = -14。

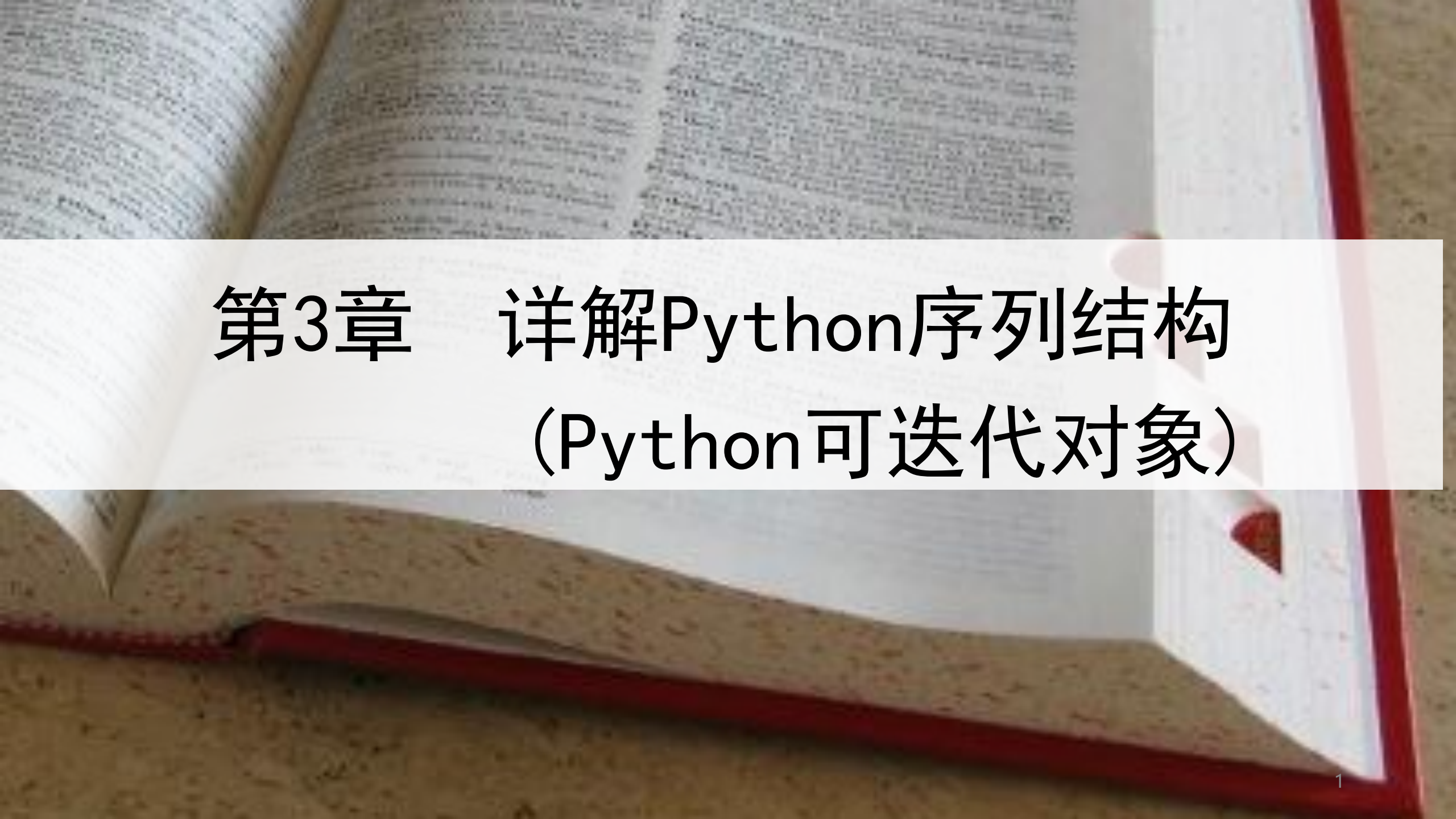
总结：正整数n的位求反运算结果为~n = -n-1

2.2.4 位运算符与集合运算符

- 集合的交集、并集、对称差集等运算符与位运算符相同，差集使用减号运算符（注意，并集运算符不是加号）。

```
>>> {1, 2, 3} | {3, 4, 5}          #并集，自动去除重复元素
{1, 2, 3, 4, 5}
>>> {1, 2, 3} & {3, 4, 5}          #交集
{3}
>>> {1, 2, 3} ^ {3, 4, 5}          #对称差集=并集-交集
{1, 2, 4, 5}
>>> {1, 2, 3} - {3, 4, 5}          #差集
{1, 2}
>>> {3, 4, 5} - {1, 2, 3}          #差集
{4, 5}
```



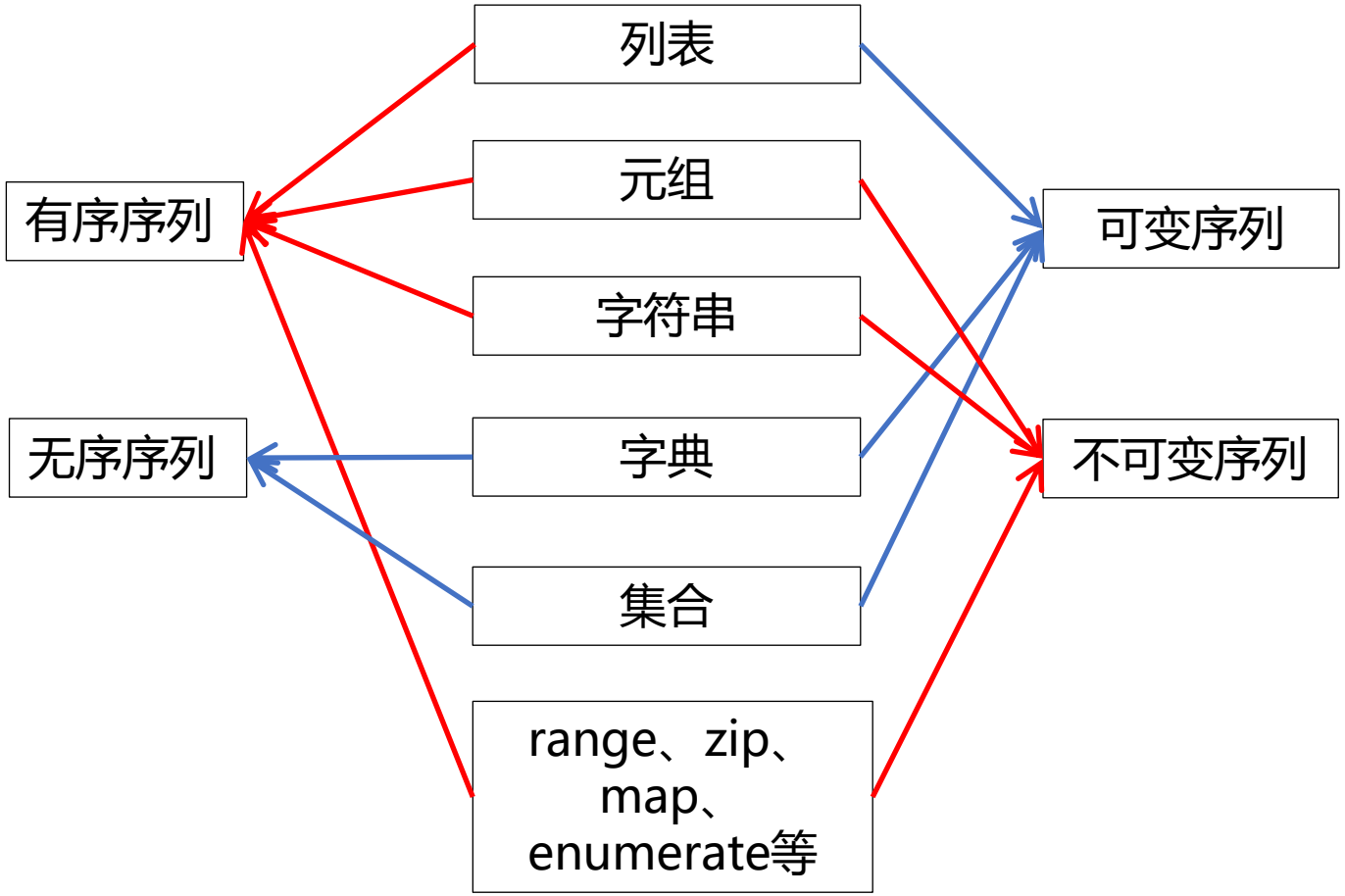


第3章　详解Python序列结构（Python可迭代对象）

第3章 玄之又玄，众妙之门: 详解Python序列结构

- 3.1列表: 打了激素的数组
- 3.2元组: 轻量级列表
- 3.3字典: 反映对应关系的映射类型
- 3.4集合: 元素之间不允许重复

第3章 详解Python序列结构



- 记蓝色的

第3章 详解Python序列结构

	列表(list)	元组(tuple)	字典(dict)	集合(set)
类型名称	list	tuple	dict	set
定界符	方括号[]	圆括号()	大括号{}	大括号{}
是否可变	是	否	是	是
是否有序	是	是	否	否
是否支持下标	是（使用序号作为下标）	是（使用序号作为下标）	是（使用“键”作为下标）	否
元素分隔符	逗号	逗号	逗号	逗号
对元素形式的要求	无	无	键:值	必须可哈希
对元素值的要求	无	无	“键”必须可哈希（不可改变，例：数字、字符串、元组）	必须可哈希
元素是否可重复	是	是	“键”不允许重复，“值”可以重复	否
元素查找速度	非常慢	很慢	非常快	非常快
新增和删除元素速度	尾部操作快 其他位置慢	不允许	快	快

第3章 详解Python序列结构

- 3.1 列表：打了激素的数组
  - 3.1.1 列表创建与删除
  - 3.1.2 列表元素访问
  - 3.1.3 列表常用方法
  - 3.1.4 列表对象支持的运算符
  - 3.1.5 内置函数对列表的操作
  - 3.1.6 列表推导式语法与应用案例
  - 3.1.7 切片操作的强大功能

3.1 列表：打了激素的数组

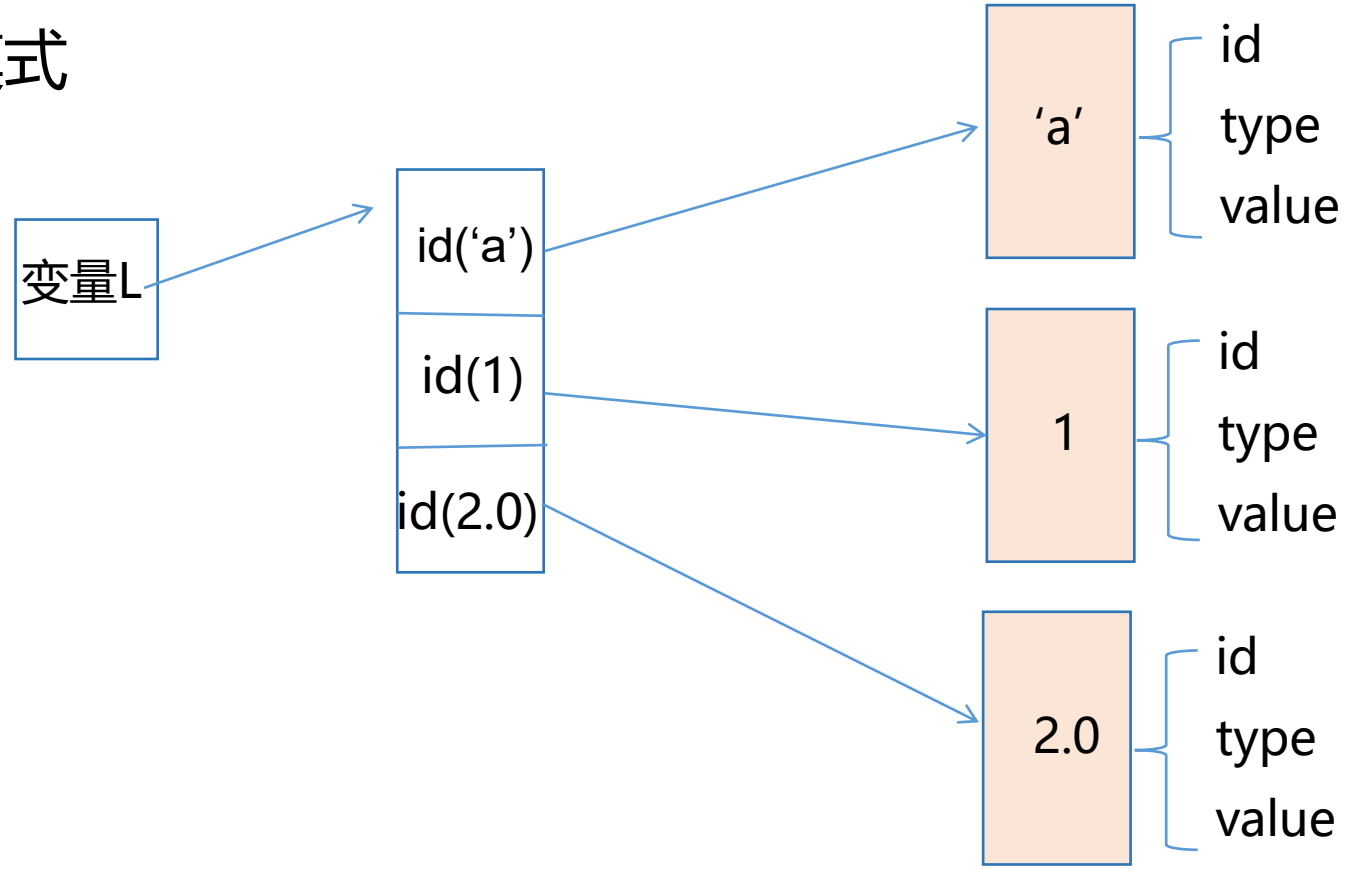
■列表（list）是包含若干元素的**有序连续**内存空间。

■Python采用基于值的自动内存管理模式，变量并不直接存储值，而是存储**值的引用或内存地址**。

■Python列表中的元素也是值的引用，所以列表中各元素可以是不同类型的数据。

3.1 列表：打了激素的数组

列表的内存管理模式



理解：L = ['a',1,2.0]，三个元素（对象）在python中有各自的存储空间，然后列表L只是把L[0]，L[1]，L[2]三个标签贴它们身上，三个标签是连续存储的，最后再用一个变量L指向这三个标签。

3.1 列表：打了激素的数组

- 当列表增加或删除元素时，列表对象自动进行内存的扩展或收缩，从而保证相邻元素之间没有缝隙。

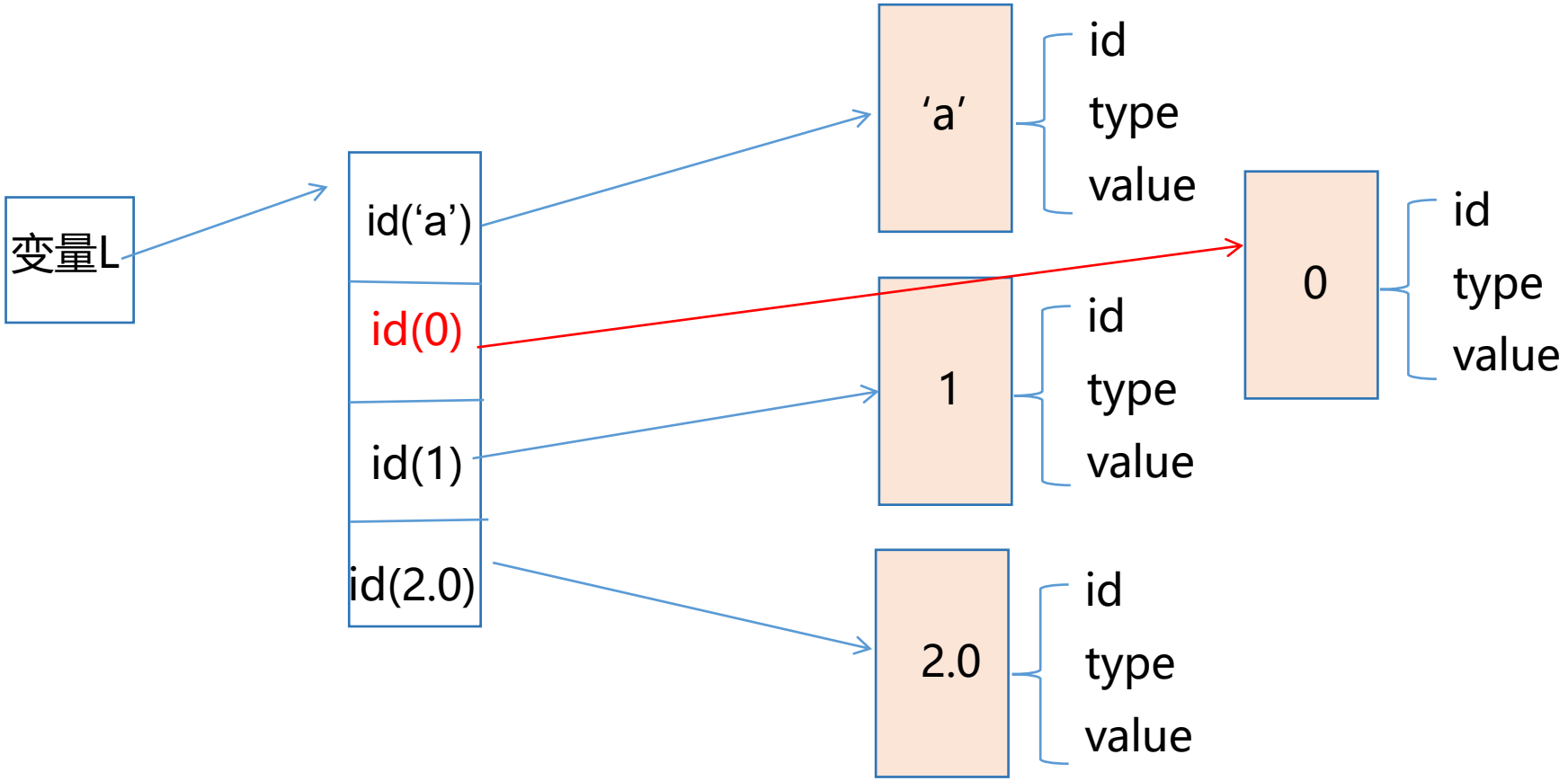
- Python列表的这个内存**自动**管理功能可以大幅度减少程序员的负担，但插入和删除非尾部元素时涉及到列表中大量元素的移动，会严重影响效率。

- 在非尾部位置插入和删除元素时会改变该位置后面的元素在列表中的索引，这对于某些操作可能会导致意外的错误结果。

- 除非确实有必要，否则应尽量从列表尾部进行元素的追加与删除操作。

3.1 列表：打了激素的数组

列表增加中间元素



在L = ['a',1,2.0]中index=1位置插入元素0，则列表自动扩展预备好的很大的空间，先把原来的L[1]，L[2]两个标签复制到新的L[2]和L[3]位置，再把id(0)放在L[1]位置。

3.1 列表：打了激素的数组

- 在Python中，同一个列表中元素的数据类型可以各不相同。
- 列表可以同时包含整数、实数、字符串等基本类型的元素，也可以包含列表、元组、字典、集合、函数以及其他**任意对象**。
- 如果只有一对方括号而没有任何元素则表示**空列表**。
- 列表的功能虽然非常强大，但是开销较大，在实际开发中，最好根据实际的问题选择一种合适的数据类型，要尽量避免过多使用列表？(该用就用)

```
[10, 20, 30, 40]
['crunchy frog', 'ram bladder', 'lark vomit']
['spam', 2.0, 5, [10, 20]]
[['file1', 200,7], ['file2', 260,9]]
[{3}, {5:6}, (1, 2, 3)]
[]
```

第3章 详解Python序列结构

- 3.1 列表：打了激素的数组
  - **3.1.1 列表创建与删除**
  - 3.1.2 列表元素访问
  - 3.1.3 列表常用方法
  - 3.1.4 列表对象支持的运算符
  - 3.1.5 内置函数对列表的操作
  - 3.1.6 列表推导式语法与应用案例
  - 3.1.7 切片操作的强大功能

3.1.1 列表创建与删除

- 使用“=”直接将一个列表赋值给变量即可创建列表对象。

```
>>> a_list = ['a', 'b', 'mpilgrim', 'z', 'example']
>>> a_list = []                                  #创建空列表
```

3.1.1 列表创建与删除

- 可以使用list()函数把元组、range对象、字符串、字典、集合或其他可迭代对象转换为列表。

```
>>> list(range(1, 10, 2))                    #将range对象转换为列表
[1, 3, 5, 7, 9]

>>> list('hello world')                      #将字符串转换为列表
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']

>>> list({3,7,5})                            #将集合转换为列表
[3, 5, 7]

>>> list({'a':3, 'b':9, 'c':78})             #将字典的“键”转换为列表
['a', 'c', 'b']

>>> x = list()                                  #创建空列表
```

3.1.1 列表创建与删除

- 当一个列表不再使用时，可以使用del命令将其删除，这一点适用于所有类型的Python对象。

```
>>> x = [1, 2, 3]
>>> y = x                          #复制了标签x
>>> del x                          #删除列表标签
>>> x                              #对象删除后无法再访问，抛出异常
NameError: name 'x' is not defined
>>> y
[1,2,3]                      #del x 只是删除了x标签，不是把[1,2,3]列表删除了

#常用: del x[0]
```

第3章 详解Python序列结构

- 3.1 列表：打了激素的数组
  - 3.1.1 列表创建与删除
  - **3.1.2 列表元素访问**
  - 3.1.3 列表常用方法
  - 3.1.4 列表对象支持的运算符
  - 3.1.5 内置函数对列表的操作
  - 3.1.6 列表推导式语法与应用案例
  - 3.1.7 切片操作的强大功能

3.1.2 列表元素访问

- 使用整数作为下标来访问列表的元素，其中0表示第1个元素，1表示第2个元素，2表示第3个元素，以此类推

- 列表还支持使用负整数作为下标，其中-1表示最后1个元素，-2表示倒数第2个元素，-3表示倒数第3个元素，以此类推

- 当下标超过允许的范围时，程序出错：设N为列表中元素的数目，下标的范围是0到N-1和-N到-1，即range(N)和range(-N,0)

序列	元素 1	元素 2	元素 3	元素...	元素 n-1	元素 n
正索引	0	1	2	...	n-2	n-1
负索引	-n	-(n-1)	-(n-2)	...	-2	-1

3.1.2 列表元素访问

```
>>> x = list('Python')                    #创建列表对象
>>> x
['P', 'y', 't', 'h', 'o', 'n']

>>> x[0]                                  #下标为0的元素，第一个元素
'P'
>>> x[-1]                                 #下标为-1的元素，最后一个元素
'n'
```

```
>>> x = list('Python')
>>> x[7]
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    x[7]
IndexError: list index out of range
```

Practice

- 设a是一个字符列表，当它的最后一个元素是’n’时，打印a(提示：a可能为空列表)

✓
```
a = list('listen')
if a and a[-1] == 'n':
    print(a)
```

- 哪些内置函数（参考36页表2-5）的返回值为列表？

✓ dir, sorted, list

第3章 详解Python序列结构

- 3.1 列表：打了激素的数组
  - 3.1.1 列表创建与删除
  - 3.1.2 列表元素访问
  - **3.1.3 列表常用方法**
  - 3.1.4 列表对象支持的运算符
  - 3.1.5 内置函数对列表的操作
  - 3.1.6 列表推导式语法与应用案例
  - 3.1.7 切片操作的强大功能

3.1.3 列表常用方法

方法	说明
append(x)	将x追加至列表尾部
extend(L)	将列表L中所有元素追加至列表尾部
insert(index, x)	在列表index位置处插入x，该位置后面的所有元素后移并且在列表中的索引加1，如果index为正数且大于列表长度则在列表尾部追加x，如果index为负数且小于列表长度的相反数则在列表头部插入元素x
remove(x)	在列表中删除第一个值为x的元素，该元素之后所有元素前移并且索引减1，如果列表中不存在x则抛出异常
pop([index])	删除并返回列表中下标为index的元素，如果不指定index则默认为-1，弹出最后一个元素；如果弹出中间位置的元素则后面的元素索引减1；如果index不是[-L, L]区间上的整数则抛出异常
clear()	清空列表，删除列表中所有元素，保留列表对象
index(x, [start, [stop]]))	返回列表中第一个值为x的元素的索引，若不存在值为x的元素则抛出异常
count(x)	返回x在列表中的出现次数
reverse()	对列表所有元素进行原地逆序，首尾交换
sort(key=None, reverse=False)	对列表中的元素进行原地排序，key用来指定排序规则，reverse为False表示升序，True表示降序
copy()	返回列表的浅复制（一个新列表）

方法是类内部自带的函数，引用需要声明对象，比如：alist.append()

可输入dir(list)查看list的所有方法

3.1.3 列表常用方法

（1）append()、insert()、extend() —— 增加元素的方法

方法	说明
append(x)	将元素x追加至列表尾部
extend(L)	将列表L中所有元素追加至列表尾部
insert(index, x)	在列表index位置处插入元素x，该位置后面的所有元素后移并且在列表中的索引加1，如果index为正数且大于列表长度则在列表尾部追加x，如果index为负数且小于列表长度的相反数则在列表头部插入元素x

■这3个方法都属于原地操作，不影响列表对象在内存中的起始地址

3. 1. 3　列表常用方法

```
>>> x = [1, 2, 3]
>>> id(x)                          #查看对象的内存地址
50159368

>>> x.append(4)                      #在尾部追加元素
>>> x.insert(0, 0)                  #在指定位置插入元素
>>> x.extend([5, 6, 7])              #在尾部追加多个元素
>>> x
[0, 1, 2, 3, 4, 5, 6, 7]
>>> x.insert(1000,8)

>>> x.append([4])

>>> id(x)                          #列表在内存中的地址不变
50159368

>>> x = [1,2,3]
>>> help(x)                          #查找所有应用
```

3. 1. 3　列表常用方法

append和extend方法的区别：

```
>>> x = [1,2,3]
>>> x.extend('abc')   #extend(L)默认参数是列表，所以先做了list( ‘abc’ )操作
                      #等价于 x.extend(list('abc'))
>>> x
[1, 2, 3, 'a', 'b', 'c']

>>> x = [1,2,3]
>>> x.append('abc')  #append(x)默认x是列表的一个元素
>>> x
[1, 2, 3, 'abc']
```

3.1.3 列表常用方法

（2）pop()、remove()、clear() —— 删除元素的方法

方法	说明
pop([index])	删除**并返回**列表中下标为index的元素，**如果不指定index则默认为-1，弹出最后一个元素**；如果弹出中间位置的元素则后面的元素索引减1；如果index不是[-L, L]区间上的整数则抛出异常
remove(x)	在列表中删除**第一个值为x**的元素，该元素之后所有元素前移并且索引减1，如果列表中不存在x则抛出异常
clear()	清空列表，删除列表中所有元素，保留列表对象

■这3个方法也属于原地操作

■还可以使用del命令删除列表中指定位置的元素，同样也属于原地操作

■使用for value in a遍历列表a时，尽量不remove和insert元素，实际执行过程太过复杂

3.1.3 列表常用方法

```
>>> x = [1, 2, 3, 4, 5, 6, 7]
>>> x.pop()                    #弹出并返回尾部元素
7
>>> x.pop(0)                   #弹出并返回指定位置的元素
1
>>> x.clear()                  #删除所有元素
>>> x
[]


>>> x = [1, 2, 1, 1, 2]
>>> x.remove(2)                #删除首个值为2的元素
>>> del x[3]                   #删除指定位置上的元素
>>> x
[1, 1, 1]

>>> x.remove(3)                #报错
```

3.1.3 列表常用方法

（3）count()、index()—— 元素的访问和计数方法

方法	说明
count(x)	返回x在列表中的出现次数
index(x, [start, [stop]]))	返回列表中第一个值为x的元素的索引，若不存在值为x的元素则抛出异常

```
>>> x = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
>>> x.count(3)                          #元素3在列表x中的出现次数
3
>>> x.count(5)                          #不存在，返回0
0

>>> x.index(2)                          #元素2在列表x中首次出现的索引
1
>>> x.index(5)                          #列表x中没有5，抛出异常
ValueError: 5 is not in list

>>> x.index(value) if value in x else 'None'     #三元运算符
'None'
```

3.1.3 列表常用方法

■index(value, [start, [stop]])，返回下标在start到stop-1范围内，value首次出现的位置

```
>>> colours = ["red", "green", "blue", "green", "yellow"]
>>> colours.index("green")
1
>>> colours.index("green", 2)
3
>>> colours.index("green", 3,4)
3
```

■值在列表中出现的所有位置？

```
colours = ["red", "green", "blue", "green", "yellow"]
indices = [i for i, x in enumerate(colours) if x == 'green']
```

3.1.3 列表常用方法

（4）sort()、reverse()——元素排序的方法

方法	说明
sort(key=None, reverse=False)	对列表中的元素进行原地排序，key用来指定排序规则，reverse为False表示**升序，True表示降序**
reverse()	对列表所有元素进行原地逆序，首尾交换

```
>>> x = list(range(11))                    #包含11个整数的列表
>>> import random
>>> random.shuffle(x)                      #把列表x中的元素随机乱序
>>> x
[6, 0, 1, 7, 4, 3, 2, 8, 5, 10, 9]
```

3.1.3 列表常用方法

```
>>> x.sort()                                    #按默认规则排序，升序
>>> x
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]


>>> x.sort(key=str)                             #按转换为字符串后的大小，升序排序
>>> x
[0, 1, 10, 2, 3, 4, 5, 6, 7, 8, 9]

>>> x.sort(reverse=True)                        #降序排列
>>> x
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

>>> x.reverse()                                 #把所有元素翻转或逆序
>>> x
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

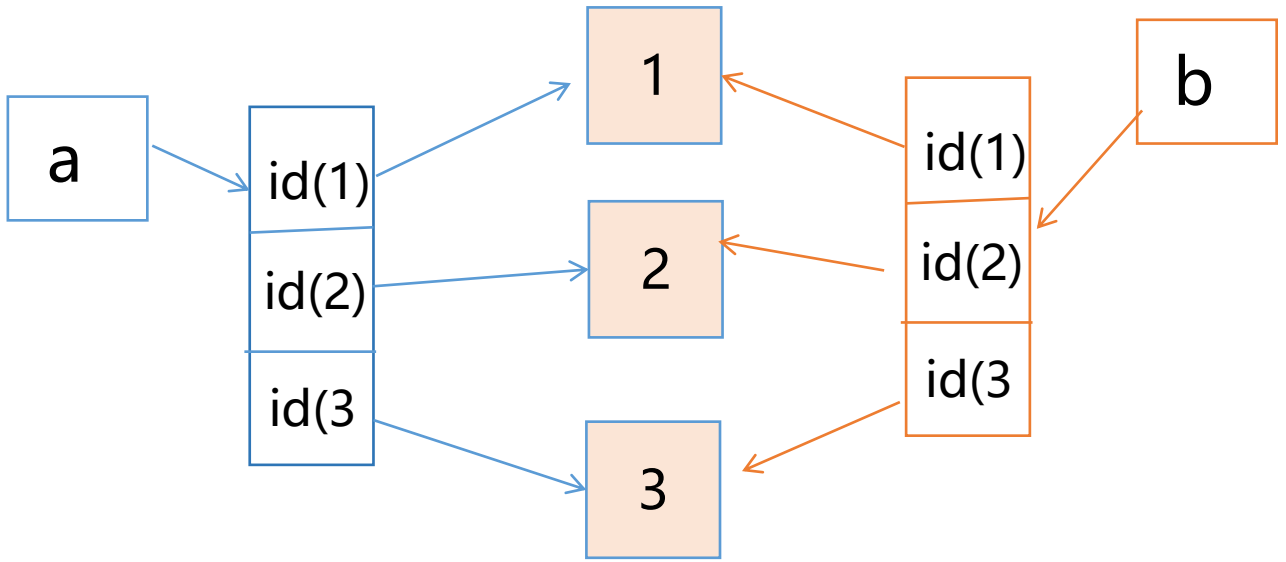
3.1.3 列表常用方法

（5）copy()——列表复制的方法

方法	说明
copy()	返回列表的浅复制（一个新列表）

■当列表中的元素为数字、字符串、元组等不可变类型时，无任何影响

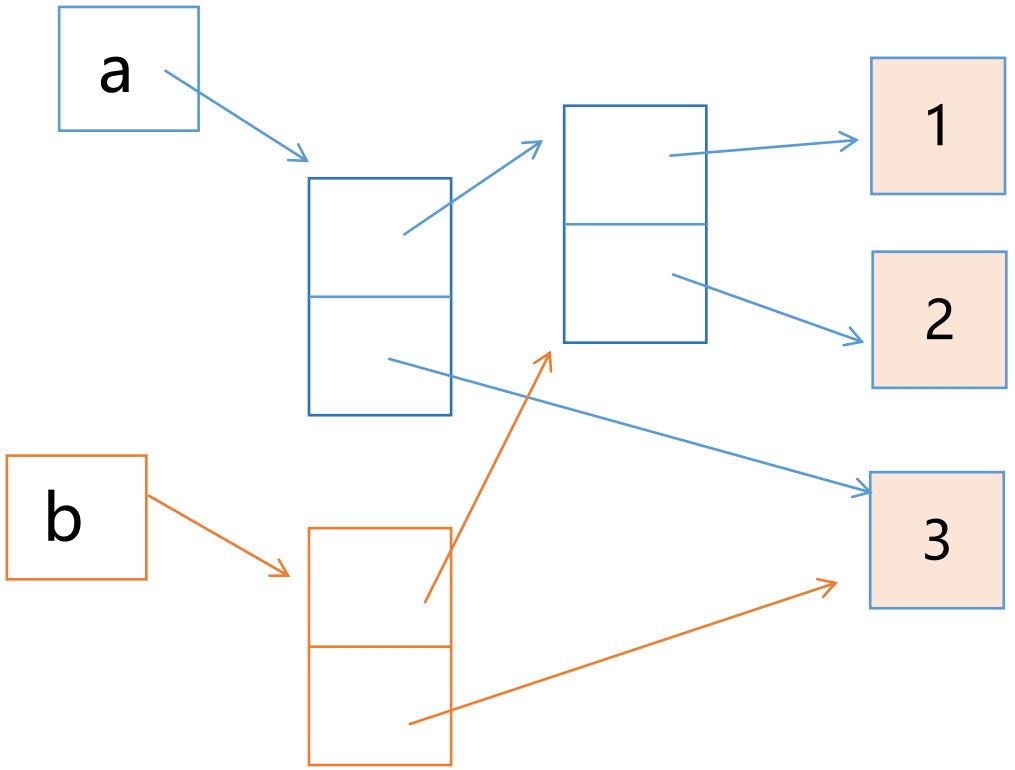
```
>>> a = [1,2,3]
>>> b = a.copy()    #b是新列表
>>> b[0] = 4
>>> b
[4, 2, 3]
>>> a
[1, 2, 3]
```



3. 1. 3　列表常用方法

■ **如果列表元素为列表等可变类型，有影响**

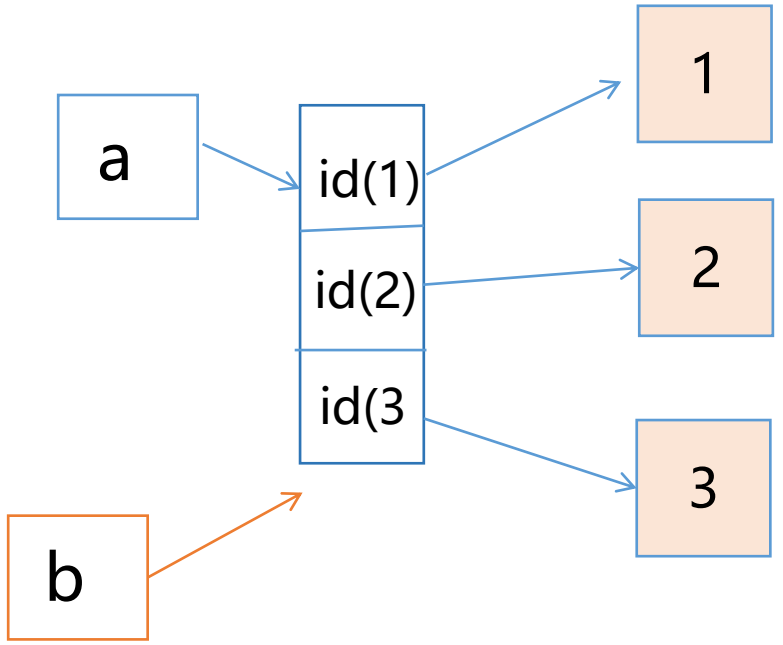
```
>>> a = [[1,2], 3]
>>> b = a.copy()
>>> b[0][0] = 4
>>> b
[[4, 2], 3]
>>> a
[[4, 2], 3]
```



3. 1. 3　列表常用方法

对比：

```
>>> a = [1,2,3]
>>> b = a        #b是新标签
>>> b[0] = 4
>>> b
[4, 2, 3]
>>> a
[4, 2, 3]
```

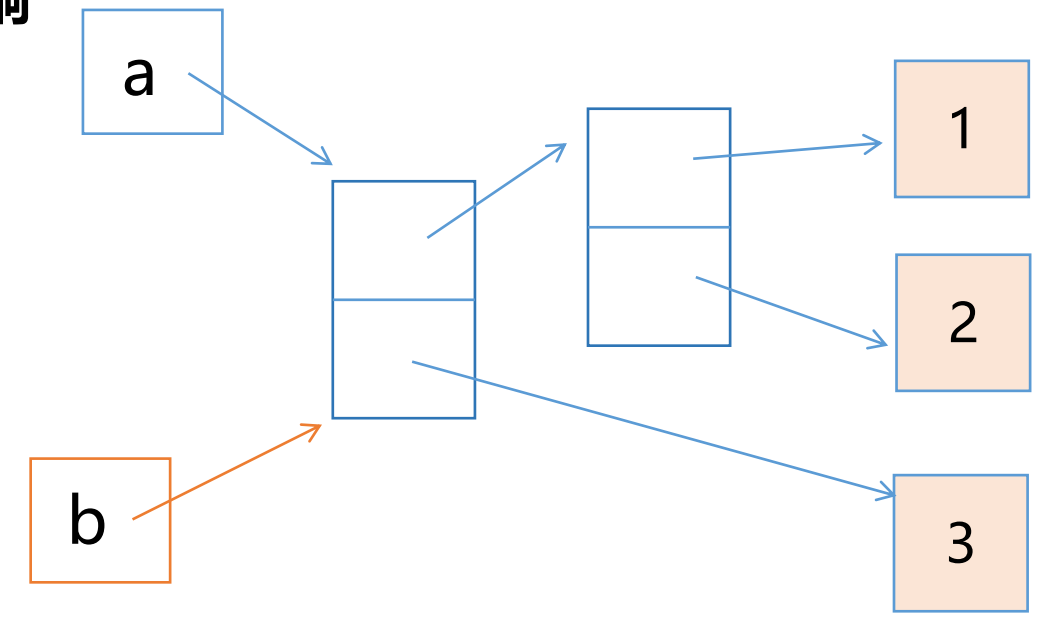


3. 1. 3　列表常用方法

对比：

#如果列表元素为列表等可变类型，有影响

```
>>> a = [[1,2], 3]
>>> b = a
>>> b[0][0] = 4
>>> b
[[4, 2], 3]
>>> a
[[4, 2], 3]
```



第3章 详解Python序列结构

- 3.1 列表：打了激素的数组
  - 3.1.1 列表创建与删除
  - 3.1.2 列表元素访问
  - 3.1.3 列表常用方法
  - **3.1.4 列表对象支持的运算符**
  - 3.1.5 内置函数对列表的操作
  - 3.1.6 列表推导式语法与应用案例
  - 3.1.7 切片操作的强大功能

3.1.4 列表对象支持的运算符

- 加法运算符+可以实现列表增加元素的目的，但不属于原地操作，而是返回新列表，涉及大量元素的复制，效率非常低

- 使用复合赋值运算符+=实现列表追加元素时属于原地操作，与append()方法一样高效，等效于extend()方法

3.1.4 列表对象支持的运算符

```
>>> x = [1, 2, 3]
>>> id(x)
53868168

>>> x = x + [4]                    #连接两个列表
>>> x
[1, 2, 3, 4]
>>> id(x)                          #内存地址发生改变
53875720

>>> x += [5]                       #为列表追加元素
>>> x
[1, 2, 3, 4, 5]
>>> id(x)                          #内存地址不变
53875720
```

3.1.4 列表对象支持的运算符

```
import time

n= 100000

start_time = time.time()
a = []
for i in range(n):
    a = a + [i * 2]
end_time = time.time()
print(end_time - start_time)
```

```
start_time = time.time()
a = []
for i in range(n):
    a += [i * 2]
end_time = time.time()
print(end_time - start_time)

start_time = time.time()
a = []
for i in range(n):
    a.append(i * 2)
end_time = time.time()
print(end_time - start_time)
```

```
17.83196735382080
0.018695592880249023
0.011968135833740234
```

3.1.4 列表对象支持的运算符

- 乘法运算符*可以用于列表和整数相乘，表示序列重复（浅复制），返回新列表
- 运算符*=也可以用于列表元素重复，属于原地操作

```
>>> x = [1, 2, 3, 4]
>>> id(x)
54497224
>>> y = x * 2                          #元素重复，返回新列表
>>> y
[1, 2, 3, 4, 1, 2, 3, 4]
>>> id(y)                              #地址发生改变
54603912

>>> y *= 2                             #元素重复，原地进行
>>> y
[1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4]
>>> id(x)                              #地址不变
54603912
```

3.1.4 列表对象支持的运算符

- 成员测试运算符in可用于测试列表中是否包含某个元素，查询时间随着列表长度的增加而线性增加
- 同样的操作对于字典和集合而言则是常数级的，与字典集合的大小无关，因为字典和集合用的是散列表

```
>>> 3 in [1, 2, 3]
True
>>> 3 in [1, 2, '3']
False
```

Practice

- 列表的那些操作是在尾部进行的？

✓ append(), extend(), pop(), +=, *=

- sorted()函数与列表的sort()方法的区别是什么？

✓ sorted()返回一个新列表，sort()方法对列表做原地排序

- 在列表a尾部附加一个元素‘ab’的三种方法？

✓ a.append('ab')

✓ a.extend(['ab'])

✓ a += ['ab']

```
>>> a = [1,4,8,2,3]
>>> id(a)
2422438866816

>>> b = sorted(a)
>>> id(b)
2422441956480
```

第3章 详解Python序列结构

- 3.1 列表：打了激素的数组
  - 3.1.1 列表创建与删除
  - 3.1.2 列表元素访问
  - 3.1.3 列表常用方法
  - 3.1.4 列表对象支持的运算符
  - **3.1.5 内置函数对列表的操作**
  - 3.1.6 列表推导式语法与应用案例
  - 3.1.7 切片操作的强大功能

3.1.5 内置函数对列表的操作

内置函数的特点：
1.全局可用性，适用于所有序列结构
2.高性能实现(C实现)

■max()、min()函数用于返回列表中所有元素的最大值和最小值

■sum()函数用于返回列表中所有元素之和

■len()函数用于返回列表中元素个数

■all()函数用来测试列表中是否所有元素都等价于True
■any()用来测试列表中是否有等价于True的元素

3.1.5 内置函数对列表的操作

```
>>> x = list(range(11))          #生成列表
>>> x
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> max(x)
10
>>> min(x)
0
>>> len(x)
11

>>> all(x)                       #测试是否所有元素都等价于True
False
>>> any(x)                       #测试是否存在等价于True的元素
True
```

3.1.5 内置函数对列表的操作

- zip()函数：是迭代器，把多个有序**序列**中**相同位置上的元素**压缩成一个**元组**，所有的元组按顺序形成一个可迭代的zip对象（迭代器），如同拉拉链一样

```
>>> list(zip([1, 2, 3], ['a', 'b', 'c']))  #list把zip对象显示出来
[(1, 'a'), (2, 'b'), (3, 'c')]
>>> for i, value in zip([1, 2, 3], ['a', 'b', 'c']):
        print(i,value)

>>> a1,a2,a3 = 'abc',[1,2,3],'ABC'
>>> for v1,v2,v3 in zip(a1,a2,a2): #可以是多个序列
    print(v1,v2,v3)

>>> a = zip([1, 2], ['a', 'b', 'c'])
>>> list(a)
[(1, 'a'), (2, 'b')]
>>> list(a)
[]    #迭代器只能用一次   注意：range()对象不是迭代器，可以多次使用
```



3.1.5 内置函数对列表的操作

- enumerate()（枚举）函数:是迭代器，用来枚举可迭代对象中的元素，返回可迭代的enumerate对象，其中每个元素都是包含索引和值的**元组**

```
>>> list(enumerate(['a', 'b', 'c']))
[(0, 'a'), (1, 'b'), (2, 'c')]

>>> list(zip(range(3), ['a', 'b', 'c'])) #要多调用一个函数，不如enumerate直观
[(0, 'a'), (1, 'b'), (2, 'c')]
```

```
colours = ["red", "green", "blue", "green", "yellow"]
indices = [i for i, x in enumerate(colours) if x == 'green']
```

3.1.5 内置函数对列表的操作

- map()把一个函数func依次映射到序列的每个元素上，并返回一个可迭代的map对象，map对象中每个元素是原序列中元素经过函数func处理后的结果

```
>>> list(map(str, range(5)))  #把列表中元素转换为字符串
['0', '1', '2', '3', '4']
```

■等价于：

```
a = []
for i in range(5):
    a.append(str(i))
```

```
>>> [ str(i) for i in range(5) ]  #map()是函数式风格，python3.0后大家更喜欢列表推导式
['0', '1', '2', '3', '4']
```

3.1.5 内置函数对列表的操作

```
>>> def add(x, y):              #可以接收2个参数的函数
    return x+y

>>> list(map(add, range(5), range(5,10)))   #把双参数函数映射到两个序列上
[5, 7, 9, 11, 13]            #返回列表

>>> [ i+j for i,j in zip(range(5), range(5,10)) ]
[5, 7, 9, 11, 13]
```

- If Guido van Rossum, the author of the programming language Python, had got his will, this chapter would be missing in our tutorial.
- In his article from May 2005 "All Things Pythonic: The fate of reduce() in Python 3000", he gives his reasons for dropping lambda, map(), filter() and reduce(). He expected resistance from the Lisp and the scheme "folks". What he didn't anticipate was the rigidity of this opposition.
- Enough that Guido van Rossum wrote hardly a year later: "After so many attempts to come up with an alternative for lambda, perhaps we should admit defeat. I've not had the time to follow the most recent rounds, but I propose that we keep lambda, so as to stop wasting everybody's talent and time on an impossible quest."

- We can see the result: lambda, map() and filter() are still part of core Python. Only reduce() had to go; it moved into the module functools.

- His reasoning for dropping them is like this:

✓ There is an equally powerful alternative to lambda, filter, map and reduce, i.e. list comprehension
✓ List comprehension is more evident and easier to understand
✓ Having both list comprehension and "Filter, map, reduce and lambda" is transgressing the Python motto "There should be one obvious way to solve a problem"

如果Python编程语言之父吉多·范罗苏姆（Guido van Rossum）当初能如愿以偿，那么本教程中这一章根本不会存在。
在2005年5月发表的《Pythonic万象：reduce()在Python 3000中的命运》一文中，他阐述了决定舍弃lambda、map()、filter()和reduce()的原因。他预料到会遭到Lisp和Scheme"阵营"的抵制，但没料到反对声浪会如此顽固。以至于不到一年后，吉多就无奈写道："经过多次尝试寻找lambda的替代方案后，也许我们该承认失败了。我最近没时间跟进最新讨论，但我建议保留lambda，别再让大家把才华和时间浪费在这场徒劳的探索上了。"
结果显而易见：lambda、map()和filter()至今仍是Python核心功能，只有reduce()被移到了functools模块。
他主张舍弃这些功能的核心理由在于：

列表解析（list comprehension）能完全替代lambda、filter、map和reduce的功能
列表解析的语法更直观易懂
同时保留列表解析和"filter/map/reduce/lambda"体系，违背了Python"解决问题的方法应该只有一种最直观方式"的设计哲学

（翻译说明：

保留技术术语原貌如"list comprehension"译为行业通用译名"列表解析"
"folks"根据语境译为"阵营"而非字面意义的"伙计们"，体现语言社区的分歧
将英语长句拆分为符合中文阅读习惯的短句结构
重要概念如Python设计哲学用引号强调
补充"核心理由在于"作为逻辑连接词，使技术论点更清晰）

Practice

- 设a是一个列表，使用enumerate()函数实现：打印所有值为v的元素的下标

✓
```
for i,item in enumerate(a):
    if item == v:
        print(i)
```

习题

3.1 表达式 [3] in [1, 2, 3, 4] 的值为_____。

3.2 列表对象的 sort() 方法用来对列表元素进行原地排序，该函数的返回值为_____。

3.3 列表对象的_____方法删除首次出现的指定元素，如果列表中不存在要删除的元素，则抛出异常。

3.14 表达式 [1, 2, 3].count(4) 的值为_____。

3.15 表达式 [1, 2, 3, 1, 2].index(2) 的值为_____。

习题

3.19 多选题：下面的列表方法中有返回值并且返回值不是空值 None 的有（　　）。

A. sort()　　B. pop()　　C. index()　　D. reverse()

3.22 多选题：下面的列表方法中不影响列表内存首地址的有（　　）。

A. sort()　　B. pop()　　C. index()　　D. reverse()

3.23 多选题：下面的列表方法中不影响列表中元素的有（　　）。

A. sort()　　B. count()　　C. index()　　D. pop()

作业4.3.1

1. 使用random库中的choice函数生成包含从’abcd’中选取的10个随机字符的列表a

2. 使用循环与index方法将列表a中’a’的下标存到列表b内

3. 使用循环和remove方法删除列表a中所有的’a’（提示：使用count方法）

第3章 详解Python序列结构

- 3.1 列表：打了激素的数组
  - 3.1.1 列表创建与删除
  - 3.1.2 列表元素访问
  - 3.1.3 列表常用方法
  - 3.1.4 列表对象支持的运算符
  - 3.1.5 内置函数对列表的操作
  - **3.1.6 列表推导式语法与应用案例**
  - 3.1.7 切片操作的强大功能

3.1.6 列表推导式语法与应用案例

■列表推导式使用非常简洁的方式来快速生成满足特定需求的列表

■代码具有非常强的可读性 #体现了python的编程风格

■列表推导式语法形式为：

```
[expression for expr1 in sequence1 if condition1
            for expr2 in sequence2 if condition2
            for expr3 in sequence3 if condition3
            ...
            for exprN in sequenceN if conditionN]
```

■ 最左边的是最外层循环，最右边的为最内层循环

```
a = [expression for v1 in seq1 if cond1
           for v2 in seq2 if cond2]

a = []
for v1 in seq1:
    if cond1:
        for v2 in seq2:
            if cond2:
                a.append(expression)
```

3.1.6 列表推导式语法与应用案例

■列表推导式在逻辑上等价于循环加选择结构，但形式上更加简洁

```
>>> aList = [x*x for x in range(10)]
```

相当于

```
>>> aList = []
>>> for x in range(10):
    aList.append(x*x)
```

3.1.6 列表推导式语法与应用案例

```
>>> freshfruit = [' banana', ' loganberry ', 'passion fruit ']
>>> aList = [w.strip() for w in freshfruit]
```

- 等价于下面的代码

```
>>> aList = []
>>> for item in freshfruit:
    aList.append(item.strip())
```

3.1.6 列表推导式语法与应用案例

- 阿凡提与国王比赛下棋，国王说要是自己输了的话阿凡提想要什么他都可以拿得出来。阿凡提说那就要点米吧，棋盘一共64个小格子，在第一个格子里放1粒米，第二个格子里放2粒米，第三个格子里放4粒米，第四个格子里放8粒米，以此类推，后面每个格子里的米都是前一个格子里的2倍，一直把64个格子都放满。需要多少粒米呢？

```
>>> sum([2**i for i in range(64)])
18446744073709551615  #约4.6e11吨
>>> int('1'*64, 2)  #生成由64个字符’1’组成的字符串，2代表2进制，转为10进制
18446744073709551615
```

3.1.6 列表推导式语法与应用案例

（1）实现嵌套列表的平铺

```
>>> alist = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> [num for sublist in alist for num in sublist]
[1, 2, 3, 4, 5, 6, 7, 8, 9]  #最左边的是最外层循环，最右边的为最内层循环
```

```
>>> alist = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> result = []
>>> for sublist in alist:
        for num in sublist:
            result.append(num)
>>> result
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

3.1.6 列表推导式语法与应用案例

演示：

```
alist = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
result = []
for sublist in alist:
    for num in sublist:
        result.append(num)
        print(result)
print('最终结果：',result)
```

演示：#子列表深度不同，

```
>>> alist = [[1, 2], [4, 5, 6]]
>>> [num for sublist in alist for num in sublist]
[1, 2, 4, 5, 6]
>>> blist = [[1, 2, 3], 4]     # blist = [[1, 2, 3], [4]]这样可以平铺
#4不是列表，平铺会涉及到广度(4)优先还是深度(1)优先问题，问题变
#复杂，无法用列表推导式
#列表推导式功能有限！
```

3.1.6 列表推导式语法与应用案例

（2）过滤不符合条件的元素

■在列表推导式中可以使用if子句对列表中的元素进行筛选，只保留符合条件的元素

```
>>> alist = [-1, -4, 6, 7.5, -2.3, 9, -11]

>>> [i for i in alist if i>0]                    #所有大于0的数字
[6, 7.5, 9]
```

3.1.6 列表推导式语法与应用案例

（3）同时遍历多个层次的列表或可迭代对象 #要求掌握到**两重**循环

```
>>> [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]  # 或 x is not y
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

```
>>> result = []
>>> for x in [1, 2, 3]:
        for y in [3, 1, 4]:
            if x != y:
            result.append((x,y))
>>> result
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

3.1.6 列表推导式语法与应用案例

演示:

```
a = []
for x in [1, 2, 3]:
    for y in [3, 1, 4]:
        if y!=x:
            a.append((x, y))
        #print(x, y, '**********', a)
print(a)

print([(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y])
```

3.1.6 列表推导式语法与应用案例

（4）列表推导式中可以使用函数或复杂表达式

```
>>> def f(x):
    if x%2 == 0:
        x = x**2
    else:
        x = x+1
    return x

>>> [f(x) for x in [2, 3, 4, -1] if x>0]
[4, 4, 16]

#三元运算符可以作为expression
>>> [x**2 if x%2 == 0 else x+1 for x in [2, 3, 4, -1] if x>0]
[4, 4, 16]
```

3.1.6 列表推导式语法与应用案例

■ **判断**列表中是否所有元素都大于5

```
>>> x = list(range(10))
>>> [item>5 for item in x]  #条件表达式
[False, False, False, False, False, False, True, True, True, True]
```

演示：

```
>>> all([item>5 for item in range(10)])

>>> all([item>5 for item in range(6,10)]) #列表推导式

>>> all((item>5 for item in range(6,10))) #元组相应的推导式叫生成器推导式

>>> all(item>5 for item in range(6,10))  #元组的()可省略
```

3.1.6 列表推导式语法与应用案例

（5）列表推导式搭配enumerate和zip函数

```
>>> colours = ["red", "green", "blue", "green", "yellow"]
>>> colours.index("green")
```

■值在列表中出现的所有位置?

```
indices = [i for i, x in enumerate(colours) if x == 'green']
```

演示:

```
colours = ["red", "green", "blue", "green", "yellow"]
indices = []
for i, x in enumerate(colours):
    if x == 'green':
        indices.append(i)
    print(i, x, '********', indices)
```

3.1.6 列表推导式语法与应用案例

```
>>> def add(x, y):              #可以接收2个参数的函数
    return x+y

>>> list(map(add, range(5), range(5,10)))   #把双参数函数映射到两个序列上
[5, 7, 9, 11, 13]
>>> [ i+j for i,j in zip(range(5), range(5,10)) ]
[5, 7, 9, 11, 13]
```

演示：

```
a = []
for i,j in zip(range(5), range(5,10)):
    a.append(i+j)
    print(i,j, '***********', a)
```

Practice

- 设a是一个列表，写列表推导式，得到：
- a中元素的平方所构成的列表

✓ `[x**2 for x in a]`

- a中的偶数的平方所构成列表

✓ `[x**2 for x in a if x%2 == 0]`

- a中的偶数的下标所构成列表

✓ `[i for i,x in enumerate(a) if x%2 == 0 ]  #可以把 i,x 写成 (i,x)`

- 设a和b是两个列表，写列表推导式，得到
- a和b相同位置上的元素相除所构成的列表

✓ `[x/y for (x,y) in zip(a,b)]`

- a和b相同位置上的元素相除所构成的列表，且判断被除数不能为零

✓ `[x/y for (x,y) in zip(a,b) if y != 0]`

第3章 详解Python序列结构

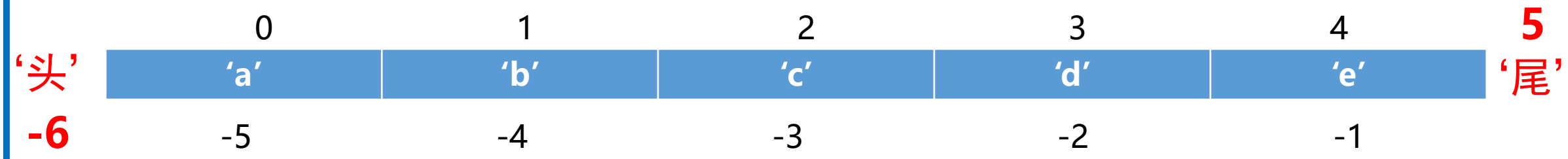
- 3.1 列表：打了激素的数组
  - 3.1.1 列表创建与删除
  - 3.1.2 列表元素访问
  - 3.1.3 列表常用方法
  - 3.1.4 列表对象支持的运算符
  - 3.1.5 内置函数对列表的操作
  - 3.1.6 列表推导式语法与应用案例
  - **3.1.7 切片操作的强大功能**

3.1.7 切片操作的强大功能

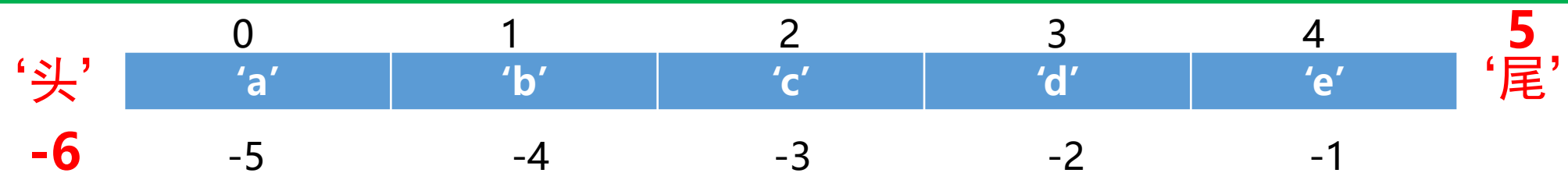
■切片的作用

✓截取序列（有序序列：列表、元组、字符串、range对象）中的一部分得到一个新序列

✓通过切片对列表进行原地操作（修改、增删元素）



3.1.7 切片操作的强大功能



■形式：使用2个冒号分隔的3个数字（只有第一个冒号是必需的）

[start:end:step]

✓第一个数字start表示切片开始位置
- （step>0省略时表示‘头’，step<0省略时表示‘尾’）

✓第二个数字end表示切片截止（但不包含）位置
- （step>0省略时表示‘尾’，step<0省略时表示‘头’）

✓第三个数字step表示切片的步长（默认为1，此时可以省略）
✓省略步长时可以同时省略最后一个冒号
✓start，end可以为负数下标

```
>>>x = ['a', 'b', 'c', 'd', 'e']
>>>x[::2]
['a', 'c', 'e']
>>>x[::-2]
['e', 'c', 'a']

>>>x[0:5:1]
['a', 'b', 'c', 'd', 'e']
>>>x[0:5]
['a', 'b', 'c', 'd', 'e']
>>>x[0:-1]
['a', 'b', 'c', 'd']
```

3.1.7 切片操作的强大功能

```
>>> a = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> len(a)
10

>>> a[1:9:2]
[4, 6, 9, 13]
>>> a[1:-1:2]
[4, 6, 9, 13]
```

```
>>> a[:2]  #与a[0:2]同
[3, 4]
>>> a[-2:] #与a[-2:-1]不同，与a[-2:100]结果相同
[15, 17]

>>> a[:]
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> a[::]
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```

3.1.7 切片操作的强大功能

- [start:end:step]当step为负整数时，表示反向切片
- 这时以start为下标的元素应该在以end为下标的元素的右侧（后面）

```
>>> a = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> len(a)
10

>>> a[::-1]                    #start,end全省略，可代表从头到尾取全部
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]

>>> a[9:0:-1]
[17, 15, 13, 11, 9, 7, 6, 5, 4]
>>> a[-1:0:-1]
[17, 15, 13, 11, 9, 7, 6, 5, 4]
>>> a[-1:-11:-1]
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
```

3.1.7 切片操作的强大功能

■ 切片也可用于字符串、元组和range对象，以获取其部分元素

```
>>> '1234'[::-1]
'4321'

>>> (1,2,3,4)[::-1]
(4, 3, 2, 1)

>>> range(1,5)[::-1]
range(4, 0, -1)
```

3.1.7 切片操作的强大功能

（1）使用切片获取列表部分元素

- 使用切片可以返回列表中部分元素组成的新列表
- 切片操作不会因为下标越界而抛出异常，而是简单地在列表尾部截断或者返回一个空列表，代码具有更强的健壮性

3.1.7 切片操作的强大功能

```
>>> a = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]

>>> a[0:100]              #切片结束位置大于列表长度时，从列表尾部截断
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> a[100]                #抛出异常，不允许越界访问
IndexError: list index out of range

>>> a[100:]               #切片开始位置大于列表长度时，返回空列表
[]

>>> a[100::-1]            #切片逆序，100变成尾部
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]

>>> a[-15:3]              #进行必要的截断处理
[3, 4, 5]
```

Practice

- 设a = [1,2,3,4,5]，下列切片的值?

- a[:-1:2]
- a[-2:]
- a[::-1]
- a[5:]
- a[:0]
- a[3:3]　#step>0时，start>=end,返回[]
- a[3:6]

```
my_list = [1,2,3]
my_list.reverse()  #原地逆序
print(my_list)      #[3,2,1]
```

```
my_list = [1,2,3]
new_list = reversed(my_list)
print(my_list)      #[1,2,3]
print(new_list)     #[3,2,1]
```

- a[::-1]与reversed(a)和a.reverse()的区别?

✓ a[::-1]为逆序组成的新列表;

✓ reversed(a)为逆序组成的reversed对象，不修改原列表，返回一个新的迭代对象;
a.reverse()对a做原地逆序,修改原列表

✓ 对a的逆序做遍历时，应该使用reversed(a)　#因为a.reverse()没有返回值

3.1.7 切片操作的强大功能

（2）使用切片为列表增加元素

■可以使用切片在列表任意位置插入新元素，属于原地操作

■赋值运算符=右侧必须为一个列表或一个序列

```
>>> a = [3, 5, 7]
>>> a[len(a):]          #理解：end=len(a)之后为空
[]
>>> a[len(a):] = [9]       #在列表尾部增加元素 #可用extend()替代
[3, 5, 7, 9]

>>> a[:0]     #理解：start=0之前为空
[]
>>> a[:0] = [1, 2]            #在列表头部插入多个元素的唯一方法
                    #等号右侧也可以为range(1,3)
```

3.1.7 切片操作的强大功能

```
>>> a
[1, 2, 3, 5, 7, 9]

>>> a[3:3]    #理解：index 3 和 3 之间也是空
[]

>>> a[3:3] = [4,4]              #在列表中间位置插入多个元素
>>> a
[1, 2, 3, 4, 4, 5, 7, 9]
```

✓ insert()一次性只能插入一个元素，用上面方法可以一次性插入**多个**元素，这是**一次插入多个元素的唯一方法！**

3.1.7 切片操作的强大功能

（3）使用切片替换列表中的元素

```
>>> a = [3, 5, 7, 9]
>>> a[:3] = [1, 2, 3]          #替换列表元素，等号两边的列表长度相等
>>> a
[1, 2, 3, 9]

>>> a[3:] = [4, 5, 6]          #切片连续，等号两边的列表长度可以不相等
>>> a
[1, 2, 3, 4, 5, 6]

>>> a[::2] = [0]*3             #隔一个修改一个 #长度必须相同，不相等则报错
>>> a
[0, 2, 0, 4, 0, 6]
```

3.1.7 切片操作的强大功能

```
>>> a[::2] = ['a', 'b', 'c']    #隔一个修改一个
>>> a
['a', 2, 'b', 4, 'c', 6]


>>> a[1::2] = range(3)          #序列解包
>>> a
['a', 0, 'b', 1, 'c', 2]

>>> a[::2] = [1]                #切片不连续时等号两边列表长度必须相等
ValueError: attempt to assign sequence of size 1 to extended slice of size 3
```

3.1.7 切片操作的强大功能

（4）使用切片删除列表中的元素

```
>>> a = [3, 5, 7, 9]
>>> a[:3] = []                    #删除列表中前3个元素
>>> a                             #remove(x)只能一次删掉一个元素
[9]
```

■也可以结合使用del命令与切片来删除列表中的部分元素，并且切片元素可以不连续

```
>>> a = [3, 5, 7, 9, 11]
>>> a[::2] = []
ValueError: attempt to assign sequence of size 0 to extended slice of size 3
>>> del a[::2]                    #切片元素不连续，隔一个删一个,只能用del + slice
>>> a
[5, 9]
```

✓切片可以实现批量操作，是很多方法无法做到的！

✓但切片的可读性不如方法。**优先使用方法！**

3.1.7 切片操作的强大功能

操作类型	语法	功能	返回值
基本切片	lst[i:j]	获取索引i到j-1的子列表	新列表
从头切片	lst[:j]	获取开始到j-1的子列表	新列表
到尾切片	lst[i:]	获取索引i到结尾的子列表	新列表
完整切片	lst[:]	获取列表完整副本	新列表
步长切片	lst[::k]	每隔k-1个元素取值	新列表
反转切片	lst[::-1]	获取反转副本	新列表
负索引切片	lst[-i:-j]	使用负索引切片	新列表
切片赋值	lst[i:j] = iterable	替换切片范围	None
切片插入	lst[i:i] = iterable	在位置i插入	None
切片删除	lst[i:j] = []	删除切片范围	None

3. 1　列表总结

1.创建列表的方式：

方法	语法示例	适用场景
字面量	[1, 2, 3]	已知元素的静态列表
list()	list(range(5))	从其他可迭代对象转换
列表推导式	[x**2 for x in range(10)]	需要处理或过滤元素的动态创建
乘法	[0] * 5	创建重复元素的列表
拼接	[1,2] + [3,4] 或 a.extend([3,4])	合并现有列表
copy()	original.copy()	创建列表的浅拷贝
map/filter	list(map(int, ['1', '2', '3']))	函数式编程风格

3. 1　列表总结

2.列表添加元素的方式：

方法	语法	特点	适用场景
append()	lst.append(x)	添加单个元素，原地修改	逐个添加元素
extend()	lst.extend(iterable)	添加多个元素，原地修改	合并列表或可迭代对象
insert()	lst.insert(i, x)	在指定位置插入，原地修改	在特定位置插入一个元素
+	lst1 + lst2	创建新列表，不修改原列表	需要保留原列表的拼接
+=	lst1 += lst2	原地修改，等价于 extend	快速合并，不关心原列表
切片赋值	lst1[i:i] = iterable	灵活插入，原地修改	在中间位置插入多个元素
	lst1[len(lst1):] = iterable	尾部插入，原地修改	在尾部插入多个元素
	lst1[:0] = iterable	头部插入，原地修改	在头部插入多个元素
列表推导式	[x for x in ...]	条件添加，创建新列表	基于条件过滤或转换

3. 1　列表总结

3.列表删除元素的方式：

方法	语法	特点	适用场景
remove()	lst.remove(x)	按值删除第一个匹配项	知道要删除的值
pop()	lst.pop([i])	按索引删除并**返回**元素，无索引时默认删除并返回最后一个	需要获取被删除的元素
del	del lst[i] 或 del lst[i:j:step]	按索引或切片删除	精确控制删除位置和范围
clear()	lst.clear()	清空整个列表	快速清空所有元素
列表推导式	[x for x in lst if condition]	条件删除，创建新列表	基于复杂条件过滤
filter()	filter(func, lst)	函数式条件过滤	使用函数进行条件判断
切片赋值	lst[i:j] = []	替换式删除	灵活的原地修改

3. 1　列表总结

4.列表替换元素的方式：

方法	语法示例	特点	适用场景
索引赋值	lst[i] = new_value	精确控制，原地修改	替换单个或少量元素
切片替换	lst[i:j] = new_list	批量替换，原地修改	替换连续范围的元素
循环替换	for i in range(len(lst)):	条件替换，可控	基于复杂条件的替换
列表推导式	[new if cond else x for x in lst]	函数式，创建新列表	简单条件替换
map函数	list(map(func, lst))	函数式，创建新列表	统一转换操作

3. 1　列表总结

5.列表元素的访问和计数方式：

方法	语法	特点	适用场景
索引访问	lst[i]	快速直接访问	知道确切位置
切片访问	lst[i:j]	获取子列表	需要连续范围的元素
循环访问	for x in lst	遍历所有元素	需要处理每个元素
解包访问	a, b, c = lst	同时赋值多个变量	知道列表确切结构

方法	语法	特点	适用场景
count()	lst.count(x)	简单快速	统计单个元素出现次数
len()	len(lst)	统计元素总数	获取列表长度
Counter	collections.Counter(lst)	功能强大	需要完整频率分析
手动计数	使用列表推导式或 filter()	灵活可控	复杂条件计数

3. 1　列表总结

6.列表元素的排序方式：

方法	语法	特点	内存使用	适用场景
sort()	lst.sort()	原地排序，修改原列表	低	不需要保留原列表
sorted()	sorted(list)	返回新列表，原列表不变	高	需要保留原列表
reverse()	lst.reverse()	原地反转	低	简单反转顺序
reversed()	reversed(list)	返回反转迭代器	低	需要反转迭代器
切片[::-1]	lst[::-1]	创建反转副本	高	需要反转副本

参数	说明	示例
reverse	是否降序排序	sort(reverse=True)
key	排序键函数	sort(key=len)

3. 1　列表总结

7.列表元素的遍历方式：

遍历方法	语法示例	特点	适用场景
直接遍历	for x in lst	简单快速	只需要元素值
enumerate	for i,x in enumerate(lst)	带索引	需要索引和元素值
range+len	for i in range(len(lst))	索引访问	需要修改原列表
zip	for a,b in zip(lst1,lst2)	并行遍历	同时处理多个列表
reversed	for x in reversed(lst)	反向遍历	需要逆序处理
切片遍历	for x in lst[start:end:step]	部分遍历	只需要部分元素
map/filter	for x in map(f, lst)	函数式编程	需要转换或过滤

3.1 列表总结

方法	是否原地操作	语法	功能	返回值
append()	✅ 原地	lst.append(x)	末尾添加元素	None
extend()	✅ 原地	lst.extend(iterable)	扩展列表	None
insert()	✅ 原地	lst.insert(i, x)	指定位置插入	None
remove()	✅ 原地	lst.remove(x)	删除第一个匹配元素	None
pop()	✅ 原地	lst.pop([i])	删除指定位置元素	被删除元素
clear()	✅ 原地	lst.clear()	清空列表	None
sort()	✅ 原地	lst.sort()	排序列表	None
reverse()	✅ 原地	lst.reverse()	反转列表	None
索引赋值	✅ 原地	lst[i] = x	修改单个元素	-
切片赋值	✅ 原地	lst[i:j:step] = iterable	修改切片范围	-

3.1 列表总结

方法	是否原地操作	语法	功能	返回值
sorted()(内置函数)	❌ 非原地	sorted(lst)	对列表排序	新的排序后列表
reversed()(内置函数)	❌ 非原地	reversed(lst)	反转列表顺序	反转迭代器对象
copy()	❌ 非原地	lst.copy()	创建列表的浅拷贝	新的列表副本
切片访问	❌ 非原地	lst[i:j:step]	获取子列表	新的子列表
+ 操作符	❌ 非原地	lst1 + lst2	列表拼接	新的拼接列表
* 操作符	❌ 非原地	lst * n	列表重复	新的重复列表
列表推导式	❌ 非原地	[x for x in lst]	转换或过滤列表	新的列表
map()	❌ 非原地	map(func, lst)	对每个元素应用函数	映射迭代器
filter()	❌ 非原地	filter(func, lst)	过滤列表元素	过滤迭代器

习题

3.4 假设列表对象 aList 的值为 [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]，那么切片 aList[3: 7] 得到的值是_____。

3.8 假设有一个列表 a，现要求从列表 a 中每 3 个元素取 1 个，并且将取到的元素组成新的列表 b，可以使用语句_____。

3.9 使用列表推导式生成包含 10 个数字 5 的列表，语句可以写为_____。

3.11 已知 vec = [[1, 2], [3, 4]]，则表达式 [col for row in vec for col in row] 的值为_____。

3.12 已知 vec = [[1, 2], [3, 4]]，则表达式 [[row[i] for row in vec] for i in range(len(vec[0]))] 的值为_____。

作业4.3.2

1. 设aList = list(range(2,12)),下列切片的值为？
    1. aList[1:9:2]
    2. aList[1:-1:2]
    3. aList[:3]
    4. aList[-3:]
    5. aList[:]
    6. aList[::]
    7. aList[::-1]
    8. aList[9:0:-1]
    9. aList[-1:0:-1]
    10. aList[0:100]
    11. aList[100:]
    12. aList[-15:3]

2. 设a是一个整数列表，写列表推导式，得到或实现：
    ① a中的奇数的平方所构成列表
    ② a中的奇数的下标所构成列表
    ③ 判断a中数是否都是偶数
    ④ 判断a中的数是否至少有一个是偶数

3. 设a和b是两个列表，写列表推导式，得到
    ① a和b相同位置上的元素不相同时相减所构成的列表
    ② 判断a和b相同位置上的元素之差的绝对值是否都小于1.0e-7

第3章 详解Python序列结构

- 3.2 元组：轻量级列表
  - 3.2.1 元组创建与元素访问
  - 3.2.2 元组与列表的异同点
  - 3.2.3 生成器推导式





3.2 元组：轻量级列表

- 列表的功能很强大，但负担也很重，在很大程度上影响了运行效率
- **当仅是对一个有序序列进行遍历或查找，而不需要对其元素进行任何修改时，建议（应该）使用“轻量级的列表”：元组（tuple）**
- 形式上，元组的所有元素放在一对圆括号中，元素之间使用逗号分隔
  - 如果元组中只有一个元素则必须在最后增加一个逗号

第3章 详解Python序列结构

- 3.2 元组：轻量级列表
  - **3.2.1 元组创建与元素访问**
  - 3.2.2 元组与列表的异同点
  - 3.2.3 生成器推导式

3. 2. 1　元组创建与元素访问

```
>>> x = (1, 2, 3)      #直接把元组赋值给一个变量  #>>> type(x)  #tuple

>>> x[0]               #元组支持使用下标访问特定位置的元素
1
>>> x[-1]              #最后一个元素，元组也支持双向索引
3

>>> x[1] = 4           #元组是不可变的
TypeError: 'tuple' object does not support item assignment

>>> x = (3)            #这和x = 3是一样的  #>>> type(x)  #int

>>> x
3
>>> x = (3,)           #如果元组中只有一个元素，必须在后面多写一个逗号
                       #>>> type(x)  #tuple
>>> x
(3,)
```

3.2.1 元组创建与元素访问

```
>>> x = ()                  #空元组    #>>> type(x)  #tuple

>>> x = tuple()             #空元组

>>> tuple(range(5))         #将其他迭代对象转换为元组
(0, 1, 2, 3, 4)
>>> tuple('abc')
('a', 'b', 'c')
>>> tuple([1,2,3])
(1, 2, 3)
>>> tuple(map(str,range(5)))
('0', '1', '2', '3', '4')
```

3.2.1 元组创建与元素访问

```
>>> x = (1, 2, 3)
>>> y = x                #复制标签
>>> y is x
True
>>> del x                #只删除标签
>>> x
NameError: name 'x' is not defined
>>> y
(1, 2, 3)
>>> del x[0]             #不能删除某个元素
NameError: name 'x' is not defined
```

3. 2. 1　元组创建与元素访问

- 很多内置函数的返回值也是包含了若干元组的可迭代对象，例如enumerate()、zip()等等。

```
>>> list(enumerate(range(5)))
[(0, 0), (1, 1), (2, 2), (3, 3), (4, 4)]
>>> list(zip(range(3), 'abcdefg'))
[(0, 'a'), (1, 'b'), (2, 'c')]
```

#enumerate(),zip()它们返回的是一个迭代器（标签），而不是具体的序列，需用list(enumerat()),tuple(enumerate())等将序列(标签对应的值)显示出来

第3章 详解Python序列结构

- 3.2 元组：轻量级列表
  - 3.2.1 元组创建与元素访问
  - **3.2.2 元组与列表的异同点**
  - 3.2.3 生成器推导式

3.2.2 元组与列表的异同点

```
>>> dir(list)
[..., 'append', 'clear', 'copy', 'count', 'extend', 'index',
'insert', 'pop', 'remove', 'reverse', 'sort']

>>> dir(tuple)
[..., 'count', 'index']
```

注意：元组没有 copy() 方法：对于不可变对象，一般无需复制操作，若需要直接赋值即可，"简单优于复杂"

```
>>>a = (1,2,3)
>>>b = a  #元组复制
>>>b is a
True
```

```
>>>a = (1,2,3)
>>>b = a[:]  #切片复制
>>>b is a
True
```

```
>>>a = (1,2,3)
>>>b = tuple(a)  #函数复制
>>>b is a
True
```

3. 2. 2　总结元组的所有操作

类别	方法/操作	语法	描述	示例
创建元组	空元组	() 或 tuple()	创建空元组	t = ()
	单元素元组	(item,) 或 item,	创建单元素元组	t = (1,) 或 t = 1,
	多元素元组	(item1, item2)	创建多元素元组	t = (1, 2, 3)或 t=1,2,3
	从序列转换	tuple(iterable)	从其他序列转换	t = tuple([1, 2, 3])
访问元素	索引访问	t[index]	通过索引访问元素	t[0]
	负索引	t[-index]	从末尾开始访问	t[-1]
	切片	t[start:end:step]	获取子元组	t[1:3]

3.2.2 总结元组的所有操作

类别	方法/操作	语法	描述	示例
查询操作	元素存在	item in t	检查元素是否存在	'a' in t
	元素不存在	item not in t	检查元素是否不存在	'x' not in t
	计数	t.count(item)	统计元素出现次数	t.count('a')
	查找索引	t.index(item)	返回元素首次出现的索引	t.index('b')
	指定范围查找	t.index(item, start, end)	在指定范围内查找	t.index('b', 2, 5)
基本信息	长度	len(t)	返回元组长度	len(t)
	最小值	min(t)	返回最小元素	min(t)
	最大值	max(t)	返回最大元素	max(t)
	求和	sum(t)	返回所有元素的和	sum(t)
	枚举遍历	for i, item in enumerate(t):	带索引遍历	for i, x in enumerate(t):

3.2.2 总结元组的所有操作

类别	方法/操作	语法	描述	示例
组合操作	连接	t1 + t2	连接两个元组	(1,2) + (3,4) → (1,2,3,4)
	重复	t * n	重复元组n次	(1,2) * 3 → (1,2,1,2,1,2)
比较操作	相等	t1 == t2	比较是否相等	(1,2) == (1,2) → True
	大小	t1 < t2	按字典序比较	(1,2) < (1,3) → True
解包操作	基本解包	a, b = t	将元组解包到变量	a, b = (1, 2)
	星号解包	a, *b, c = t	扩展解包	a, *b, c = (1,2,3,4)
遍历操作	直接遍历	for item in t:	遍历元组元素	for x in t: print(x)
	枚举遍历	for i, item in enumerate(t):	带索引遍历	for i, x in enumerate(t):
转换操作	转列表	list(t)	转换为列表	list((1,2,3)) → [1,2,3]
	转字符串	str(t)	转换为字符串	str((1,2)) → "(1, 2)"
排序操作	排序	sorted(t)	返回排序后的列表	sorted((3,1,2)) → [1,2,3]
	反向排序	sorted(t, reverse=True)	返回降序排序列表	sorted((3,1,2), reverse=True) → [3,2,1]

3. 2. 2　元组与列表的异同点

- 同：
  - 列表和元组都属于有序序列，都支持使用双向索引访问其中的元素，都支持使用切片截取一部分元素得到一个新序列
  - 都可使用count()方法统计指定元素的出现次数和index()方法获取指定元素的索引
  - 都支持len()、map()、filter()等大量内置函数和+、+=、in等运算符

3.2.2 元组与列表的异同点

■异：

- 元组属于不可变（immutable）序列，不可以直接修改元组中元素的值，也无法为元组增加或删除元素。
- 元组没有提供append()、extend()、insert()、remove()和pop()方法，也不支持对元组元素进行del操作，而**只能使用del命令删除整个元组。**
- 元组也支持切片操作，而不允许使用切片来修改元组

3.2.2 元组与列表的异同点

- Python的内部实现对元组做了大量优化，访问速度比列表更快
- 元组在内部实现上不允许修改其元素值，从而使得代码更加安全，例如调用函数时使用元组传递参数可以防止在函数中修改元组，而使用列表则很难保证这一点

3.2.2 元组与列表的异同点

特性	元组 (Tuple)	列表 (List)	说明与示例
基本定义	不可变序列	可变序列	元组创建后不能修改，列表可以
语法表示	(1, 2, 3) 或 1, 2, 3	[1, 2, 3]	元组括号可省略，列表不可
空序列	() 或 tuple()	[] 或 list()	
单元素	(1,) 或 1,	[1]	元组单元素必须加逗号
可变性	❌ 不可变	✅ 可变	元组创建后不能增删改
哈希性	✅ 可哈希	❌ 不可哈希	元组可作为字典键，列表不可
内存占用	较小	较大	元组更节省内存
性能	创建和访问更快	修改操作更快	元组适合只读场景

3. 2

操作类型	元组	列表	说明
创建	t = (1, 2, 3)	lst = [1, 2, 3]	
访问	t[0] ✅	lst[0] ✅	都支持索引访问
切片	t[1:3] ✅	lst[1:3] ✅	都支持切片
修改元素	t[0] = 1 ❌	lst[0] = 1 ✅	元组不可修改
添加元素	❌ 不支持	append(), insert() ✅	
删除元素	❌ 不支持	remove(), pop() ✅	
连接	+ ✅	+ ✅	都支持连接
重复	* ✅	* ✅	都支持重复
长度	len() ✅	len() ✅	
包含检查	in ✅	in ✅	
计数	count() ✅	count() ✅	
查找索引	index() ✅	index() ✅	
排序	sorted() ✅	sorted() ✅ sort() ✅	元组返回新列表
反转	reversed() ✅	reversed() ✅ reverse() ✅	元组返回迭代器
复制	直接赋值 ✅	copy() ✅ 或直接赋值	元组无需复杂的拷贝

3.2.2 元组与列表的异同点

特殊方法	元组	列表	描述
append()	❌	✅	列表末尾添加元素
extend()	❌	✅	扩展列表
insert()	❌	✅	插入元素
remove()	❌	✅	删除指定元素
pop()	❌	✅	删除并返回元素
clear()	❌	✅	清空列表
sort()	❌	✅	原地排序
reverse()	❌	✅	原地反转
copy()	❌	✅	浅拷贝

3.2.2 元组与列表的异同点

使用场景	推荐使用	理由
数据保护	元组 ✅	不可变性保证数据安全
字典键	元组 ✅	可哈希性
函数多返回值	元组 ✅	轻量且安全
常量集合	元组 ✅	语义明确
动态数据集合	列表 ✅	需要频繁修改
栈/队列	列表 ✅	支持增删操作
数据缓存	元组 ✅	内存效率高
迭代操作	元组 ✅	性能更好

第3章 详解Python序列结构

- 3.2 元组：轻量级列表
  - 3.2.1 元组创建与元素访问
  - 3.2.2 元组与列表的异同点
  - **3.2.3 生成器推导式**



3.2.3 生成器推导式

```
g = ((i+2)**2 for i in range(10))
```

- **生成器推导式**（generator expression），在形式上使用圆括号作为定界符，内容与列表推导式相同
- 与列表推导式**最大的不同**是，生成器推导式的结果是一个生成器对象
- 生成器对象是一种可迭代对象，但具有**惰性求值**的特点，只在需要时生成新元素，并不事先存储所有元素
- 比列表推导式具有更高的效率，空间占用非常少，因此**应优先使用**

- *生成器推导式是创建生成器的方式之一。*

3. 2. 3　生成器推导式

生成器是一种特殊的迭代器(遍历器)，它不会一次性生成所有数据，而是按需生成（惰性求值），从而节省内存。

生成器的特点：

- 惰性计算：只在需要时生成值
- 内存高效：不会一次性存储所有数据
- 一次性使用：生成器只能迭代一次
- 状态保持：记住上次执行的位置

性能指标	生成器	迭代器
开发效率	高（代码简洁）	低（需要更多代码）
执行效率	相当	相当
内存效率	高	取决于实现
可读性	高	低

生成器是更强大、更简洁的迭代器，在大多数情况下是首选！

3. 2. 3　生成器推导式

```
>>> a = [0,1,2,3]
>>> ls = [(i+2)**2 for i in a]     #type(ls) ->list
[4, 9, 16, 25]

>>> g = ((i+2)**2 for i in a)     #type(g)->generator
>>> next(g)
4
>>> next(g)
9
```

3.2.3 生成器推导式

生成器推导式 vs 列表推导式

特性	生成器推导式	列表推导式	说明
内存占用	✅ 极低	❌ 高	生成器逐个产生元素，不占用大量内存
执行方式	✅ 惰性求值	❌ 立即执行	生成器需要时才计算，列表立即计算所有元素
大数据处理	✅ 适合	❌ 不适合	生成器可处理无限序列或超大数据集
性能	✅ 启动快	❌ 启动慢	生成器立即返回，列表需要先构建完整列表
复用性	❌ 一次性	✅ 可复用	生成器遍历一次后耗尽，列表可重复使用

3. 2. 3　生成器推导式

生成器推导式 vs 列表推导式

```
# 列表推导式 - 立即计算，占用内存
list_comp = [x**2 for x in range(1000000)]
# 立即创建100万个元素的列表
print(f"列表大小: {len(list_comp)}")

# 生成器推导式 - 惰性计算，节省内存
gen_comp = (x**2 for x in range(1000000))
# 不立即创建，只在需要时生成
print(f"生成器大小: 无法直接获取长度")
```

```
In [20]: list_comp = [x**2 for x in range(1000000)]

In [21]: len(list_comp)
Out[21]: 1000000

In [22]: gen_comp = (x**2 for x in range(1000000))

In [23]: len(gen_comp)
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[23], line 1
----> 1 len(gen_comp)

TypeError: object of type 'generator' has no len()
```

3. 2. 3　生成器推导式

生成器推导式 vs 列表推导式（内存效率对比）

```
import sys

# 列表推导式 - 占用大量内存
list_memory = sys.getsizeof([x for x in range(1000000)])
print(f"列表内存占用: {list_memory} bytes")

# 生成器推导式 - 占用极少内存
gen_memory = sys.getsizeof((x for x in range(1000000)))
print(f"生成器内存占用: {gen_memory} bytes")
```

```
In [24]: import sys

In [25]: list_memory = sys.getsizeof([x for x in range(1000000)])

In [26]: print(list_memory)
8448728

In [27]: gen_memory = sys.getsizeof((x for x in range(1000000)))

In [28]: print(gen_memory)
200
```

3.2.3 生成器推导式

■生成器对象的作用：

- ✓ 高效率的遍历或查找序列的特殊形式
- ✓ 用作all(), any(), sorted()等内置函数的参数
- ✓ 赋值给列表的切片
- ✓ 其他可迭代对象出现的场合

■ 生成器对象支持的用法：

- ✓ 使用生成器对象的__next__()方法（了解）或者内置函数next()逐个获取元素
- ✓ 使用for循环来遍历
- ✓ 使用in运算符做查找

3.2.3 生成器推导式

■使用for循环直接遍历生成器对象中的元素

```
>>> g = ((i+2)**2 for i in range(10))
>>> for item in g:
        print(item, end=' ')
4 9 16 25 36 49 64 81 100 121

>>> for item in ((i+2)**2 for i in range(10)):  #更直接
        print(item, end=' ')

#实现对生成器的多次使用(了解，不常用)
>>> g = tuple(((i+2)**2 for i in range(10))) #转化为元组或列表 #内层()可去掉

>>> g = '((i+2)**2 for i in range(10))'  #把生成器转成字符串，每次使用调用eval()
>>> for item in eval(g):
       print(item, end=' ')
```

3.2.3 生成器推导式

```
>>> g = '((i+2)**2 for i in range(10))'
>>> a = [1,2,3]
>>> a[3:] = eval(g)    #把生成器赋值给列表的切片来替换列表中元素
>>> a
[1, 2, 3, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121]

>>> sorted(eval(g), reverse=True)  #用作all(), any(), sorted()等内置函数的参数

[121, 100, 81, 64, 49, 36, 25, 16, 9, 4]

>>> all((i+2)**2 < 100 for i in range(10))  #生成器的()可省略
False
>>> all((i+2)**2 < 200 for i in range(10))
True
```

3.2.3 生成器推导式

■in运算：访问过的元素不再存在

```
>>> g = (i for i in range(10))
>>> 1 in g
True
>>> 2 in g
True
>>> 1 in g
False
>>> list(g)
[]
```

3. 2. 3　生成器推导式

```
# 验证生成器的惰性
>>> a = [1,2,3,4]
>>> g = (i**2 for i in a)
>>> a.clear()  #谨慎操作
>>> list(g)
[]
```

- 生成器推导式 (i**2 for i in a) 在创建时：
  - 不会立即执行计算
  - 不会复制列表 a 的数据
  - 只是保存了对列表 a 的引用
  - 实际计算推迟到迭代时进行
- 结果为空列表！ 因为当真正迭代生成器时，原列表 a 已经被清空了。

3.2.3 生成器推导式

■生成器对象的特性：

- 对生成器对象，没有任何方法可以再次访问已访问过的元素，也不支持使用下标访问其中的元素
- 当所有元素访问结束以后，生成器变“空”

3.2.3 生成器推导式

■使用内置函数next()依次获取生成器对象中的元素

```
>>> g = ((i+2)**2 for i in range(10))  #创建生成器对象
>>> type(g)
<class 'generator'>
>>> tuple(g)                          #将生成器对象转换为元组
(4, 9, 16, 25, 36, 49, 64, 81, 100, 121)
>>> list(g)                  #生成器对象已遍历结束，没有元素了
[]

>>> g = ((i+2)**2 for i in range(10))  #重新创建生成器对象
>>> next(g)                  #使用函数next()获取生成器对象中的元素
4
```

Practice

- 使用哪个内置函数来逐个访问生成器对象的元素？

✓ next()

- 下列语句输出？

```
>>> g = (i**2 for i in range(0,10,2))
>>> 10 in g
>>> list(g)
```

✓ False

✓ []　　#第二句in 操作会消耗元素：直到找到目标或确定不存在
　　#上一句访问10的时候已经便利一遍了

```
>>> g = (i**2 for i in range(0,10,2))
>>> 4 in g    #in 操作找到目标后立即停止，生成器指针会停留在找到位置的下一个元素之前
>>> list(g)   #list(g) 总是从当前指针位置开始收集剩余元素
```

✓ True

✓ [16, 36, 64]

习题

3.10 _____（可以、不可以）使用 del 命令来删除元组中的部分元素。

休息一下

子在川上曰：逝者如斯夫，不舍昼夜。

——《论语·子罕》

My mom always said life was like a box of chocolates. You never know what you're gonna get.
人生就像一盒巧克力，你永远不知道下一颗是什么味道.

——《阿甘正传》

第3章 详解Python序列结构

- 3.1字典：反映对应关系的映射类型
  - 3.3.1字典创建
  - 3.3.2字典元素的访问
  - 3.3.3元素的添加、修改与删除
  - 3.3.4标准库collections中与字典有关的类

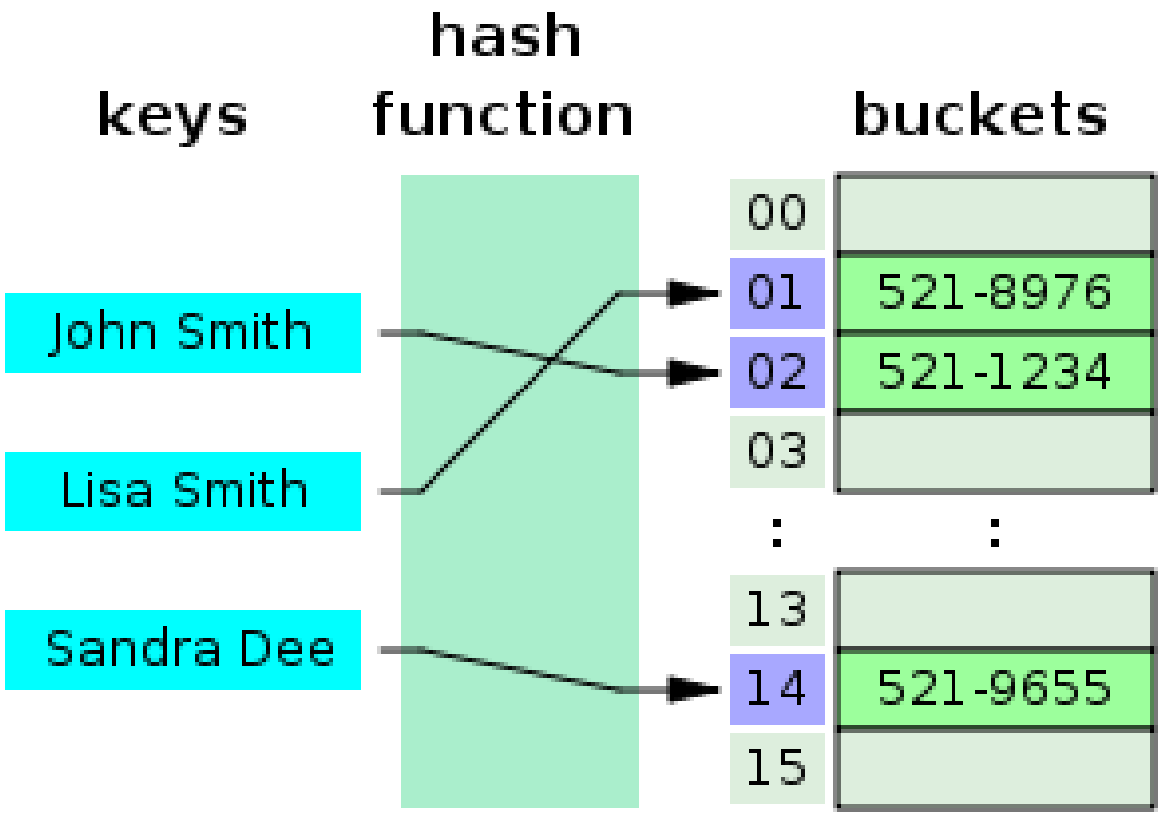
3.3 字典：反映对应关系的映射类型

```
>>> d = {'server': 'db.diveintopython3.org', 'database': 'mysql'}
>>> d = {('appphys', 1):'Xiong', ('jidiban', 1):'Xiong'} #元组可做键
```

- 字典（dictionary）是包含若干“键:值”元素的**无序**可变序列，表示一种映射关系
- 字典中元素的“键”必须是不可变（可哈希）数据类型，例如数字、字符串、元组等，但**不能**使用**列表**等可变类型**作为字典的“键”**
- 字典中的“键”不允许重复，而“值”是可以重复的：字典是一种离散函数

3.3 字典：反映对应关系的映射类型

- CPython对字典的实现（了解）：散列表（Hash table，也叫哈希表），是根据键（Key）而直接访问内存存储位置的数据结构



3.3 字典：反映对应关系的映射类型

■ Python字典一个应用（了解）：HTTP请求的“头”（header）--典型的网页爬虫基础模板

```
import requests

headers = {
'User-Agent': 'Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/70.0.3538.110 Safari/537.36',
'Accept':
'text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8',
'Accept-Encoding': 'gzip, deflate',
'Accept-Language': 'zh-CN,zh;q=0.9,en;q=0.8,ja;q=0.7,zh-TW;q=0.6,th;q=0.5,ko;q=0.4,es;q=0.3',
}

url = r'https://book.douban.com/subject/30173345/'
r = requests.get(url, headers = headers)
print(r.text)
```

3.3 字典：反映对应关系的映射类型

■ Python字典另一个例子：摩尔斯电码

```
morse = {
"A" : ".-",
"B" : "-...",
"C" : "-.-.",
"D" : "-..",
"E" : ".",
"F" : "..-.",
"G" : "--.",
"H" : "....",
"I" : "..",
"J" : ".---",
"K" : "-.-",
"L" : ".-..",
"M" : "--",
"N" : "-.",
"O" : "---",
"P" : ".--.",
"Q" : "--.-",
"R" : ".-.",
"S" : "...",
"T" : "-",
"U" : "..-",
"V" : "...-",
"W" : ".--",
"X" : "-..-",
"Y" : "-.--",
"Z" : "--..",
"0" : "-----",
"1" : ".----",
"2" : "..---",
"3" : "...--",
"4" : "....-",
"5" : ".....",
"6" : "-....",
"7" : "--...",
"8" : "---..",
"9" : "----.",
"." : ".-.-.-",
"," : "--..--"
}
```

演示：

```
>>> len(morse)
38

>>> "a" in morse
False
>>> "A" in morse
True
>>> "a" not in morse
True

morse1 = dict(zip(morse.values(), morse.keys()))
```

第3章 详解Python序列结构

- 3.1字典：反映对应关系的映射类型
  - **3.3.1字典创建**
  - 3.3.2字典元素的访问
  - 3.3.3元素的添加、修改与删除
  - 3.3.4标准库collections中与字典有关的类

3. 3. 1　字典创建

■可以使用内置类dict以不同形式创建字典　#dir(dict)

```
>>> x = dict()                          #空字典
>>> type(x)
<class 'dict'>
>>> x = {}

>>> keys = ['a', 'b', 'c', 'd']
>>> values = [1, 2, 3, 4]
>>> dictionary = dict(zip(keys, values))     #根据已有数据创建字典
```

3. 3. 1　字典创建

```
>>> d = dict(name='Dong', age=39)          #以关键参数的形式创建字典
>>> d = {'name': 'Dong', 'age': 39}


>>> aDict = dict.fromkeys(['name', 'age', 'sex'])
                        #类方法的调用，不同于常见的实例方法
                        #以给定内容为“键”，创建“值”为空的字典
>>> aDict
{'age': None, 'name': None, 'sex': None}  #可用于初始化字典

>>> aDict = dict.fromkeys(['name', 'age', 'sex'],0)
                         #以给定内容为“键”，创建“值”为空的字典
>>> aDict
{'name': 0, 'age': 0, 'sex': 0}  #用于所有键需要相同的初始值的情况
```

3. 3. 1　字典创建

■使用字典对象的copy()方法获得字典的浅复制（了解）

```
>>> aDict = {'age': 39, 'score': [98, 97], 'name': 'Dong', 'sex': 'male'}
>>> d = aDict.copy()
>>> d['score'][0] = 100
>>> aDict
{'age': 39, 'score': [100, 97], 'name': 'Dong', 'sex': 'male'}
                                #原理类似于列表中对可变元素的浅拷贝，本ppt32页
```

第3章 详解Python序列结构

- 3.1字典：反映对应关系的映射类型
  - 3.3.1字典创建
  - **3.3.2字典元素的访问**
  - 3.3.3元素的添加、修改与删除
  - 3.3.4标准库collections中与字典有关的类

3. 3. 2　字典元素的访问

■以“键”为下标访问对应的“值”
■如果字典中不存在这个“键”会抛出异常

```
>>> aDict = {'age': 39, 'score': [98, 97], 'name': 'Dong', 'sex': 'male'}
>>> aDict['age']                    #指定的“键”存在，返回对应的“值”
39


>>> 'address' in aDict
False

>>> aDict['address']                #指定的“键”不存在，抛出异常
KeyError: 'address'
```

3.3.2 字典元素的访问

- 字典对象的get(k[,d])方法用来返回指定“键”对应的“值”
- 允许指定该键不存在时返回特定的“值”

```
>>> aDict.get('age')                          #如果字典中存在该“键”则返回对应的“值”
39
>>> aDict.get('address', 'Not Exists.') #指定的“键”不存在时返回指定的默认值
'Not Exists.'
```

- (了解)字典对象的setdefault(k[,d])方法与get(k[,d])方法用法基本相同
- 但当键k不存在时，设置其对应的值为d

```
>>> aDict.setdefault('address', 'Not Exists.')
'Not Exists.'
>>> aDict
{'age': 39, 'score': [98, 97], 'name': 'Dong', 'sex': 'male', 'address': 'Not Exists.'}
```

d[key]访问：用于“必须存在”场景，强调数据的完整性
get()访问：用于“可选存在”场景，提供安全访问
setdefault()访问：用于“初始化或获取”场景，简化代码

3. 3. 2　字典元素的访问

■对字典直接进行遍历时，默认是遍历字典的“键”

```
>>> aDict = {'age': 39, 'score': [98, 97], 'name': 'Dong', 'sex': 'male'}
>>> for k in aDict:
        print(k, aDict[k])

age 39
score [98, 97]
name Dong
sex male
```

■遍历字典的“键值对”、“值”，必须使用字典对象的items()和values()方法

3. 3. 2　字典元素的访问

- 使用字典对象的items()方法可以返回字典的键、值对序列（可迭代的dict_items对象）

```
>>> aDict.items()
dict_items([('age', 39), ('score', [98, 97]), ('name', 'Dong'), ('sex', 'male'), ('address', 'Not Exists.')])        #(键,值)以元组形式输出
```

使用字典对象的values()方法可以返回字典的值序列

```
>>> aDict.values()
dict_values([39, [98, 97], 'Dong', 'male', 'Not Exists.'])
```

使用字典对象的keys()方法可以返回字典的键序列

```
>>> aDict.keys()
dict_keys(['age', 'score', 'name', 'sex', 'address'])
```

3. 3. 2　字典元素的访问

- **问题解决：**已知有一个包含一些同学成绩的字典，现在需要计算所有成绩的最高分、最低分、平均分，并查找所有最高分同学

```
>>> scores = {"Zhang San": 99, "Li Si": 78, "Wang Wu": 40, "Zhou Liu": 96,
              "Zhao Qi": 65, "Sun Ba": 90, "Zheng Jiu": 78, "Wu Shi": 99,
              "Dong Shiyi": 60}

>>> highest = max(scores.values())                    #最高分
>>> lowest = min(scores.values())                     #最低分
>>> average = sum(scores.values()) / len(scores)      #平均分
>>> highest, lowest, average
(99, 40, 72.33333333333333)

>>> highestPerson = [name for name, score in scores.items() if score == highest]
>>> highestPerson
['Wu Shi']
```

3. 3. 2　字典元素的访问

演示：

```
scores = {"Zhang San": 99, "Li Si": 78, "Wang Wu": 40, "Zhou Liu": 96,
          "Zhao Qi": 65, "Sun Ba": 90, "Zheng Jiu": 78, "Wu Shi": 99,
          "Dong Shiyi": 60}

print(list(scores.values()))
print(list(scores.items()))

highest = max(scores.values())
print(highest)

highestPerson = []
for name, score in scores.items():
    if score == highest:
        highestPerson.append(name)
    print(name, score, highestPerson)

print([name for name, score in scores.items() if score == highest])
```

3. 3. 2　字典元素的访问

- 对字典scores按姓名排序?

```
>>> sorted(scores.items())　#默认按键排序
```

- 对字典scores按分数排序？(了解)

```
>>> sorted(scores.items(), key=lambda v: v[1])
                                    #每个v是元组 (键，值)
```

第3章 详解Python序列结构

- 3.1字典：反映对应关系的映射类型
  - 3.3.1字典创建
  - 3.3.2字典元素的访问
  - **3.3.3元素的添加、修改与删除**
  - 3.3.4标准库collections中与字典有关的类

3.3.3 元素添加、修改与删除

■当以指定“键”为下标为字典元素赋值时，有两种含义

✓若该“键”存在，则表示修改该“键”对应的值

✓若不存在，则表示添加一个新的“键:值”对，也就是添加一个新元素

```
>>> aDict = {'age': 35, 'name': 'Dong', 'sex': 'male'}
>>> aDict['age'] = 39                    #修改元素值
>>> aDict
{'age': 39, 'name': 'Dong', 'sex': 'male'}

>>> aDict['address'] = 'SDIBT'   #山东工商学院      #添加新元素
>>> aDict
{'age': 35, 'name': 'Dong', 'sex': 'male', 'address': 'SDIBT'}
```

3.3.3 元素添加、修改与删除

- 使用字典对象的update()方法可以将另一个字典的“键:值”一次性全部添加到当前字典对象 #类似于列表中的extend()方法
- 如果两个字典中存在相同的“键”，则以新加字典中的“值”为准对当前字典进行更新

```
>>> aDict = {'age': 37, 'score': 98, 'name': 'Dong', 'sex': 'male'}
>>> aDict.update({'a':97, 'age':39})  #修改'age'键的值，同时添加新元素'a':97
>>> aDict
{'age': 39, 'score': 98, 'name': 'Dong', 'sex': 'male', 'a': 97}
#也可以是aDict.update(a=97, age=39)
#或者aDict.update([('a',97), ('age',39)])
```

3.3.3 元素添加、修改与删除

■如果需要删除字典中指定的元素，可以使用del命令 #列表中的del方法相同

```
>>> del aDict['age']                    #删除字典元素
>>> aDict
{'score': [98, 97], 'sex': 'male', 'a': 97, 'name': 'Dong'}
```

■也可以使用字典对象的pop()和popitem()方法弹出并删除指定的元素

```
>>> aDict = {'age': 37, 'score': 98, 'name': 'Dong', 'sex': 'male'}
>>> aDict.popitem()                     #弹出一个元素，对空字典会抛出异常
('age', 37)
>>> aDict.pop('sex')                    #弹出指定键对应的元素
'male'
>>> aDict
{'score': 98, 'name': 'Dong'}
```

■clear()可清空字典 #与列表中的clear方法相同

3. 3. 3　元素添加、修改与删除

- 问题解决：首先生成包含1000个随机字符的字符串，然后统计每个字符的出现次数

```
>>> import string
>>> import random
>>> x = string.ascii_letters + string.digits + string.punctuation
>>> y = [random.choice(x) for i in range(1000)]

>>> d = dict()                    #使用字典保存每个字符出现次数
>>> for ch in y:
        d[ch] = d.get(ch, 0) + 1    #否则返回0
                                    #ch第一次出现在d中会报错，因为d初始为[]，所以用get()
>>> print(d)
```

3.3.3 元素添加、修改与删除

- 问题解决：使用**字典推导式**快速创建字典(了解)
  {键表达式:值表达式 for value in seq if cond}

```
>>> import string
>>> import random
>>> x = string.ascii_letters + string.digits + string.punctuation
>>> y = [random.choice(x) for i in range(1000)]

>>> d = {ch:y.count(ch) for ch in x}
# 或 d = dict((ch, y.count(ch)) for ch in x)
>>> print(d)
```

- #查dir(dict)

Practice

- 如何创建空字典？

✓ `dict()`

✓ `{}`

- 如何以序列k的元素为键，以序列d的元素为值，创建字典？

✓ `dict(zip(k,d))`

- 分别使用哪种方法获取字典的键、值、键值对所构成的序列？

✓ `keys(), values(), items()`

- 如何以序列k的元素为键，以0为对应的值，创建字典？

✓ `dict(zip(k, (0 for i in range(len(k)))))`

✓ `{key:0 for key in k}`

✓ `dict.fromkeys(k, 0)`

习题

3.5 在 Python 中，字典和集合都是用一对_____作为界定符，字典的每个元素由两部分组成，即_____和_____，其中_____不允许重复。

3.6 使用字典对象的_____方法可以返回字典的“键:值”对，使用字典对象的_____方法可以返回字典的所有“键”，使用字典对象的_____方法可以返回字典的所有“值”。

3.7 假设有列表 a = ['name', 'age', 'sex'] 和 b = ['Dong', 38, 'Male']，请使用一条语句将这两个列表的内容转换为字典，并且以列表 a 中的元素为“键”，以列表 b 中的元素为“值”，这个语句可以写为_____。

第3章 详解Python序列结构

- 3.1字典：反映对应关系的映射类型
  - 3.3.1字典创建
  - 3.3.2字典元素的访问
  - 3.3.3元素的添加、修改与删除
  - **3.3.4标准库collections中与字典有关的类**

3.3.4 标准库collections中与字典有关的类

■collections模块的Counter类：频次统计
■提供了更多的功能，例如查找出现次数最多的元素

```
>>> import string
>>> import random
>>> x = string.ascii_letters + string.digits + string.punctuation
>>> y = [random.choice(x) for i in range(1000)]

>>> from collections import Counter
>>> frequences = Counter(y)
>>> frequences.items()
>>> frequences.most_common(1)        #返回出现次数最多的1个字符及其频率
>>> frequences.most_common(3)        #返回出现次数最多的前3个字符及其频率
```

3. 3　字典总结

1. 创建字典

方法	语法	示例	说明
直接创建	{key: value}	d = {'a': 1, 'b': 2}	最常用
dict() 函数	dict()	d = dict(a=1, b=2)	键必须是合法标识符
键值对序列	dict([(k,v)])	d = dict([('a',1),('b',2)])	从元组列表创建
		或 d = dict(zip(['a','b'],[1,2]))	利用zip函数
fromkeys()	dict.fromkeys(seq, val)	d = dict.fromkeys(['a','b'], 0)	所有键相同值
copy()	d.copy()	a = d.copy	浅复制
字典推导式	{k:v for}	d = {x: x**2 for x in range(3)}	动态创建
		或 d = dict(x: x**2 for x in range(3))	

3. 3　字典总结

2. 访问元素

方法	语法	示例	返回值	键不存在时
键访问	d[key]	d['a']	值	KeyError
get()	d.get(key, default)	d.get('a', 0)	值或default	返回default
setdefault()	d.setdefault(key, default)	d.setdefault('a', 0)	值或设置default	设置并返回default

3. 查询信息

方法	语法	示例	返回值	说明
in / not in	key in d	'a' in d	bool	键是否存在
len()	len(d)	len(d)	int	键值对数量
keys()	d.keys()	d.keys()	键视图	所有键
values()	d.values()	d.values()	值视图	所有值
items()	d.items()	d.items()	键值对视图	所有键值对

3.3 字典总结

4. 添加或修改字典

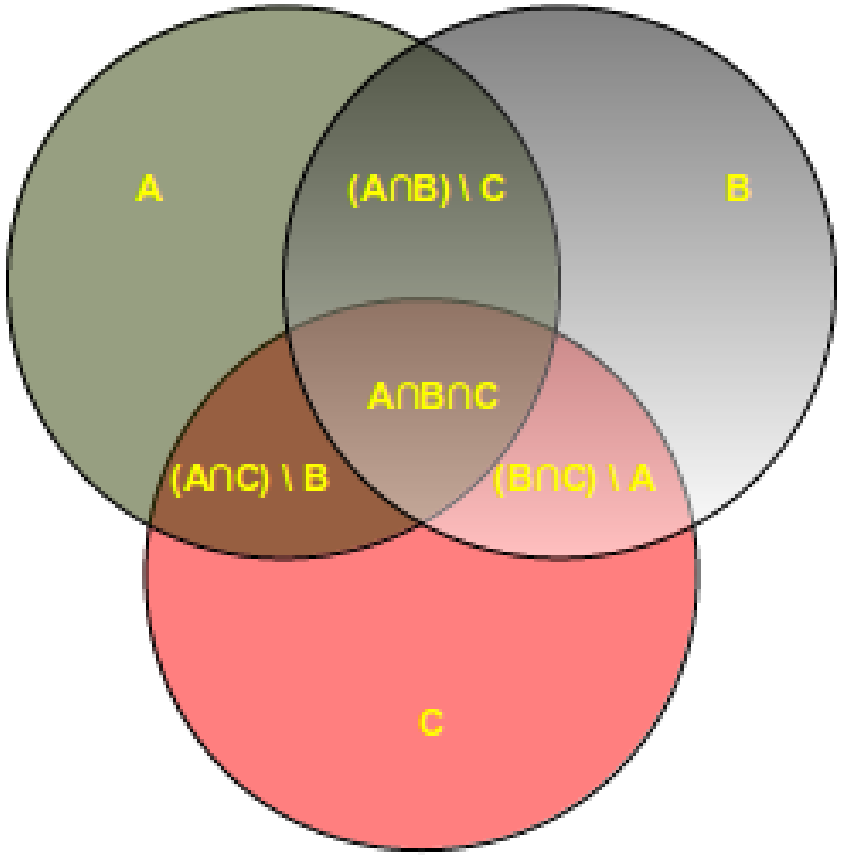
方法	语法	示例	说明
直接赋值	d[key] = value	d['a'] = 1	添加或修改
update()	d.update(other)	d.update({'c':3})	合并字典
	d.update(key=value)	d.update(c=3, d=4)	关键字参数
	d.update([(k,v)])	d.update([('c',3)])	键值对序列

5. 删除元素

方法	语法	示例	返回值	键不存在时
del	del d[key]	del d['a']	None	KeyError
pop()	d.pop(key, default)	d.pop('a')	删除的值	KeyError或缺省值
popitem()	d.popitem()	d.popitem()	(key, value)	KeyError(空字典)
clear()	d.clear()	d.clear()	None	清空所有元素

第3章 详解Python序列结构

- 3.4集合：元素之间不允许重复
  - 3.4.1集合对象的创建
  - 3.4.2集合操作与运算
  - 3.4.3集合应用案例
- 3.5序列解包的多种形式和用法



3.4 集合

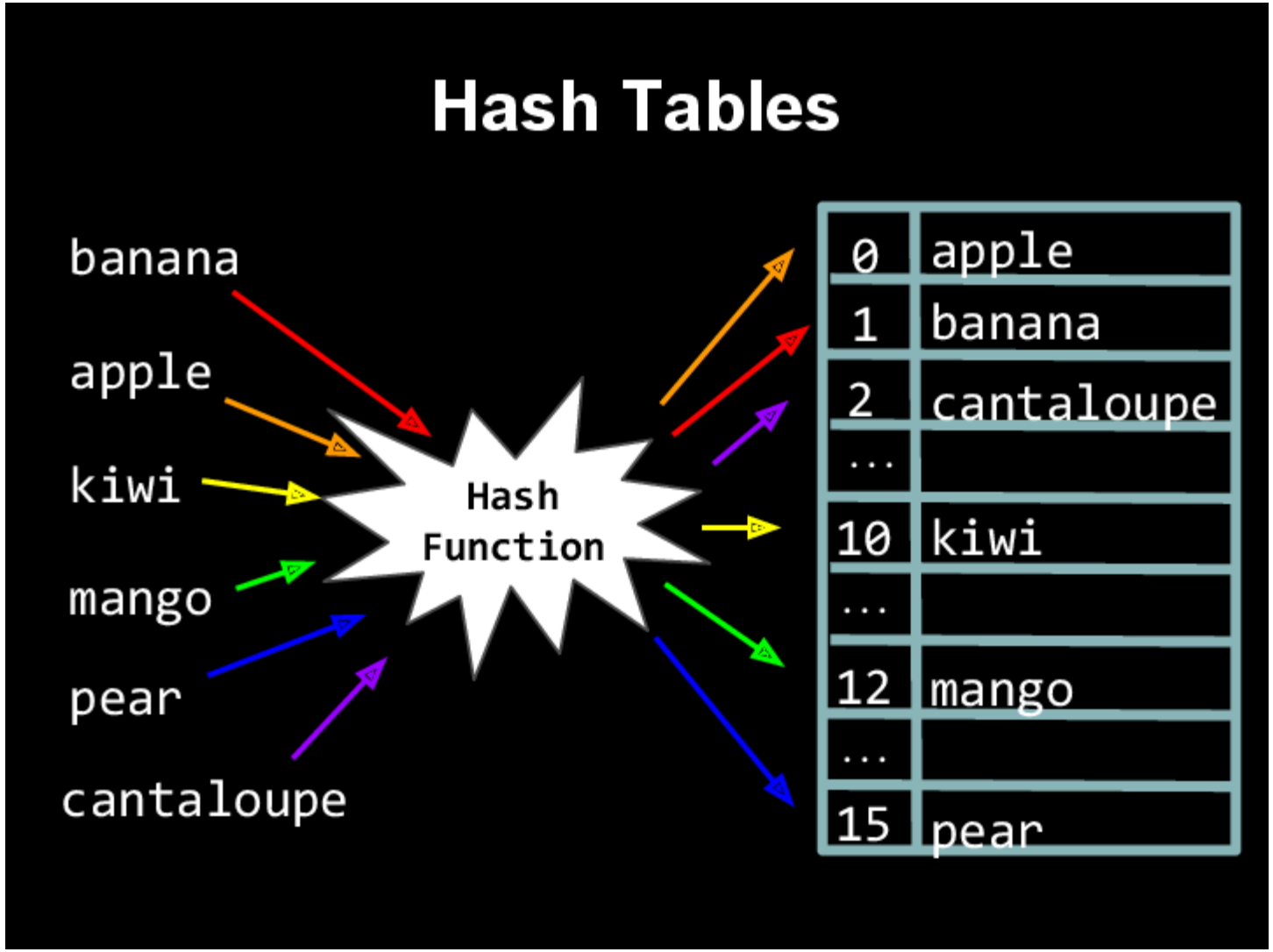
```
>>> fruits = {'banana', 'apple', 'pear'}
```

- 集合（set）属于Python**无序可变**序列
- 一个集合内的每个元素都是唯一的，元素之间不允许重复
- 集合中只能包含数字、字符串、元组等不可变类型（或者说可哈希）的数据，而不能包含列表、字典、集合等可变类型的数据

第3章 详解Python序列结构

- 3.4集合：元素之间不允许重复
  - **3.4.1集合对象的创建**
  - 3.4.2集合操作与运算
  - 3.4.3集合应用案例
- 3.5序列解包的多种形式和用法

3.4 集合



集合底层是字典实现的，

集合的所有元素都是字典的”键”，

因此是不能重复且唯一的。

3.4.1 集合对象的创建

■直接将集合赋值给变量即可创建一个集合对象

```
>>> a = {3, 5}                    #创建集合对象
>>> type(a)                       #查看对象类型
<class 'set'>
```

■可以使用函数set()函数将**列表**、元组、字符串、range对象等其他可迭代对象转换为集合

- 如果原序列中存在重复元素，则在转换为集合的时候只保留一个
- 如果原序列中有不可哈希的值，无法转换成为集合，抛出异常

```
set() -> new empty set objectset(iterable)

set() -> new empty set object set(iterable) -> new set
object

Build an unordered collection of unique elements.
```

```
In [4]: set(0, 1, 2, 3, 0, 1, 2, 3, 7, 8)
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[4], line 1
----> 1 set(0, 1, 2, 3, 0, 1, 2, 3, 7, 8)

TypeError: set expected at most 1 argument, got 10
```

3. 4. 1　集合对象的创建

```
>>> a_set = set(range(8, 14))                    #把range对象转换为集合
>>> a_set
{8, 9, 10, 11, 12, 13}

>>> b_set = set([0, 1, 2, 3, 0.0, 1.0, 2, 3, 7, 8])   #转换时自动去掉重复元素
>>> b_set
{0, 1, 2, 3, 7, 8}            # 0 == 0.0，整数会被提升为浮点数，再进行比较

>>> x = set()                                  #空集合  #{}代表空字典

>>> set('aaabbccdd')
{'a', 'b', 'c', 'd'}

  (了解)
>>> set([[1,2],[3,4]])  #TypeError: unhashable type: 'list'
>>> set([{1,2},{3,4}])  #TypeError: unhashable type: 'set'
>>> set([{'a':1},{'b':2}])  #TypeError: unhashable type: 'dict'
>>> set([(1,2),(3,4)])   #{(1, 2), (3, 4)}
```

3. 4. 1　集合对象的创建

■使用集合推导式快速生成集合；也可以自动去重（了解）

{表达式 for v in seq if cond}

```
>>> set(range(10))
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

>>> {i**2%5 for i in range(10)}          #自动去重
{0, 1, 4}

>>> set(i**2%5 for i in range(10))  #{}换为set()也可以
{0, 1, 4}
```

第3章 详解Python序列结构

- 3.4集合：元素之间不允许重复
  - 3.4.1集合对象的创建
  - **3.4.2集合操作与运算**
  - 3.4.3集合应用案例
- 3.5序列解包的多种形式和用法

3. 4. 2　集合操作与运算

（1）集合元素增加与删除

- add()方法可以增加**新元素**，如果该元素已存在则忽略该操作，不会抛出异常
  #类似于列表中的append()
- update()方法用于**合并**另外一个**集合**中的元素到当前集合中，并自动去除重复元素
- add()和update()均为**原地**操作，并集运算符|生成**新**集合

```
>>> s = {1, 2, 3}
>>> s.add(3)                    #添加元素，重复元素自动忽略
>>> s
{1, 2, 3}

>>> s.update({3,4})             #更新当前字典，自动忽略重复的元素
>>> s
{1, 2, 3, 4}
```

3. 4. 2　集合操作与运算

- pop()方法用于**随机**删除并返回集合中的一个**元素**，如果集合为空则抛出异常
- remove()方法用于删除集合中的**元素**，如果指定元素不存在则**抛出异常**(严肃)
- discard()用于从集合中删除一个**特定**元素，如果元素不在集合中则忽略该操作
- clear()方法清空集合删除所有元素

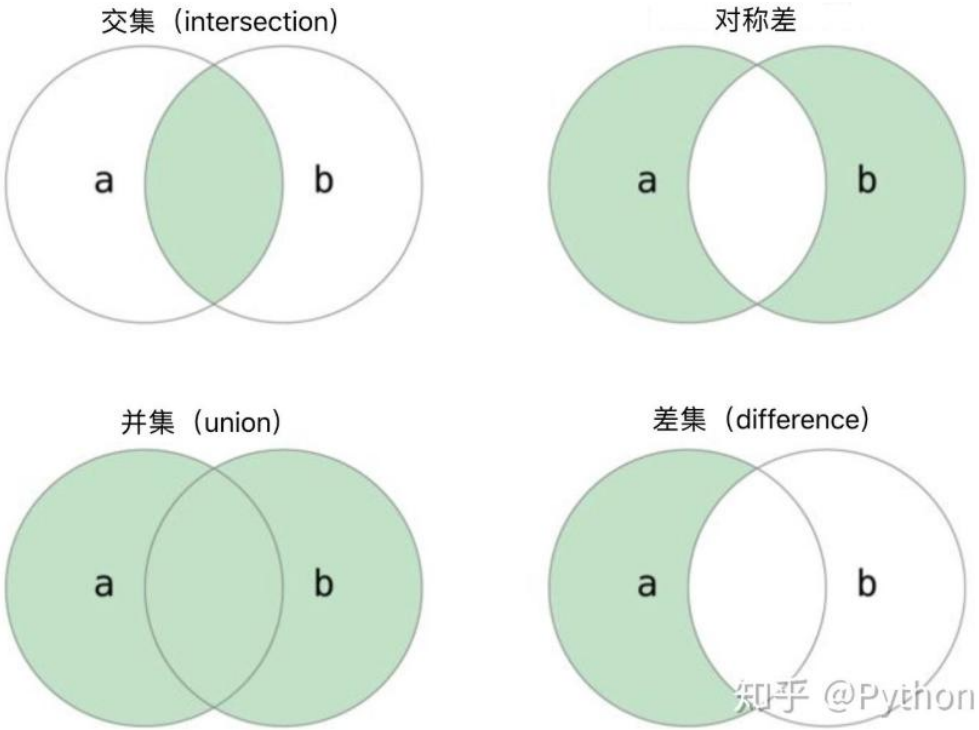
```
>>> s.discard(5)        #删除元素，不存在则忽略该操作
>>> s
{1, 2, 3, 4}
>>> s.remove(5)         #删除元素，不存在就抛出异常
KeyError: 5
>>> s.pop()             #删除并返回一个元素
1
>>> s.clear()
```

•理解与记忆：
•pop(弹出)：像从袋子里随机拿出一个东西（不知道是哪一个）。
•remove(移除)：你要移除一个具体的东西，如果找不到，你会感到异常（报错）。
•　　态度很严肃！！！
•discard(丢弃)：你要丢弃一个东西，如果找不到，那就算了（不报错）。
•　　态度很随意！！！

3.4.2 集合操作与运算

- （复习）集合的交集、并集、对称差集等运算符与位运算符相同，差集使用减号运算符（注意，并集运算符不是加号）。

```
>>> {1, 2, 3} | {3, 4, 5}          #并集，自动去除重复元素
{1, 2, 3, 4, 5}
>>> {1, 2, 3} & {3, 4, 5}          #交集
{3}
>>> {1, 2, 3} ^ {3, 4, 5}          #对称差集
{1, 2, 4, 5}
>>> {1, 2, 3} - {3, 4, 5}          #差集
{1, 2}
```



3. 4. 2　集合操作与运算

（2）集合运算

```
>>> a_set = {8, 9, 10, 11, 12, 13}
>>> b_set = {0, 1, 2, 3, 7, 8}

>>> a_set | b_set                          #并集
{0, 1, 2, 3, 7, 8, 9, 10, 11, 12, 13}
>>> a_set.union(b_set)                     #并集（了解）
{0, 1, 2, 3, 7, 8, 9, 10, 11, 12, 13}

>>> a_set & b_set                          #交集
{8}
>>> a_set.intersection(b_set)              #交集（了解）
{8}
```

3.4.2 集合操作与运算

```
>>> a_set.difference(b_set)                #差集（了解）
{9, 10, 11, 12, 13}
>>> a_set - b_set
{9, 10, 11, 12, 13}

>>> a_set.symmetric_difference(b_set) #对称差集（了解）
{0, 1, 2, 3, 7, 9, 10, 11, 12, 13}
>>> a_set ^ b_set
{0, 1, 2, 3, 7, 9, 10, 11, 12, 13}
>>> (a_set|b_set) - (a_set&b_set)
```

3.4.2 集合操作与运算

```
>>> x = {1, 2, 3}
>>> y = {1, 2, 5}
>>> z = {1, 2, 3, 4}

>>> x < y                                  #比较集合包含关系
False
>>> x < z                                  #真子集
True
>>> y < z
False

>>> {1, 2, 3} <= {1, 2, 3}                 #子集
True
```

3.4 集合总结

基本方法

方法	语法	描述	返回值	元素不存在时	示例
add()	set.add(element)	添加元素到集合	None	-	s.add(5)
update()	set.update(iterable)	批量添加元素	None	-	s.update([1,2,3])
pop()	set.pop()	随机移除并返回元素	被移除的元素	集合为空时报错	x = s.pop()
remove()	set.remove(element)	移除指定元素	None	报错(KeyError)	s.remove(5)
discard()	set.discard(element)	安全移除元素	None	静默忽略	s.discard(5)
clear()	set.clear()	清空所有元素	None	-	

3.4 集合总结

查询操作

方法	语法	描述	返回值	示例
in	element in set	检查元素是否存在	bool	5 in s
not in	element not in set	检查元素是否不存在	bool	5 not in s
len()	len(set)	返回元素个数	int	len(s)

集合运算

方法	语法	描述	数学符号	示例
union()	set1.union(set2)	并集	A ∪ B	s1.union(s2)
intersection()	set1.intersection(set2)	交集	A ∩ B	s1.intersection(s2)
difference()	set1.difference(set2)	差集	A - B	s1.difference(s2)
symmetric_difference()	set1.symmetric_difference(set2)	对称差集	A Δ B	s1.symmetric_difference(s2)

3.4 集合总结

集合关系判断（了解）

方法	语法	描述	返回值	示例
issubset()	set1.issubset(set2)	子集判断	bool	s1.issubset(s2)
issuperset()	set1.issuperset(set2)	超集判断	bool	s1.issuperset(s2)
isdisjoint()	set1.isdisjoint(set2)	是否无交集	bool	s1.isdisjoint(s2)

修改原集合的方法（了解）

方法	语法	描述	示例
intersection_update()	set1.intersection_update(set2)	交集并更新	s1.intersection_update(s2)
difference_update()	set1.difference_update(set2)	差集并更新	s1.difference_update(s2)
symmetric_difference_update()	set1.symmetric_difference_update(set2)	对称差集并更新	s1.symmetric_difference_update(s2)

3.4 集合总结

运算符版本

方法	运算符	描述	示例
并集	|	全部元素去重	集合合并
交集	&	共同元素	s1 & s2
差集	-	只在第一个集合中的元素	s1 - s2
对称差集	^	不同时在两个集合中的元素	s1 ^ s2
子集	<=	子集判断	s1 <= s2
真子集	<	真子集判断	s1 < s2
超集	>=	超集判断	s1 >= s2
真超集	>	真超集判断	s1 > s2

3. 4　集合总结

集合同时提供方法和运算符两种版本：

主要区别和适用场景

方面	方法版本	运算符版本
可读性	语义明确，像英语句子	简洁，数学感强
灵活性	可接受任意可迭代对象	只能用于集合类型
链式调用	支持链式调用	不支持链式调用
多个操作	可处理多个集合	只能两个集合操作

```
# 方法版本更灵活 - 接受任何可迭代对象
set_a = {1, 2, 3}
result1 = set_a.union([3, 4, 5])      # 列表
result2 = set_a.intersection(range(5)) # 范围对象
result3 = set_a | [3, 4, 5]         # 错误!
```

3. 4　集合总结

```
set_a = {1, 2}
set_b = {2, 3}
set_c = {3, 4}
set_d = {4, 5}

# 方法版本支持链式调用
result = (set_a.union(set_b).intersection(set_c).difference(set_d))

# 运算符版本不支持优雅的链式
result = ((set_a | set_b) & set_c) - set_d  # 括号嵌套，可读性差


# 方法版本 - 支持多个集合
union_all = set_a.union(set_b, set_c)      # {1, 2, 3, 4}
# 运算符版本 - 只能两个集合
union_ab = set_a | set_b | set_c           # 需要多次操作
```

第3章 详解Python序列结构

- 3.4集合：元素之间不允许重复
  - 3.4.1集合对象的创建
  - 3.4.2集合操作与运算
  - **3.4.3集合应用案例**
- 3.5序列解包的多种形式和用法

3.4.3 集合应用案例

- 可以使用集合快速提取序列中单一元素，即提取出序列中所有不重复元素。如果使用传统方式的话，需要编写下面的代码：

```
>>> import random
#生成100个介于0到9999之间的随机数
>>> listRandom = [random.choice(range(10000)) for i in range(100)]
          #or listRandom = random.choices(range(10000), k=100)
          #or listRandom = [random.randint(0, 9999) for _ in range(100)]
>>> noRepeat = []
>>> for i in listRandom :
        if i not in noRepeat :
           noRepeat.append(i)
```

- 使用集合只需要下面这么一行代码

```
>>> newSet = set(listRandom)
```

3.4.3 集合应用案例

- **问题解决：**返回指定范围内一定数量的不重复数字。

```
import random

def randomNumbers(number, start, end):
    '''使用集合来生成number个介于start和end之间的不重复随机整数'''
    data = set()
    while len(data)<number: #当len(data)=number-1时，add最后一个数
        element = random.randint(start, end) #可能会产生重复数字
        data.add(element)    #自动去重
    return data
```

3.4.3 集合应用案例

```
import random  #更高效

def randomNumbers(number, start, end):
    '''使用集合来生成number个介于start和end之间的不重复随机整数'''
    data = set()
    all_data = set(range(start, end+1)) #所有可能数字
    for _ in range(number):
        if all_data-data:
            element = random.choice(tuple(all_data-data)) #choice(seq)
                #直接从剩余数字中选择，避免重复尝试已选中的数字，保证100%效率
            data.add(element)
    return data
```

```
import itertools
lst = list(itertools.combinations(range(10),3))  #给出所有可能的选择
                                                 #可用import random；random.choice(lst)随机选一个
for i,j,k in lst:                                #或用import random；random.choices(lst,k=N)随机选N个
    print(i,j,k)
```

3.4.3 集合应用案例

- **问题解决：**下面两段代码用来测试指定列表中是否包含非法（不规范）数据，很明显第二段使用集合的代码更高效一些。

```
import random

lstColor = ('red', 'green', 'blue')
colors = [random.choice(lstColor) for i in range(10000)]
colors[9000]='white' #给一个非法数据来测试

#测试1
for item in colors:                                    #遍历列表中的元素并逐个判断
    if item not in lstColor:
        print('error:', item)
        break
#测试2
if (set(colors)-set(lstColor)):#set(colors)之后自动去重，应该和set(lstCorlor)相同
    print('error')                          #即两个集合差集应为空，如不为空，则存在不合法数据
```

3. 4. 3　集合应用案例

- **问题解决：**假设已有若干用户名字及其喜欢的电影清单，现有某用户，已看过并喜欢一些电影，现在想找个新电影看看，又不知道看什么好。
- 思路：根据已有数据，查找与该用户爱好最相似的用户，也就是看过并喜欢的电影与该用户最接近，然后从那个用户喜欢的电影中选取一个当前用户还没看过的电影，进行推荐。

- 演示：豆瓣　#https://movie.douban.com/
- **数据才是关键！**　#大数据时代，搜集数据比处理数据更重要

3.4.3 集合应用案例

```
from random import randrange

# 其他用户喜欢看的电影清单
data = {'user'+str(i):{'film'+str(randrange(1, 10))  #字典推导式
                       for j in range(randrange(15))} #集合存电影清单，不重复
        for i in range(10)}    #10个用户,10部电影
                    #15 是一个上限值，控制每个用户观看电影数量的随机范围，提高命中率
print('历史数据: ')
for u, f in data.items():
    print(u, f, sep=':')
```

3.4.3 集合应用案例

```
# 待测用户曾经看过并感觉不错的电影
user = {'film1', 'film2', 'film3'}

#待测用户与其他用户喜欢看的电影的交集的大小
[ (k, len(value & user)) for k, value in data.items() ] #列表推导式 #此句可不要

# 查找与待测用户最相似的用户和Ta喜欢看的电影
similarUser, films = max(data.items(), key=lambda item: len(item[1]&user))
                                                    #item[1]是电影集合，即values
print('和您最相似的用户是：', similarUser)
print('Ta最喜欢看的电影是：', films)
print('Ta看过的电影中您还没看过的有：', films-user)
```

Practice

- 如何创建空集合？
✓ set()

- 使用哪两个方法为集合增加元素？
✓ add(), update()

- 使用哪两个方法删除集合中的特定元素，这两个方法有什么区别？
✓ remove(), discard()
✓ discard()方法当元素不存在时不会抛出异常，remove()反之

- 集合的^运算符做什么运算？ {1,2,3,4} ^ {2,3,5}的值是？
✓ 对称差集
✓ {1,4,5}

习题

3.13 表达式 {1, 2, 3} < {3, 4, 5} 的值为_____。

3.16 单选题：表达式 set().union([1,2,3], (4,5), {6,7}) 的值为（ ）。

A. {1, 2, 3, 4, 5, 6, 7}　　B. {1, 2, 3, 4, 5}

C. {4, 5, 6, 7}　　D. {4, 5}

3.17 单选题：已知 x = {1: 'a', 2: 'b', 3: 'c'} 和 y = {1, 3, 4}，那么表达式 x.keys() - y 的值为（ ）。

A. {2}　　B. {3}

C. {1, 3}　　D. 表达式错误，无法计算

3.18 单选题：已知 x = {1: 3, 2: 1, 3: 1} 和 y = {1, 3, 4}，那么表达式 x.values() - y 的值为（ ）。

A. {2}　　B. {3}

C. {1, 3}　　D. set()

习题

3.26　多选题：下面可以使用表示位置或序号的整数做下标访问其中元素的有（　　）。

A．列表　　B．元组　　C．字典　　D．集合

E．字符串

3.27　多选题：下面可以支持下标运算的有（　　）。

A．列表　　B．元组　　C．字典　　D．集合

E．字符串

总结：Python序列结构

1. 基本特性对比

特性	List（列表）	Tuple（元组）	Dict（字典）	Set（集合）
可变性	✅ 可变	❌ **不可变**	✅ 可变	✅ 可变
有序性	✅ 有序	✅ 有序	❌ **无序**	❌ **无序**
语法	[]	()	{key: value}	{} 或 set()
元素要求	任意类型	任意类型	**键：唯一且不可变**	**唯一且不可变**
空对象创建	list() 或 []	tuple() 或 ()	dict() 或 {}	set()

总结：Python序列结构

2. 创建方法

操作	List	Tuple	Dict	Set
空对象	lst = [] 或 list()	tup = () 或 tuple()	d = **{}** 或 dict()	s = set(), **无{}**
带值创建	[1, 2, 'a']	(1, 2, 'a')	{'a': 1, 'b': 2}	{1, 2, 3}
单元素	[1]	**(1,)（必须加逗号）**	{'key': value}	{element}
从可迭代对象	list('abc')	tuple([1, 2, 3])	dict([('a',1)])	set([1,2,2,3])

总结：Python序列结构

3. 增加/修改操作

操作	List	Tuple	Dict	Set
添加单个	append(element)	❌ 不支持	**d[key] = value**	add(element)
添加多个	extend(iterable)	❌ 不支持	update(dict)	update(iterable)
插入指定位置	insert(index, element)	❌ 不支持	❌ 不支持	❌ 不支持
修改元素	lst[index] = new_value	❌ 不支持	d[key] = new_value	❌ 不支持

总结：Python序列结构

4. 删除元素

集合专注于成员存在与否的测试，提供安全discard()和严格remove()两种删除方式

操作	List	Tuple	Dict	Set
按**值**删除	remove(element)	❌ 不支持	❌ **不支持**	remove(element) 或 **discard**(element)
按**位置/键**删除	pop(index)	❌ 不支持	pop(key)	❌ 不支持
删除末尾/随机	pop()（末尾）	❌ 不支持	popitem()	**pop()（随机）**
清空所有	clear()	❌ **不支持**	clear()	clear()
del语句	del lst[index]	❌ **不支持删除元素** 只可删除标签del t	del d[key]	❌ **不支持删除元素** 只可删除标签del s

总结：Python序列结构

5. 访问与查询

d[key]访问：用于“必须存在”场景，强调数据的完整性
get()访问：用于“可选存在”场景，提供安全访问
setdefault()访问：用于“初始化或获取”场景，简化代码

操作	List	Tuple	Dict	Set
按**索引/键**	lst[index]	tup[index]	d[key] 或 **d.get(key)** 或 **d.setdefault(key)**	❌ **不支持**
切片操作	lst[start:end]	tup[start:end]	❌ 不支持	❌ 不支持
检查存在	element in lst	element in tup	key in d	element in s
长度	len(lst)	len(tup)	len(d)	len(s)

总结：Python序列结构

6. 复制

方法	List	Tuple	Dict	Set
赋值	new = original	new = original **new = original[:]** **new = tuple(original)**	new = original	new = original
浅拷贝	new = original.copy() new = original[:] new = list(original)		new = original.copy() new = dict(original)	new = original.copy() new = set(original)

操作类型	描述	内存关系	修改影响
赋值	创建对象的**引用**（别名）	共享同一内存地址	**完全同步**，修改任一都会影响另一个
浅拷贝	创建新对象，复制第一层元素	不同内存地址，**但嵌套的可变对象共享**	修改第一层元素互不影响，**修改嵌套的可变对象会相互影响**

总结：Python序列结构

7. 其他操作与方法

操作	List	Tuple	Dict	Set
排序	**sort()**	❌ 不支持	❌ 不支持	❌ 不支持
反转	**reverse()**	❌ 不支持	❌ 不支持	❌ 不支持
计数	count(element)	count(element)	❌ 不支持	❌ 不支持
索引查找	index(element)	index(element)	❌ 不支持	❌ 不支持
集合运算	❌ 不支持	❌ 不支持	❌ 不支持	**并集(|), 交集(&), 差集(-)等**
获取所有键	❌ 不支持	❌ 不支持	**keys()**	❌ 不支持
获取所有值	❌ 不支持	❌ 不支持	**values()**	❌ 不支持

总结：Python序列结构

8. 核心用途

- List：**有序**序列，频繁增删改
- Tuple：**不可变**数据，安全可靠
- Dict：键值映射，快速**查找**
- Set：**去重**存储，集合运算

第3章 详解Python序列结构

- 3.4集合：元素之间不允许重复
  - 3.4.1集合对象的创建
  - 3.4.2集合操作与运算
  - 3.4.3集合应用案例
- **3.5序列解包的多种形式和用法**



3.5 序列解包的多种形式和用法

序列解包（Sequence Unpacking）是指将一个序列（如列表、元组、字符串等）或可迭代对象的元素分解并赋值给多个变量的操作。

1. 列表（List）

```
>>> my_list = [1, 2, 3]
>>> a, b, c = my_list
>>> a, b, c      # 输出: (1, 2, 3)
>>> a            # 输出: 1
>>> b            # 输出: 2
>>> c            # 输出: 3
```

2. 元组（Tuple）

```
>>> my_tuple = (4, 5, 6)
>>> x, y, z = my_tuple
>>> x, y, z  # 输出: (4, 5, 6)
```

3. 5　序列解包的多种形式和用法

3. 字符串（String）

```
>>> my_str = "abc"
>>> x, y, z = my_str
>>> x, y, z  # 输出: ('a', 'b', 'c')
```

4. 字典（Dictionary）

```
>>> my_dict = {'a': 1, 'b': 2, 'c': 3}
>>> k1, k2, k3 = my_dict  # 默认解包键
>>> k1, k2, k3   # 输出:  ('a', 'b', 'c')
>>>for key, value in my_dict.items():
        print(key, value)  # 输出: a 1
                                  b 2
                                  c 3  #解包键值对
```

3. 5　序列解包的多种形式和用法

5. 集合（Set）

注意：集合是无序的，解包顺序不确定。#不建议用

```
>>> my_set = {7, 8, 9}
>>> a, b, c = my_set
>>> a, b, c                    # 顺序可能随机，如(8, 9, 7)
```

6. 生成器（Generator）

```
>>> gen = (x**2 for x in range(3))
>>> u, v, w = gen
>>> u, v, w   # 输出: (0, 1, 4)
```

3. 5　序列解包的多种形式和用法

7. range对象

```
>>> r = range(3)
>>> i, j, k = r
>>> i, j, k  # 输出: (0, 1, 2)
```

8. 嵌套序列

```
>>> data = [1, (2, 3), 4]
>>> a, (b, c), d = data
>>> a, b, c, d  # 输出: (1, 2, 3, 4)
```

3.5 序列解包的多种形式和用法

9. 使用星号（*）处理不定长元素

```
>>>numbers = [1, 2, 3, 4, 5]
>>>first, *middle, last = numbers
>>>first      # 输出: 1
>>>middle  # 输出: [2, 3, 4]
>>>last       # 输出: 5
```

10. map对象

```
>>>  x, y, z = map(str, range(3))
>>> x, y, z   #输出: ('0', '1', '2')
```

11. sorted对象

```
>>>  x, y, z = sorted([1, 3, 2])
>>> x, y, z   #输出: (1, 2, 3)
```

3.5 序列解包的多种形式和用法

12. 以两个循环变量遍历序列中的元组的形式也称为序列解包

```
>>> keys = ['a', 'b', 'c', 'd']
>>> values = [1, 2, 3, 4]
>>> for k, v in zip(keys, values):
        print(k, v)

#输出:
a 1
b 2
c 3
d 4

>>> for i, v in enumerate(keys):
        print(i, v)

#输出:
0 a
1 b
2 c
3 d

>>> s = {'a':1, 'b':2, 'c':3}
>>> for k, v in s.items():
    print(k, v)

#输出:
a 1
b 2
c 3
```

3.5 序列解包的多种形式和用法

注意事项： 元素数量匹配

```
>>> x, y = [1, 2, 3]  # ValueError: too many values to unpack
```

总结：

- ✓ 任何可迭代对象都支持序列解包，包括但不限于列表、元组、字符串、字典、集合、生成器等。
- ✓ 解包时需注意结构匹配和元素数量的一致性。

3.5 序列解包的多种形式和用法

重要应用：实现两个变量的交换

```
>>> x = [1,2]
>>> y = [3,4,5]
>>> x, y = y, x      #交换两个变量的值
                     #本质 (x, y) = (y, x) 或 (x, y) = [y, x]
>>> x
[3,4,5]
>>> y
[1,2]
```

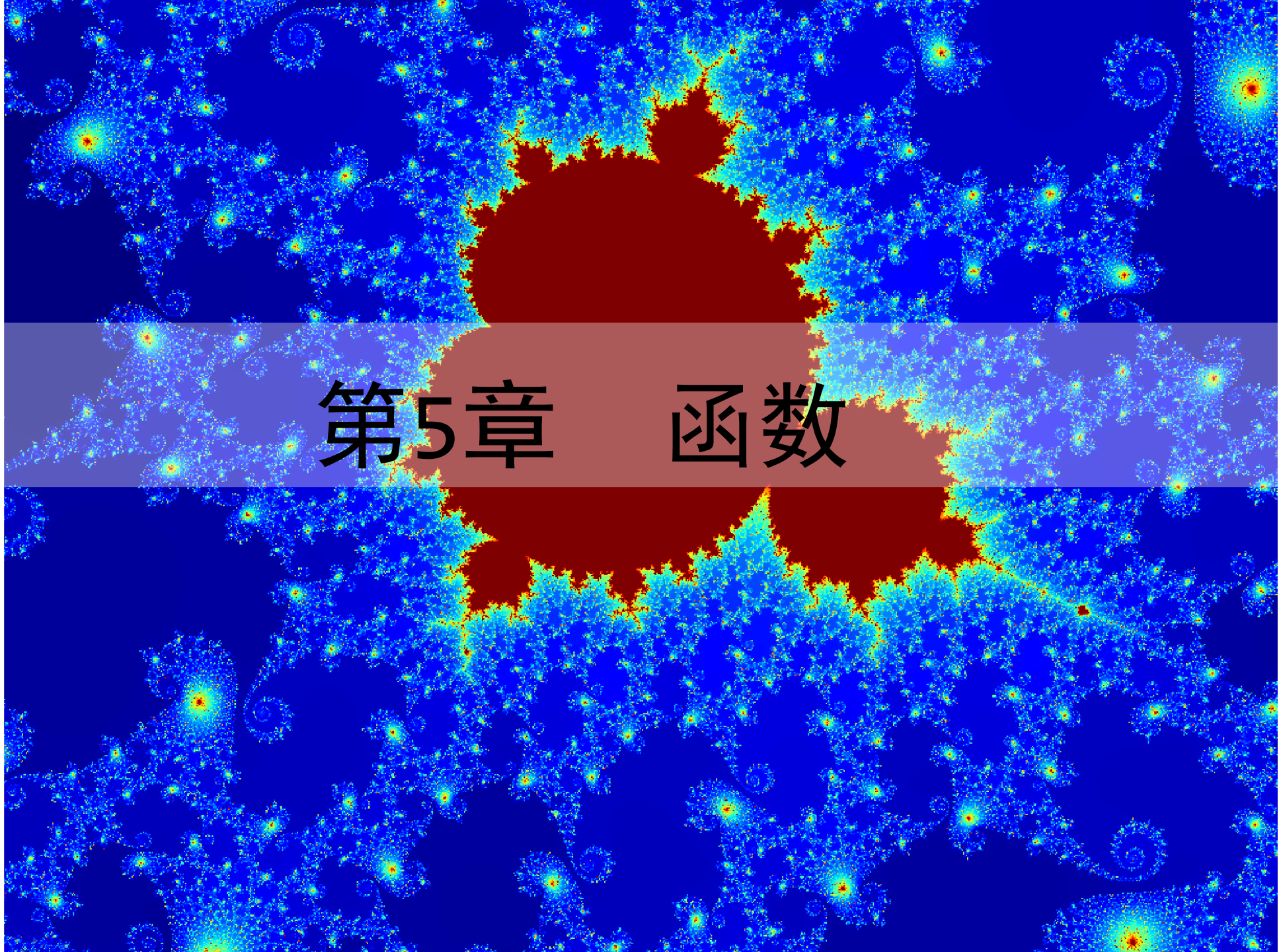
作业4.3.3

1.创建一个字典d，键为26个小写英文字母，值为区间[0,10]中的随机整数

2.print字典d的值的最大值、最小值、平均值

3.使用列表推导式生成在[6,9]区间内的值所对应的键组成的列表

4.设a和b是两个列表，利用集合运算写表达式判断(1)a的所有元素都属于b，（2）a的所有元素都不属于b，（3）a有至少一个元素属于b



第5章　函数

第5章 python中函数与c语言的区别：

特性	C 语言	Python
类型声明	必需	可选（动态类型）
参数传递	值传递、指针传递	对象引用传递
函数作为对象	通过函数指针	函数是对象，可为参数或返回值
默认参数	不支持	支持
可变参数	通过 va_list	支持 *args, **kwargs
作用域	词法作用域	词法作用域 + 闭包(函数嵌套)
高级特性	有限	修饰器、生成器、lambda等

第5章 代码复用技术(一)：函数

- 5.1 函数定义与使用
- 5.2 函数参数
- 5.3 变量作用域
- 5.4 lambda表达式
- 5.5 生成器函数设计要点
- 5.6 精彩案例赏析

第5章　函数

- 将可能需要反复执行的代码封装为函数，并在需要该功能的地方进行调用
- 使用函数可以实现代码复用，更重要的是可以保证代码的一致性，只需要修改该函数代码则所有调用均受到影响
- 设计函数时，应注意提高模块的内聚性，同时降低模块之间的隐式耦合
  - 只让一个函数完成一个高度相关且大小合适的功能
  - 减少全局变量的使用，使得函数之间仅通过调用和参数传递来显式体现其关系
  - Python可以定义嵌套函数，它通常会使用函数体外的变量

第5章 代码复用技术(一)： 函数

- 5.1 函数定义与使用
  - 5.1.1基本语法
  - 5.1.2函数嵌套定义、可调用对象与修饰器
  - 5.1.3函数递归调用
- 5.2 函数参数
- 5.3 变量作用域
- 5.4 lambda表达式
- 5.5 生成器函数设计要点
- 5.6 精彩案例赏析

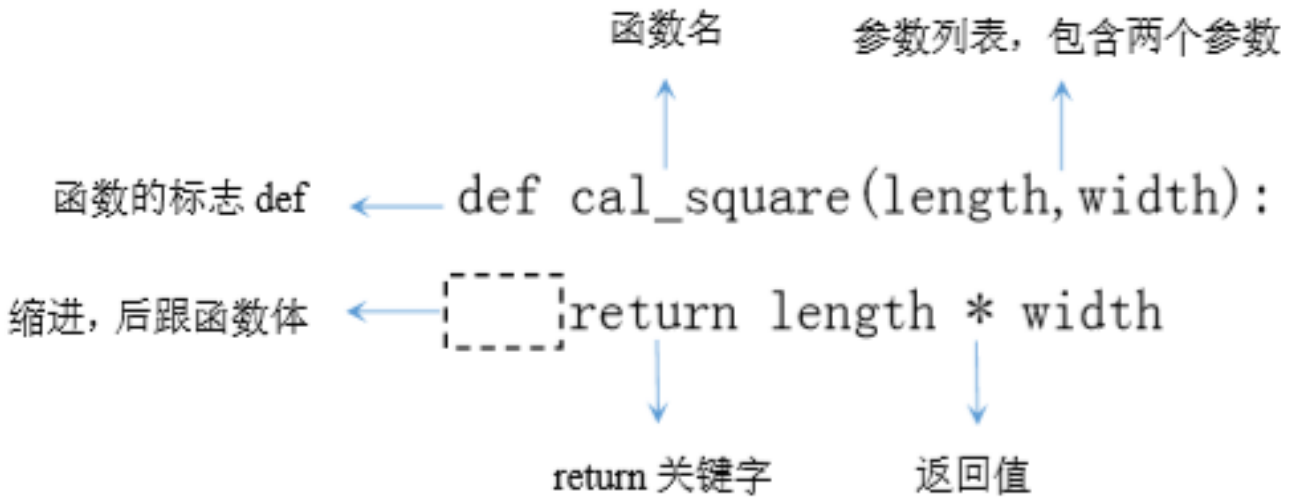
5.1.1 函数定义与调用基本语法

在定义函数时的参数列表里面的参数是形式参数，简称为“**形参**”，指length和width；当调用函数时，要传给函数内部的参数是实际参数，简称为“**实参**”，指cal_square(4,3)中的4和3。

定义计算矩形面积的函数

```
def cal_square(length,width):
    return length * width
result = cal_square(4,3)
print(result)
```

12



5.1.1 函数定义与调用基本语法

❖ 函数定义语法：使用def语句定义函数

```
def 函数名([参数(形参)列表]):
    '''注释'''
    函数体
    return 返回值列表
```

```
def cal_square(length,width):
    return length * width
result = cal_square(4,3)
print(result)
```

❖ 注意事项

✓ 函数形参不需要声明类型，也不需要指定函数返回值类型；

✓ **参数列表**可以有零个、一个或多个，没有参数时圆括号也必须保留；

✓ 括号后面的冒号必不可少；

✓ 函数体相对于def关键字必须保持一定的空格缩进；

✓ **return语句**可以省略，当需要返回值时，return是函数结束的标志，会将返回值列表返回给调用者。

5.1.1 函数定义与调用基本语法

- **问题解决：**编写打印斐波那契数列的函数并调用。

n是形参

函数体

函数名

```
def fib(n):
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()

fib(1000)
```

- $F_0 = 0$
- $F_1 = 1$
- $F_n = F_{n-1} + F_{n-2}$ （n≧2）

b a

#函数调用的具体执行过程？

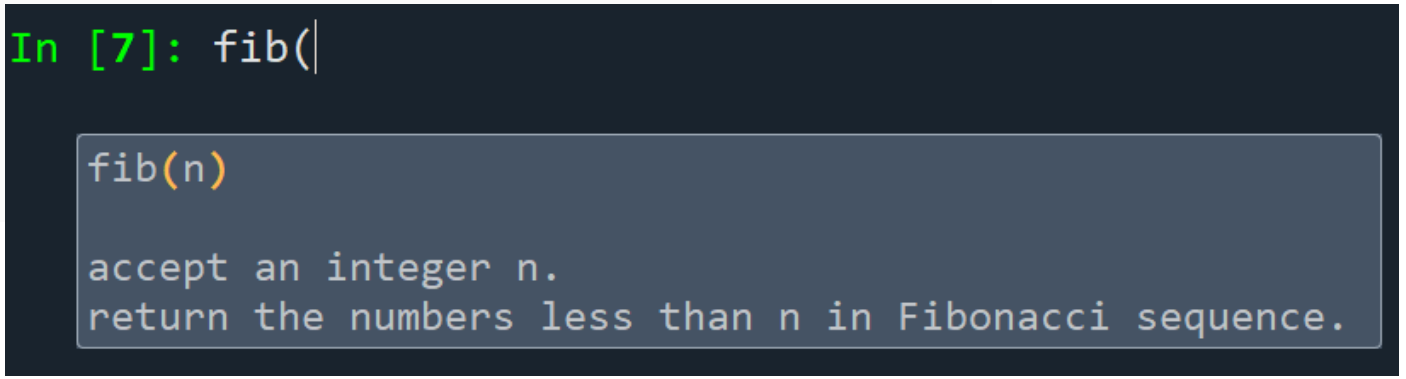
```
>>> type(fib)
<class 'function'>
```

	0	1	1	2	3
循环外	a	b			
第一次循环		a	b		
第二次循环			a	b	

5.1.1 函数定义与调用基本语法

■在定义函数时，开头部分的注释并不是必需的，但如果为函数的定义加上注释的话(可用中文)，可以为用户提供友好的提示。

```
def fib(n):
    ''' accept an integer n.
        return the numbers less than n in Fibonacci sequence.
    '''
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()
```



或用help(fib).__doc__

5.1.1 函数定义与调用基本语法

- Python语言使用return语句返回函数的值
- 定义函数时也不需要声明函数的返回值类型，函数返回值类型与return语句返回表达式的类型一致，同一个函数的返回值可能是任意类型
- **不论return语句出现在函数的什么位置，一旦得到执行将直接退出函数**
- 如果函数没有return语句、有return语句但是没有执行到或者执行了不返回任何值的return语句，解释器都会认为该函数以return None结束，即返回空值

5.1.1 函数定义与调用基本语法

```
1. def borrow_books(name,book,day):
2.     print(f"{name}借阅了书籍《{book}》,预计借阅{day}天")
3. borrow_books("张三","红楼梦",14)
4. result = borrow_books("李四","西游记",10)  #把return的结果赋值给result
5. print(result)
```



```
张三借阅了书籍《红楼梦》,预计借阅14天
李四借阅了书籍《西游记》,预计借阅10天
None
```

5.1.1 函数定义与调用基本语法

■ 例：

```
def max_fun(seq):
    '''这是一个实现最大值查找的函数'''
    v_max = None #初始化最大值为none
    for v in seq:
        if v_max is None or v > v_max:
        #如果是第一个元素或找到更大的值
            v_max = v #更新最大值
    return v_max

a = 'THE world puts off its mask of vastness to its lover.'.split()
print(max_fun(a)) #返回一个字符串

a = (1,2,3,2,1)
print(max_fun(a)) #返回一个数字

print(max_fun(i**3%5 for i in range(10))) #参数为一个生成器

a = []
print(max_fun(a)) #返回None
```

Practice

- Python函数在什么情况下返回None？

✓ 函数没有return语句
✓ 有return语句但是没有执行到
✓ 执行了不返回任何值的return语句

- 函数的形参和返回值的类型由什么决定？

✓ 实参的类型决定形参的类型
✓ 返回值的类型由return语句中表达式的值决定

- 写一个名为len_fun的函数计算一个序列的长度，参数为任意类型的序列？

```
def len_fun(seq):
    n=0
    for v in seq:
        n += 1
    return n
```

第5章 代码复用技术(一)： 函数

- 5.1 函数定义与使用
  - 5.1.1基本语法
  - 5.1.2函数嵌套定义、可调用对象与修饰器
  - 5.1.3函数递归调用
- 5.2 函数参数
- 5.3 变量作用域
- 5.4 lambda表达式
- 5.5 生成器函数设计要点
- 5.6 精彩案例赏析



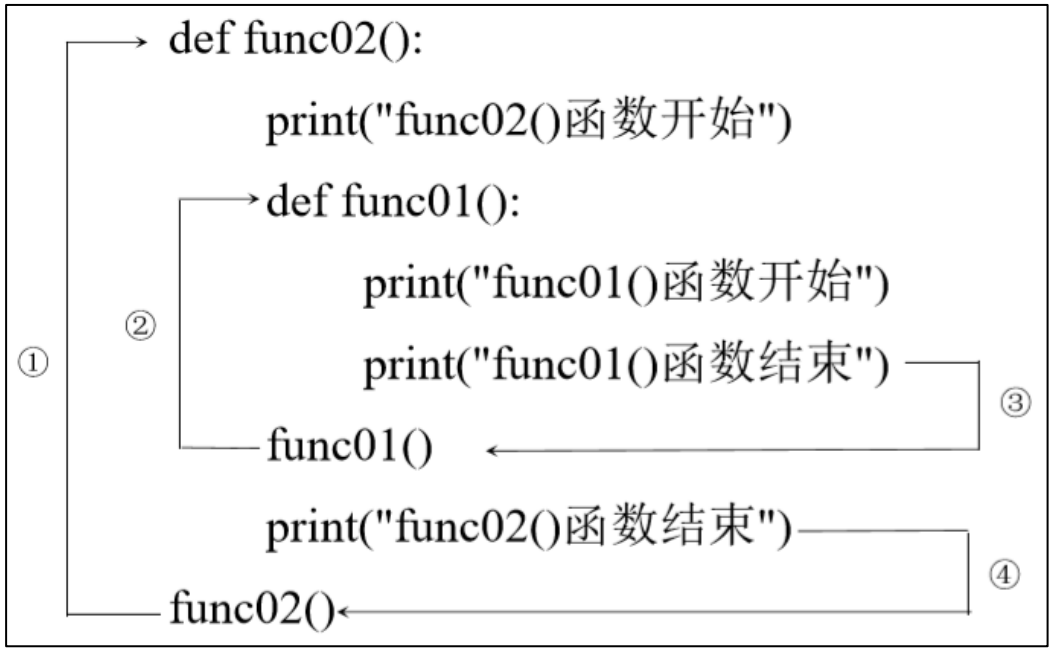
5.1.2 函数嵌套定义、可调用对象与修饰器

(1) 函数嵌套定义

- Python允许函数的嵌套定义，在函数内部可以再定义另外一个函数
- Python函数可以使用函数体外的变量，但要保证其内部不存在同名变量（将在5.3变量作用域中详细讲解）

```
1. def func02():
2.     print("func02()函数开始")
3.     def func01():
4.         print("func01()函数开始")
5.         print("func01()函数结束")
6.     func01()  #使用在函数内部定义的函数
7.     print("func02()函数结束")
8. func02()
```

```
func02()函数开始
func01()函数开始
func01()函数结束
func02()函数结束
```



5.1.2 函数嵌套定义、可调用对象与修饰器

repr(obj)

Return the canonical string representation of the object.

For many object types, including most builtins, eval(repr(obj)) == obj.

```
>>> repr('ab')
"'ab'"
>>> repr([1,2,3])
'[1, 2, 3]'
```

```
>>> eval(repr('ab'))
'ab'
>>> eval(repr([1,2,3]))
[1, 2, 3]
```

```
>>> eval(str('ab'))
NameError: name 'ab' is not defined
>>> eval(str([1,2,3]))
[1, 2, 3]
```

5.1.2 函数嵌套定义、可调用对象与修饰器

```
def myMap(iterable, op, value):     #自定义函数
    if op not in '+-*/':
        return 'Error operator'
    def nested(item):                    #嵌套定义函数
        return eval(repr(item) + op + repr(value))
    return map(nested, iterable)        #使用在函数内部定义的函数
    #return (nested(v) for v in iterable)
```

```
def myMap(iterable, op, value):
#自定义函数
    if op not in '+-*/':
        return 'Error operator'
    return [eval(repr(item) + op + repr(value)) for item in iterable]
```

```
>>> list(myMap(range(5), '+', 5))       #调用外部函数，不需要关心其内部实现
[5, 6, 7, 8, 9]
>>> list(myMap(range(5), '-', 5))
[-5, -4, -3, -2, -1]
>>> list(myMap(['hello', 'world'], '+', ' good'))
['hello good', 'world good']
>>> myMap(range(5), '%', 2)
'Error operator'
```

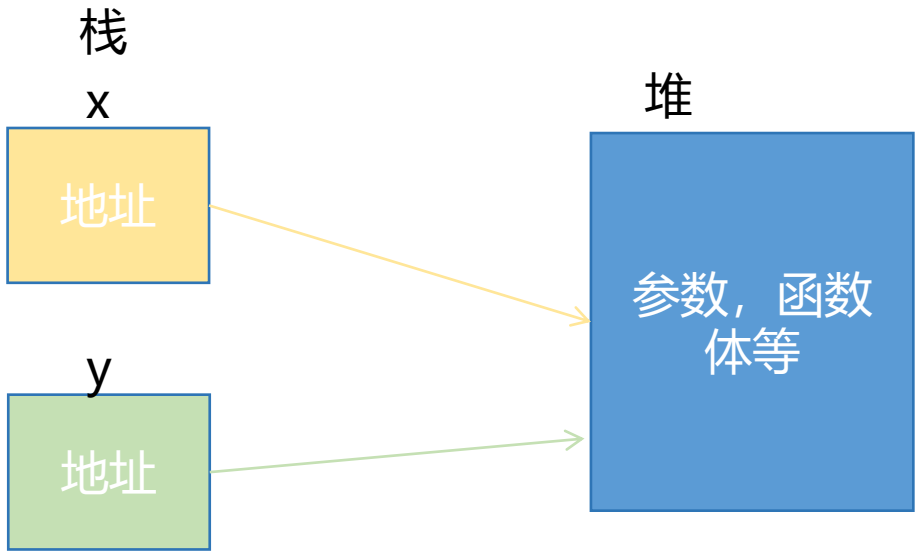
5.1.2 函数嵌套定义、可调用对象与修饰器

（2）可调用对象：

函数是一个对象，可作为参数和返回值。

```
def x(a, b):
    result = a + b
    return result

y = x
print(y(1,2))
print(id(x))
print(id(y))
print(type(x))
print(type(y))
```



5.1.2 函数嵌套定义、可调用对象与修饰器

可调用对象是Python中一个强大的概念，它让对象可以**像函数一样被调用**。

■可通过函数的嵌套定义实现可调用对象的定义

```
def linear(a, b):
    def result(x):                    #在Python中，函数是可以嵌套定义的
        return a * x + b
    return result                     #返回可被调用的函数
                                      #return返回值可以是一个函数
```

5.1.2 函数嵌套定义、可调用对象与修饰器

■使用上面的嵌套函数可以创建一个可调用对象：

taxes = linear(0.3, 2)

■然后就可以把taxes当做函数来调用：

```
taxes(5)
```

■ 也可以直接调用：

```
linear(0.3, 2)(5)
```

■使用callable()函数可以检查对象是否可调用：

```
>>> callable(taxes)  #True
>>> callable(linear)  #True
>>> callable(result)  #NameError: name 'result' is not defined
>>> taxes.__name__   #'result' #获取本来的名字
```

5.1.2 函数嵌套定义、可调用对象与修饰器

- **函数名**本质上是一个对象名，它指向了一个函数，**可以作为参数**传递给另外一个函数

```
def g():
    print("Hi, it's me 'g'")
    print("Thanks for calling me")

def f(func):
    print("Hi, it's me 'f'")
    print("I will call 'func' now")
    func()
    print("func's real name is " + func.__name__)


f(g)
```

- 如何判断一个变量表示一个函数？

```
>>> callable(g)
True
```

- 设变量fun代表一个函数，如何获取它本来的名字?

```
>>> fun = g
>>> fun.__name__
'g'
```

5.1.2 函数嵌套定义、可调用对象与修饰器

（3）修饰器

- 修饰器（decorator）是函数嵌套定义的另一个重要应用。修饰器本质上也是一个函数，只不过这个函数接收其他函数作为参数并对其进行一定的改造之后使用新函数替换原来的函数

- Decorators belong most probably to the most beautiful and most powerful design possibilities in Python, but at the same time the concept is considered by many as complicated to get into.

5.1.2 函数嵌套定义、可调用对象与修饰器

- 在没有修饰器之前，如果多个函数需要相同的附加功能（如日志、计时、权限检查），代码会变得重复：

```
# 没有修饰器的写法 - 代码重复

def function1():
    start_time = time.time()  # 重复的计时代码
    print("开始执行 function1")
    # 核心逻辑
    result = "function1 result"
    end_time = time.time()  # 重复的计时代码
    print(f"function1 执行时间: {end_time - start_time}")
    return result
```

```
def function2():
    start_time = time.time()  # 重复的计时代码
    print("开始执行 function2")
    # 核心逻辑
    result = "function2 result"
    end_time = time.time()  # 重复的计时代码
    print(f"function2 执行时间: {end_time - start_time}")
    return result
```

5.1.2 函数嵌套定义、可调用对象与修饰器

使用修饰器解决方案：

```
import time
def timer_decorator(func):   # 参数 func: 被装饰的原始函数
    def wrapper(*args, **kwargs): #wrapper是内部执行函数，它将替换原始函数
        start_time = time.time()      #*args, **kwargs 确保包装器可以处理任何参数组合
        print(f"开始执行 {func.__name__}") #func.__name__ 获取原始函数的名称
        result = func(*args, **kwargs)  #核心调用：将接收到的所有参数原样传递给原始函数
        end_time = time.time()
        print(f"{func.__name__} 执行时间: {end_time - start_time}")
        return result  #返回原始函数的执行结果
    return wrapper  #返回内部函数 wrapper，它将替换原始函数
```

- 修饰器本质上是一个函数(timer_decorator(func))，它接受一个函数（func）作为参数，并返回一个包装过新函数(wrapper())。

5.1.2 函数嵌套定义、可调用对象与修饰器

```
@timer_decorator  #(语法糖)
def function1():
    time.sleep(1)
    return "function1 over!"

@timer_decorator
def function2():
    time.sleep(5)
    return "function2 over!"

print(function1())

print(function2())
```

```
等价于:
def function1():
     time.sleep(1)
    return "function1 over!"
function1 = timer_decorator(function1)
```

执行过程分解:

1.定义原始函数：function1 被定义

2.应用装饰器：timer_decorator(function1) 被调用

3.返回包装器：timer_decorator 返回 wrapper 函数

4.重新赋值：function1 现在指向 wrapper 函数

5.1.2 函数嵌套定义、可调用对象与修饰器

■例：修饰第三方库函数

```
from random import random, randint, choice

def our_decorator(func):
    def function_wrapper(*args, **kwargs):
        print("Before calling " + func.__name__)
        res = func(*args, **kwargs)
        print(res)
        print("After calling " + func.__name__)
        return res
    return function_wrapper

random = our_decorator(random)  #手动应用装饰器，而不是使用 @ 语法糖
randint = our_decorator(randint)
choice = our_decorator(choice)

random()
randint(3, 8)
choice([4, 5, 6])
```

5.1.2 函数嵌套定义、可调用对象与修饰器

■一个应用：检查参数类型

```
def argument_test_natural_number(f):
    def helper(x):
        if type(x) == int and x > 0:
            return f(x)
        else:
            raise Exception("Argument is not an integer")
    return helper

@argument_test_natural_number
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)    #递归调用

print(factorial(-1))
```

Practice

- 我们学过的那些内置或标准库函数可以以函数名为参数？

✓ max(),min()和sorted()的key参数、map()的第一个参数、re.sub的repl参数

- 什么是修饰器函数？

✓ 接收其他函数作为参数，并对其进行一定的改造之后，以修饰后的新函数为返回值的函数

- 设jit()是一个修饰器函数，如何用它来修饰自定义的函数？

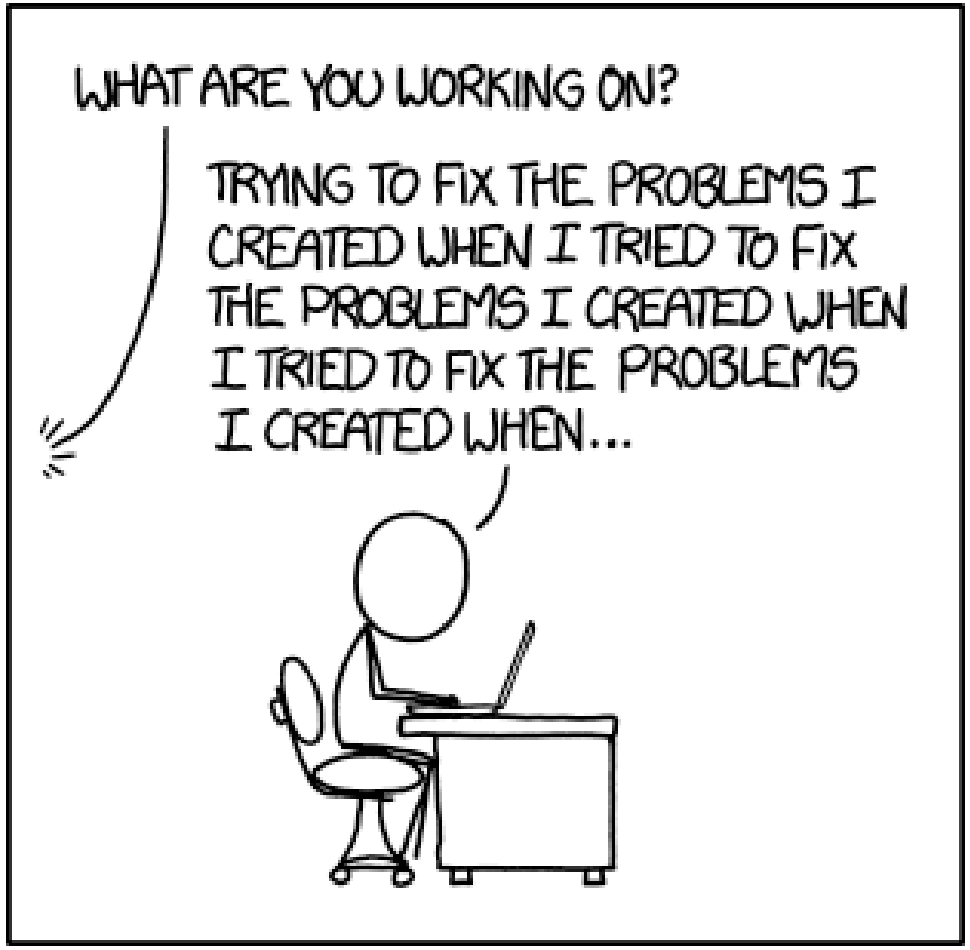
✓ 在函数定义前面添加语句@jit

第5章 代码复用技术(一)： 函数

- 5.1 函数定义与使用
  - 5.1.1基本语法
  - 5.1.2函数嵌套定义
  - **5.1.3函数递归调用**
- 5.2 函数参数
- 5.3 变量作用域
- 5.4 lambda表达式
- 5.5 生成器函数设计要点
- 5.6 精彩案例赏析

5.1.3 函数递归调用

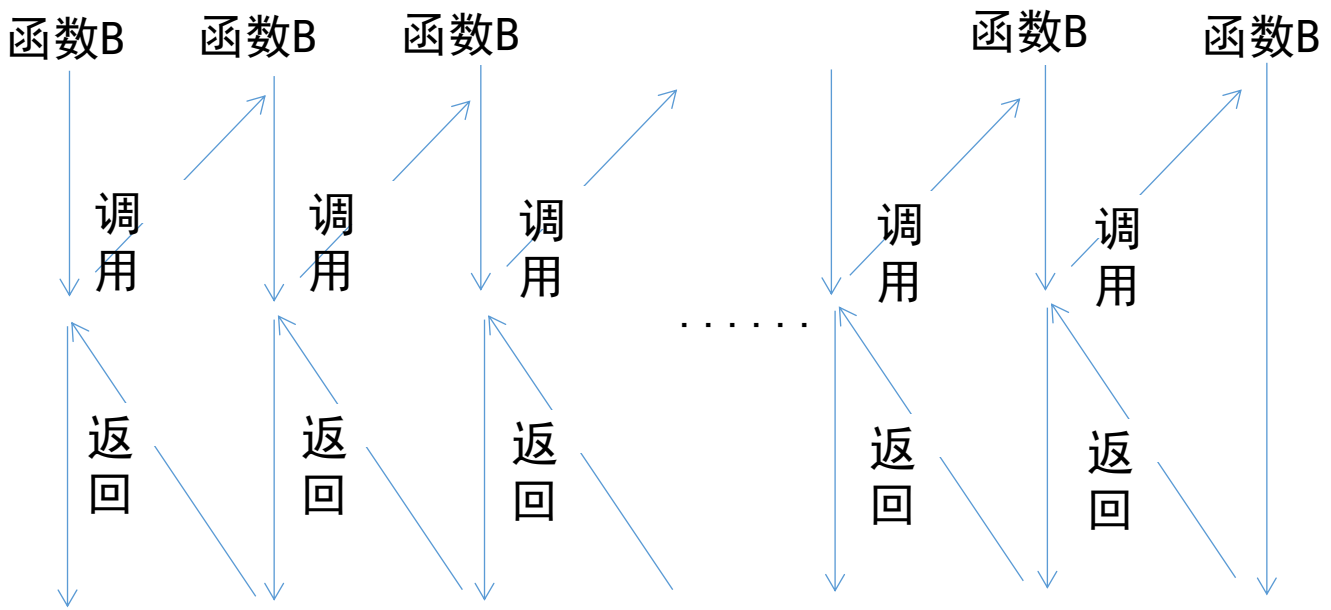
- 调用了自己的函数称为递归函数

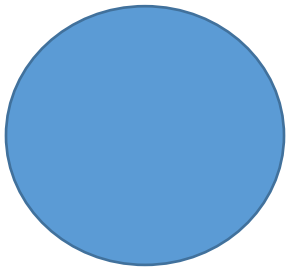




5.1.3 函数递归调用

- 函数的递归调用过程：函数调用自己，自己再调用自己，自己再调用自己，. . . ，当某个条件得到满足的时候就不再调用了，然后再一层一层地返回直到该函数第一次调用的位置





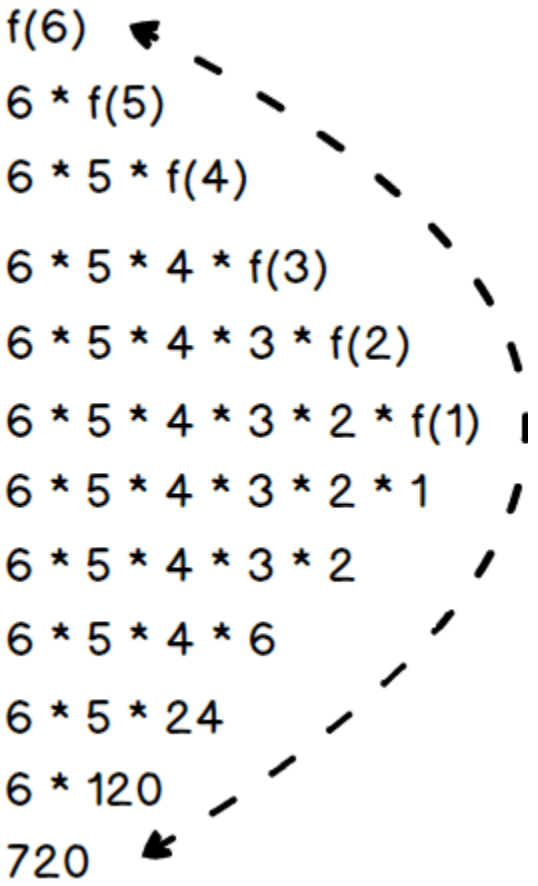
5.1.3 函数递归调用

- **问题解决：**使用递归法计算阶乘

`n! = n * (n-1)!, if n > 1 and 0! = 1`

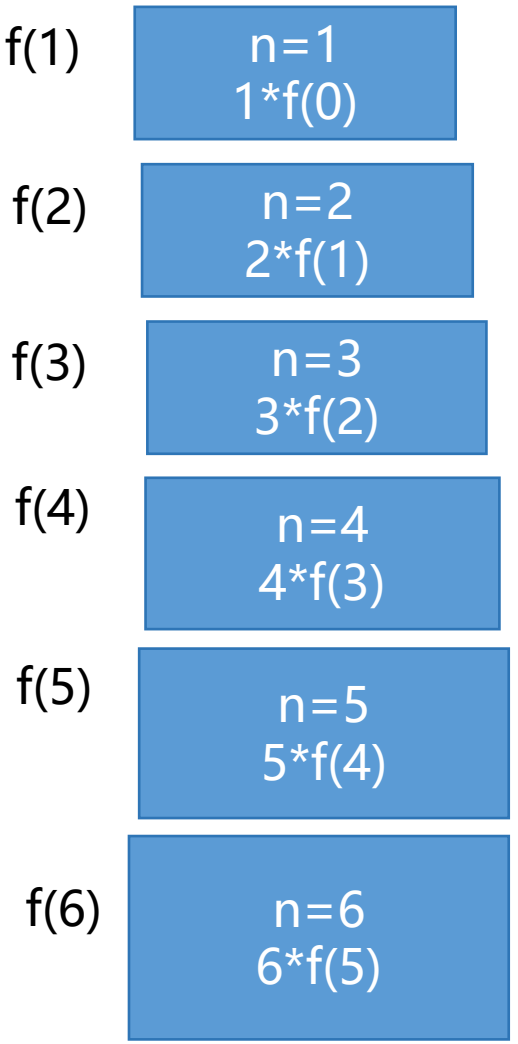
```
def factorial(n):
    print("factorial has been called with n = " + str(n))
    if n == 0:
        return 1
    else:
        res = n * factorial(n-1)
        print("intermediate result for ", n, " * factorial(" ,n-1, "): ",res)
        return res

print(factorial(6))
```



5.1.3 函数递归调用

- 内存执行：



n==0, return 1: 该调用板块执行完毕

return 2*1: 该调用板块执行完毕

return 3*2*1: 该调用板块执行完毕

return 4*3*2*1: 该调用板块执行完毕

return 5*4*3*2*1: 该调用板块执行完毕

return 6*5*4*3*2*1: 该调用板块执行完毕

```
factorial has been called with n = 6
factorial has been called with n = 5
factorial has been called with n = 4
factorial has been called with n = 3
factorial has been called with n = 2
factorial has been called with n = 1
factorial has been called with n = 0
intermediate result for  1  * factorial( 0 ):  1
intermediate result for  2  * factorial( 1 ):  2
intermediate result for  3  * factorial( 2 ):  6
intermediate result for  4  * factorial( 3 ):  24
intermediate result for  5  * factorial( 4 ):  120
intermediate result for  6  * factorial( 5 ):  720
720
```

5.1.3 函数递归调用

- **问题解决：**用函数嵌套定义和递归实现帕斯卡公式C(n,i) = C(n-1, i) + C(n-1, i-1)，进行组合数C(n,i)的快速求解。

```
def C(n,i):
    cache = dict()
    # 创建缓存字典，用于存储已计算的结果
    # 字典为{(n,i):f(n,i)}  #元组为键

    def f(n,i): # 内部递归函数
        if i==n or i==0:
            print(f"计算 C({n},{i})，加入缓存")
            return 1
        elif (n,i) not in cache: # 如果结果未缓存
            print(f"计算 C({n},{i})，加入缓存")
            cache[(n,i)] = f(n-1, i) + f(n-1, i-1)
        else:
            print(f"从缓存读取 C({n},{i}) = {cache[(n, i)]}")
        return cache[(n,i)]

    return f(n,i) # 调用内部函数并返回结果
```

缓存（记忆化）技术通过存储已计算的结果来避免重复计算，大大提高了递归算法的效率。以计算 f(4, 2) 为例：

```
f(4,2) -> 不在缓存中
  f(3,2) -> 不在缓存中
    f(2,2) -> 返回 1
    f(2,1) -> 不在缓存中
      f(1,1) -> 返回 1
      f(1,0) -> 返回 1
      cache[(2,1)] = 1 + 1 = 2
    cache[(3,2)] = 1 + 2 = 3
  f(3,1) -> 不在缓存中
    f(2,1) -> 已在缓存中，返回 2
    f(2,0) -> 返回 1
    cache[(3,1)] = 2 + 1 = 3
  cache[(4,2)] = 3 + 3 = 6
返回 6
```

5.1.3 函数递归调用

■问题解决：展平嵌套列表

```
def flatten(a):
    a_flat = [] # 用于存储展平后的结果

    def fun(x): # 内部递归函数
        for v in x: # 遍历输入的可迭代对象
            if type(v) in (list, tuple):
                fun(v)
                #如果当前元素是列表或元组，
                #递归调用，继续展平
            else: # 如果是基本元素（数字、字符串等）
                a_flat.append(v)  # 直接添加到结果列表

    fun(a) # 调用内部函数开始处理
    return a_flat # 返回展平后的结果

seq = [[1,2,(3,4)], (5,6), 7]
print(flatten(seq))
```

第1层：遍历 [[1,2,(3,4)], (5,6), 7]
 v = [1,2,(3,4)] → 是列表 → 递归进入第2层
  第2层：遍历 [1,2,(3,4)]
   v = 1 → 不是列表/元组 → 添加到 a_flat: [1]
   v = 2 → 不是列表/元组 → 添加到 a_flat: [1, 2]
   v = (3,4) → 是元组 → 递归进入第3层
    第3层：遍历 (3,4)
     v = 3 → 添加到 a_flat: [1, 2, 3]
     v = 4 → 添加到 a_flat: [1, 2, 3, 4]
  返回第1层
 v = (5,6) → 是元组 → 递归进入第2层
  第2层：遍历 (5,6)
   v = 5 → 添加到 a_flat: [1, 2, 3, 4, 5]
   v = 6 → 添加到 a_flat: [1, 2, 3, 4, 5, 6]
  返回第1层
 v = 7 → 不是列表/元组 → 添加到 a_flat: [1, 2, 3, 4, 5, 6, 7]

关键思路：
遇到列表或元组就递归调用自身
遇到基本元素就直接收集

5.1.3 函数递归调用

■问题解决：由序列中的元素组成的长度为m的排列，且排列中的元素可重复：
如由’ABCD’生成长度为2的排列：AA AB AC AD BA BB BC BD CA CB CC CD DA DB DC DD

```
def prod_m(a, m):
    if m == 1:
        return [(item,) for item in a)]

    return [(v,) + item for v in a for item in prod_m(a, m-1)]
```

- 标准库itertools中的product()、permutations()、combinations()可分别生成所有乘积序列、排列、组合的迭代器
- 如itertools.product('ABCD', repeat=2)

5.1.3 函数递归调用

以 a = [1, 2], m = 2 为例：

prod_m([1,2], 2)
→ 递归调用 prod_m([1,2], 1) 得到：[(1,), (2,)]
→ 然后对每个 v in [1,2] 和每个 item in [(1,), (2,)] 进行组合：

v=1: (1,) + (1,) = (1, 1)
(1,) + (2,) = (1, 2)
v=2: (2,) + (1,) = (2, 1)
(2,) + (2,) = (2, 2)
→ 返回：[(1,1), (1,2), (2,1), (2,2)]

以 m = 3 为例：

prod_m([1,2], 3)
→ 先计算 prod_m([1,2], 2) = [(1,1), (1,2), (2,1), (2,2)]
→ 然后对每个 v in [1,2] 和每个 item in 上述结果组合：
v=1: (1,1,1), (1,1,2), (1,2,1), (1,2,2)
v=2: (2,1,1), (2,1,2), (2,2,1), (2,2,2)

Practice

- 试写一个递归函数fib(n)计算第n个斐波那契数？

- $F_0 = 0$
- $F_1 = 1$
- $F_n = F_{n-1} + F_{n-2}$（n≧2）

```
def fib(n):
    if n == 0 or n == 1:
        return n
    return fib(n-1) + fib(n-2)
```

第5章 代码复用技术(一)： 函数

- 5.1 函数定义与使用
- **5.2 函数参数**
  - 5.2.1位置参数
  - 5.2.2默认值参数
  - 5.2.3关键参数
  - 5.2.4可变长度参数
  - 5.2.5传递参数时的序列解包
- 5.3 变量作用域
- 5.4 lambda表达式
- 5.5 生成器函数设计要点
- 5.6 精彩案例赏析

5.2 函数参数

- 函数定义时圆括弧内是使用逗号分隔开的形参（parameters）
- 函数没有参数时，定义和调用时一对圆括弧也必须要有，表示这是一个函数并且不接收参数

- 调用函数时向其传递实参（arguments），是将实参的**引用**传递给形参
- 定义函数时不需要声明参数类型，解释器会根据实参的类型自动推断形参类型

第5章 代码复用技术(一)： 函数

- 5.1 函数定义与使用
- 5.2 函数参数
  - **5.2.1位置参数**
  - **5.2.2默认值参数**
  - **5.2.3关键参数**
  - 5.2.4可变长度参数
  - 5.2.5传递参数时的序列解包
- 5.3 变量作用域
- 5.4 lambda表达式
- 5.5 生成器函数设计要点
- 5.6 精彩案例赏析

5.2.1 位置参数

- 调用函数时实参和形参的**顺序**必须严格一致，并且实参和形参的**数量**必须相同的参数称为**位置（positional）参数，包括形参和实参**

```
>>> def demo(a, b, c):
    print(a, b, c)
>>> demo(3, 4, 5)                          #按位置传递参数
3 4 5

>>> demo(1, 2, 3, 4)                       #实参与形参数量必须相同
TypeError: demo() takes 3 positional arguments but 4 were given
```

5.2.2 默认值形参

def 函数名(……，**形参**名=默认值)：　　e.g. def f(a,b,c=10)
　　函数体

■在调用函数时，可以不用为设置了默认值的形参传递实参，此时函数内形参取函数定义时设置的默认值；即默认值参数是可选的(optional)

■需要注意的是，在定义带有默认值参数的函数时，任何一个默认值参数右边都不能再出现位置参数，否则会提示语法错误：位置参数位于所有默认值参数的左边

print(*args, sep=' ', end='\n', file=None, flush=False)

5.2.2 默认值形参

```
def favorite_place(name,place="万里长城"):
    print(f"我的名字是{name}")
    print(f"我最喜欢的名胜古迹是{place}")
favorite_place(name="张三")
```



```
我的名字是张三
我最喜欢的名胜古迹是万里长城
```

```
def favorite_place(name,place="万里长城"):
    print(f"我的名字是{name}")
    print(f"我最喜欢的名胜古迹是{place}")
favorite_place(name="张三",place="桂林山水")
```



```
我的名字是张三
我最喜欢的名胜古迹是桂林山水
```

```
def favorite_place(name,place="万里长城",age):
    print(f"我的名字是{name}")
    print(f"我最喜欢的名胜古迹是{place}")
favorite_place(name="张三",place="桂林山水",age=19)
```



```
SyntaxError: non-default argument follows default argument
```

place 有默认值，但后面的 age 没有默认值，这是不允许的。

5.2.2 默认值形参

- 可以使用“函数名.__defaults__”随时查看函数所有默认值参数的当前值，其返回值为一个元组，其中的元素依次表示每个默认值参数的当前值

```
def example_func(a, b=2, c=3, d=4):
    """示例函数，有多个默认参数"""
    return a + b + c + d

# 查看默认参数的当前值
print(example_func.__defaults__)  # 输出：(2, 3, 4)

# 使用函数
print(example_func(1))          # 输出：10 (1+2+3+4)
print(example_func(1, 10))      # 输出：18 (1+10+3+4)
```

5.2.2 默认值形参

■函数参数的默认值是在函数定义时解释的，所以只会被初始化一次

```
>>> i = 3
>>> def f(n=i):      #参数n的值仅取决于i的当前值    #这里已经完成了默认值初始化
    print(n)

>>> f()
3

>>> i = 5            #函数定义后修改i的值不影响参数n的默认值
>>> f()
3

>>> f(i)             #重新赋值，这时计算的是f(5)
5

>>> f()              #n的默认值不变
3
```

5.2.2 默认值形参

■对于列表、字典这样可变类型的默认值参数，这一点可能会导致很意外的共享状态

```
>>> def demo(newitem, old_list=[]):
    old_list.append(newitem)
    return old_list

>>> print(demo('aaa', ['a', 'b'])) #这里没有调用默认值，默认列表 [] 没有变化
['a', 'b', 'aaa']
>>> print(id(demo('aaa', ['a', 'b'])))  #2040440092800

>>> x=demo('a')
>>> print(x)                  #第一次调用默认值，则默认值被初始化了
['a']
>>> print(id(x))          #2040443445312
>>> y=demo('b')
>>> print(y)              #注意这里的输出结果 #这时默认值已经是[‘a’]了
['a', 'b']
>>> print(id(y))          #2040443445312
>>> z=demo('c')
>>> print(z)
['a', 'b', 'c']                #每次调用都使用同一个列表对象，之前的修改会保留
>>> print(id(z))          #2040443445312
```

5.2.2 默认值形参

■一般来说，要**避免**使用列表、字典、集合等可变序列作为函数参数默认值

■接受可变序列的形参的默认值一般设为None

```
def demo(newitem, old_list=None):
    if old_list is None:
        old_list = []                    #相当于每次使用默认值时，都创建新列表
    old_list.append(newitem)
    return old_list

>>> print(demo('a'))
['a']
>>> print(demo('b'))                     #注意这里的输出结果
['b']
```

5.2.3 关键词实参(keyword arguments)

- 通过关键词实参可以按形参名字传递参数，明确指定哪个值传递给哪个参数，实参顺序可以和形参顺序不一致
- 某些函数要求不能使用关键词实参为位置参数传递参数，比如很多内置函数
- 函数调用时，位置参数必须置于所有关键词实参左边

```
>>> def demo(a, b, c=5):
    print(a, b, c)
>>> demo(a=7, b=3, c=6)
7 3 6
>>> demo(c=6, a=7, b=3)
7 3 6
>>> demo(a=2,3,4)
SyntaxError: positional argument follows keyword argument
>>> demo(2,3,c=4)
2 3 4
```

```
>>> help(id)
id(obj, /)
    Return the identity of an object.

id(obj=1) #id()是用C语言写的函数，只接收位置形参
TypeError: id() takes no keyword arguments
```

Practice

- def fun(a, b=0, c)的形参表正确吗，为什么？

✓ 不正确。默认值参数必须置于所有位置参数的右边

- 函数形参的默认值在什么时间初始化？

✓ 在函数定义时

- 试写一个函数min_fun(seq, key=None)计算列表seq的最小值，比较大小的键key为一个函数，默认值为None

```
def min_fun(seq, key=None):
    seq1 = ([key(v) for v in seq] if key else seq)
    if seq1:
        k_min = seq1[0]
        for k in seq1[1:]:
            if k < k_min:
                k_min = k
        return seq[seq1.index(k_min)]
    else:
        return None
```

```
# 基本用法
numbers = [3, 1, 4, 1, 5, 9]
print(min_fun(numbers))  # 输出: 1

# 使用key参数
words = ["apple", "banana", "cherry", "date"]
print(min_fun(words, key=len))  # 输出: "date"（最短的字符串）

# 空序列情况
empty_list = []
print(min_fun(empty_list))  # 输出: None
```

第5章 代码复用技术(一)： 函数

- 5.1 函数定义与使用
- 5.2 函数参数
  - 5.2.1位置参数
  - 5.2.2默认值参数
  - 5.2.3关键参数
  - **5.2.4可变长度参数**
  - 5.2.5传递参数时的序列解包
- 5.3 变量作用域
- 5.4 lambda表达式
- 5.5 生成器函数设计要点
- 5.6 精彩案例赏析

5.2.4 可变长度形参

- 可变长度（个数）**形参**的两种形式：

➢ *args用来接收多个**位置参数**并将其放在元组args中

➢ **kwargs接收多个**关键词参数**并存放到字典kwargs中

```
>>> def foo(*args):
    print(args)

>>> foo(1, 2, 3)
(1, 2, 3)
```

```
>>> def foo(*args, **kwargs):
    print(args)
    print(kwargs)


>>> foo(1,2,3, a=1, b=2)
(1, 2, 3)
{'a': 1, 'b': 2}
```

5.2.4 可变长度参数

```
>>> def demo(**p):
    for item in p.items():
        print(item)

>>> demo(x=1,y=2,z=3)
('x', 1)
('y', 2)
('z', 3)
```

■ print()函数参数的个数是可变的

#print(***args**, sep=' ', end='\n', file=None, flush=False)

```
>>> print(1, 2, 3, [1,2,3], 'abc')
1 2 3 [1, 2, 3] abc
```

5.2.4 可变长度参数

- 几种不同类型的参数可以混合使用，但是不建议这样做（了解）

```
>>> def func_4(a, b, c=4, *aa, **bb):
    print(a,b,c)
    print(aa)
    print(bb)

>>> func_4(1,2,3,4,5,6,7,xx='1',yy='2',zz=3)
(1, 2, 3)
(4, 5, 6, 7)
{'xx': '1', 'yy': '2', 'zz': 3}
```

第5章 代码复用技术(一)： 函数

- 5.1 函数定义与使用
- 5.2 函数参数
  - 5.2.1位置参数
  - 5.2.2默认值参数
  - 5.2.3关键参数
  - 5.2.4可变长度参数
  - **5.2.5传递参数时的序列解包**
- 5.3 变量作用域
- 5.4 lambda表达式
- 5.5 生成器函数设计要点
- 5.6 精彩案例赏析

5.2.5 传递参数时的序列解包

- 调用函数传递参数时，可以通过在**实参序列**前加一个星号将其解包，然后**'批量'**传递给多个形参

```
>>> def demo(a, b, c):
    print(a+b+c)


>>> seq = [1, 2, 3]
>>> demo(*seq)
6
>>> tup = (1, 2, 3)
>>> demo(*tup)
6
```

```
>>> dic = {1:'a', 2:'b', 3:'c'}
>>> demo(*dic) #只解包字典的keys，顺序取决于内部实现
6
>>> Set = {1, 2, 3}
>>> demo(*Set)
6
>>> demo(*dic.values())
abc
```

5.2.5 传递参数时的序列解包

- 可以在字典前面加两个星号进行解包，以’批量’传递关键词参数
- 字典的键为形参名的字符串

```
>>> def demo(a, b, c):
    print(a+b+c)

>>> dic = {'a':1, 'b':2, 'c':3}
>>> demo(**dic)
6
>>> demo(a=1, b=2, c=3)                #与上面等价
6
```

5.2.5 传递参数时的序列解包

■注意：调用函数时，使用一个星号*进行解包后的实参将会被当做普通位置参数对待，会在关键词参数和使用两个星号**进行解包的参数之前进行处理（了解）

```
>>> def demo(a, b, c):                          #定义函数
    print(a, b, c)

>>> demo(1, *(2, 3))                            #位置参数和序列解包同时使用
1 2 3

>>> demo(a=1, *(2, 3))                  #序列解包相当于位置参数，优先处理
TypeError: demo() got multiple values for argument 'a'
```

5.2.5 传递参数时的序列解包

```
>>> demo(**{'a':1, 'b':2}, *(3,)) #序列解包不能在关键词参数解包之后
SyntaxError: iterable argument unpacking follows keyword argument
unpacking

>>> demo(*(3,), **{'a':1, 'b':2})
TypeError: demo() got multiple values for argument 'a'

>>> demo(*(3,), **{'c':1, 'b':2})
3 2 1
```

5.2.5 传递参数时的序列解包

■例：修饰器函数中*和**，使得修饰器可以修饰任意函数

```
from random import randint

def our_decorator(func):
    def function_wrapper(*args, **kwargs): #这里args为可变长度形参
        print(“Before calling ” + func.__name__)
        result=func(*args, **kwargs) #这里args为实参，*args表示将序列args解包
        print("After calling " + func.__name__)
        return result
    return function_wrapper

randint = our_decorator(randint)

>>> randint(0,10)
```

Practice

- def fun(*args, **kw)，通过fun(0, b=1, c=2)调用函数后，fun内部args和kw的值为？

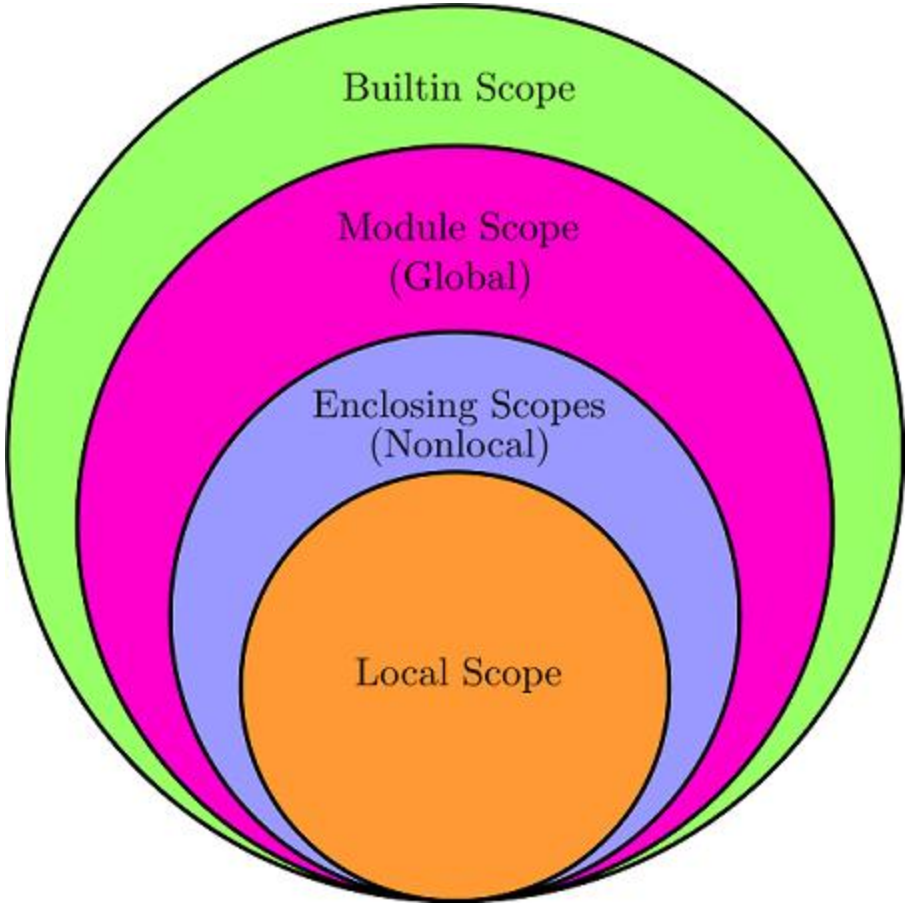
✓ (0,)和{‘b’:1, ‘c’:2}

- def fun(a, b)，通过调用fun(**kw)将参数0和1分别传递给a和b，kw的值应为？

✓ {‘a’:0, ‘b’:1}

第5章 代码复用技术(一)： 函数

- 5.1 函数定义与使用
- 5.2 函数参数
- **5.3 变量作用域**
- 5.4 lambda表达式
- 5.5 生成器函数设计要点
- 5.6 精彩案例赏析



5.3 变量作用域

变量起作用的代码范围称为**变量的作用域**，根据变量作用域的不同，变量可以分为:

- **局部变量**：在函数内部使用的变量，只在**函数内部**起作用
- **全局变量**: 在函数之外进行定义的变量，能在**整个程序**中使用

5.3 变量作用域

■ **不同函数中的局部变量**名称可以相同，且**互不影响**

```
1. def person_name(name):
2.     print("宠物主人的名字是：",name)
3. def pet_name(name):
4.     print("宠物的名字是：",name)
5. person_name("张三")
6. pet_name("旺财")
```



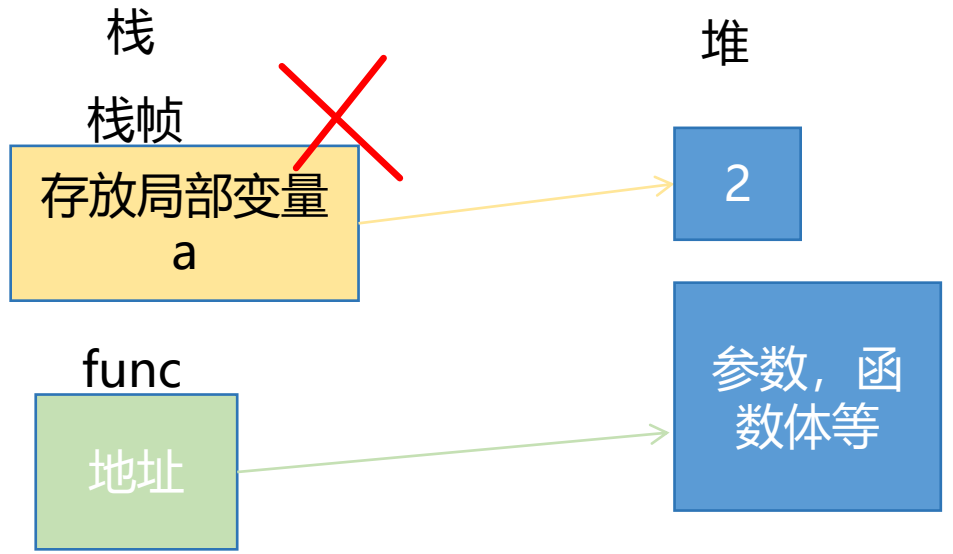
```
宠物主人的名字是： 张三
宠物的名字是： 旺财
```

5.3 变量作用域

- 当函数执行结束后，局部变量自动删除，不再可以使用

```
def func(a):
    return a
result = func(2)
print(a)
```

NameError: name 'a' is not defined



局部变量的内存管理

栈帧：类似于电影胶卷的一帧，这一帧放完，函数调用完就删掉

5.3 变量作用域

- 列表推导式中的循环变量为局部变量

```
>>> a = [i**3 for i in range(10)]
>>> i
NameError: name 'i' is not defined

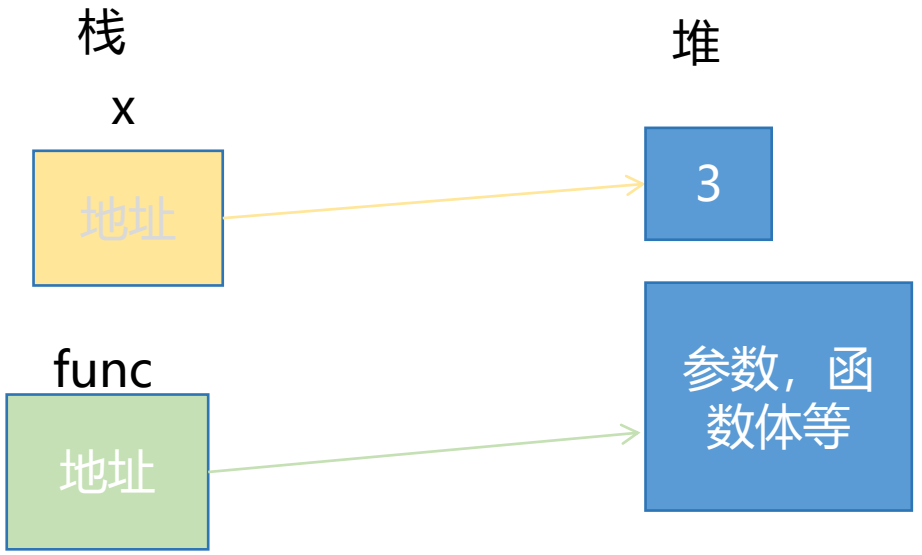
>>> i = 0
>>> a = [ i**3 for i in range(10)]
>>> i
0
```

5.3 变量作用域

全局变量是指在函数之外进行定义的变量，能在整个程序中使用

```
x = 3
def func():
    print(x)
func()
```

3



全局变量的内存管理

在func()函数中能改变全局变量x的值吗?

5.3 变量作用域

在func()函数中，给x的值增加2

```
x = 3
def func():
    x=x+2
    print(x)
func()
```



UnboundLocalError: cannot access local variable 'x' where it is not associated with a value

在函数里实现全局变量x增加2，要如何操作呢?

5.3 变量作用域

- 一个变量已在函数外定义，如果在函数内需要为这个变量赋值，并要将这个赋值结果反映到函数外，可以在函数内使用**global**将其声明为全局变量

```
x = 3
def func():
    global x
    x = x+2
    print(x)
func()
print(x)
```



```
5
5
```

5.3 变量作用域

- 如果一个变量在函数外没有定义，在函数内部也可以用global直接将一个变量声明为全局变量，该函数执行后，将增加一个**新的**全局变量

```
def demo():
    global x
    x = 3
    y = 4
    print(x,y)
```

```
>>> x = 5
>>> demo()
3  4
>>> x
3
>>> y
NameError: name 'y' is not defined
```

```
>>> demo()
3  4
>>> x = 5
>>> x
5
```

5.3 变量作用域

- 如果局部变量与全局变量具有相同的名字，那么该局部变量会在自己的作用域内隐藏同名的全局变量，即函数会**优先使用局部变量**

```
>>> x = 5                    #创建了全局变量

>>> def demo():
        x = 3                 #创建了同名的局部变量，会自动隐藏了同名的全局变量
        print('内部x:',x)


>>> demo()                   #函数执行只使用了其内部的局域变量
内部x: 3
>>> x                        #不影响外面全局变量的值
5
```

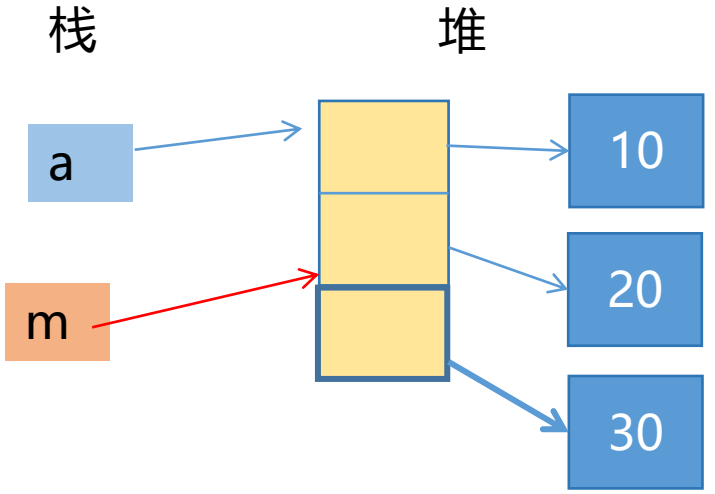
5.3 变量作用域

- 当**列表、字典等可变**数据类型作为全局变量时，在函数内部可以对全局变量进行修改，**不需要global关键字进行声明**。

```
a = [10,20]                #定义全局变量a
print(id(a))
def func(m):
    print(id(m))
    m.append(30)    #在函数中修改全局变量a

func(a)                    #调用函数
print(a)
```

```
2040440624832
2040440624832
[10, 20, 30]
```



传递可变对象的引用的内存管理

5.3 变量作用域

在函数外创建列表后，在函数内创建同名列表

```
a = [10,20]                #定义全局变量a

def func():
    a = [ ]                #在函数内创建同名变量
    a.append(30)           #在函数中修改变量a
    print('内部a:',a)

func()                     #调用函数
print('外部a:',a)
```



```
内部a: [30]
外部a: [10, 20]
```

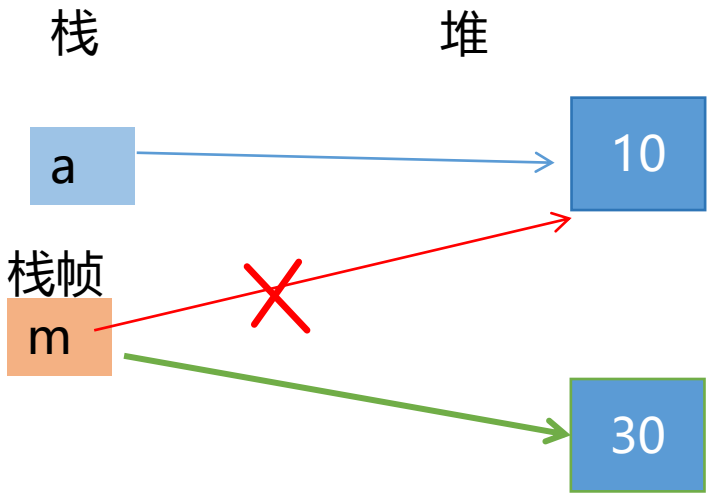
函数内部仅对局部变量进行操作，函数结束后局部变量被释放，不影响全局变量的值

5.3 变量作用域

- **对比**：当**数字等不可变**数据类型作为全局变量时，由于不可变对象无法修改，系统内部会创建一个**新对象**。

```
a = 10                    #定义全局变量a
print('外部a的地址:',id(a))
def func(m):
    print('内部m的地址:',id(m))
    m=30        #在函数内创新新对象
    print('内部m:',m)
    print('内部m的新地址:',id(m))
func(a)                   #调用函数
print('外部a:',a)
```

```
外部a的地址: 140721810207816
内部m的初始地址: 140721810207816
内部m: 30
内部m的新地址: 140721810208456
外部a: 10
```



传递不可变对象的引用的内存管理

5.3 变量作用域

- **不可变的数据类型作为局部变量**，仅能在函数内部创建和使用，函数执行结束后，变量就会被释放，如果全局变量中有与其同名的变量，不会受到影响，函数优先使用内部变量。

- **不可变的数据类型可以使用global关键字**，转换为全局变量，函数执行结束后，此变量能够**保留且能改变**同名全局变量的值。

- **当可变的数据类型作为全局变量时**，如果在函数中**没有创建同名的局部变量**，则**函数内部**可以直接**使用并修改**全局变量的值。

- **当可变的数据类型作为全局变量时**，如果在函数中**已经创建同名的局部变量**，则**函数内部仅对局部变量进行操作**，函数结束后局部变量被释放，不影响全局变量的值。

5.3 变量作用域

■ 如果需要查看局部变量与全局变量，可以使用**globals()**与**locals()**函数获取。

```
def join_str(brand_str):
    brand_str = "www." + brand_str + ".com"
    print("局部变量：",locals())
    return brand_str

brand = "taobao"
url = join_str(brand)
print("全局变量：",globals())
```



局部变量： {**'brand_str': 'www.taobao.com'**}
全局变量： {'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <_frozen_importlib_external.SourceFileLoader object at 0x000001DEBE186D00>, '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, '__file__': 'C:\\1000phone\\parter7\\check_varible.py', '__cached__': None, 'join_str': <function join_str at 0x000001DEBE1CF040>, **'brand': 'taobao', 'url': 'www.taobao.com'**}

5.3 变量作用域

- 对**嵌套函数**,nonlocal语句可以将变量的作用域扩展到外层函数内（了解）

```
def func02():
    x = 5
    def func01():
        print(x)
    func01()
func02()
```

如果增加一个赋值语句x = x+1，程序将会出现异常

```
5
```

```
def func02():
    x = 5
    def func01():
        nonlocal x
        x = x + 1
        print(x)
    func01()
    print(x)
func02()
```

```
6
6
```

5.3 变量作用域

- 在通用的模块中，全局变量与global语句是极其罕见的；全局变量难以维护
- 除非是嵌套函数或极特殊的情况，应避免使用函数体外的变量！！！
- 局部变量的引用比全局变量速度快，应优先考虑使用

Practice

- 在函数外定义的变量x，在什么情况下会使x在函数内部成为局部变量？

✓ 变量x未被global声明且x被重新赋值

- 哪一类函数经常使用函数体外的变量？

✓ 嵌套函数

```
x = 10  # 全局变量

def func():
    x = 20  # 这会在函数内部创建新的局部变量 x
    print(f"局部 x: {x}")

func()      # 输出: 局部 x: 20
print(f"全局 x: {x}")  # 输出: 全局 x: 10
```

第5章 代码复用技术(一)：函数

- 5.1 函数定义与使用
- 5.2 函数参数
- 5.3 变量作用域
- **5.4 lambda表达式**
- 5.5 生成器函数设计要点
- 5.6 精彩案例赏析

作业

1. 写一个递归函数print_int(a)，以嵌套列表a为参数，打印a及其所有子列表中的整数。并以实参[[1,2,(3.0,4)], (5.0,6), 7]调用它，写下程序和输出结果。

2. (1) 试写一个修饰器函数our_decorator(func)，被修饰的函数的参数数目任意，且调用被修饰函数时可能使用关键词参数。要求被修饰的函数依次打印各个位置参数的值，各个关键词参数的名和值，和修饰函数的返回值，均打印在一行里面。

(2)使用our_decorator修饰

```
def f(n,i):
    if n==i or i==0:
        return 1
    return f(n-1, i=i) + f(n-1, i=i-1)
```

(3)调用修饰过的f(8, i=3)， 写下程序和输出结果的最后5行。

作业

2(1)修饰器函数的模板：请填写留空的部分

```
def our_decorator(func):
    def function_wrapper(*args, **kwargs):
        res = func(*args, **kwargs)
        #留空
        #留空
        return res
    return function_wrapper
```

5.4 lambda表达式

- 格式：lambda 形参表：表达式

```
# 普通函数
def add(x, y):
    return x + y
```

```
# Lambda等价形式
add = lambda x, y: x + y

print(add(3, 5))  # 输出: 8
```

```
# 定义并立即调用
result = (lambda x, y: x + y)(3, 5)
print(result)  # 输出: 8
```

5.4 lambda表达式

- lambda表达式可以用来声明匿名函数，也就是没有函数名字的临时使用的小函数，尤其适合需要一个函数作为另一个函数参数的场合
- lambda表达式只可以包含一个表达式，以该表达式的计算结果作为匿名函数的返回值，在表达式中可以调用其他函数；但不允许包含复合语句
- 只有当匿名函数被调用时，其中的表达式才会被计算；定义匿名函数时，只做默认值形参的初始化

5.4 lambda表达式

```
>>> f = lambda x, y, z: x+y+z          #可以给lambda表达式起名字
>>> f(1,2,3)                           #像函数一样调用
6
>>> g = lambda x, y=2, z=3: x+y+z      #默认值形参
>>> g(1)
6

>>> L = [(lambda x: x**2),             #里面圆括号并不是必需的，但增强了可读性
         (lambda x: x**3),
         (lambda x: x**4)]
>>> L
[<function <lambda> at 0x00000000005C1EA0>,
<function <lambda> at 0x0000000003009A60>,
<function <lambda> at 0x00000000030092F0>]

>>> print(L[0](2),L[1](2),L[2](2))
4 8 16
```

5.4 lambda表达式

```
>>> D = {‘f1’:(lambda:2+3),              #lambda函数可以没有参数
         'f2':(lambda:2*3),
         'f3':(lambda:2**3)}
>>> print(D['f1'](), D['f2'](), D['f3']())  #()必须
5 6 8
```

■当给lambda函数具名时，应**避免**在表达式中使用全局变量！

```
>>> f = lambda x: (x + n)**2    #如真的需要，可以定义两个参数 x,n
>>> n = 0
>>> f(1)
1
>>> n = 1
>>> f(1)
4
```

5.4 lambda表达式

■ 与 sort()/sorted() 一起使用

```
>>> data = [4, 3, 11, 13, 12, 15, 9, 2, 10, 6, 19, 18, 14, 8, 0, 7, 5, 17, 1, 16]
>>> data.sort(key=lambda x: len(str(x)))     #按转换成字符串以后的长度排序
>>> data
[4, 3, 9, 2, 6, 8, 0, 7, 5, 1, 11, 13, 12, 15, 10, 19, 18, 14, 17, 16]

>>> data = [(2, 3), (2, 2), (1, 1)]
>>> sorted(data, key=lambda v: (v[0], v[1])) #先按第一个元素排序
[(1, 1), (2, 2), (2, 3)]                     #若第一个元素相同，则按第二个元素排序
```

5.4 lambda表达式

■ 与 filter() 一起使用

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)  # 输出: [2, 4, 6, 8]
```

```
# 等价的列表推导式
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
even_numbers = [x for x in numbers if x % 2 == 0]
print(even_numbers)  # 输出: [2, 4, 6, 8]
```

5.4 lambda表达式

■ 与 map() 一起使用

```
# 平方所有数字
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x ** 2, numbers))
print(squares)  # 输出: [1, 4, 9, 16, 25]
```

```
#等价的列表推导式
numbers = [1, 2, 3, 4, 5]
squares = [x ** 2 for x in numbers]
print(squares)  # 输出: [1, 4, 9, 16, 25]
```

5.4 lambda表达式

■ 与 reduce() 一起使用

```
from functools import reduce

# 计算乘积
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
print(product)  # 输出: 120
```

reduce() 对序列中的元素从左到右依次应用函数，逐步减少序列，最终返回一个单一的结果。
reduce(f, [a, b, c, d]) = f(f(f(a, b), c), d)

5.4 lambda表达式

何时使用Lambda

- 可以使用lambda自定义小函数，但标准库operator提供了很多高效的小函数，他们一般对应Python内置运算符的计算（了解）
- 主要用于sort(),filter(),map(), reduce()等函数，但很多适合列表推导式更方便

```
>>> import operator
>>> operator.__all__
['abs', 'add', 'and_', 'attrgetter', 'concat', 'contains', 'countOf',
  'delitem', 'eq', 'floordiv', 'ge', 'getitem', 'gt', 'iadd', 'iand',
  'iconcat', 'ifloordiv', 'ilshift', 'imatmul', 'imod', 'imul', 'index',
  'indexOf', 'inv', 'invert', 'ior', 'ipow', 'irshift', 'is_', 'is_not',
  'isub', 'itemgetter', 'itruediv', 'ixor', 'le', 'length_hint', 'lshift',
  'lt', 'matmul', 'methodcaller', 'mod', 'mul', 'ne', 'neg', 'not_',
  'or_', 'pos', 'pow', 'rshift', 'setitem', 'sub', 'truediv', 'truth',
  'xor']
```

第5章 代码复用技术(一)：函数

- 5.1 函数定义与使用
- 5.2 函数参数
- 5.3 变量作用域
- 5.4 lambda表达式
- **5.5 生成器函数设计要点**
- 5.6 精彩案例赏析

5.5 生成器函数设计要点

生成器是一种特殊的迭代器，它不会一次性生成所有数据，而是**按需生成**（惰性求值），从而节省内存。

生成器的特点：

- 惰性计算：只在需要时生成值
- 内存高效：不会一次性存储所有数据
- 一次性使用：生成器只能迭代一次
- 状态保持：记住上次执行的位置

5.5 生成器函数设计要点

生成器 vs 迭代器

特性	迭代器	生成器
创建方式	需要定义类并实现 __iter__ 和 __next__ 方法	使用 **yield 关键字**或**生成器推导式**
代码复杂度	相对复杂，需要完整的类定义	简单，几行代码即可实现
内存使用	取决于实现方式	极低，惰性计算，**按需生成**
状态管理	需要手动管理状态	自动管理状态，记住上次执行位置
使用频率	较少直接使用	**非常常用**

```
# 生成器表达式（惰性计算）
squares_gen = (x**2 for x in range(5))  # 生成器对象

print(list(squares_gen))  # 输出: [0, 1, 4, 9, 16]
```

5.5 生成器函数设计要点

- 生成器函数是Python中一种特殊的函数，它使用 yield 语句而不是 return 来返回值，并且可以暂停和恢复执行状态。
- 与以return语句返回值的普通函数不同，生成器函数每次执行到yield语句并返回一个值之后，会暂停或挂起后面代码的执行
- 下次通过生成器对象的__next__()方法、内置函数next()、for循环遍历显式“索要”其数据时恢复执行
- 生成器具有惰性求值的特点，适合大数据处理

5.5 生成器函数设计要点

```
# 普通函数
def normal_function():
    return [1, 2, 3, 4, 5]
```

```
#生成器函数
def simple_generator():
    yield 1  #暂停执行，需要时再产生一个新元素
    yield 2  #暂停执行，需要时再产生一个新元素
    yield 3   #暂停执行，需要时再产生一个新元素

# 使用生成器
gen = simple_generator()
print(next(gen))  # 输出: 1
print(next(gen))  # 输出: 2
print(next(gen))  # 输出: 3
```

5.5 生成器函数设计要点

```
def f():                    #斐波那契数列生成器
    a, b = 0, 1
    while True:                 #无穷循环
        yield a                 #暂停执行，需要时再产生一个新元素
        a, b = b, a+b
aa = f()                    #创建生成器对象aa；aa是无限的
for i in range(10):
    print(next(aa), end=' ')
```

```
0 1 1 2 3 5 8 13 21 34
```

5.5 生成器函数设计要点

```
>>> def f():
        n = 0
        while n <= 10:    #有限循环
                yield n
                n = n + 1

>>> a = f()
>>> for i in range(20):  #超出循环
        print(next(a))
StopIteration
```

Practice

- 哪些内置或标准库函数的返回值为一个生成器或迭代器？

✓ enumerate, zip, map, iter, re.finditer,...

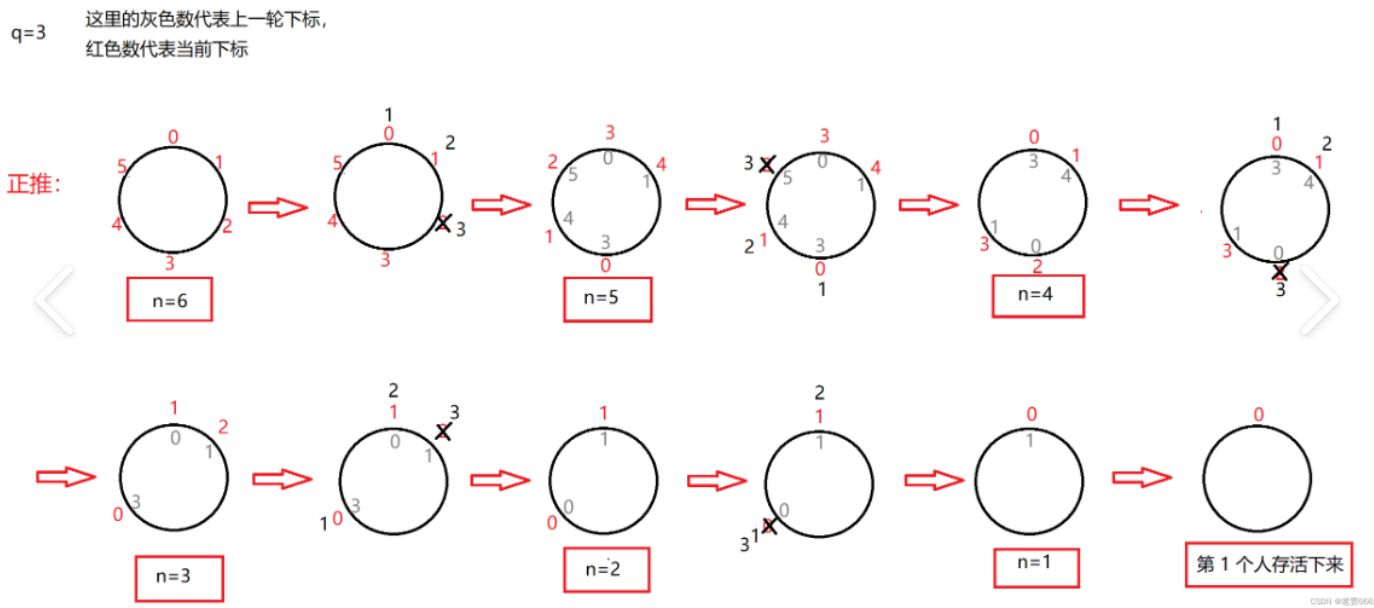
第5章 代码复用技术(一)：函数

- 5.1 函数定义与使用
- 5.2 函数参数
- 5.3 变量作用域
- 5.4 lambda表达式
- 5.5 生成器函数设计要点
- **5.6 精彩案例赏析**

5.6 精彩案例赏析

- **示例5-13** 编写函数模拟报数游戏。有n个人围成一圈，顺序编号，从第一个人开始从1到k（假设k=3）报数，报到k的人退出圈子，然后圈子缩小，从下一个人继续游戏，问最后留下的是原来的第几号。

这是约瑟夫环问题（Josephus problem），一个经典的数学和计算机科学问题.



5.6 精彩案例赏析

```
from itertools import cycle

def demo(t_lst, k):

    #游戏一直进行到只剩下最后一个人
    while len(t_lst)>1:

        #为当前圈子创建无限循环的迭代器
        c = cycle(t_lst)


        #从1到k报数
        for i in range(k):
            t = next(c)  #获取第k人

        #一个人出局，圈子缩小
        index = t_lst.index(t) # 找到出局者的位置
        t_lst = t_lst[index+1:] + t_lst[:index] # 重新排列列表

#游戏结束
    return t_lst[0]  # 返回最后的幸存者
```

5.6 精彩案例赏析

解释：
1. cycle() 函数创建一个无限循环的迭代器，比如：

```
c = cycle([1, 2, 3, 4, 5])
# c 现在是一个无限循环：1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, ...
```

2. for i in range(k): t = next(c) 的逐步执行：

以 t_lst = [1, 2, 3, 4, 5], k = 3 为例：

第一次循环：

```
c = cycle([1, 2, 3, 4, 5])  # 创建迭代器

# for循环开始
i = 0: t = next(c) → t = 1
i = 1: t = next(c) → t = 2
i = 2: t = next(c) → t = 3 ← 最终t的值是3
```

最终结果示例：
t_lst = [1, 2, 3, 4, 5], k = 3：

初始: [1, 2, 3, 4, 5]
第1轮: 数到3 → 3出局 → 新圈子: [4, 5, 1, 2]
第2轮: 数到3 → 1出局 → 新圈子: [2, 4, 5]
第3轮: 数到3 → 5出局 → 新圈子: [2, 4]
第4轮: 数到3 → 2出局 → 幸存者: [4]
返回: 4

5.6 精彩案例赏析

- 示例5-16　24点游戏是指随机选取4张扑克牌（不包括大小王），然后通过四则运算来构造表达式，如果表达式的值恰好等于24就赢一次。下面的代码定义了一个函数用来测试随机给定的4个数是否符合24点游戏规则，如果符合就输出所有可能的表达式。

5.6 精彩案例赏析

```
from random import randint
from itertools import permutations, product

#4个数字和2个运算符可能组成的一种表达式形式
exps = '(%s %s %s) %s (%s %s %s)'  #表达式模板，格式为 (a op b) op (c op d)
ops = '+-*/'

def test24(v):  #v: 输入参数，包含4个数字的列表（如 [3, 3, 8, 8]）

    #这个函数对字符串表达式求值并验证是否等于24
    def check(exp):
        try:
            #有可能会出现除0异常，所以放到异常处理结构中
            return abs(eval(exp) - 24) < 1.0e-6
        except:
            return False  #捕获除零等异常，返回False

    #全排列，枚举4个数的所有可能顺序
    for a in permutations(v):  #生成数字的所有排列顺序,如(3,8,3,8), (3,8,8,3)等
        for op in product(ops, repeat=3):  #生成3个运算符的所有可能组合
            exp_str = exps % (a[0], op[0], a[1], op[1], a[2], op[2], a[3])
            if check(exp_str):  #上一行: 填入: (a[0] op[0] a[1]) op[1] (a[2] op[2] a[3])
                print(exp_str)
```

```
lst = [randint(1, 14) for j in range(4)]
test24(lst)
```

习题

5.1 在函数内部可以通过关键字_____来定义全局变量。

5.2 如果函数中没有 return 语句或者 return 语句但不带任何返回值，那么该函数的返回值为_____。

5.3 包含_____语句的函数可以用来创建生成器对象。

5.4 下面代码的运行结果为_____。

```
def func(para):
    para = 3

n = 5
func(n)
print(n)
```

5.5 下面代码的运行结果为_____。

```
def func():
    para = 3

para = 5
func()
print(para)
```

习题

5.6 下面代码的运行结果为_____。

```
def func():
    global para
    para = 3

para = 5
func()
print(para)
```

5.7 下面代码的运行结果为_____。

```
def func():
    para[0] = 3
para = [5]
func()
print(para)
```

5.8 下面代码的运行结果为_____。

```
value = 3
def func(para=value):
    print(para)

value = 5
func()
```

习题

5.9 下面代码的运行结果为_____。

```
def func(para=[]):
    para.append(3)
    return para

func()
func()
print(func())
```

5.10 下面代码的运行结果为_____。

```
def func(a, b):
    return {b: a}

print(func(3, 5))
```

5.11 下面代码的运行结果为_____。

```
from operator import eq

s1 = 'Python'
s2 = 'python'
print(sum(map(eq, s1, s2)))
```

习题

5.12 下面代码的运行结果为_____。

```
def func(a, n):
    result, each = a, a
    for _ in range(n-1):
        each = each*10 + a
        result = result + each
        return result
print(func(2, 3))
```

5.13 下面代码的运行结果为_____。

```
def func():
    for i in range(10):
        if i > 3:
            return i
        yield i

r = func()
print(next(r), next(r))
```

习题

5.14 下面代码的运行结果为_____。

```
def func():
    for i in range(10):
        if i > 3:
            return i
        yield i

r = func()
print(next(r), *r)
```

5.15 下面代码的运行结果为_____。

```
def func():
    for i in range(10):
        if i > 3:
            return i
        yield i

r = func()
print(*r)
```

习题

5.16 假设已从标准库functools导入reduce()函数，那么表达式reduce(lambda x, y: x*y, [1, 2, 3])的值为_____。

5.17 假设已成功执行语句 from functools import reduce 和 from operator import or_，那么表达式 reduce(or_, [{1}, {2}, {3}]) 的值为_____。

5.18 假设已成功执行语句 from functools import reduce 和 from operator import and_，那么表达式 reduce(and_, [{1}, {2}, {3}]) 的值为_____。

5.19 已知函数定义

```
def func(a, b, c, *p):
    print(len(p))
```

那么语句 func(1, 2, 3, 4) 的输出结果为_____。

习题

5.20 下面代码的运行结果为_____。

```
x = 3
def modify():
    x = 5

modify()
print(x)
```

5.21 下面代码的运行结果为_____。

```
x = [3]
def modify():
    x[0] = 5
modify()
print(x)
```

5.22 单选题：下面代码的运行结果为（　　）。

```
x: int = [3]
print(x)
```

A. 3　　B. [3]

C. {3}　　D. 代码错误，无法运行

习题

5.23 单选题：下面代码的运行结果为（　　）。

```
def func():
    global x

func()
print(x)
```

A. 0　　B. []　　C. {}　　D. 出错抛出异常

5.24 单选题：已知函数定义 def func(a, b, c, *p): pass，那么通过语句 func(1, 2, 3, 4, 5) 调用函数时，在函数内部形参 p 的值为（　　）。

A. 4　　B. 5　　C. (4, 5)　　D. 出错抛出异常

5.25 多选题：关于函数的描述错误的有（　　）。

A. 只能使用关键字 def 定义函数，没有其他方式了

B. 函数属于可调用对象

C. Python 不支持嵌套定义函数

D. Python 程序必须有 main() 函数作为程序执行的入口

5.26 多选题：关于函数参数的描述正确的有（　　）。

A. 函数的形参在函数内可以作为局部变量直接使用

B. 如果在函数内修改了形参变量的引用，对应实参的引用也会被修改

C. 调用函数时是把实参的引用传递给形参

D. 定义函数时不需要声明形参的类型，Python 会根据实参的值自动推断形参的类型

习题

5.27 多选题：关于变量作用域的描述正确的有（　　）。

A. 在函数中可以直接使用已定义的全局变量的值

B. 在函数中使用已定义的全局变量的值必须先使用关键字 global 进行声明

C. 如果在函数中有局部变量与外部的全局变量同名，会优先使用全局变量

D. 在函数内试图修改已定义的全局变量的值必须先使用关键字 global 进行声明

5.28 判断对错：生成器函数的调用结果是一个确定的值。

5.29 判断对错：使用关键参数调用函数时，也必须记住每个参数的顺序和位置。

5.30 判断对错：已知不同的三个函数 A、B、C，在函数 A 中调用了 B，函数 B 中又调用了 C，这种调用方式称作递归调用。

5.31 判断对错：定义 Python 函数时，如果函数中没有 return 语句，则默认返回空值 None。

5.32 判断对错：如果在函数中有语句 return 3,那么该函数一定会返回整数 3。

5.33 判断对错：在函数内部没有任何声明的情况下直接为某个变量赋值，这个变量一定是函数内部的局部变量。

5.34 判断题：调用带有默认值参数的函数时，不能为默认值参数传递任何值，必须使用函数定义时设置的默认值。

5.35 判断题：lambda 表达式只能用来创建匿名函数，不能为这样的函数起名字。

第5章　函数

- 曼德博集合（Mandelbrot set）是一种在复平面上组成分形的点的集合

曼德博集合可以用复二次多项式来定义：

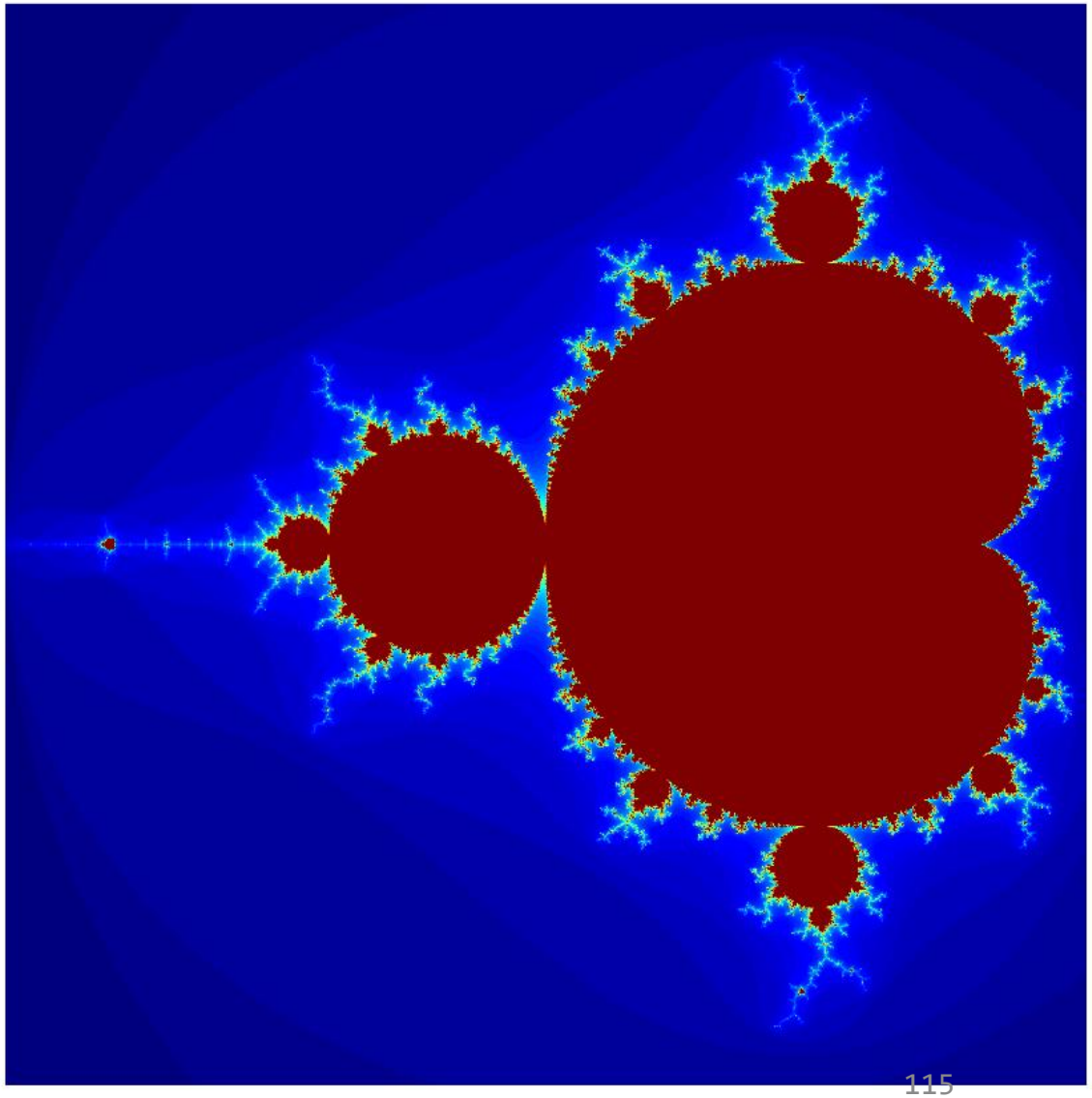
$$f_c(z) = z^2 + c$$

其中 c 是一个复数参数。

从 $z = 0$ 开始对 $f_c(z)$ 进行迭代：

$$z_{n+1} = z_n^2 + c, n = 0, 1, 2, \ldots$$

- 曼德博集合就是使迭代不发散的所有复数c的集合



第5章　函数

```
import numpy as np
from matplotlib import cm
import matplotlib.pyplot as plt

#1.核心函数
def iter_point(c):
    z = c
    # 最多迭代次数80或360
    for i in range(1, 80):
        if abs(z) > 2: # 半径大于2则认为发散
            break
        z = z * z + c
    return i # 返回迭代次数(用于颜色映射)

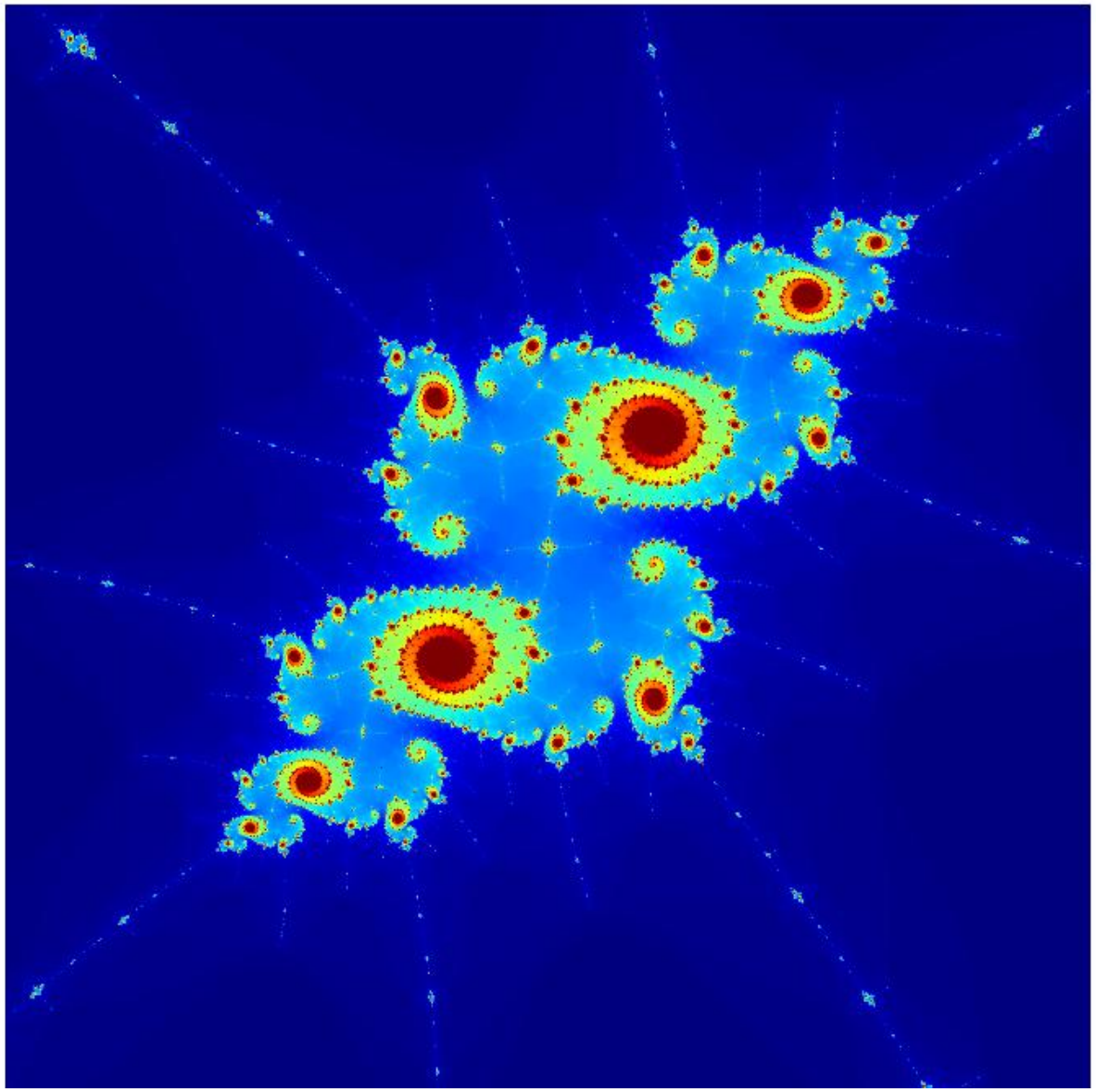
#2.参数设置
cx, cy, d = -0.75, 0, 1.25 #查看整个MSet
#cx, cy, d = -1.768620774, 0.002428273, 5e-6

fig = plt.figure()
```

```
#绘制点(cx, cy)附近正负d的范围的Mandelbrot
x0, x1, y0, y1 = cx-d, cx+d, cy-d, cy+d
y, x = np.mgrid[y0:y1:1000j, x0:x1:1000j]
                #创建1000*1000的网格
c = x + y * 1j #将实数网格转为复数网格

mandelbrot =
np.frompyfunc(iter_point,1,1)(c).astype(np.float)
   #np.frompyfunc将Python函数转换为numpy的ufunc，可以向量化操作

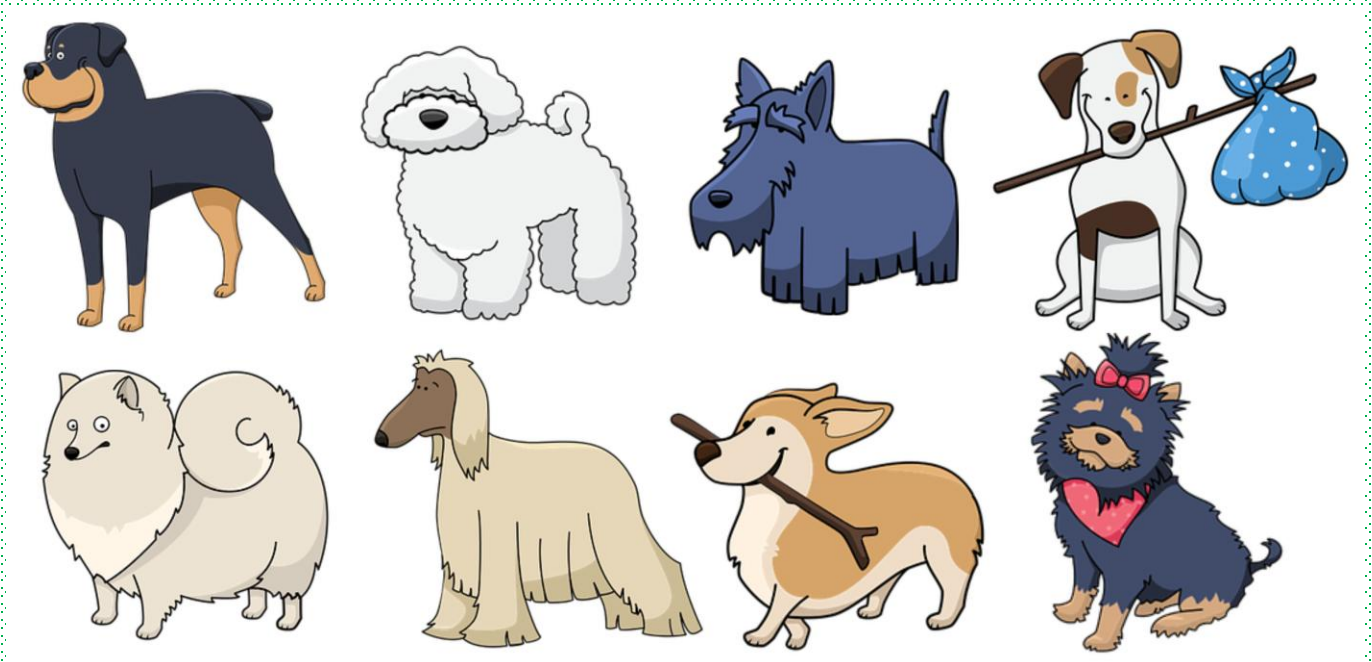
#可视化
plt.imshow(mandelbrot, cmap=cm.jet,
extent=[x0,x1,y0,y1])
plt.gca().set_axis_off()  #隐藏坐标轴
plt.show()
```



```
cx, cy, d = -1.768620774,
0.002428273, 5e-6
```

最大迭代次数：360
像素：2000 * 2000

第6章　面向对象程序设计



第6章　面向对象程序设计

面向过程的程序设计（Procedure-Oriented Programming）是关注**“如何做”(步骤)**的设计方法，围绕事件进行，分析出解决问题的步骤，划分任务并逐步实现功能。**面向过程的程序设计往往一个过程只能解决一个问题**，对于复杂问题难以进行设计，如果有需求变动，则会牵一发而动全身。**(执行者思维)**

面向对象是关注**“是什么”(对象)和“能做什么”(方法)**的设计方法。面向对象程序设计（Object-Oriented Programming,OOP）用于描述客观世界中的事物，每一个具体的事物都是一个对象，同类型的对象拥有共同的属性和行为，则形成了类，这一过程就是抽象。**抽象是面向对象程序设计的本质，类是其关键，类与对象是抽象与具体的对应。面向对象程序设计将数据(属性)和操作(方法)看成一个整体**，可以提高程序的开发效率，使得程序结构清晰，增加程序的可维护性，提高代码的可复用性。**（设计者思维）**

第6章　面向对象程序设计

■Python是面向对象的解释型高级动态编程语言，完全支持面向对象的基本功能，如封装、继承、多态以及对基类方法的覆盖或重写

□创建类时用变量形式表示对象**特征**的成员称为属性（attribute）[书：数据成员]

```
>>> x = 1 + 2j
>>> x.imag
```

□用函数形式表示对象**行为**的成员称为方法（method）[书：成员方法]

```
>>> x.conjugate()
```

□属性[数据成员]和方法[成员方法]统称为**类的成员**

注：python支持面向对象、面向过程、函数式编程等多种编程范式。

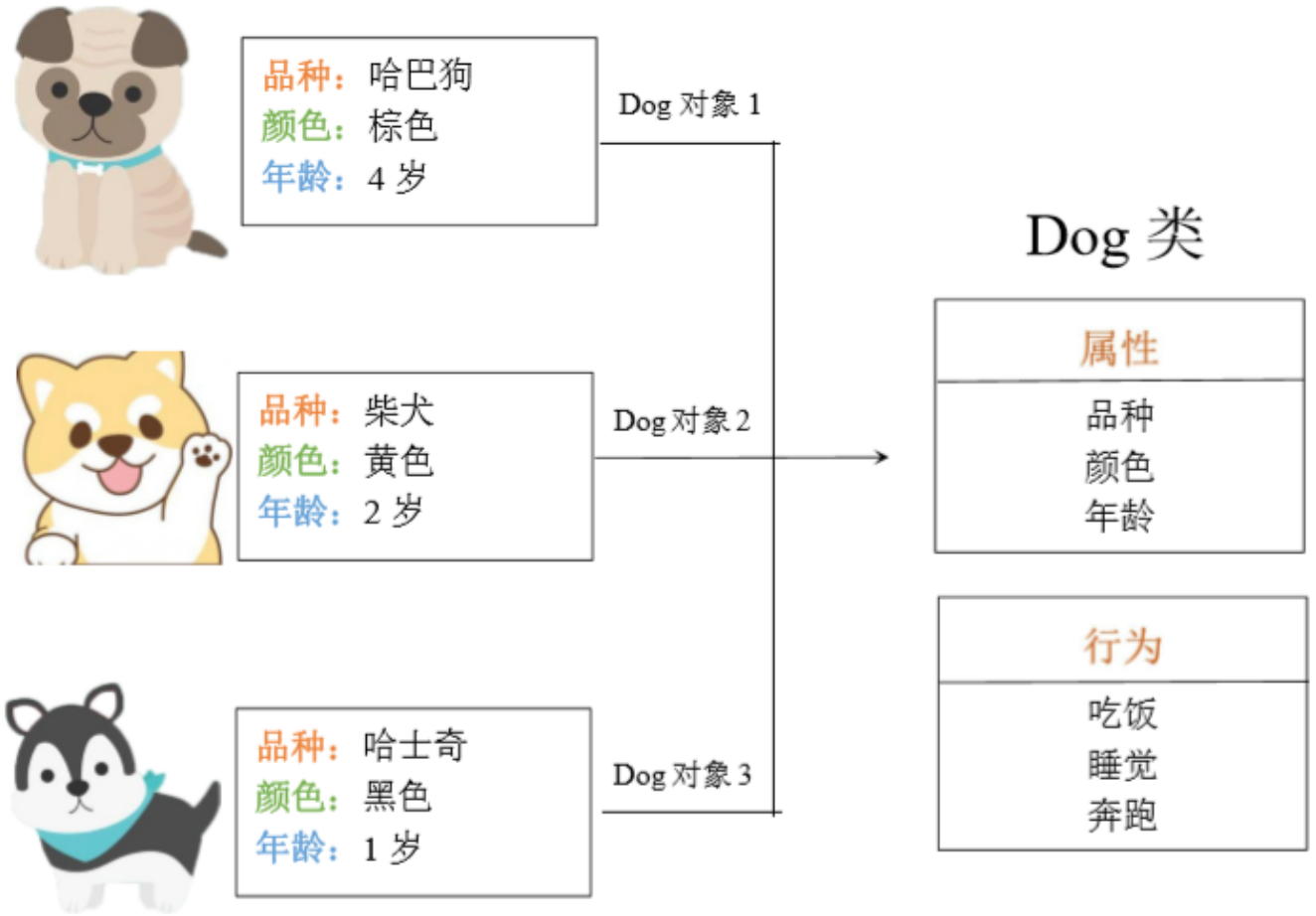
第6章代码复用技术(二)：面向对象程序设计

- 6.1 类和对象的概述
- 6.2 类的基本使用—实例属性和实例方法
- 6.3 类属性和类方法、静态方法
- 6.4 私有属性和私有方法
- 6.5 面向对象的三大特性
- 6.6 特殊方法与运算符重载

6.1 类和对象的概述

对象是类的一个实例，有属性和方法(行为)。

类则表示一个模板，用于描述一类对象的属性和方法(行为)。



6.1 类和对象的概述

有一家宠物医院，需要记录所有的宠物信息，要怎么记录呢？难道要每个都单独记录吗？

姓名：小巴
品种：哈巴狗
颜色：棕色
年龄：4 岁
可以吃 1 碗狗粮，睡觉时间约 8 小时，不喜欢跑。

姓名：小柴
品种：柴犬
颜色：黄色
年龄：2 岁
可以吃 1 碗狗粮，睡觉时间约 8 小时，喜欢跑。

姓名：小喵
品种：暹逻猫
颜色：黑白
年龄：1 岁
可以吃 1 碗猫粮，睡觉时间约 8 小时，喜欢玩猫玩具。

6.1 类和对象的概述

面向对象程序设计的思想**需要以对象来思考问题**，将现实中的实体抽象为对象，并考虑这个对象对应的**属性和方法(行为)**。

根据面向对象程序设计的思想，可以将此宠物信息的记录问题分解以下步骤。

- 首先**从此问题中抽象出对象**，对象是宠物医院的每一只宠物。
- 然后**识别出对象的属性**，比如每一只宠物都有姓名、品种、颜色以及年龄。
- 最后**识别对象的动态行为(方法)**。

6.1 类和对象的概述

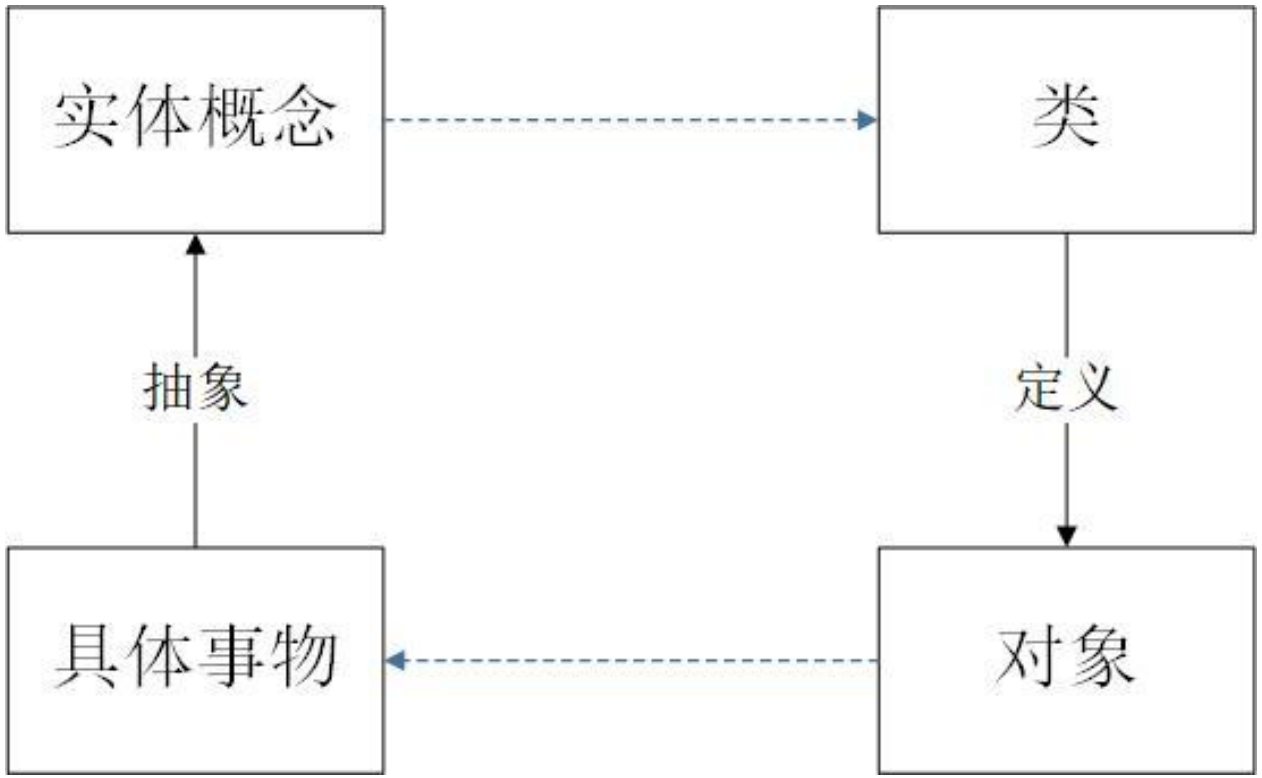
以面向对象的方式记录宠物信息

类别：狗						
属性				行为		
姓名	品种	颜色	年龄	吃狗粮（碗）	睡觉（小时）	跑步（喜欢√，不喜欢则不填）
小巴	哈巴狗	棕色	4	1	8	
小柴	柴犬	黄色	2	1	8	√

类别：猫						
属性				行为		
姓名	品种	颜色	年龄	吃猫粮（碗）	睡觉（小时）	猫玩具(喜欢√,不喜欢则不填)
小喵	暹逻猫	黑白	1	1	8	√

6.1 类和对象的概述

类实质上就是封装对象属性和方法(行为)的载体（模板），而对象则是类中具体的一个实例。



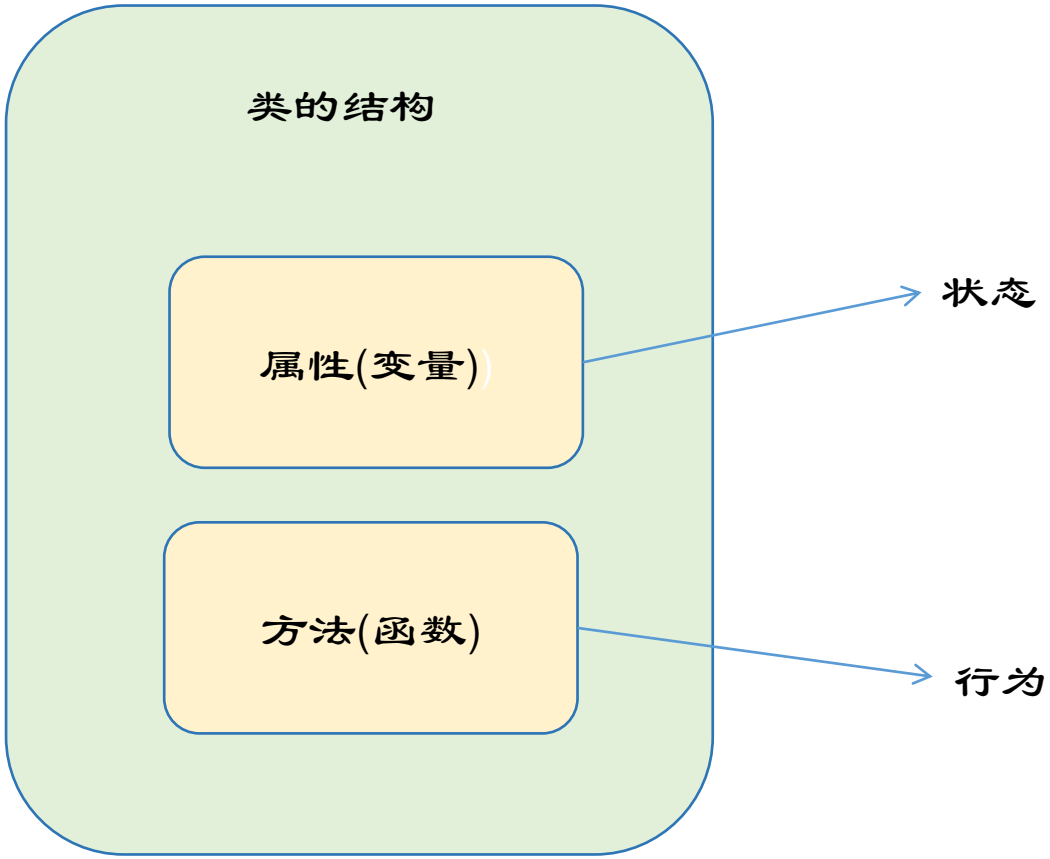
前面学习的可变对象和不可变对象，都是包含了各自方法的对象。

第6章代码复用技术(二)：面向对象程序设计

- 6.1 类和对象的概述
- **6.2 类的基本使用—实例属性和实例方法**
- 6.3 类属性和类方法、静态方法
- 6.4 私有属性和私有方法
- 6.5 面向对象的三大特性
- 6.6 特殊方法与运算符重载

6. 2 类的基本使用

6.2.1 类的定义



6. 2 类的基本使用

6.2.1 类的定义

定义类的语法是个如下：

```
class 类名：
    类体
```

1.类名必须符合”标识符”的规则，一般规定，首字母必须大写，多个单词使用”驼峰原则”；

2. 类体中可以定义属性和方法；

3. 属性用来描述数据，方法(函数)用来描述这些数据的相关操作。

6.2 类的基本使用

6.2.1 类的定义

```
class Dog:   #类名首字母大写
    def __init__(self,name,breed,age):    #self必须位于第一个参数  #__init__构造方法
        """初始化属性name、breed和age"""
        self.name = name                    #实例属性
        self.breed = breed                  #self.实例属性名=初始值
        self.age = age
    def eat(self):                          #实例方法
        """小狗正在吃狗粮"""                  #self必须位于第一个参数
        print(f"{self.name}正在吃狗粮")
    def run(self):                         #实例方法
        """小狗正在奔跑"""                    #self必须位于第一个参数
        print(f"{self.name}在奔跑玩耍")
```

6. 2 类的基本使用

6.2.1 类的定义

1.构造方法__init__()

- 构造方法(**特殊方法**)一般用于类的**初始化**操作，它包含了实例**属性**，在创建实例对象时被**自动**调用和执行。
- 初始化程序的名字是固定的，只能写成“__init__”。因此，只要保证它出现在类定义语句块的里面就行，至于是块中的第1个方法，还是第几个方法，完全无关紧要；甚至如果不需要传递什么参数，在类的定义中不编写初始化程序也没有什么关系。

2.self的作用

- **self**指当前的**实例对象**。
- 实例属性定义时，左边的self.name是实例属性名(变量)，右边是初始值name(实参)。原则上，左边可写为其他变量名，如self.nm。但习惯上，把二个写同名。

3.实例方法

- 第一个参数必须为self，调用实例对象时，不需要也不能给self传参。self由解释器自动传参。

6.2 类的基本使用—实例属性和实例方法

6.2.2 创建实例对象

```
class Dog:
    def __init__(self,name,breed,age):
        """初始化属性name、breed和age"""
        self.name = name
        self.breed = breed
        self.age = age
    def eat(self):
        """小狗正在吃狗粮"""
        print(f"{self.name}正在吃狗粮")
    def run(self):
        """小狗正在奔跑"""
        print(f"{self.name}在奔跑玩耍")
d1 = Dog("小巴","哈巴狗",4)        #创建实例对象，自动调用和执行__init__()
                                   #self无需也不能传参，解释器自动将d1的地址传给它
```

类的使用与函数类似，当定义了一个类时，其中的代码不会被执行，**当调用类来创建对象时，类中的代码才真正起作用。**

6.2 类的基本使用—实例属性和实例方法

6.2.2 创建实例对象-**访问属性**

访问属性，需要使用**“实例名.属性”**的方式

```
class Dog:
    """类的定义与前文一致，在此处省略部分代码"""
d1 = Dog("小巴","哈巴狗",4)          #创建实例对象
print(f"狗狗的姓名是{d1.name}")      #访问name属性
print(f"狗狗的品种是{d1.breed}")     #访问breed属性
print(f"狗狗的品种是{d1.age}")       #访问age属性
```



```
狗狗的姓名是小巴
狗狗的品种是哈巴狗
狗狗的品种是4
```

6.2 类的基本使用—实例属性和实例方法

6.2.2 创建实例对象-**调用实例方法**

创建实例对象后，可以调用类中的实例方法，形式是“**实例名.实例方法**”

```
class Dog:
    """类的定义与前文一致，在此处省略部分代码"""
d1 = Dog("小巴","哈巴狗",4)      #创建实例对象
d1.eat()                         #调用eat()方法，无需也不能给self传参
d1.run()                         #调用run()方法，self由解释器自动传参
```



```
小巴正在吃狗粮
小巴在奔跑玩耍
```

6.2 类的基本使用—实例属性和实例方法

6.2.2 创建实例对象-**调用实例方法**

```
1. class Dog:
2.     """类的定义与前文一致，在此处省略部分代码"""
3. d1 = Dog("小巴","哈巴狗",4)      #创建实例对象
4. d1.eat()                          #调用eat()方法，无需也不能给self传参
5. Dog.eat(d1)                       #等效调用，解释器会将第4行翻译成第5行执行
6.                                   # 即解释器自动将d1传参给self
```



```
小巴正在吃狗粮
小巴正在吃狗粮
```

6.2 类的基本使用—实例属性和实例方法

6.2.2 创建实例对象-**创建多个实例对象**

可以创建**多个实例对象**，每个实例对象之间相互独立，有自己的属性，且都可以调用类中的方法

```
class Dog:
    """类的定义与前文一致，在此处省略部分代码"""
d1 = Dog("小巴","哈巴狗",4)      #创建一个实例对象d1
print(f"{d1.name}的品种是{d1.breed}，年龄是{d1.age}")
d1.eat()
d2 = Dog("小柴","柴犬",1)        #创建一个实例对象d2
print(f"{d2.name}的品种是{d2.breed}，年龄是{d2.age}")
d2.eat()
```

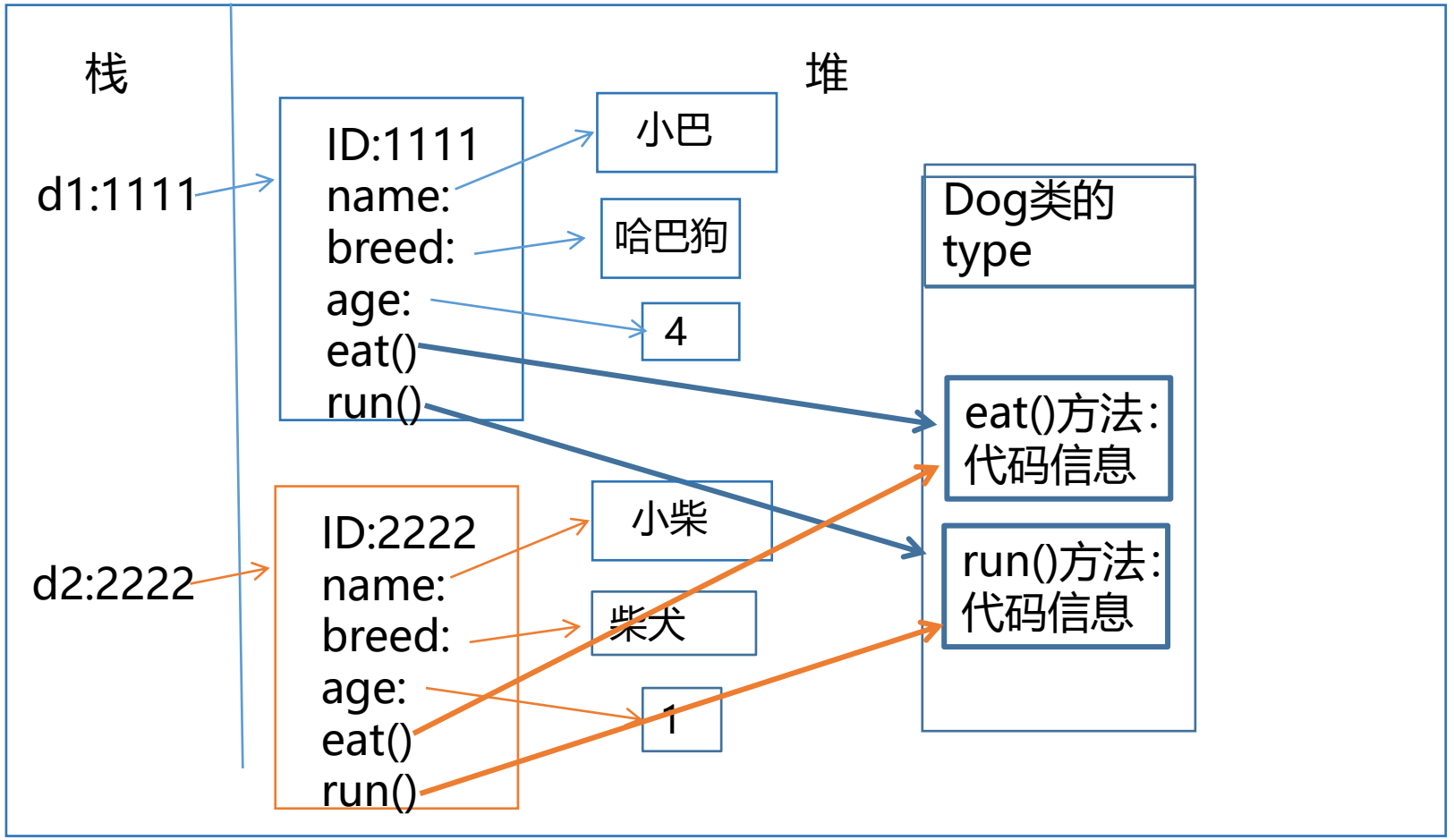


```
小巴的品种是哈巴狗，年龄是4
小巴正在吃狗粮
小柴的品种是柴犬，年龄是1
小柴正在吃狗粮
```

6.2 类的基本使用—实例属性和实例方法

6.2.2 创建实例对象-**创建多个实例对象**

从一个类创建对象时，每个对象会共享这个类的行为(类中定义的方法)，但拥有自己的属性值(不共享状态)，即“**方法代码是共享的，属性数据不共享**”。



6.2 类的基本使用—实例属性和实例方法

6.2.3 设置属性的默认值

创建Student类，并将age属性设置为默认值19

```
1. class Student:
2.     def __init__(self,name,id,age=19):
3.         self.name = name
4.         self.id = id
5.         self.age = age
6. s1 = Student("小明",202401)
7. print(f"{s1.name}的年龄是{s1.age}")
```

小明的年龄是19

如果不想使用实例属性self.age的默认值19，可以在第6行创建实例对象时，传入3个参数。

```
s1 = Student("小明",202401,20)
```

给age属性**设置默认值**，**还可以通过在构造方法内部给age赋值的形式**，将第2~5行代码改为如下形式。

```
def __init__(self, name, id): #这里无age
    self.name = name
    self.id = id
    self.age = 19      #可以动态添加新属性（Python特性）
                       #可通过s1.__dict__查看所有属性
```

6.2 类的基本使用—实例属性和实例方法

6.2.4 修改属性的值

创建Student类

```
class Student:
    def __init__(self,name,id):
        self.name = name
        self.id = id
        self.age = 19  #可以动态添加新属性（Python特性）
```

如果在Student类的构造方法内部直接给实例属性赋值，那如何修改此属性的值呢?

6.2 类的基本使用—实例属性和实例方法

6.2.4 修改属性的值-**直接修改属性的值**

通过给实例对象的属性进行直接赋值的方式，修改实例对象的值

```
class Student:
    def __init__(self,name,id):
        self.name = name
        self.id = id
        self.age = 19                #动态添加新属性
s1 = Student("小明",202401)          #创建一个实例对象
print(f"{s1.name}的年龄初始为{s1.age}")
s1.age = 20                          #修改实例对象的属性age
print(f"修改{s1.name}的年龄为{s1.age}")
print(s1.__dict__)                   #查看所有属性
del s1.age                           #删除属性
print(s1.__dict__)
```

```
小明的年龄初始为19
修改小明的年龄为20
{'name': '小明', 'id': 202401, 'age': 20}
{'name': '小明', 'id': 202401}
```

6.2 类的基本使用—实例属性和实例方法

6.2.4 修改属性的值-**通过方法修改属性的值**

在类中写一个方法，通过调用此方法，来修改属性的值

```
class Student:
    def __init__(self,name,id):
        self.name = name
        self.id = id
        self.age = 19
    def update_age(self,age):
        """用于修改属性age的值"""
        self.age = age
s1 = Student("小明",202401)          #创建实例对象
print(f"{s1.name}的年龄是{s1.age}")
s1.update_age(20)                    #调用修改年龄的方法
print(f"{s1.name}的年龄修改为{s1.age}")
```



```
小明的年龄是19
小明的年龄修改为20
```

通过方法来修改属性，而不是直接访问，体现了面向对象编程的封装思想，

6.2 类的基本使用—实例属性和实例方法

6.2.4 修改属性的值-**通过方法修改属性的值**

- 通过方法来修改属性，而不是直接访问，体现了面向对象编程的封装思想！

- 为什么 update_age 比直接赋值更好?

  - ✓ 封装性：隐藏实现细节，只暴露必要接口
  - ✓ 可扩展性：未来可以轻松添加验证、日志、通知等逻辑
  - ✓ 可维护性：修改逻辑只需在一个地方
  - ✓ 安全性：可以控制数据的有效性

- 后面学习@property修饰器将进一步实现封装

类比：

- 以前你要调空调温度，得走到空调前按按钮（直接修改属性）。
- 现在你可以说："小爱同学，调到26度"（调用方法）。
- 智能音箱会：
  - 检查：26度是否合理？（验证）
  - 记录：记录这次温度调整（日志）
  - 通知：如果从制热切到制冷，提醒你（业务逻辑）
  - 执行：最后才真正调整温度

6.2 类的基本使用—实例属性和实例方法

总结:

- Python中所有**实例属性**和**实例方法**默认都是**公有**的，即[书]**公有成员**
- 公有成员是可以**公开使用**的，既可以在类的**内部**进行访问，也可以在**外部**程序中使用
- 实例属性和实例方法可以在运行时灵活地**创建、修改和管理。**
- 注意：__xxx__(以两个下划线开头且结束)：系统定义的类的**特殊成员**，也是公有成员；如构造方法__init__()

Practice

- 从fractions库中使用Fraction(1, 2)创建一个分数对象时，参数1和2传递给了类Fraction的哪个方法？在类的定义中，该方法的第一个参数是什么？

✓ 传递给了__init__

✓ self，代表当前对象

```
>>> from fractions import Fraction
>>> print(type(Fraction))
<class 'abc.ABCMeta'>

>>> print(type(str))
<class 'type'>
```

- 在Python中，所有类都是由元类（metaclass） 创建的
- 元类：Python面向对象的终极武器(元类 = 类的类 = 制造类的工厂)
- 内置基础元类：type是Python最基础的元类
- 标准库中的核心元类：abc.ABCMeta - 抽象基类元类

第6章代码复用技术(二)：面向对象程序设计

- 6.1 类和对象的概述
- 6.2 类的基本使用—实例属性和实例方法
- **6.3 类属性和类方法、静态方法**
- 6.4 私有属性和私有方法
- 6.5 面向对象的三大特性
- 6.6 特殊方法与运算符重载

6. 3 类属性和类方法、静态方法

6.3.1 类对象

我们在前面讲类定义格式中，“class 类名：”，实际上，当解释器执行class语句时，就会创建一个**类对象**, 比如下面的 **Room 和 Book**。

```
class Room:
    pass            #空语句

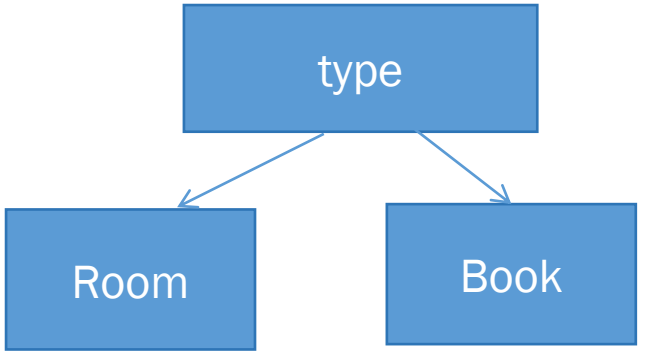
class  Book:
    pass

print(type(Room))
print(id(Room))

print(type(Book))
print(id(Book))
```



```
<class 'type'>
2443717714832
<class 'type'>
2443717704096
```



6.3 类属性和类方法、静态方法

6.3.2 类属性

类属性是从属于“类对象”的属性，也称为“类变量”。

- 对于类属性，可以通过“**类名.类属性**”进行直接**访问或修改。**
- 类属性是该类**所有对象共享**的，不属于任何一个对象；**但不常用。**

```
class Book:
    number = 0                  #创建一个类属性number
    def __init__(self,name):
        self.name = name        #self.name是实例属性
print(Book.number)              #直接用类名访问类属性
Book.number += 1                #修改类属性
print(Book.number)
```

```
0
1
```

6.3 类属性和类方法、静态方法

6.3.2 类属性

- **通过实例对象也可以访问类属性。**
- 当实例对象访问属性时，首先会查找是否有**同名的实例属性**，如果没有，则会去查找是否有**同名类属性**，如果再找不到，就会*抛出异常*。

```
class Book:
    number = 0                          #定义类属性number
    def __init__(self,name):
        self.name = name
b1 = Book("《红楼梦》")                  #创建实例对象b1
print(b1.number)
```



```
0
```

6.3 类属性和类方法、静态方法

6.3.2 类属性

实例对象不能对类属性进行修改，只有类对象能修改类属性的值。

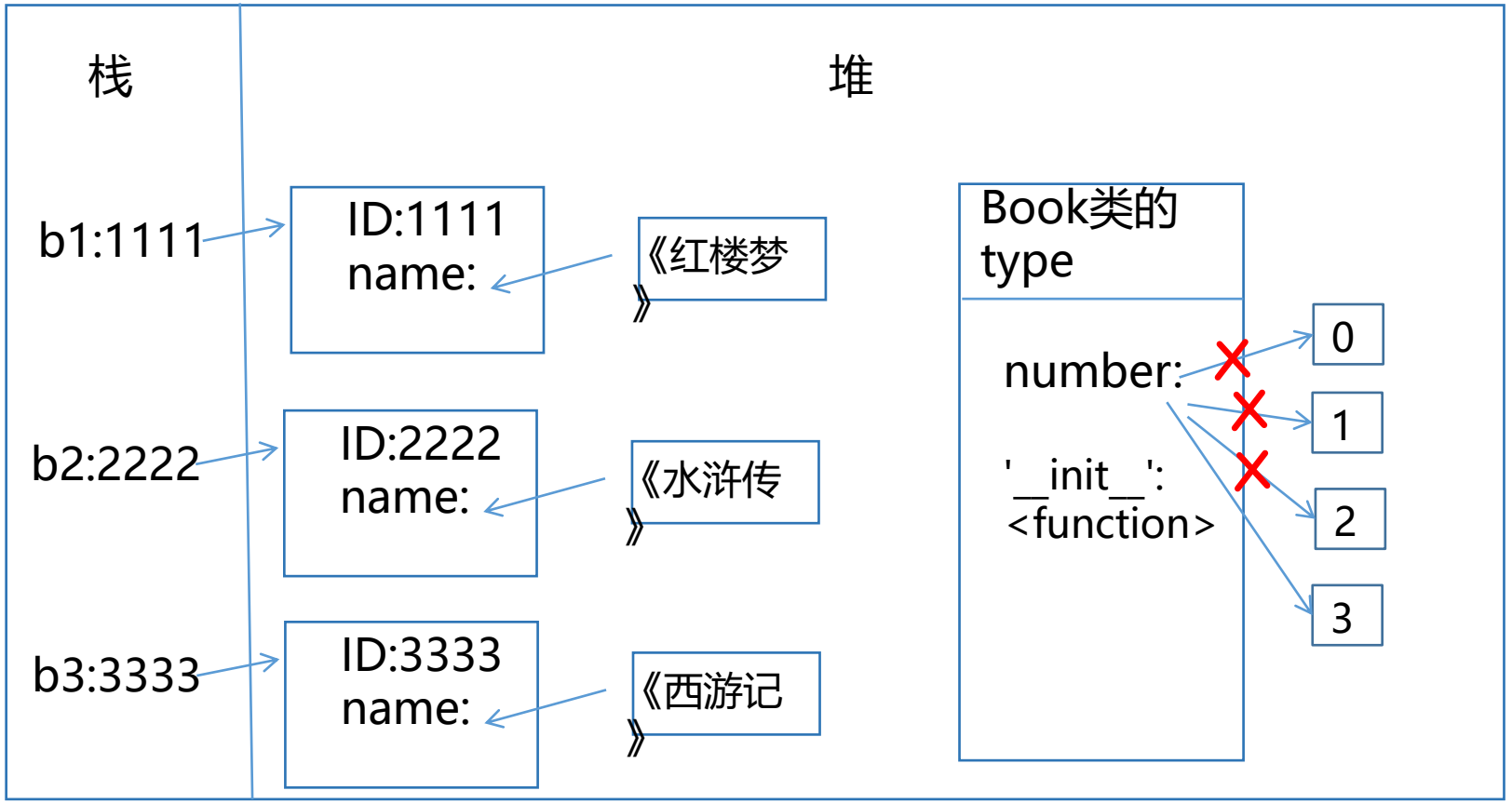
```
class Book:
    number = 0                  #创建一个类属性number
    def __init__(self,name):
        self.name = name
        Book.number += 1        #通过类对象修改类属性number增加1
b1 = Book("《红楼梦》")          #这里Book.number在__init__()内部
print(b1.number)                #每次创建新实例时，__init__ 方法执行Book.number += 1
b2 = Book("《水浒传》")          #注意：如果Book.number在__init__()外部，不会执行 +1
print(b2.number)
b3 = Book("《西游记》")
print(b3.number)
print("书籍的总数是：",Book.number)
```

```
1
2
3
书籍的总数是： 3
```

6.3 类属性和类方法、静态方法

6.3.2 类属性

实例对象不能对类属性进行修改，只有**类对象能修改**类属性的值。



6.3 类属性和类方法、静态方法

6.3.3 类方法

在类中可以定义**类方法**，通过装饰器**@classmethod**进行定义。

- 类方法不属于任何实例，不会绑定任何实例。
- 调用类方法可以用**类名**也可以用**实例对象**，它**可以**访问**类属性(方法)**，但**不能**访问**实例属性(方法)**。

```
class 类名:
    @classmethod
    def 类方法名(cls [,形参列表]):   #第一个参数cls必须有，cls指类对象本身(类名)
        方法体                       #不需要也不能给cls传值
```

6.3 类属性和类方法、静态方法

6.3.3 类方法

计算员工的总数

```
class Employee:
    number = 0                          #类属性用于计算员工的总数
    def __init__(self,name):
        self.name = name
        Employee.number += 1            #类属性number加1
    @classmethod
    def count_num(cls):                 #类方法用于打印类属性
        print("员工个数：",cls.number)  #cls表示类本身，即Employee
e1 = Employee("张三")                   #类方法只能调用类属性
e2 = Employee("李四")
e3 = Employee("王五")
e1.count_num()                          #用实例对象调用类方法
Employee.count_num()                    #用类名调用类方法
Employee.number                         #也可直接通过类访问类属性
```



```
员工个数： 3
员工个数： 3
3
```

6.3 类属性和类方法、静态方法

6.3.4 静态方法

静态方法需要用装饰器**@staticmethod**进行定义，静态方法不同于实例方法，**不需要填写形参self**；也不同于类方法，**不需要填写形参cls**。

- 与类方法相同，静态方法不属于任何实例，不会绑定任何实例。
- 与类方法相同，静态方法可以通过**类名**或**实例对象**进行调用，它**可以**访问**类属性(方法)**，**不能**访问**实例属性(方法)**。

```
class 类名:
    @staticmethod
    def 静态方法名([形参列表]):
        方法体
```

6.3 类属性和类方法、静态方法

6.3.4 静态方法

```
class Employee:
    number = 0                          #类属性用于计算员工的总数
    def __init__(self,name):
        self.name = name
        Employee.number += 1            #类属性number加1
    @staticmethod
    def count_num():                    #静态方法用于打印类属性
        print("员工个数：",Employee.number) #静态方法只能访问类属性
e1 = Employee("张三")
e2 = Employee("李四")
e3 = Employee("王五")
e1.count_num()                          #用实例对象调用静态方法
Employee.count_num()                    #用类名调用静态方法
```



```
员工个数： 3
员工个数： 3
```

6.3 类属性和类方法、静态方法

用静态方法：

- 纯粹的实用工具函数
- 不依赖类状态的辅助函数
- 逻辑上与类相关，但不需要类上下文

用类方法：

- 需要创建类实例
- 需要访问或修改类属性
- 实现工厂模式
- 方法需要在子类中被重写并改变行为

```
class Date:
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day

    @classmethod
    def from_string(cls, date_string):
        # 需要cls来创建实例
        year, month, day = map(int, date_string.split('-'))
        return cls(year, month, day)

    @staticmethod
    def is_valid_date(year, month, day):
        # 不需要cls或实例，只是验证逻辑
        if month < 1 or month > 12:
            return False
        return True

date1 = Date(2025, 12, 15)          # 传统方式
date2 = Date.from_string("2025-12-15")  # 工厂方法方式

print(f"date1: {date1.year}-{date1.month}-{date1.day}")
print(f"date2: {date2.year}-{date2.month}-{date2.day}")
```

6.3 类属性和类方法、静态方法

总结:

- Python中所有**类属性和类方法、静态方法**默认也都是**公有**的，即[书]**公有成员**
- 公有成员是可以**公开使用**的，既可以通过**类对象**进行访问，也可以通过**实例对象**访问；既可以在**类内部**访问，也可以在**类外部**访问。
- 只有**类对象能修改类属性**的值，**实例对象不能**对**类属性**进行**修改**
- 类方法和静态方法**只能**访问**类属性(方法)**，**不能**访问**实例属性(方法)**

Practice

- 什么是类方法和静态方法，其区别是什么？

✓ 通过修饰器@classmethod和@staticmethod声明的方法分别为类方法和静态方法

✓ 类方法一般以cls作为类方法的第一个参数表示该类自身；静态方法则无此要求，可以不接收任何参数

第6章代码复用技术(二)：面向对象程序设计

- 6.1 类和对象的概述
- 6.2 类的基本使用—实例属性和实例方法
- 6.3 类属性和类方法、静态方法
- **6.4 私有属性和私有方法**
- 6.5 面向对象的三大特性
- 6.6 特殊方法与运算符重载

6.4 私有属性和私有方法

- 前面讲的公有属性和公有方法不利于类的封装原则：数据使用的安全性。Python提供了一种自我保护办法: **私有属性**和**私有方法**(统称[书]私有成员)。
- 私有成员一般在**类的内部**进行访问和操作，在类的外部(外部程序以及子类对象)不能直接访问
- Python不存在严格意义的私有成员，类外部可以通过”**对象名._类名__私有成员名**”访问(Python 遵循“我们都是负责任的成年人”原则。它不强制限制访问，而是通过约定来规范行为。)
- 约定，**私有成员**以两个下划线开头：**__xxx**，但不以两个下划线结束
- 注: **_xxx**(以单个下划线开头)是**约定的私有(保护)成员**，Python不强制执行，全靠开发者自觉，其行为可能随着版本变化

6.4 私有属性和私有方法

6.4.1 私有属性

在Python中**在属性前加两个下划线**可以定义**私有属性**。所谓私有属性，是指不允许外界进行访问，**只能在类内访问**。

私有属性不能在类外进行访问

```
class Room:
    def __init__(self,name):      #左右两边各有两个下划线是特殊方法
        self.__name = name         #定义私有属性__name
r1 = Room("房间1")                 #创建一个实例对象
print(r1.__name)                   #类外部访问私有属性
```



```
AttributeError: 'Room' object has no attribute '__name'
```

6.4 私有属性和私有方法

6.4.1 私有属性

那么如何在类内访问私有属性呢?

```
class Room:
    def __init__(self,name):
        self.__name = name          #定义私有属性__name
    def print_name(self):           #定义实例(公有)方法访问私有属性
        print(f"房间的名称为{self.__name}")
r1 = Room("房间1")                  #创建一个实例对象
r1.print_name()                     #通过实例(公有)方法访问私有属性
```



```
房间的名称为房间1
```

6.4 私有属性和私有方法

6.4.1 私有属性

Python**并没有从语法上严格的保证属性是私有的**，只是给私有属性换了一个名称，来**妨碍**对私有属性的访问。如果知道私有属性的命名规则，仍然可以访问到它们，命名规则为“**对象名._类名__属性**”。

```
class Room:
    def __init__(self,name):
        self.__name = name        #定义私有属性__name
r1 = Room("房间1")                #创建一个实例对象
name = r1._Room__name             #访问Room中的私有属性，并赋值给name
print(name)
```



```
房间1
```

6.4 私有属性和私有方法

6.4.2 私有方法

私有方法的定义与私有属性类似，**以两个下划线开头进行定义**，**在类外不能进行访问**。与私有属性类似，**Python也没有在语法上严格保证方法是私有的**，在类外也能访问私有方法，访问方式是“**对象名._类名__方法**”。

```
class Employee:
    def __init__(self,name,year,performance):
        self.__name = name                          #定义私有属性姓名
        self.__year = year                          #定义私有属性工作年限
        self.__performance = performance            #定义私有属性绩效
    def __cal_salary(self):                         #定义私有方法来访问私有属性
        """用于计算工资"""
        salary = self.__year*2000 + self.__performance
        return salary
    def print_salary(self):                         #自定义实例(公有)方法访问私有方法
        print(f"{self.__name}的工资是{self.__cal_salary()}")
e1 = Employee("张三",2,3000)
e1.print_salary()                                   #类外可访问实例方法，但不能访问私有方法
```



```
张三的工资是7000
```

6.4 私有属性和私有方法

总结:

- 私有属性和私有方法都是**私有**的，即[书]**私有成员**
- 私有成员一般在类的内部进行访问和操作，在类的外部(外部程序以及子类对象)不能直接访问
- Python不存在严格意义的私有成员，类外部可以通过”**对象名._类名__私有成员名**”访问
- 私有属性和私有方法是实现类的**封装**的工具

Practice

- 在一个类的定义中，定义了名为__add__, _add, __test的方法，请问哪些是公有成员，哪些是私有成员？

✓ __add__公有(特殊成员)，_add约定私有，__test私有

6.4 私有属性和私有方法

```
class Example:
    def __init__(self):
        self.public = 1      # 公开
        self._protected = 2  # 保护（约定）
        self.__private = 3   # 私有

    def get_private(self):
        return self.__private  # 私有属性只能内部访问

obj = Example()
print(obj.public)       # ✅ 1
print(obj._protected)    # ⚠2（不推荐）
# print(obj.__private)   # ❌ AttributeError
print(obj._Example__private)  # ⚠3（强烈不推荐！破坏封装）
```

6.4 私有属性和私有方法

```
class Example:
    def public_method(self):
        print("公开方法")

    def _protected_method(self):  # 这只是约定
        print("保护方法")

    def __private_method(self):   # 名称改编，稍强的"私有"
        print("私有方法")

obj = Example()
obj.public_method()       # ✅ 1
obj._protected_methond()   # ⚠2（仍然可以调用！只是约定不要调用）
#obj.__private_methond()   # ❌ AttributeError
```

小结：

属性/方法权限	实例方法	类方法	静态方法	私有(实例)方法
访问实例属性	✅ 直接：**self.**attr	❌ 不能直接访问	❌ 不能访问	✅ 同实例方法
修改实例属性	✅ 直接：**self.**attr = value	❌ 不能直接修改	❌ 不能修改	✅ 同实例方法
访问类属性	✅ 通过类名：**MyClass.**attr	✅ 直接：**cls.**attr	✅ 通过类名：**MyClass.**attr	✅ 同实例方法
修改类属性	✅ 通过类名：**MyClass.**attr = value	✅ 直接：**cls.**attr = value	✅ 通过类名：**MyClass.**attr = value	✅ 同实例方法
访问私有属性	✅ 通过名称修饰：**c1.**_ClassName__attr			
修改私有属性	✅ 通过名称修饰：**c1.**_ClassName__attr = value			
主要用途	**操作实例数据**，最常见的类型	**操作类级别的数据**，工厂方法	**工具函数**，与类相关但不依赖实例或类状态	**内部实现细节**，名称会被修饰
定义方式	def method(self)	@classmethod def method(cls)	@staticmethod def method()	def __private(self)

小结：

重要原则总结：

- **实例方法**是**唯一**可以**直接**访问**实例属性**的方法类型
- **类方法和静态方法**都**无法直接**访问**实例属性**
- **所有方法**类型都可以访问**类属性**（通过不同的方式）
- 私有属性对所有方法都是可访问的（通过名称修饰）
- 私有(实例)方法与实例方法在权限上没有区别，只是名称被修饰了

作业

1.设计一个学生类Student,包括：

(1). 作为属性的姓名name、年龄age和成绩score.

(2). 实现实例方法isPassed方法，当score大于等于60分则返回True，否则返回False。

(3). 生成实例对象姓名：小明，年龄19岁，成绩80分，并调用isPassed方法进行score判断。

2.设时间类Time的部分定义如下：在方法__init__内定义了三个数据成员_h、_m和_s表示时间的时、分和秒。

(1). 为类Time写一个类方法from_str，以字符串s为参数。当s是类似'10:5:20'的字符串时，返回相应的对象Time(10, 5, 20)。

(2). 为类Time写一个实例方法is_time，若Time对象的分和秒在0到59之间，则返回True，否则返回False。

第6章代码复用技术(二)：面向对象程序设计

- 6.1 类和对象的概述
- 6.2 类的基本使用—实例属性和实例方法
- 6.3 类属性和类方法、静态方法
- 6.4 私有属性和私有方法
- **6.5 面向对象的三大特性**
- 6.6 特殊方法与运算符重载

6.5 面向对象的三大特性

6.5 面向对象的三大特性

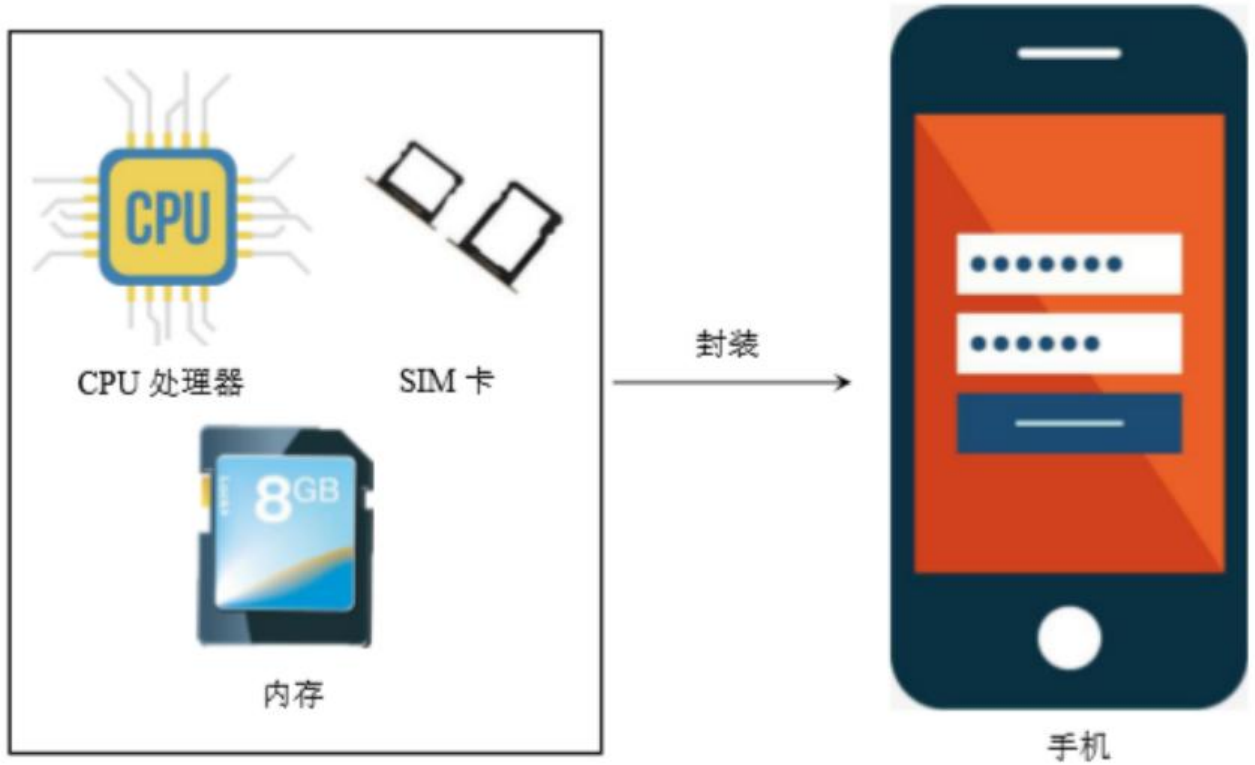
面向对象程序设计的主要特征可以概括为:

- **封装**
- **继承**
- **多态**

6.5 面向对象的三大特性

6.5.1 面向对象的三大特性-**封装**

封装(encapsulation)是面向对象程序设计的核心思想，将对象的属性和方法封装起来，避免了外界**直接访问**对象而造成的过度依赖，也阻碍了外界修改对象的内部数据而可能引发的难以预见的结果。



6.5.1 面向对象的三大特性——封装

6.5.1.1 封装的概念

私有属性和私有方法都属于Python**封装的形式**，通过在属性和方法前加上两个下划线的方式，**阻碍类外**对**类内**属性和方法的访问和修改。

但是Python中的私有属性和私有方法不是完全私有化的，是一种通过**改变名称**而**阻碍类外访问**的方式，实际上，还是可以通过“**_类名__属性名**”或者“**_类名__方法名**”的形式对其进行访问。

6.5.1 面向对象的三大特性——封装

6.5.1.1 封装的概念

定义Student类的私有属性和私有方法

```
class Student:
    def __init__(self,score):
        self.__score = score    #私有属性
    def __get_score(self):    #私有方法
        return self.__score
s1 = Student(100)
print(s1._Student__score)                #访问私有属性__score
print(s1._Student__get_score())          #访问私有方法__get_score()
```

私有属性和私有方法不能直接访问，否则就会报错。通过加上类名的形式进行访问，能够成功访问，但是并**不建议这种访问方式**。

```
100
100
```

6.5.1 面向对象的三大特性——封装

6.5.1.2 @property的使用

@property是Python内置的装饰器，可以将**方法调用**变成“**属性访问**”，**实现修改属性的读写权限(只能引用值，不能修改)**(外界无法直接修改，实现封装属性)

```
class Score:
    def __init__(self,data):
        self.__data = data          #定义私有属性__data，外部无法直接访问
    @property                       #使用@property将私有属性变为只读的
    def data(self):
        return f"私有属性的值为{self.__data}"
s1 = Score(60)
print(s1.data)                      #方法调用改为属性访问，私有属性变为(公有)只读
```

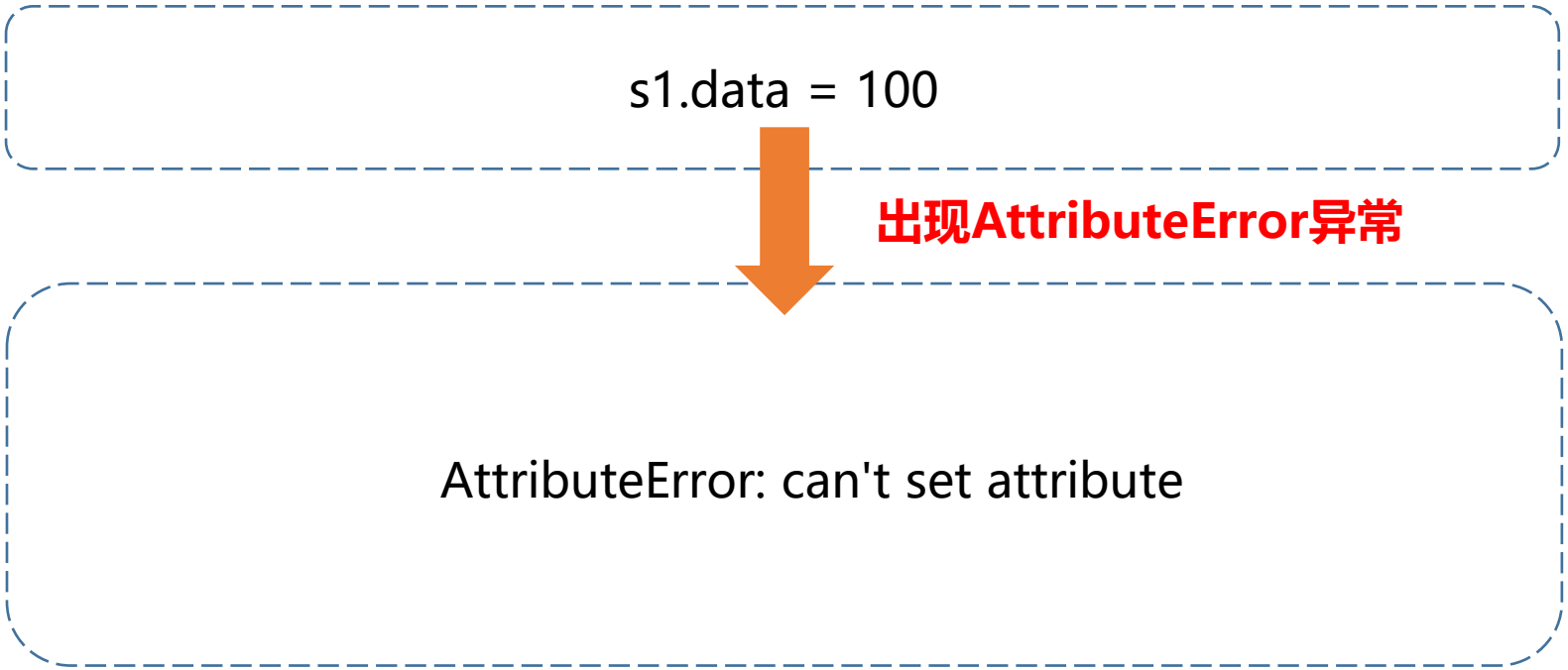
定义**私有**属性self.__data，@property**将其变为只读的**

私有属性的值为60

那么可以修改data的值吗?

6.5.1 面向对象的三大特性——封装

6.5.1.2 @property的使用



如何使data变为可以修改的?

6.5.1 面向对象的三大特性——封装

6.5.1.2 @property的使用

如果希望能将Score类中的私有属性变为可修改的，需要通过@property装饰的方法名后加“**.setter**”，实现读写权限的控制，从而实现封装

```
1.  class Score:
2.      def __init__(self,data):
3.          self.__data = data              #定义私有属性__data
4.      @property                           #使用@property将属性变为只读的
5.      def data(self):
6.          return f"私有属性的值为{self.__data}"
7.      @data.setter                        #使用data.setter将属性变为可修改的
8.      def data(self,value):
9.          if value < 0 or value > 100:
10.             raise ValueError("您的输入有误，输入的范围应该在0~100之间")
11.         else:
12.             self.__data = value
13. s1 = Score(60)
14. print(s1.data)      #调用 @property 装饰的 data() 方法
15. s1.data = 100       #调用 @data.setter 装饰的 data() 方法
16. print(s1.data)      #调用 @property 装饰的 data() 方法
```

第10行中的raise是什么意思?

运行结果：

私有属性的值为60

私有属性的值为100

6.5.1 面向对象的三大特性——封装

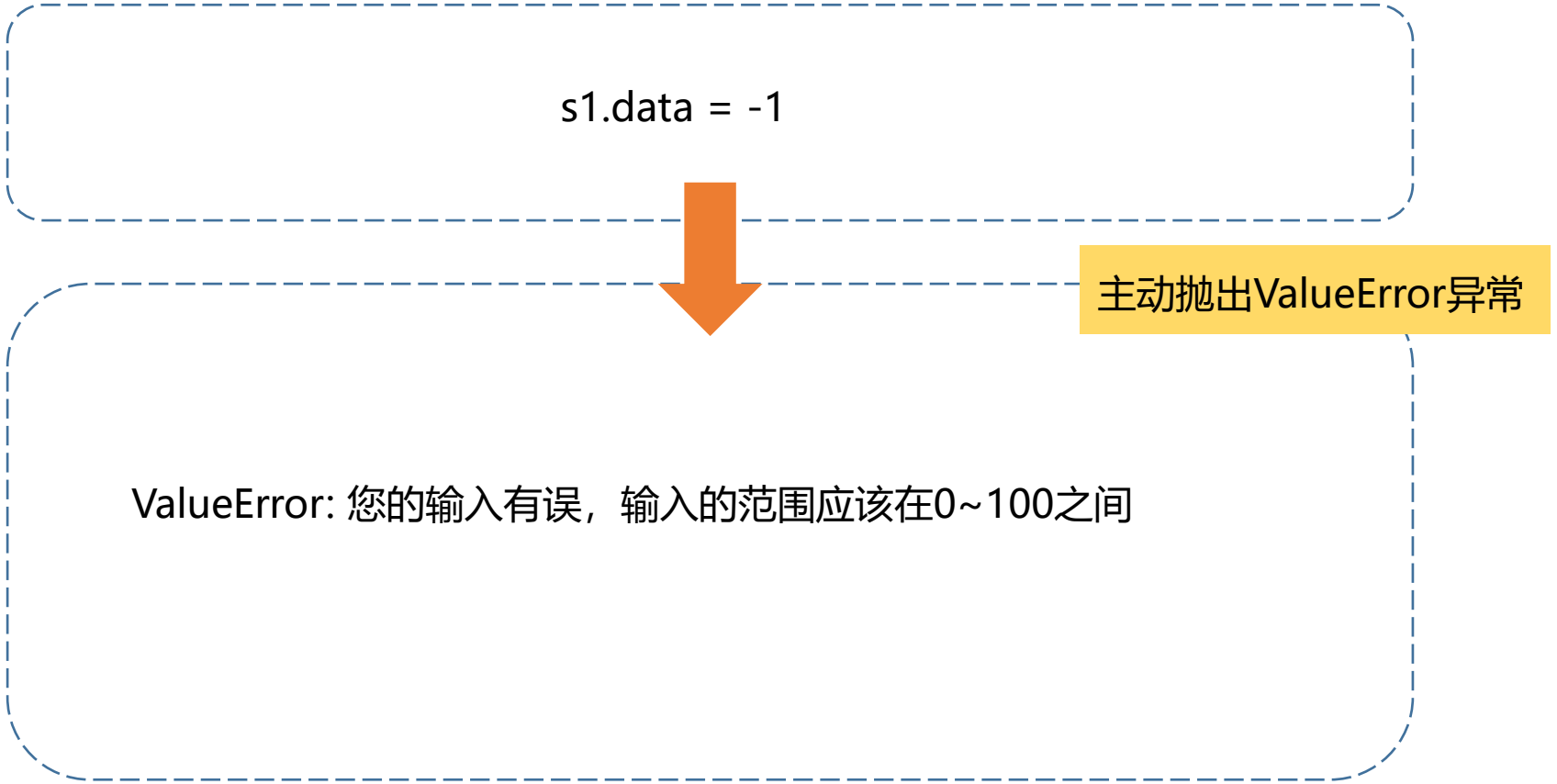
6.5.1.2 @property的使用-raise语句

raise语句，用于主动抛出异常

raise 异常类型(异常说明)

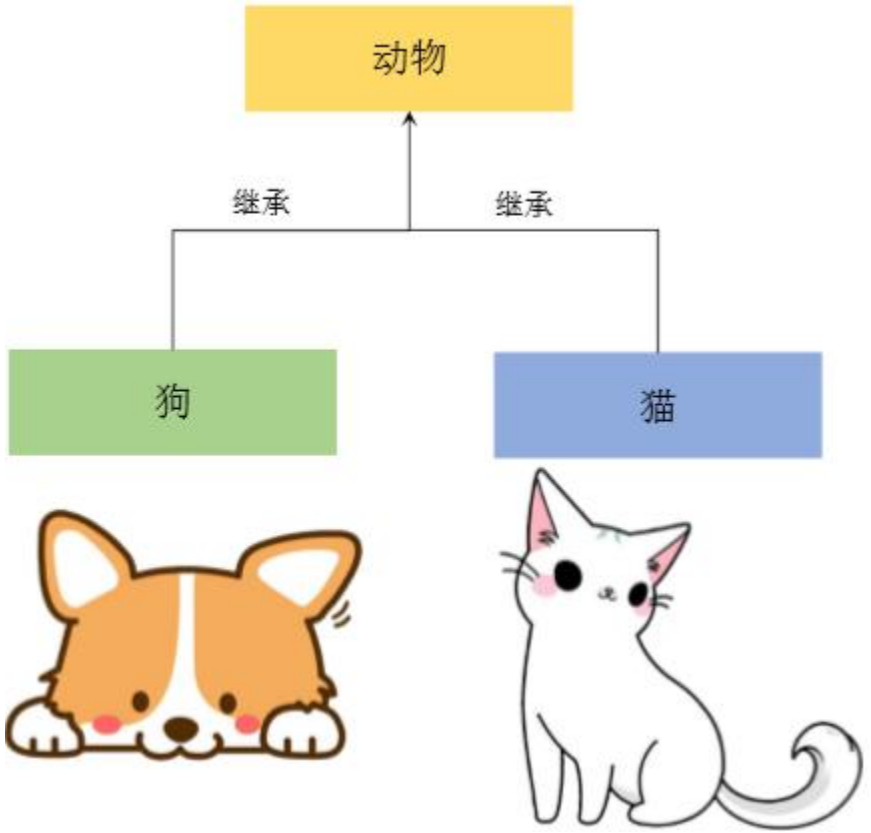
6.5.1 面向对象的三大特性——封装

6.5.1.2 @property的使用-raise语句



6.5 面向对象的三大特性

6.5.2 面向对象的三大特性-**继承**



继承(inheritance)主要描述的是**类与类之间的关系**，是面向对象程序设计中提高重用性的重要措施。**继承可以在无需编写原有类的情况下，对原有类的功能进行扩展**，既包括原有类的所有属性和行为，又可以定义自身的属性和行为，称为继承了原有类。**原有类称为父类或者基类，继承原有类的这个类称为子类或派生类。**

6.5.2 面向对象的三大特性——继承

6.5.2.1 单一继承

单一继承是指定义的派生类只有一个基类

```
class 基类名:                    #也可以写成class 基类名(object):
    类体
class 派生类名(基类名):
    类体
```

- 派生类继承自基类，**可以使用基类中的所有公有成员，不能使用基类中的私有成员。**
- 派生类可以定义**新的**属性和方法，从而完成对基类的**扩展。**
- 注意Python中所有的基类都继承自object类，这种继承写法可以省略。

6.5.2 面向对象的三大特性——继承

6.5.2.1 单一继承

```
class Animal:
    def __init__(self,name,age):
        self.name = name          #姓名属性
        self.age = age            #年龄属性
    def eat(self):                #吃饭行为
        print(f"{self.name}正在吃东西")
    def sleep(self):              #睡觉行为
        print(f"{self.name}正在睡觉")
class Dog(Animal):                #Dog类继承Animal类的属性和方法
    pass
class Cat(Animal):                #Cat类继承Animal类的属性和方法
    def play(self):               #在Cat类中创建新的方法
        print(f"{self.name}正在玩耍")
if __name__ == "__main__":        #只有直接运行(非导入)时，执行性能测试
    my_dog = Dog("小汪",4)        #创建Dog类的实例对象
    my_dog.eat()                  #调用基类中的方法
    my_cat = Cat("小喵",2)        #创建Cat类的实例对象
    my_cat.play()                 #调用Cat类中的方法
```

运行结果：

小汪正在吃东西

小喵正在玩耍

6.5.2 面向对象的三大特性——继承

6.5.2.1 单一继承

if __name__ == "__main__": 是 Python 中非常重要的一个惯用法，用于控制模块的执行方式。

- 每个 Python 模块都有一个内置属性 __name__：

- 当模块被直接运行时，__name__ 的值为 "__main__"
- 当模块被导入到其他模块时，__name__ 的值为模块的文件名（不含扩展名）

```
def hello():
    print(f"你好！来自 {__name__}")

print(f"直接运行时，__name__ 是: {__name__}")
```

```
直接运行时，__name__ 是: __main__
```

```
#将左边函数保存在hello_fun.py中，并运行

import hello_fun

print(f"=== 被导入时的名字：{hello_fun.__name__} ===")
```

```
=== 被导入时的名字：hello_fun ===
```

6.5.2 面向对象的三大特性——继承

6.5.2.2 方法重写-重写构造方法

重写Dog类中的构造方法

```
1.  class Animal:
2.      def __init__(self,name,age):
3.          self.name = name                #姓名属性
4.          self.age = age                  #年龄属性
5.  class Dog(Animal):                      #Dog类继承Animal类的属性和方法
6.      def __init__(self,name,age,vaccine):
7.          super().__init__(name,age)    #继承Animal类中的属性
8.          self.vaccine = vaccine          #是否打过疫苗属性
9.  if __name__ == "__main__":              #直接运行(非导入)时，进行性能测
    试
10.     my_dog = Dog("小汪",4,"True")
11.     if my_dog.vaccine:
12.         print(f"{my_dog.name}打过疫苗了")
13.     else:
14.         print(f"{my_dog.name}还没打过疫苗")
```

运行结果：

小汪打过疫苗了

super()是一个特殊函数，能够调用基类的方法

第7行代码的三种形式：

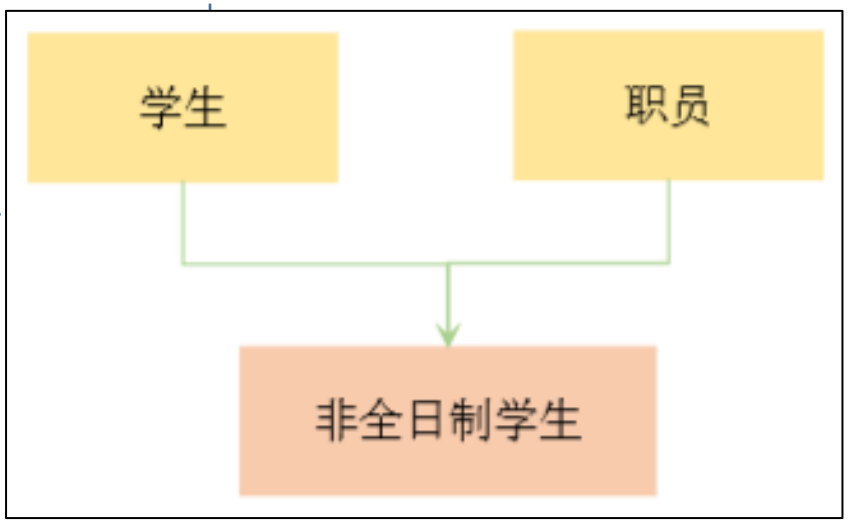
Python版本	语法	说明
Python 2	super(Dog, self).__init__(name, age)	显式指定类和实例
Python 3	super().__init__(name, age)	自动推断类和实例
等价写法	Animal.__init__(self, name, age)	直接调用，**不推荐**

6.5.2 面向对象的三大特性——继承

6.5.2.3 多重继承

多重继承是指派生类继承了多个基类

```
class 基类1:
    类体
class 基类2:
    类体
class 派生类(基类1,基类2):
    类体
```



6.5.2 面向对象的三大特性——继承

6.5.2.3 多重继承

职员、学生与非全日制学生的多重继承关系

```
class Student:
    def __init__(self,name,score):
        self.name = name
        self.score = score
    def stu_info(self):
        print(f"姓名：{self.name}，分数：{self.score}")
class Staff:
    def __init__(self,id,salary):
        self.id = id
        self.salary = salary
    def staff_info(self):
        print(f"ID：{self.id}，薪资：{self.salary}")
class PartTimeStudent(Student,Staff):
    def __init__(self,name,score,id,salary):
        Student.__init__(self,name,score)
        Staff.__init__(self,id,salary)
p1 = PartTimeStudent("小明",100,"202401",10000)
p1.stu_info()
p1.staff_info()
```

运行结果：

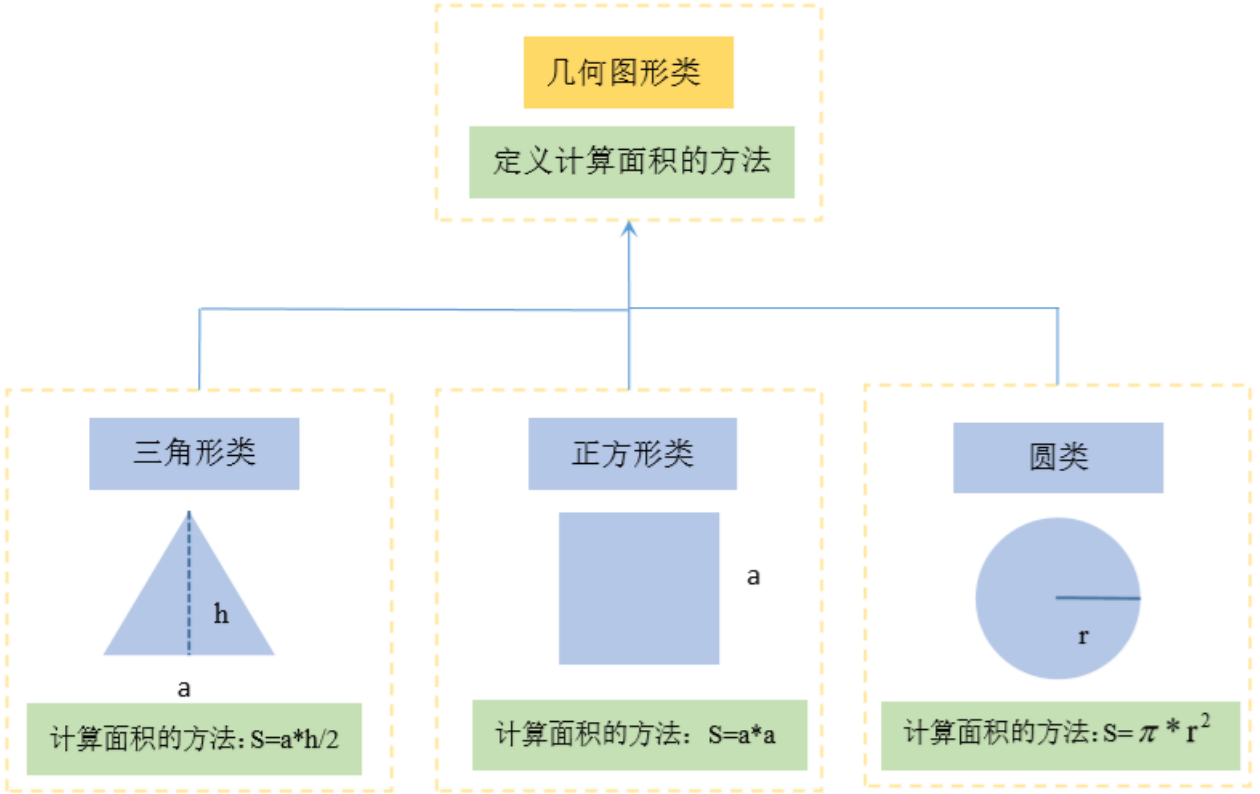
姓名：小明，分数：100

ID：202401，薪资：10000

6.5 面向对象的三大特性

6.5.3 面向对象的三大特性-**多态**

多态(polymorphism)是指属性或者方法在基类及其派生类中具有不同的含义或者形式，多态特性使得开发更加科学以及符合人类的思维习惯。



6.5.3 面向对象的三大特性——多态

6.5.3.1 多态的概念

多态的含义是“**有多种形式**”，在面向对象程序设计中，**多态是指基类的同一个方法在不同派生类对象中具有不同的表现和行为**，当调用该方法时，程序会根据对象选择合适的方法。

加法运算符对于不同类型的对象执行了不同的操作，也是多态的一种体现

```
1 + 2
"素质" + "教育"
```

6.5.3 面向对象的三大特性——多态

6.5.3.1 多态的概念

```
import math
class Graphic:
    def __init__(self,name):
        self.name = name
    def cal_square(self):
        pass
class Triangle(Graphic):
    def __init__(self,name,height,border):
        super().__init__(name)
        self.height = height
        self.border = border
    def cal_square(self):
        square = 1/2 * self.height * self.border
        print(f"{self.name}的面积是{square:.2f}")
class Circle(Graphic):
    def __init__(self,name,radius):
        super().__init__(name)
        self.radius = radius
    def cal_square(self):
        square = math.pi * pow(self.radius,2)
        print(f"{self.name}的面积是{square:.3f}")
t1 = Triangle("三角形",6,8)
t1.cal_square()
c1 = Circle("圆",3)
c1.cal_square()
```

几何图形求面积的多态体现

运行结果如下:

三角形的面积是24.00

圆的面积是28.274

6.5.3 面向对象的三大特性——多态

6.5.3.2 内置函数重写

在Python中，**不仅能对自定义类中的方法进行重写，还可以对Python已定义的内置函数进行重写**。比较常见的内置函数的重写包括前文已经介绍过的str()和repr()函数。这两个函数原本是用于将其它数据类型转换为字符串形式，在类中对其重写后，就能**用于将对象转化为字符串**。

重写str()和repr()函数，**需要在函数名前加两个下划线，函数名后加两个下划线**：

- **__str__()函数**会将对象转化为**人更容易理解**的字符串
- **__repr__()函数**会将对象转换为**解释器可识别**的字符串

6.5.3 面向对象的三大特性——多态

6.5.3.2 内置函数重写

定义一个Clock类，未重写内置函数

```
class Clock:
    def __init__(self,hour,minute,second):
        self.hour = hour
        self.minute = minute
        self.second = second
c1 = Clock(2,30,20)
print(c1)
```



```
<__main__.Clock object at 0x0000028F0E3418E0>
```

6.5.3 面向对象的三大特性——多态

6.5.3.2 内置函数重写

定义一个Clock类，重写内置函数

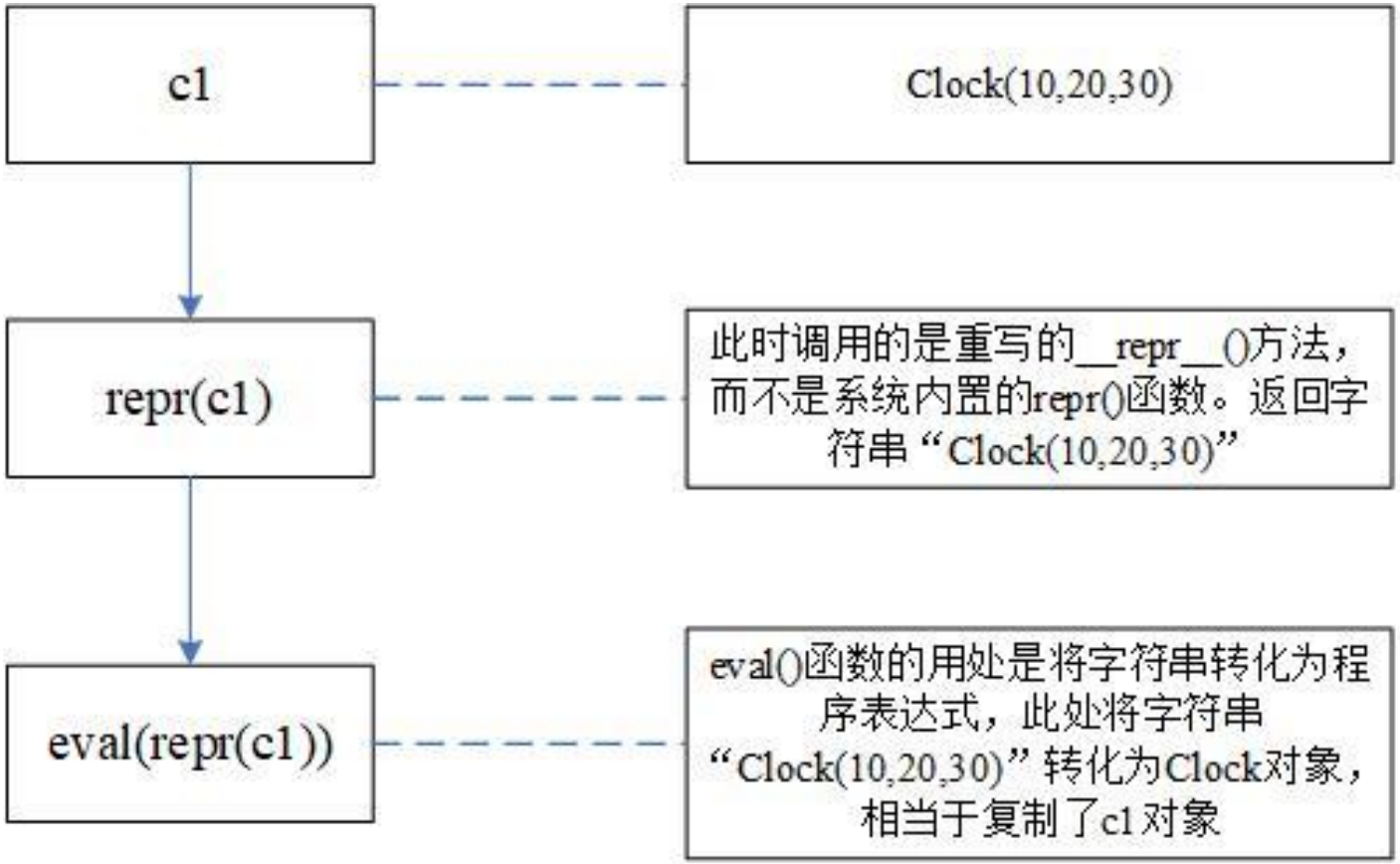
```
class Clock:
    def __init__(self,hour,minute,second):
        self.hour = hour
        self.minute = minute
        self.second = second
    def __str__(self):                  #重写__str__()
        return f"时{self.hour}，分{self.minute}，秒{self.second}"
    def __repr__(self):                 #重写__repr__()
        return f"Clock({self.hour},{self.minute},{self.second})"
c1 = Clock(10,20,30)                    #定义实例对象c1
print(c1)                               #打印对象c1
c2 = eval(repr(c1))                     #克隆c1对象赋值给c2  #c2 = c1亦可以
c2.hour = 1                             #修改c2属性
print(c2)                               #打印对象c2
```



```
时10，分20，秒30
时1，分20，秒30
```

6.5.3 面向对象的三大特性——多态

6.5.3.2 内置函数重写



第6章代码复用技术(二)：面向对象程序设计

- 6.1 类和对象的概述
- 6.2 类的基本使用—实例属性和实例方法
- 6.3 类属性和类方法、静态方法
- 6.4 私有属性和私有方法
- 6.5 面向对象的三大特性
- **6.6 特殊方法与运算符重载**

6.6 特殊方法与运算符重载(了解)

- Python解释器默默的为类提供了有大量的特殊方法，如构造函数、析构函数、实现运算符重载的特殊方法

✓ Python中类的构造函数是__init__()，一般用来为数据成员设置初值或进行其他必要的初始化工作，在创建对象时被自动调用。如果用户没有设计构造函数，Python将提供一个默认的构造函数用来进行必要的初始化工作。

✓ Python中类的析构函数是__del__()(了解)，一般用来释放对象占用的资源，在Python删除对象和收回对象空间时被自动调用。如果用户没有编写析构函数，Python将提供一个默认的析构函数进行必要的清理工作。

6.6 特殊方法与运算符重载

- 所有特殊方法的用法可参考https://docs.python.org/3.7/reference/datamodel.html

方法	功能说明
__new__()	类的静态方法，用于确定是否要创建对象
__init__()	**构造方法，创建对象时自动调用**
__del__()	析构方法，释放对象时自动调用
__add__(self, other)	**+，计算self + other**
__sub__(self, other)	**-，计算self - other**
__mul__()	*
__truediv__()	/
__floordiv__()	//
__mod__()	%
__pow__()	**
__eq__()、__ne__()、 __lt__()、__le__()、 __gt__()、__ge__()	==、!=、 <、<=、 >、>=
__lshift__()、__rshift__()	<<、>>
__and__()、__or__()、 __invert__()、__xor__()	&、|、 ~、^

6.6 特殊方法与运算符重载

方法	功能说明
__iadd__()、__isub__()	+=、-=，很多其他运算符也有与之对应的复合赋值运算符
__pos__()	一元运算符+，正号
__neg__()	一元运算符-，负号
__contains__(self, item)	**与成员测试运算符in对应，计算item in self**
__radd__()、__rsub__	反射加法、反射减法，一般与普通加法和减法具有相同的功能，但操作数的位置或顺序相反，很多其他运算符也有与之对应的反射运算符
__abs__()	与内置函数abs()对应
__bool__()	与内置函数bool()对应，要求该方法必须返回True或False
__bytes__()	与内置函数bytes()对应
__complex__()	与内置函数complex()对应，要求该方法必须返回复数
__dir__()	与内置函数dir()对应
__divmod__()	与内置函数divmod()对应
__float__()	与内置函数float()对应，要求该该方法必须返回实数
__hash__()	与内置函数hash()对应
__int__()	与内置函数int()对应，要求该方法必须返回整数

6.6 特殊方法与运算符重载

方法	功能说明
__len__()	与内置函数len()对应
__next__()	与内置函数next()对应
__reduce__()	提供对reduce()函数的支持
__reversed__()	与内置函数reversed()对应
__round__()	对内置函数round()对应
__str__()	与内置函数str()对应，要求该方法必须返回str类型的数据
__repr__()	打印、转换，要求该方法必须返回str类型的数据
__getitem__(self, key)	**按照索引获取值，计算self[key]**
__setitem__(self, key, value)	**按照索引赋值，等价于self[key] = value**
__delattr__()	删除对象的指定属性
__getattr__()	获取对象指定属性的值，对应成员访问运算符“.”

6.6 特殊方法与运算符重载

方法	功能说明
__getattribute__()	获取对象指定属性的值，如果同时定义了该方法与__getattr__()，那么__getattr__()将不会被调用，除非在__getattribute__()中显式调用__getattr__()或者抛出AttributeError异常
__setattr__()	设置对象指定属性的值
__base__	该类的基类
__class__	返回对象所属的类
__dict__	对象所包含的属性与值的字典
__subclasses__()	返回该类的所有子类
__call__(self [,args…])	**包含该特殊方法的类的实例可以像函数一样调用**
__get__()	定义了这三个特殊方法中任何一个的类称作描述符（descriptor），描述符对象一般作为其他类的属性来使用，这三个方法分别在获取属性、修改属性值或删除属性时被调用
__set__()	
__delete__()	

6.6 特殊方法与运算符重载

```
import math

class Fraction:
    def __init__(self, numerator=0, denominator=None):
        if denominator is None:
            self._numerator = numerator
            self._denominator = 1
            return

        g = math.gcd(numerator, denominator)
        if denominator < 0:
            g = -g
        numerator //= g
        denominator //= g
        self._numerator = numerator
        self._denominator = denominator

    def to_float(self):
        return self._numerator/self._denominator

    @classmethod
    def from_str(cls, s):
        if '/' not in s:
            numerator = int(s)
            return cls(numerator)
        else:
            s = s.split('/')
            numerator = int(s[0].strip())
            denominator = int(s[1].strip())
            return cls(numerator, denominator)
```

```
    @staticmethod
    def gcd(a, b):
        # Supports non-integers for backward compatibility.
        while b:
            a, b = b, a%b
        return a

    @property
    def numerator(a):
        return a._numerator

    @property
    def denominator(a):
        return a._denominator

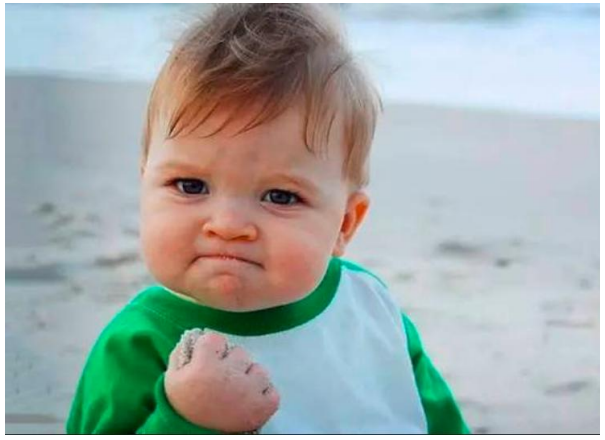
    def __repr__(self):
        """repr(self)"""
        return '%s(%s, %s)' % (self.__class__.__name__,
                               self._numerator, self._denominator)

    def __str__(self):
        """str(self)"""
        if self._denominator == 1:
            return str(self._numerator)
        else:
            return '%s/%s' % (self._numerator,
self._denominator)
```

```
def __add__(a, b):
    """a + b"""
    da, db = a.denominator, b.denominator
    return Fraction(a.numerator * db + b.numerator * da,
                    da * db)

def __sub__(a, b):
    """a - b"""
    da, db = a.denominator, b.denominator
    return Fraction(a.numerator * db - b.numerator * da,
                    da * db)
```

Python中完整的Fraction类，包含本章所有知识！



6.6 特殊方法与运算符重载

```
class Fraction:

    def __add__(self, other):
        """self + other"""
        a, b = self, other
        da, db = a.denominator, b.denominator
        return Fraction(a.numerator * db + b.numerator * da, da * db)

    def __sub__(self, other):
        """self - other"""
        a, b = self, other
        da, db = a.denominator, b.denominator
        return Fraction(a.numerator * db - b.numerator * da, da * db)
```

- 在这两个特殊方法中，调用了类的构造方法

6.6 特殊方法与运算符重载

```
class Colormap:  #颜色插值器，用于在两个颜色之间进行线性插值

    def __init__(self, segmentdata):
        """segmentdata: 长度为2的列表，每个元素为一个(R,G,B)颜色值元组；
           颜色值在0到255之间，比如[(255, 0, 0), (0, 0, 255)]
        """
        self._colorb = segmentdata[0]  #基础颜色（红）
        self._colort = segmentdata[1]  # 目标颜色（蓝）

    def __call__(self, x):       #使用__call__方法使实例可调用
        return tuple(int(x * self._colort[i] + (1-x)* self._colorb[i])
                     for i in range(3))    #根据比例获取插值颜色
```

Hex: # FF0000
Red: 255
Green: 0
Blue: 0

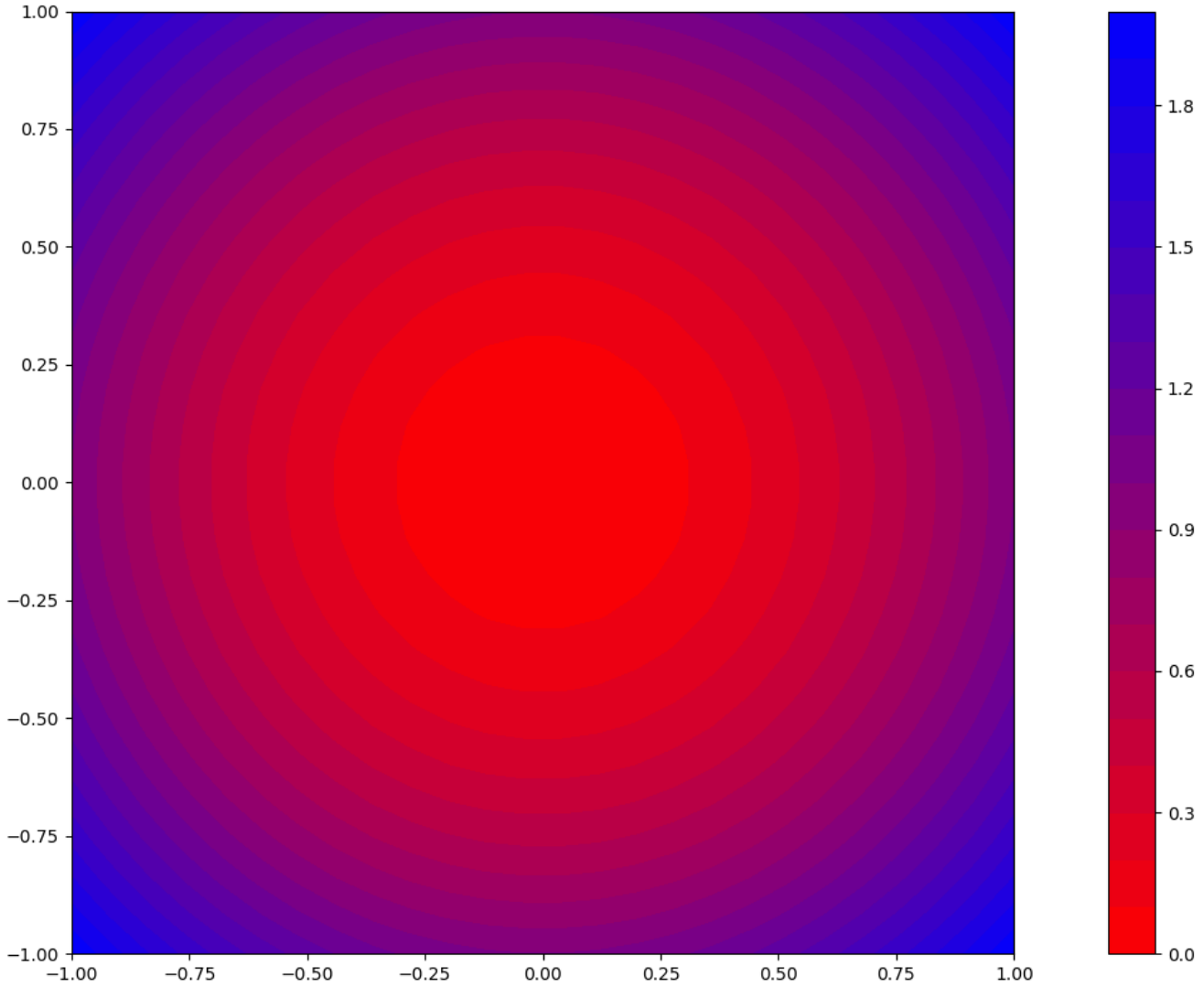


Hex: # 0000FF
Red: 0
Green: 0
Blue: 255



```
>>> cm = Colormap([(255, 0, 0), (0, 0, 255)]) #对象名(参数)，参数传递给__init__
>>> cm(0)   #对象名(参数)，参数传递给__call__
(255, 0, 0)
>>> cm(1)
(0, 0, 255)
>>> cm(0.5)
(127, 0, 127)
```

6.6 特殊方法与运算符重载



6.6 特殊方法与运算符重载

6.6.2 运算符重载

“+”运算符的本质是**调用了内置的__add__()方法**。那么**类的对象之间可以进行“+”操作吗?** “-”、“*”、“>”、“<”等运算符操作呢?

运算符对应的方法

运算符	方法	说明	示例
+	__add__(self,other)	加法	a + b
-	__sub__(self,other)	减法	a - b
*	__mul__(self,other)	乘法	a * b
/	__truediv__(self,other)	除法	a / b
%	__mod__(self,other)	求余	a % b
>	__gt__(self,other)	大于	a > b
>=	__ge__(self,other)	大于等于	a >= b
==	__eq__(self,other)	等于	a == b
<	__lt__(self,other)	小于	a < b
<=	__le__(self,other)	小于等于	a <= b
!=	__ne__(self,other)	不等于	a != b

6.6 特殊方法与运算符重载

6.6.2 运算符重载

计算接力赛的赛跑长度及时长，并判断获胜队伍

```
1.  class Race:
2.      def __init__(self,length,time):
3.          self.length = length
4.          self.time = time
5.      def __add__(self, other):              #重载+运算
6.          return Race(self.length + other.length,self.time + other.time)
7.      def __gt__(self, other):               #重载>运算
8.          if self.length == other.length:
9.              return self.time > other.time
10.         else:
11.             raise TypeError("无法比较")
12.     def __eq__(self, other):               #重载==运算
13.         if self.length == other.length:
14.             return self.time == other.time
15.         else:
16.             raise TypeError("无法比较")
17.     def __str__(self):
18.         return f"赛跑长度为{self.length}，赛跑时长为{self.time}"
19. if __name__ == "__main__":
20.     team1_1 = Race(200,20)                 #team1的1号队员
21.     team1_2 = Race(200,19.8)               #team1的2号队员
22.     team1 = team1_1 + team1_2              #team1的总赛跑长度和时长
23.     print(team1)
24.     team2_1 = Race(200,18.9)               #team2的1号队员
25.     team2_2 = Race(200,20.1)               #team2的2号队员
26.     team2 = team2_1 + team2_2              #team2的总赛跑长度和时长
27.     print(team2)
28.     if team1 > team2:
29.         print("team2取得胜利")
30.     elif team1 == team2:
31.         print("team1和team2平局")
32.     else:
33.         print("team1取得胜利 ")
```

运行结果：

赛跑长度为400，赛跑时长为39.8

赛跑长度为400，赛跑时长为39.0

team2取得胜利

6.6 特殊方法与运算符重载

6.6.2 运算符重载

- 运算符重载是面向对象编程中的一个重要特性，它允许我们自定义类对内置运算符的行为。
- 所有运算符重载方法都以双下划线开头和结尾：__xxx__
- 运算符重载让自定义类型用起来像内置类型一样自然，用直观的数学符号表达复杂的操作。

```
# 没有重载的代码
result = obj1.add(obj2)  # 不直观

# 使用重载的代码
result = obj1 + obj2     # 直观！Pythonic！
```

- 所有运算符重载方法都是特殊方法，但特殊方法不全是运算符重载方法。

第7章　字符串



第7章　文本处理(一)：字符串

- 7.1字符串编码格式简介
- 7.2转义字符与原始字符串
- 7.3字符串格式化
- 7.4字符串常用操作
- 7.5字符串常量
- 7.6中英文分词
- 7.8精彩案例赏析

第7章 字符串

■在Python中，字符串属于不可变有序序列，使用单引号、双引号、三单引号或三双引号作为定界符，并且不同的定界符之间可以互相嵌套。

```
>>> s = '''Tom said,"Let's go"'''
```

◆**一句话的字符串**

第7章 字符串

```
>>> s = """STRAY birds of summer come to my window
to sing and fly away.
And yellow leaves of autumn,
which have no songs,
flutter and fall there with a sigh.

O TROUPE of little vagrants of the world,
leave your footprints in my words.

THE world puts off its mask of vastness to its lover.
It becomes small as one song,
as one kiss of the eternal.

IT is the tears of the earth
that keep her smiles in bloom.

THE mighty desert is burning
for the love of a blade of grass
who shakes her head and laughs
and flies
away.

IF you shed tears when you miss the sun,
you also miss the stars.

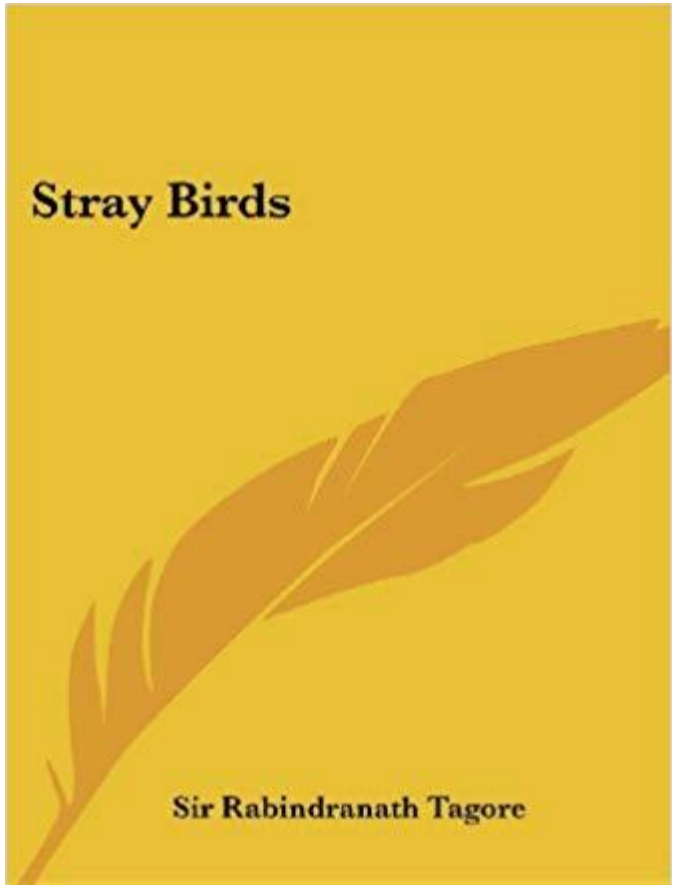
THE sands in your way beg for your song
and your movement,
dancing water.
Will you carry the burden of their lameness?

HER wistful face haunts my dreams
like the rain at night.

ONCE we dreamt that we were strangers.
We wake up to find that we were dear to each other.

SORROW is hushed into peace in my heart
like the evening among the silent trees."""
```

◆ 多行字符串



第7章 字符串

◆一本书的字符串

```
>>> fp = open(r'C:\Users\xiong\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\红楼梦.txt', 'r', encoding='utf-8')
>>> hlm = fp.read()
>>> fp.close()

>>> len(hlm)
873792

>>> print(hlm[:500]) #前500字
```

第一回甄士隐梦幻识通灵贾雨村风尘怀闺秀

——此开卷第一回也。作者自云：曾历过一番梦幻之后，故将真事隐去，而借



第7章 字符串

- 字符串支持序列的通用操作：包括双向索引、比较大小、计算长度、切片、成员测试等操作

```
>>> s = '浙江工业大学'
>>> s[-2:]
'大学'
```

- 字符串还支持一些特有的操作方法，例如字符串格式化、查找、替换、排版
- 字符串属于不可变序列，不能直接对字符串对象进行原地字符增加、修改与删除等操作，各操作均创建新的字符串

```
>>> s.replace('工业', '')
'浙江大学'
```

第7章 字符串

操作类别	操作名	语法示例（Python）	适用序列类型
通用操作	索引	seq[0]	所有序列
	切片	seq[1:4]	所有序列
	成员检查	‘a' in seq	所有序列
	长度	len(seq)	所有序列
	最小值/最大值	min(seq), max(seq)	元素可比较的序列
	连接/重复	seq1 + seq2, seq * 3	所有序列
	迭代	for x in seq:	所有序列
	查找索引	seq.index(‘a')	所有序列
	计数	seq.count(‘a')	所有序列
可变序列操作	赋值	seq[0] = 1, seq[1:3] = [4,5]	可变序列（如list）
	添加元素	seq.append(x), seq.insert(i, x)	可变序列
	删除元素	del seq[0], seq.remove(x)	可变序列
	排序/反转	seq.sort(), seq.reverse()	可变序列

第7章 字符串

- 字符串中的字符属于**Unicode字符集**，每个字符都有一个唯一的编号（**码点**）

```
>>> for ch in s:
        print(ch, ord(ch))
```

```
浙 27993
江 27743
工 24037
业 19994
大 22823
学 23398
```

- Python默认使用**UTF-8**(8-bit Unicode Transformation Format)编码格式以二进制的形式**存储**字符

```
>>> import sys
>>> sys.getdefaultencoding()
'utf-8'
```

第7章 字符串

■除了支持str类型之外，Python还支持字节串类型bytes

■**字符串**可以通过encode()方法使用指定的编码格式编码成为**bytes对象**

■**bytes对象**则可以通过decode()方法使用指定编码格式解码成为**str字符串**

```
>>> s = '浙江工业大学'
>>> s.encode()
b'\xe6\xb5\x99\xe6\xb1\x9f\xe5\xb7\xa5\xe4\xb8\x9a\xe5\xa4\xa7\xe5\xad\xa6 ‘
>>> len(s)
6
>>> len(s.encode())
18  #三个字节构成一个汉字
>>> s.encode().decode()
'浙江工业大学'
```

第3章　文本处理(一)：字符串

- **7.1字符串编码格式简介**
- 7.2转义字符与原始字符串
- 7.3字符串格式化
- 7.4字符串常用操作
- 7.5字符串常量
- 7.6中英文分词
- 7.8精彩案例赏析

字符	码点	UTF-8 编码	说明
m (拉丁文小写m)	U+006D	6D	ASCII字符，1字节
م (阿拉伯文字母Meem)	U+062D	D8 AD	阿拉伯字符，2字节
Ș(拉丁文大写S带逗号)	U+0219	C8 99	欧洲字符，2字节
看 (中文字符"看")	U+770B	E7 9C 8B	中文字符，3字节

7.1 字符串编码格式简介

- 最早的字符串编码是美国标准信息交换码ASCII，仅对10个数字、26个大写英文字母、26个小写英文字母及一些其他符号进行了编码
- ASCII码采用1个字节来对字符进行编码，共编码128个字符

7.1 字符串编码格式简介

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Source: www.LookupTables.com

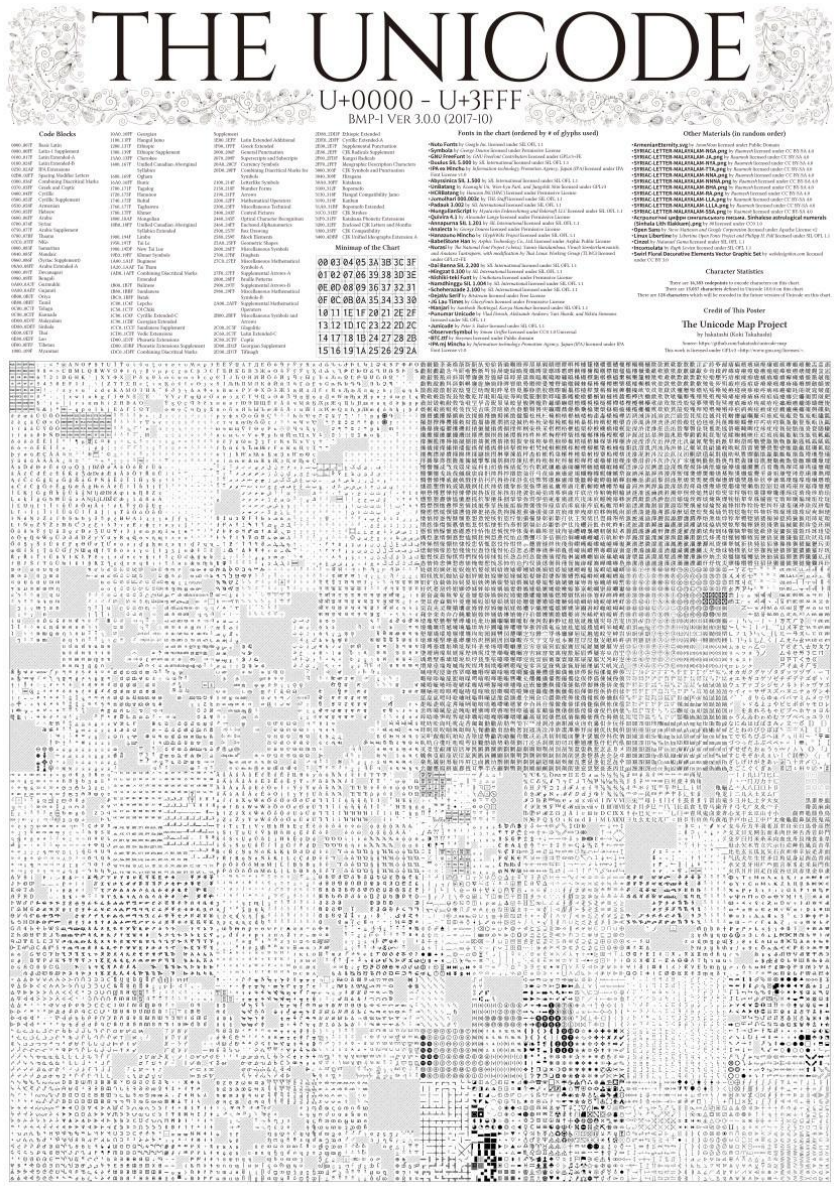
Dec：Decimal → 10
Hx：Hexadecimal → 16
Oct：Octal → 8

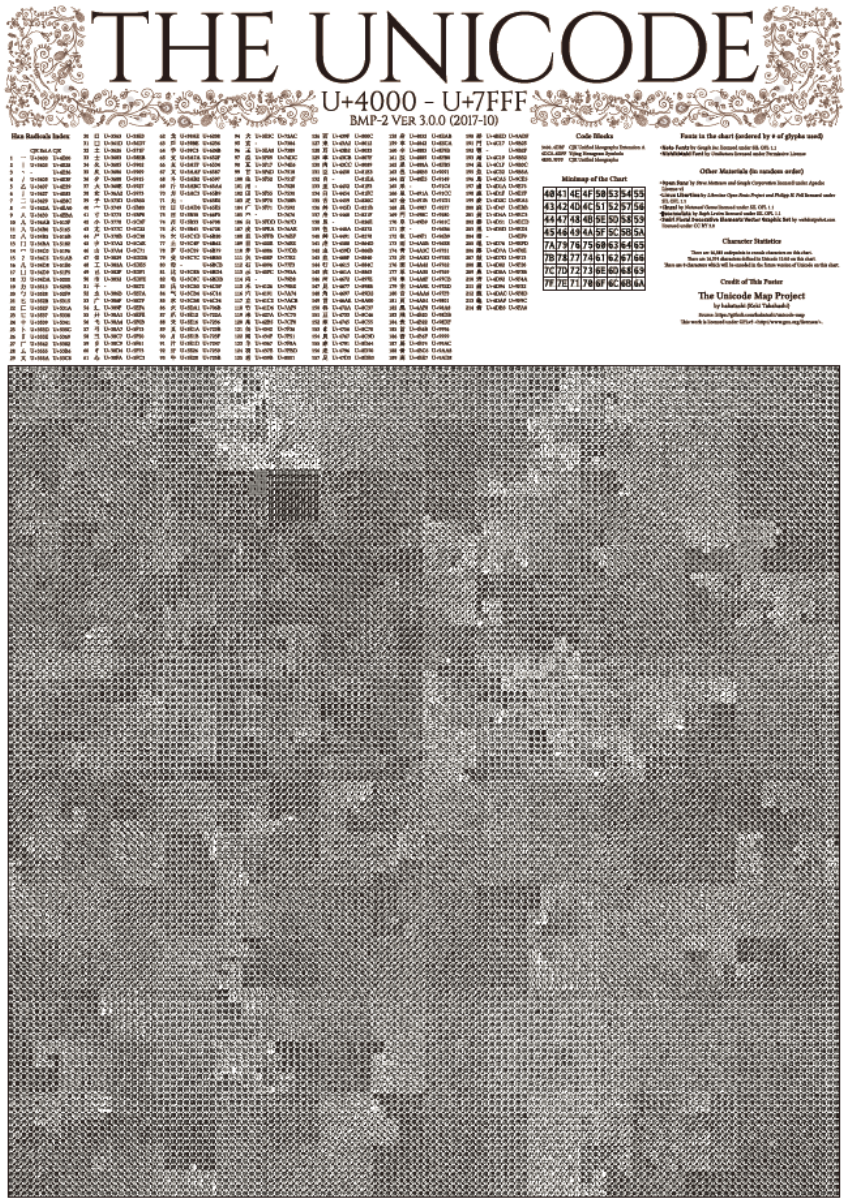
7.1 字符串编码格式简介

- UTF-8对全世界所有国家需要用到的字符（Unicode字符集）进行了编码
- 以1个字节表示英语字符(兼容ASCII)，以3个字节表示中文
- 还有些语言的符号使用2个字节（例如俄语和希腊语符号）或4个字节

- GB2312是我国制定的中文编码，使用1个字节表示英语，2个字节表示中文；GBK(国标扩展)是GB2312的扩充，而CP936是微软在GBK基础上开发的编码方式
- GB2312、GBK和CP936都是使用2个字节表示中文
- GB2312等不支持一些其他语言的符号

7.1 字符串编码格式简介





7.1 字符串编码格式简介

■ UTF-8编码是如何识别一个字符使用几个字节存储的（了解）

```
>>> ord('熊')
29066 #unicode码点
          #默认十进制显示
>>> bin(ord('熊')) #二进制
'0b0111000110001010'
```

特性	Unicode码点	UTF-8编码
本质	字符的抽象编号	字符的具体字节表示
用途	字符的唯一标识	存储和传输格式
值类型	整数（0-0x10FFFF）	字节序列（1-4个字节）
确定性	一个字符对应一个码点	一个码点对应一种编码

7.1 字符串编码格式简介

- UTF-8编码是如何识别下一个字符使用几个字节存储的（了解）

```
>>> s = '熊'.encode()  #编译为字节
>>> s
b'\xe7\x86\x8a'  #默认显示十六进制字节值
>>> bin(int('0x' + s.hex(), base=16))
            # 添加 0x 前缀表示这是一个十六进制字符串
             #s.hex()字节对象转为十六进制字符串
             #int(,base=16)将十六进制字符串转为十进制整数
'0b111001111100001101010001010'  #二进制的UTF-8编码
```

```
11100111 10000110 10001010  ← 分成3个字节
\__e7___/ \__86___/ \__8a___/  ← 对应的十六进制
```

7.1 字符串编码格式简介

表示方式	值	说明
字符	'熊'	实际的汉字
Unicode码点	29066 (0x718c)	字符在Unicode表中的编号
码点二进制	0b0111000110001010	编号的二进制表示
UTF-8编码	b'\xe7\x86\x8a'	字符在UTF-8编码中的字节序列
UTF-8二进制	0b**1110**0111**10**000110**10**001010	编码的二进制表示

Unicode码点：字符的抽象编号，与具体编码无关

UTF-8编码：具体的字节表示，**用于存储和传输**

UTF-8是一种变长编码，汉字通常需要3个字节：

首字节：1110xxxx（表示3字节序列）

后续字节：10xxxxxx

7.1 字符串编码格式简介

- 不同编码格式之间相差很大，采用不同的编码格式意味着不同的存储形式
- 把同一字符以不同编码格式存入文件时，写入的内容可能会不同，在试图理解其内容时必须了解编码规则并进行正确的解码
- 如果解码方法不正确就无法还原信息，从这个角度来讲，字符串编码也具有加密的效果

7.1 字符串编码格式简介

```
>>> s = '浙江工业大学'
>>> s.encode('utf-8').decode('gbk')
Traceback (most recent call last):
  File "<pyshell#23>", line 1, in <module>
    s.encode('utf-8').decode('gbk')
UnicodeDecodeError: 'gbk' codec can't decode byte 0xad in position 16:
illegal multibyte sequence

>>> 'abc'.encode('gbk').decode('utf-8')
'abc'  #因为'abc'是ASCII码，GBK和UTF-8在ASCII字符集上是兼容的
```

UTF-8编码 -> UTF-8解码 (正确)
GBK编码 -> GBK解码 (正确，但前提是s包含的字符都在GBK字符集内)

第3章　文本处理(一)：字符串

- 7.1字符串编码格式简介
- **7.2转义字符与原始字符串**
- 7.3字符串格式化
- 7.4字符串常用操作
- 7.5字符串常量
- 7.6中英文分词
- 7.8精彩案例赏析

7.2 转义字符与原始字符串

保持字符串原样**输出**有3种方法：元字符串、repr函数和转义字符（\）。

元字符串

元字符串即在任意字符串之前添加字母**r或R**，那么，当前字符串中所有转义字符在使用时都**不会进行转义操作。正则表达式**中常见该格式。 eg. print(r'abc\n')

repr函数

如果希望按原始字符串输出，也可使用**repr函数**，不过，此时原字符串会使用**一对单引号括起来。**eg. print(repr("abc\n"))

转义字符（\）

如果只想原样输出转义字符，如\n、\t等，则可以在前面**再加一个反斜杠**，也就是使用两个反斜杠，这样，第一个反斜杠后的\n、\t符号会被认为是普通字符，而不是转义字符。 eg. print('abc\\n')

7.2 转义字符与原始字符串

转义字符	含义	转义字符	含义
\b	退格，把光标移动到前一列位置	\\	一个斜线\
\f	换页符	\'	单引号'
\n	换行符	\"	双引号"
\r	回车	\ooo	3位八进制数对应的字符
\t	水平制表符	\xhh	2位十六进制数对应的字符
\v	垂直制表符	\uhhhh	4位十六进制数表示的Unicode字符

7.2 转义字符与原始字符串

■转义字符用法

```
>>> print('Hello\nWorld')                    #包含转义字符的字符串
Hello
World
>>> print('\101')                            #三位八进制数对应的字符
A
>>> print('\x41')                            #两位十六进制数对应的字符
A
>>> print('\u6d59\u6c5f\u5de5\u4e1a\u5927\u5b66')四位十六进制数表示Unicode
  字符
浙江工业大学
```

7.2 转义字符与原始字符串

■在字符串前面加上字母r或R表示**原始字符串**，其中的所有字符都表示原始的含义而不会进行任何转义

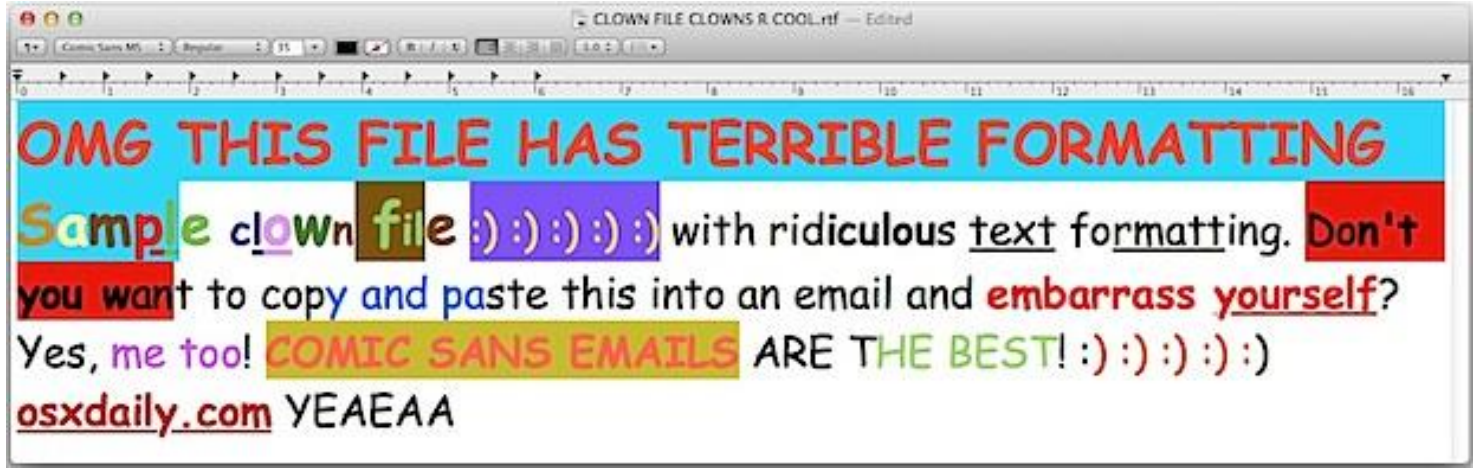
```
>>> path = 'C:\Windows\notepad.exe'
>>> print(path)                          #字符\n被转义为换行符
C:\Windows
otepad.exe

>>> path = r'C:\Windows\notepad.exe'   #原始字符串，任何字符都不转义
>>> print(path)
C:\Windows\notepad.exe

>>> path = 'C:\\Windows\\notepad.exe'   #r不是万能的，也可以用\\转义
>>> print(path)
C:\Windows\notepad.exe
```

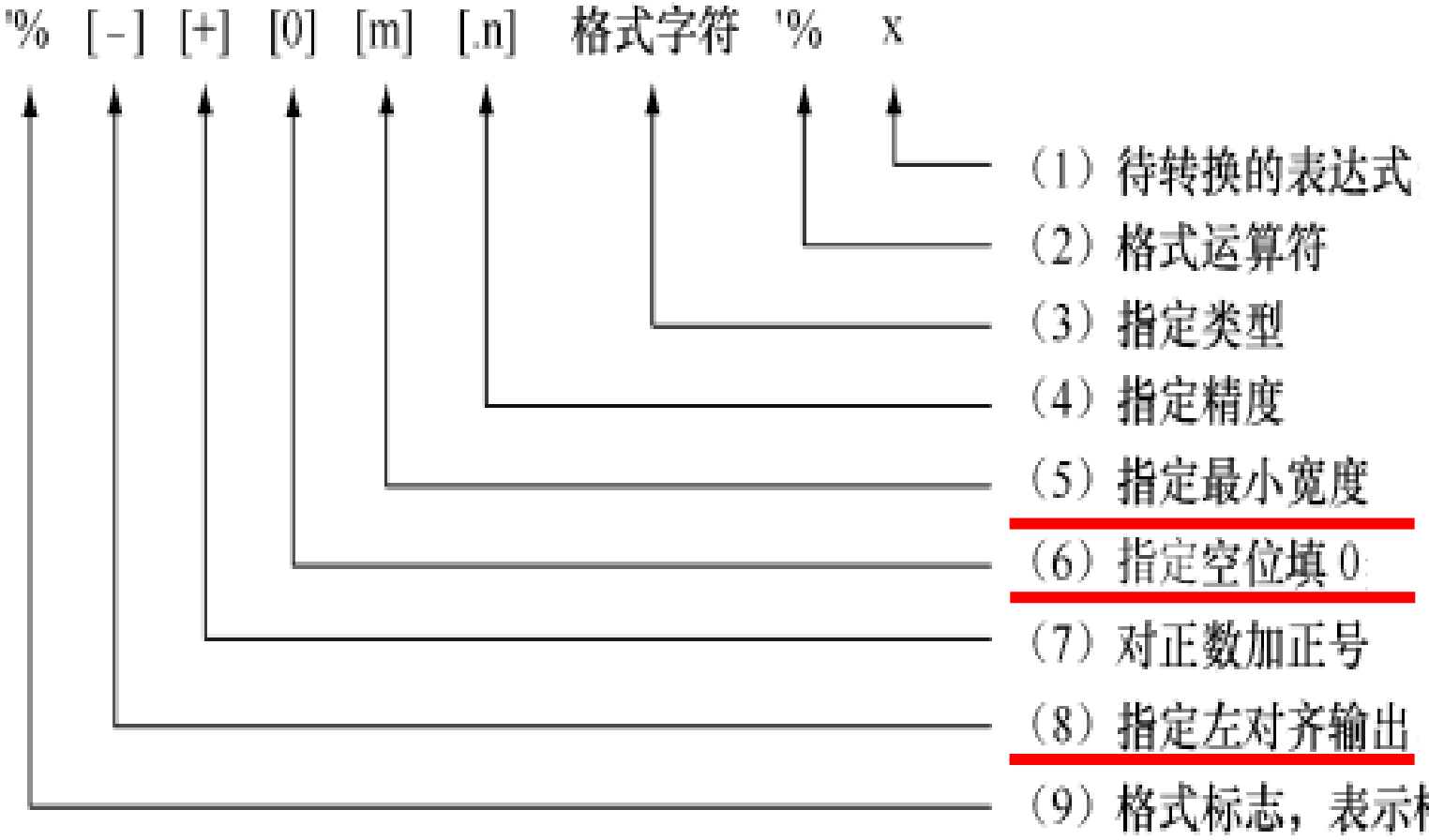
第3章　文本处理(一)：字符串

- 7.1字符串编码格式简介
- 7.2转义字符与原始字符串
- 7.3字符串格式化
  - 7.3.1使用%符号进行格式化
  - 7.3.2使用format()方法进行字符串格式化
  - 7.3.3格式化的字符串常量



7.3.1 使用%运算符进行格式化

■ 通过格式化创建以特定格式表示数据的字符串



默认右对齐

'''将一个浮点数格式化为小数点后6位，并在一个至少20字符宽的字段中左对齐'''

```
>>>s = '%-20.6f'%12.5
>>> s
'12.500000           '
>>>s ='%020.6f'%12.5
 >>> s
 '00000000000012.500000'
```

7.3.1 使用%运算符进行格式化

■常用格式字符

格式字符	说明
%s	字符串 (采用str()的显示)
%r	字符串 (采用repr()的显示)
%c	单个字符
%d	十进制整数
%i	十进制整数
%o	八进制整数
%x	十六进制整数
%e	指数 (基底写为e)
%E	指数 (基底写为E)
%f、%F	浮点数
%g	指数(e)或浮点数 (根据显示长度)
%G	指数(E)或浮点数 (根据显示长度)
%%	一个字符"%"

7.3.1 使用%运算符进行格式化

```
>>> for i in range(9,12):
  print('f%02d'%i)

f09
f10
f11

>>> '(%d, %d)'%(65,65)
'(65, 65)'
>>> '[%d, %d]'%[65,65]
Traceback (most recent call last):
  File "<pyshell#45>", line 1, in <module>
    '[%d, %d]'%[65,65]
TypeError: %d format: a number is required, not list
# % 操作符实际上期望的是一个元组（tuple） 作为右侧的参数
```

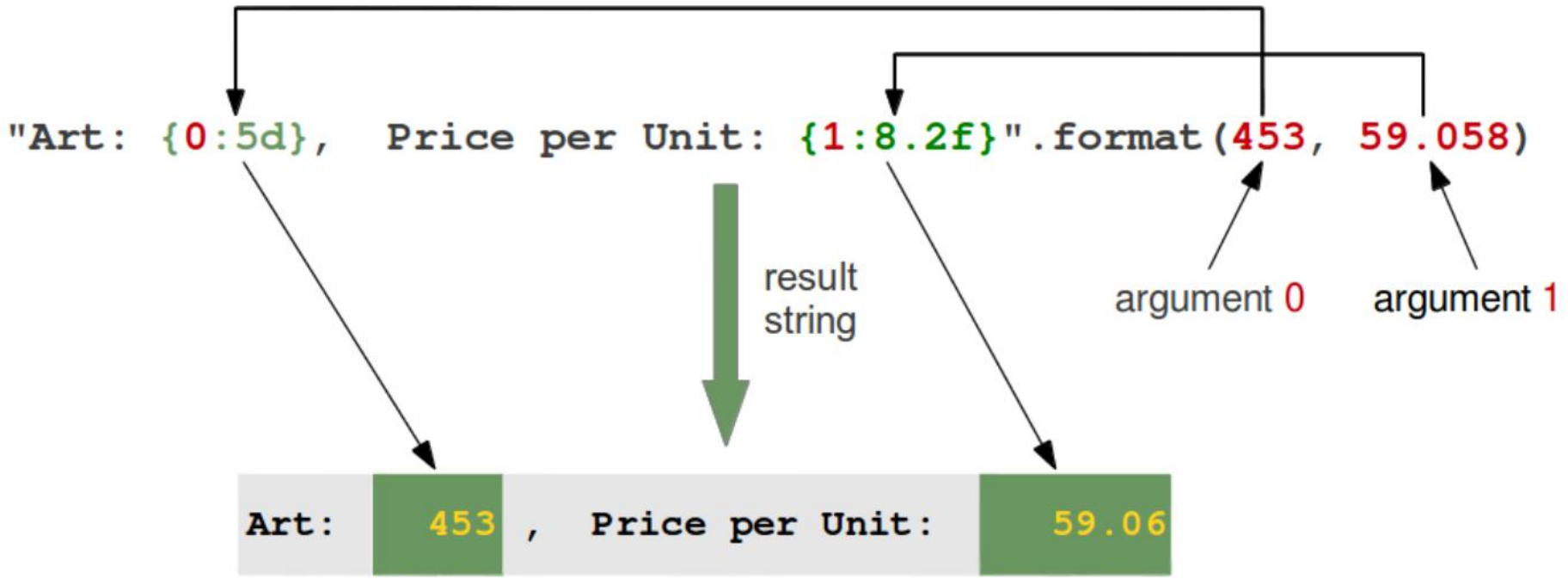
第3章　文本处理(一)：字符串

- 7.1字符串编码格式简介
- 7.2转义字符与原始字符串
- 7.3字符串格式化
  - 7.3.1使用%符号进行格式化
  - **7.3.2使用format()方法进行字符串格式化**
  - 7.3.3格式化的字符串常量

7.3.2 使用format()方法进行格式化

- 用format()方法格式化字符串更符合Python的风格，且更灵活，**推荐使用**

str.format()



7.3.2 使用format()方法进行格式化

- 占位符（placeholder）的形式为{参数:格式码}

```
>>> 1/3
0.3333333333333333
>>> print('{0:.3f}'.format(1/3))          #保留3位小数
0.333
>>> print('{:.3f}'.format(1/3))           #参数0可省略
0.333
>>> print('{0}'.format(1/3))
0.3333333333333333
>>> print('{}'.format(1/3))               #0可省略，只要占位即可
0.3333333333333333

>>> '{0:.2%}'.format(0.035)               #格式化为百分数
'3.50%'          #  %：格式类型。这是一个特殊的格式类型，它会将数字乘以100，并添加一个百分号%。
```

7.3.2 使用format()方法进行格式化

- 参数可以是序号或**关键字参数**的名字，且若参数表示序列时，可使用下标

```
>>> print("my name is {name}, my age is {age}, and my QQ is
  {qq}".format(name = "Dong Fuguo",age = 40,qq = "30646****"))
my name is Dong Fuguo, my age is 40, and my QQ is 30646****

>>> position = (5, 8, 13)
>>> print("X:{0[0]};Y:{0[1]};Z:{0[2]}".format(position))
X:5;Y:8;Z:13

>>> position = (5, 8, 13)
>>> print('X:{}; Y:{}; Z:{}'.format(position[0],position[1],position[2]))
X:5;Y:8;Z:13
>>> print("X:{};Y:{};Z{}".format(*position))    #用*做序列解包，前面下标可省略
X:5;Y:8;Z:13
```

7.3.2 使用format()方法进行格式化

```
weather = [("Monday","rainy"),("Tuesday","sunny"),
           ("Wednesday", "sunny"),("Thursday","rainy"),
           ("Friday","cloudy")]

>>> formatter = "Weather of '{0[0]}' is '{0[1]}'"
>>> for item in weather:
        print(formatter.format(item))
```

运行结果：

```
Weather of 'Monday' is 'rainy'
Weather of 'Tuesday' is 'sunny'
Weather of 'Wednesday' is 'sunny'
Weather of 'Thursday' is 'rainy'
Weather of 'Friday' is 'cloudy'
```

#或用序列解包方式

```
>>> formatter = "Weather of '{}' is '{}'"
>>> for item in weather:
        print(formatter.format(*item))
```

第3章　文本处理(一)：字符串

- 7.1字符串编码格式简介
- 7.2转义字符与原始字符串
- 7.3字符串格式化
  - 7.3.1使用%符号进行格式化
  - 7.3.2使用format()方法进行字符串格式化
  - **7.3.3格式化的字符串常量**

7.3.3 格式化的字符串常量（强烈推荐！）

- 从Python 3.6.x开始支持一种新的字符串格式化方式，官方叫做Formatted String Literals，在字符串前加字母f，常被称为**f字符串，**含义与字符串对象format()方法类似。

```
>>> name = 'Dong'
>>> age = 39
>>> f'My name is {name}, and I am {age} years old.'
'My name is Dong, and I am 39 years old.'
>>> print(_)
My name is Dong, and I am 39 years old.
```

7.3.3 格式化的字符串常量

```
>>> width = 10
>>> precision = 4
>>> value = 11/3
>>> f'result:{value:{width}.{precision}f}'
'result:    3.6667'

>>> 'result:{0:{1}.{2}f}'.format(value, width, precision)
'result:    3.6667'
```

Practice

- UTF-8和GB2312分别使用几个字节存储英文字母和汉字？

✓ utf-8：1和3字节

✓ gb2312：1和2字节

- ‘f{…}-{…}.txt’.format(12, 2)的输出为‘f012-02.txt’，两个占位符分别怎么写？

✓ 'f{0:03d}-{1:02d}.txt'.format(12, 2) #参数0和1可省

习题

7.1 已知列表对象 x = ['11', '2', '3']，则表达式 max(x) 的值为_____。

7.2 已知列表对象 x = ['11', '2', '3']，则表达式 max(x, key=len) 的值为_____。

7.6 表达式 'a' + 'b' 的值为_____。

7.7 表达式 len('中国共产党'.encode('utf-8')) 的值为_____。

7.11 单选题：表达式 '{},{}'.format(3, 5) 的值为（ ）。

A. '3.5' B. '3,5' C. '35' D. '5,3'

7.12 单选题：表达式 '{1},{0}'.format(3, 5) 的值为（ ）。

A. '1,0' B. '3,5' C. '0,1' D. '5,3'

第3章　文本处理(一)：字符串

- 7.1字符串编码格式简介
- 7.2转义字符与原始字符串
- 7.3字符串格式化
- **7.4字符串常用操作**
- 7.5字符串常量
- 7.6中英文分词
- 7.8精彩案例赏析

7.4 字符串常用操作

- Python字符串对象提供了大量方法用于字符串的切分、连接、替换和排版等操作，另外还有大量内置函数和运算符也支持对字符串的操作 #dir(str)
- 字符串对象是不可变的，所以字符串对象提供的涉及到字符串“修改”的方法都是返回修改后的新字符串，并不对原始字符串做任何修改，无一例外

7.4 字符串常用操作

方法	功能描述
capitalize()	返回新字符串，第一个英文字母大写，其余小写
center(width, fillchar=' ', /) ljust(width, fillchar=' ', /) rjust(width, fillchar=' ', /)	返回指定长度的新字符串，当前字符串在新字符串中居中/居左/居右，如果参数width指定的新字符串长度大于当前字符串长度就在两侧/右侧/左侧使用参数fillchar指定的字符（默认为空格）进行填充；如果参数width指定的长度小于或等于当前字符串的长度，直接返回当前字符串，不会进行截断
count(sub[, start[, end]])	返回字符串sub在当前字符串下标范围[start,end)内不重叠出现的次数，参数start默认值为0，参数end默认值为字符串长度。例如，'abababab'.count('aba')的值为2
encode(encoding='utf-8', errors='strict')	返回当前字符串使用参数encoding指定的编码格式编码后的字节串。对于非ASCII字符，UTF-8编码得到的字节串较长，网络传输时占用带宽较大，保存为文件时占用空间较大；GBK编码得到的字节串更短一些，但有些字符不在GBK字符集中，无法使用GBK编码
endswith(suffix[, start[, end]]) startswith(prefix[, start[, end]])	如果当前字符串下标范围[start,end)的子串以某个字符串suffix/prefix或元组suffix/prefix指定的几个字符串之一结束/开始则返回True，否则返回False

可dir(str)或'a'.查看所有

7.4 字符串常用操作

expandtabs(tabsize=8)	返回新字符串，所有tab键都替换为指定数量的空格。例如，'abcd\te'.expandtabs()返回'abcd e'，'abcdef\te'.expandtabs()返回'abcdef e'
find(sub[, start[, end]]) **rfind(sub[, start[, end]])**	返回字符串sub在当前字符串下标范围[start,end)内出现的最小/最大下标，不存在时返回-1
format(*args, **kwargs)	返回对当前字符串进行格式化（格式化是指把字符串中大括号以及内部变量或编号指定的占位符替换为实际值并以指定的格式呈现）后的新字符串，其中args表示位置参数，kwargs表示关键参数，见5.2节
index(sub[, start[, end]]) **rindex(sub[, start[, end]])**	返回字符串sub在当前字符串下标范围[start,end)内出现的最小/最大下标，不存在时抛出ValueError异常并提示子串不存在
isalnum()、**isalpha()**、 isascii()、isdecimal()、 **isdigit()**、isidentifier()、 islower()、**isnumeric()**、 isprintable()、isspace()、 istitle()、isupper()	测试字符串中字符的类型

7.4 字符串常用操作

join(iterable, /)	使用当前字符串作为连接符把参数iterable中的所有字符串连接成为一个长字符串并返回连接之后的长字符串，要求参数iterable指定的可迭代对象中所有元素全部为字符串
lower() upper()	返回当前字符串中所有字母都变为小写/大写之后的新字符串，非字母字符保持不变
lstrip(chars=None, /) rstrip(chars=None, /) strip(chars=None, /)	返回当前字符串删除左侧/右侧/两侧的空白字符或参数chars中所有字符之后的新字符串
maketrans(...)	根据参数给定的字典或者两个等长字符串对应位置的字符，构造并返回字符映射表（形式上是字典，“键”和“值”都是字符的Unicode编码），如果指定了第三个参数（必须为字符串）则该参数中所有字符都被映射为空值None。该方法是字符串类str的静态方法，可以通过任意字符串进行调用，也可以直接通过字符串类str进行调用
partition(sep, /) rpartition(sep, /)	使用参数sep指定字符串的第一次/最后一次出现作为分隔符，把当前字符串分隔为3个子串，返回包含3个子串的元组。如果指定的子串不存在，返回当前字符串和两个空串组成的元组

7.4 字符串常用操作

removeprefix(prefix, /) removesuffix(suffix, /)	返回删除前缀/后缀之后的新字符串
replace(old, new, count=-1, /)	返回当前字符串中所有子串old都被替换为子串new之后的新字符串，参数count用来指定最大替换次数，默认值-1表示全部替换
rsplit(sep=None, maxsplit=-1) **split(sep=None, maxsplit=-1)**	使用参数sep指定的字符串对当前字符串从后向前/从前向后进行切分，返回包含切分后所有子串的列表。参数sep=None表示使用所有空白字符作为分隔符并丢弃切分结果中的所有空字符串，参数maxsplit表示最大切分次数，默认值-1表示没有限制
splitlines(keepends=False)	使用换行符切分字符串，返回列表
swapcase()	交换英文字母大小写，返回新字符串
title()	返回新字符串，每个单词的首字母大写，其余字母小写
translate(table, /)	根据参数table指定的映射表对当前字符串中的字符进行替换并返回替换后的新字符串，不影响原字符串，参数table一般为字符串方法maketrans()创建的映射表，其中映射为空值None的字符将会被删除而不出现在新字符串中

7.4.1 find()、rfind()、index()、rindex()、count()

- find(sub[, start[, end]])和rfind(sub[, start[, end]])方法分别用来查找另一个字符串在当前字符串指定范围（默认是整个字符串）中首次和最后一次出现的**位置**，如果不存在则返回-1；
- index(sub[, start[, end]])和rindex(sub[, start[, end]])方法用来返回另一个字符串在当前字符串指定范围中首次和最后一次出现的**位置**，如果不存在则抛出异常；
- count(sub[, start[, end]])方法用来返回另一个字符串在当前字符串中出现的次数。

7.4.1 find()、rfind()、index()、rindex()、count()

```
>>> s="apple,peach,banana,peach,pear"
>>> s.find("peach")
6
>>> s.find("peach",7)
19
>>> s.find("peach",7,20) #7-19之间
-1  #不存在
>>> s.rfind('p')
25
>>> s.index('p')
1
>>> s.index('pe')
6
```

```
>>> s.index('pear')
25
>>> s.index('ppp')
Traceback (most recent call last):
  File "<pyshell#11>", line 1, in
<module>
    s.index('ppp')
ValueError: substring not found
>>> s.count('p')
5
>>> s.count('pp')
1
>>> s.count('ppp')
0
```

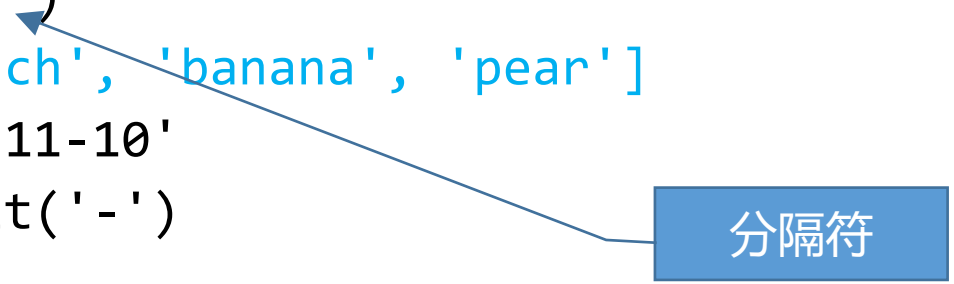
a	p	p	l	e	,	p	e	a	c	h	,	b	a	n	a	n	a	,	p	e	a	c	h	,	p	e	a	r
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

7.4.2 split()、rsplit()

- split(sep=None, maxsplit=-1)和rsplit(sep=None, maxsplit=-1)方法分别用来以参数字符串为分隔符，把当前字符串从左往右或从右往左分隔成多个字符串，返回包含分隔结果的列表。

7.4.2 split()、rsplit()

```
>>> s = 'apple,peach,banana,pear'
>>> s.split(',')
['apple', 'peach', 'banana', 'pear']
>>> s = '2025-11-10'
>>> t = s.split('-')
>>> print(t)
['2025', '11', '10']
>>> print(list(map(int, t)))
[2025, 11, 10]
```



7.4.2 split()、rsplit()

- split()和rsplit()方法允许通过参数maxsplit指定最大分割次数，该参数默认值-1表示没有限制。

```
>>> s = '\n\nhello\t\t world \n\n\n My name is Dong    '
>>> s.split(None, 1)              # 第一个参数None表示使用任意空白字符做分隔符
['hello', 'world \n\n\n My name is Dong    ']
>>> s.rsplit(None, 2)
['\n\nhello\t\t world \n\n\n My name', 'is', 'Dong']
>>> s.split(maxsplit=6)           # 使用关键参数直接指定第二个参数
                                  # 这时第一个参数使用默认值None
['hello', 'world', 'My', 'name', 'is', 'Dong']
>>> s.split(maxsplit=100)         # 最大分隔次数大于可分隔次数时，相当于-1
['hello', 'world', 'My', 'name', 'is', 'Dong']
```

7.4.2 split()、rsplit()

- 对于split()和rsplit()方法，如果不指定分隔符，则字符串中的**任何空白字符（空格、换行符、制表符等）**都将被认为是分隔符，并删除分割结果中的所有空字符串。

```
>>> s = 'hello world \n\n My name is Dong   '
>>> s.split()
['hello', 'world', 'My', 'name', 'is', 'Dong']
>>> s = '\n\nhello world \n\n\n My name is Dong   '
>>> s.split()
['hello', 'world', 'My', 'name', 'is', 'Dong']
>>> s = '\n\nhello\t\t world \n\n\n My name\t is Dong   '
>>> s.split()
['hello', 'world', 'My', 'name', 'is', 'Dong']
```

7.4.2 split()、rsplit()

◆然而，明确传递参数指定split()使用的分隔符时，情况是不一样的。

```
>>> 'a,,,bb,,ccc'.split(',')          # 每个逗号都被作为独立的分隔符
                                       # 相邻逗号直接会得到一个空字符串
['a', '', '', 'bb', '', 'ccc']
>>> 'a\t\t\tbb\t\tccc'.split('\t') # 每个制表符都被作为独立的分隔符
                                       # 相邻制表符之间会得到一个空字符串
['a', '', '', 'bb', '', 'ccc']
>>> 'a\t\t\tbb\t\tccc'.split()      # 连续多个制表符被作为一个分隔符
['a', 'bb', 'ccc']
```

7.4.3 join()

- 字符串方法join(iterable, /)用于连接可迭代对象中的字符串，以当前字符串作为连接符，返回连接后的新字符串。




```
>>> li = ["apple", "peach", "banana", "pear"]
>>> ','.join(li)
'apple,peach,banana,pear'
>>> '.'.join(li)
'apple.peach.banana.pear  '

>>> '.'.join(map(str, [192, 168, 1, 0]))
'192.168.1.0'
>>> ' '.join(map(str, [192, 168, 1, 0]))
'192 168 1 0'
```

7.4.3 join()

- **问题解决：**使用split()和join()方法删除字符串中多余的空白字符，连续多个空白字符只保留一个

```
>>> x = 'aaa      bb      c d e   fff    '
>>> ' '.join(x.split())                  #使用空格作为连接符
'aaa bb c d e fff '
```

当 split() 不提供分隔符时，它会：

✓使用任何空白字符作为分隔符

✓自动合并连续的空白字符

✓忽略开头和结尾的空白字符

这是一个非常经典且实用的字符串处理技巧，用于规范化空白字符！

7.4.3 join()

■**问题解决：**判断两个字符串在Python意义上是否等价

```
>>> def equivalent(s1, s2):          #
      if s1 == s2:
            return True  #字符串内容完全相同
      elif ' '.join(s1.split()) == ' '.join(s2.split()): #此句冗余，可不要
            return True  #多个连续空格被压缩为单个空格后相等
      elif ''.join(s1.split()) == ''.join(s2.split()):
            return True  #移除所有空格后相等
      else:
            return False
>>> equivalent('pip list', 'pip    list')
True
```

Practice

- 字符串的find()和index()方法的区别？

✓ find()方法当被查找字符串不存在时，返回-1

✓ index()则抛出异常

- 'a,b,,d,,'.split(',')返回的列表包含几个元素？

✓ 6个 #5个逗号分出来是6个元素 #['a', 'b', '', 'd', '', '']

- 设x为一个整数列表，将其转化为逗号分隔的字符串

✓ ‘, '.join(map(str, x))

7.4.4 lower()、upper()、capitalize()、title()、swapcase()

- **lower()、upper()**、capitalize()、title()、swapcase()

```
>>> s = "What is Your Name?"
>>> s.lower()                    #返回小写字符串
'what is your name?'
>>> s.upper()                    #返回大写字符串
'WHAT IS YOUR NAME?'
>>> s.capitalize()               #字符串首字符大写
'What is your name?'
>>> s.title()                    #每个单词的首字母大写
'What Is Your Name?'
>>> s.swapcase()                 #大小写互换
'wHAT IS yOUR nAME?'
```

7.4.5 replace()、maketrans()、translate()

- **查**字符串对象的replace(old, new, count=-1, /)方法把指定子字符串的所有出现都替换为另一个字符串，类似于Word中的“**全部替换**”功能，返回替换后的新字符串。必要时可以通过参数count指定替换次数。

```
>>> s = "中国，中国"
>>> s.replace(“中国”，“中华人民共和国”）#两个参数都作为一个整体整理
中华人民共和国，中华人民共和国

>>> s.replace("中国", "中华人民共和国", 1)
'中华人民共和国，中国'
```

7.4.5 replace()、maketrans()、translate()

- **问题解决**：测试用户输入中是否有敏感词，如果有的话就把敏感词替换为3个星号***。

```
>>> words = ('测试', '非法', '暴力', '话')
>>> text = '这句话里含有非法内容'
>>> for word in words:
    if word in text:
        text = text.replace(word, '***')
>>> text
'这句***里含有***内容'
```

7.4.5 replace()、maketrans()、translate()

- 字符串对象的maketrans()方法用来生成字符映射表（**了解**）
- translate()方法用来根据映射表中定义的对应关系转换字符串中的字符。（**了解**）

```
#创建映射表，将字符"abcdef123"一一对应地转换为"uvwxyz@#$"
>>> table = ''.maketrans('abcdef123', 'uvwxyz@#$')
>>> type(table)
<class 'dict'>
>>> table
{97: 117, 98: 118, 99: 119, 100: 120, 101: 121, 102: 122, 49: 64, 50: 35, 51:
  36}
```

7.4.5 replace()、maketrans()、translate()

```
>>> s = "Python is a greate programming language. I like it!"
>>> s.translate(table)
'Python is u gryuty progrumming lunguugy. I liky it!'

>>> table1 = dict(zip(table.values(), table.keys()))
>>> s = 'Python is u gryuty progrumming lunguugy. I liky it!'
>>> s.translate(table1)
'Pethon is a greate programming langaage. I like it!'
```

7.4.5 replace()、maketrans()、translate()

- **问题解决：**凯撒加密，每个字母替换为后面第k个（所有字母排成一个圆）。

```
>>> import string
>>> def kaisa(s, k):
    lower = string.ascii_lowercase          #小写字母
    upper = string.ascii_uppercase          #大写字母
    before = string.ascii_letters
    after = lower[k:] + lower[:k] + upper[k:] + upper[:k]
    table = ''.maketrans(before, after)     #创建映射表
    return s.translate(table)

>>> s = "Python is a greate programming language. I like it!"
>>> s1 = kaisa(s, 3)     #加密
>>> s1
'Sbwkrq lv d juhdwh surjudpplqj odqjxdjh. L olnh lw!'

>>> s2 = kaisa(s1, -3)   #解密
>>> s2
"Python is a greate programming language. I like it!"
```

7.4.5 replace()、maketrans()、translate()

示例：k = 3

```
a b c d e f g h i j k l m n o p q r s t u v w x y z
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
d e f g h i j k l m n o p q r s t u v w x y z a b c

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
D E F G H I J K L M N O P Q R S T U V W X Y Z A B C
```

```
lower = "abcdefghijklmnopqrstuvwxyz"
upper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

# 分解计算:
lower[3:] = "defghijklmnopqrstuvwxyz"    # 从第3位开始到结尾
lower[:3] = "abc"                        # 从开始到第3位（不包括）
upper[3:] = "DEFGHIJKLMNOPQRSTUVWXYZ"    # 从第3位开始到结尾
upper[:3] = "ABC"                        # 从开始到第3位（不包括）

# 组合结果:
after = "defghijklmnopqrstuvwxyz" + "abc" + "DEFGHIJKLMNOPQRSTUVWXYZ" + "ABC"
      = "defghijklmnopqrstuvwxyzabc" + "DEFGHIJKLMNOPQRSTUVWXYZABC"
      = "defghijklmnopqrstuvwxyzabcDEFGHIJKLMNOPQRSTUVWXYZABC"
```

7.4.6 strip()、rstrip()、lstrip()

方法	说明
s.strip([chars])	在其**左侧和右侧**去除chars中列出的字符
s.lstrip([chars])	在其**左侧**去除chars中列出的字符
s.rstrip([chars])	在其**右侧**去除chars中列出的字符

- chars为可选参数，用于指定需要去除的字符，可以指定多个。如果不指定此参数，则默认去除空格、制表符\t、回车符\r和换行符\n等。
- **但不能删除中间部分的字符。**

7.4.6 strip()、rstrip()、lstrip()

- **strip()、rstrip()**、lstrip()

```
>>> s = " abc  "
>>> s.strip()                          #删除两侧空白字符
'abc'
>>> '\n\nhello world   \n\n'.strip()     #删除两侧空白字符以及换行符\n，但中间部分不删除
'hello world'
>>> "aaaassddf".strip("a")             #删除两侧指定字符
'ssddf'

>>> "aaaassddfaaa".rstrip("a")         #删除字符串右侧指定字符
'aaaassddf'
>>> "aaaassddfaaa".lstrip("a")         #删除字符串左侧指定字符
'ssddfaaa'
```

7.4.7 startswith()、endswith()

- s.startswith(t)、s.endswith(t)，判断字符串是否以指定字符串开始或结束

```
>>> s = 'Beautiful is better than ugly.'
>>> s.startswith('Be')                    #检测整个字符串
True
>>> s.startswith('Be', 5)                 # 指定检测范围起始位置
False
>>> s.startswith('Be', 0, 5)              # 指定检测范围起始和结束位置
True
```

#应用1：检测文件名

```
>>> import os
>>> [filename for filename in os.listdir('c:\\') if filename.endswith(('.bmp','.txt','.dll'))]
```

#应用2：检测文件内容

```
>>> [line for line in s.split('\n') if line.strip().endswith('.')]  #以Stray Birds为例
```

第7章 字符串

```
>>> s = """STRAY birds of summer come to my window
to sing and fly away.
And yellow leaves of autumn,
which have no songs,
flutter and fall there with a sigh.

O TROUPE of little vagrants of the world,
leave your footprints in my words.

THE world puts off its mask of vastness to its lover.
It becomes small as one song,
as one kiss of the eternal.

IT is the tears of the earth
that keep her smiles in bloom.

THE mighty desert is burning
for the love of a blade of grass
who shakes her head and laughs
and flies
away.

IF you shed tears when you miss the sun,
you also miss the stars.

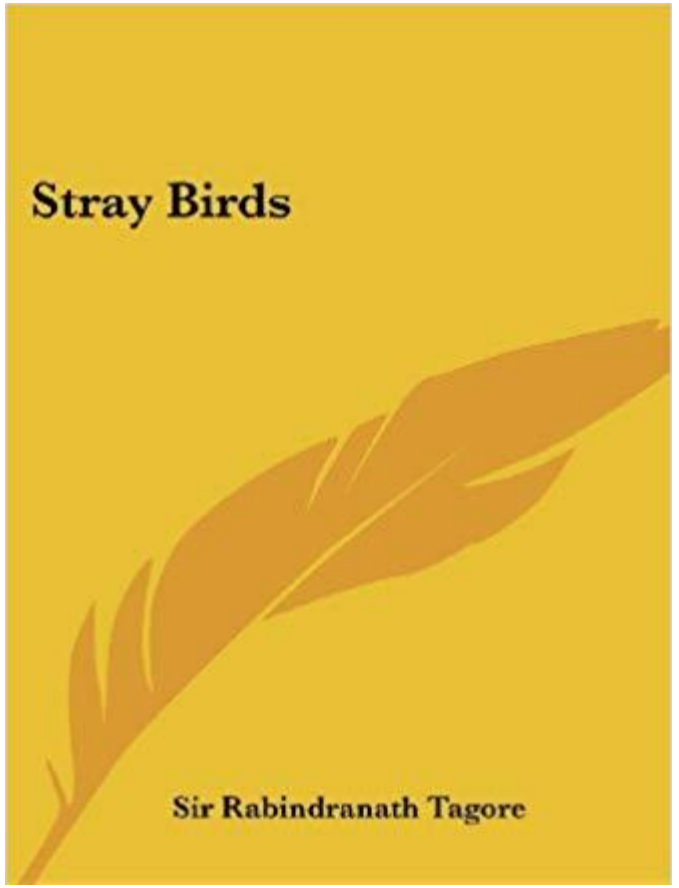
THE sands in your way beg for your song
and your movement,
dancing water.
Will you carry the burden of their lameness?

HER wistful face haunts my dreams
like the rain at night.

ONCE we dreamt that we were strangers.
We wake up to find that we were dear to each other.

SORROW is hushed into peace in my heart
like the evening among the silent trees."""
```

◆ 多行字符串



7.4.7 startswith()、endswith()

#应用2：检测文件内容

```
fp = open(r'C:\Users\xiong\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\红楼梦.txt', 'r', encoding='utf-8')
hlm = fp.read()
for line in hlm.split('\n'):
    if line.startswith('第'):
        print(line)
```

第二回贾夫人仙逝扬州城冷子兴演说荣国府

第三回托内兄如海荐西宾接外孙贾母惜孤女

第二个削肩细腰，长挑身材，鸭蛋脸儿，俊眼修眉，顾盼神飞，文彩精华，见之忘

第四回薄命女偏逢薄命郎葫芦僧判断葫芦案

第五回贾宝玉神游太虚境警幻仙曲演红楼梦

7.4.8 isalnum()、isalpha()、isdigit()、isdecimal()、isnumeric()、isspace()、isupper()、islower()(了解)

- isalnum()、isalpha()、**isdigit()**、isdecimal()、isnumeric()、isspace()、isupper()、islower()，用来测试字符串是否为数字或字母、是否为字母、是否为数字字符、是否为空白字符、是否为大写字母以及是否为小写字母。

```
>>> '1234abcd'.isalnum()          #检查字符串是否只包含字母和数字
True
>>> '1234abcd'.isalpha()          #全部为英文字母时返回True
False

>>> '1234'.isdigit()              #是否为数字字符
True
>>> '九'.isnumeric()               #isnumeric()方法支持汉字数字
True
>>> 'ⅣⅢX'.isnumeric()             #支持罗马数字
True
```

isdigit()	检查字符串是否只包含数字字符
isdecimal()	检查字符串是否只包含十进制数字字符，比 isdigit() 更严格
isnumeric()	检查字符串是否只包含数字字符（包括分数、上标等），范围最广，包含汉字数字、罗马数字等

7.4.9 center()、ljust()、rjust()、zfill()

- center()、ljust()、rjust()，返回指定宽度的新字符串，原字符串居中、左对齐或右对齐出现在新字符串中
- 如果指定宽度大于字符串长度，则使用指定的字符（默认为空格）进行填充
- zfill()返回指定宽度的字符串，在左侧以字符0进行填充

```
>>> 'Hello world!'.center(20)        #居中对齐，以空格进行填充
'    Hello world!     '
>>> 'Hello world!'.center(20, '=')   #居中对齐，以字符=进行填充
'====Hello world!====='
>>> 'Hello world!'.ljust(20, '=')    #左对齐
'Hello world!========='
>>> 'Hello world!'.rjust(20, '=')    #右对齐
'=========Hello world!'
```

7.4.9 center()、ljust()、rjust()、zfill()

```
weather = [("Monday","rainy"),("Tuesday","sunny"),
           ("Wednesday", "sunny"),("Thursday","rainy"),
           ("Friday","cloudy")]

>>> for item in weather:
        print('Weather of {0} is {1}'.format(item[0].rjust(10), item[1].rjust(8)))
```

运行结果：

```
Weather of    Monday is   rainy
Weather of   Tuesday is   sunny
Weather of Wednesday is   sunny
Weather of  Thursday is   rainy
Weather of    Friday is  cloudy
```

7.4.9 center()、ljust()、rjust()、zfill()

```
>>> for line in s.split('\n'):
        print(line.strip().rjust(50))

                 STRAY birds of summer come to my window
                                to sing and fly away.
                          And yellow leaves of autumn,
                                  which have no songs,
                      flutter and fall there with a sigh.

                O TROUPE of little vagrants of the world,
                       leave your footprints in my words.
```

7.4.9 center()、ljust()、rjust()、zfill()

```
for i in range(10):
    print('f' + str(i).zfill(4))
    # print('f' + str(i).rjust(4,'0'))  #亦可
```

```
f0000
f0001
f0002
f0003
f0004
f0005
f0006
f0007
f0008
f0009
```

Practice

- 使用字符串的哪个方法去掉字符串右端的空白字符？

✓ rstrip()

- 设fn为一文件名，如何判断它的后缀为'.exe'??

✓ fn.endswith('.exe')

- 设s = 'IF you shed tears when you miss the sun,you also miss the stars.'，如何得到其中以字母’s’开头的单词组成的列表？

✓
```
s_words = [word for word in s.split() if word.lower().startswith('s')]
print(s_words)
```

✓ ['shed', 'sun,', 'stars.'] #可进一步用replace把’,’ ’.’去掉

✓
```
s_words = [word for word in s.replace(',',' ').replace('.',' ').split() if word.lower().startswith('s')]
print(s_words)
```

✓ ['shed', 'sun', 'stars']

习题

7.5 表达式 len('Hello world!'.ljust(20)) 的值为_____。

7.8 已知 text = '延安精神是中国共产党创造的一种革命精神,主要内容包括:实事求是、理论联系实际的精神，全心全意为人民服务的精神和自力更生艰苦奋斗的精神。'，那么表达式 text.count('精神') 的值为_____。

7.9 表达式 len('::'.join(['a', 'b', 'c'])) 的值为_____。

7.13 单选题：表达式 'Beautiful is better than ugly.'.index('beautiful') 的值为（　　）。

A. 0　　B. -1　　C. 1　　D. 引发异常

习题

7.14 单选题：表达式 len('a,,b'.split(',')) 的值为（　　）。

A. 1　　B. 2　　C. 3　　D. 4

7.15 单选题：表达式 len('a\t\t\tb'.split()) 的值为（　　）。

A. 1　　B. 2　　C. 3　　D. 4

7.16 单选题：表达式 '1234'.upper() 的值为（　　）。

A. '1234'　　B. '一二三四'

C. '壹贰叁肆'　　D. 表达式错误

7.17 单选题：表达式 'abababababa'.rindex('aba') 的值为（　　）。

A. 0　　B. 2　　C. 6　　D. -2

习题

7.18 单选题：表达式 'abababababa'.count('aba') 的值为（　　）。

A. 1　　B. 2　　C. 3　　D. 4

7.20 单选题：已知 table = ''.maketrans('abc', 'ABC')，那么表达式 'dong fuguo'.translate(table) 的值为（　　）。

A. 'dong fuguo'　　B. 'Dong Fuguo'

C. 'Dong FuGuo'　　D. 'Dong fuguo'

7.4.10 字符串对象支持的运算符

- Python字符串支持加法运算符，表示两个字符串连接，生成新字符串。

```
>>> 'hello ' + 'world'
'hello world‘
```

- 成员判断，关键字in

```
>>> "a" in "abcde"      #测试一个字符中是否存在于另一个字符串中
True
>>> 'ac' in 'abcde'     #关键字in左边的字符串作为一个整体对待
False
```

7.4.10 字符串对象支持的运算符

- Python字符串支持与整数的乘法运算，表示字符串内容的重复，得到新字符串

```
>>> 'abcd' * 3
'abcdabcdabcd'
```

7.4.11 适用于字符串对象的内置函数

```
>>> x = 'Hello world.'
>>> list(zip(x,x))                  # 组合对应位置上的字符
[('H', 'H'), ('e', 'e'), ('l', 'l'), ('l', 'l'), ('o', 'o'), (' ', ' '), ('w', 'w'),
('o', 'o'), ('r', 'r'), ('l', 'l'), ('d', 'd'), ('.', '.')]
>>> sorted(x)                       # 按字符Unicode编码排序
[' ', '.', 'H', 'd', 'e', 'l', 'l', 'l', 'o', 'o', 'r', 'w']
>>> list(reversed(x))               # 翻转
['.', 'd', 'l', 'r', 'o', 'w', ' ', 'o', 'l', 'l', 'e', 'H']
>>> list(enumerate(x))              # 枚举字符串中的字符
[(0, 'H'), (1, 'e'), (2, 'l'), (3, 'l'), (4, 'o'), (5, ' '), (6, 'w'), (7, 'o'), (8,
'r'), (9, 'l'), (10, 'd'), (11, '.')]
>>> list(map(lambda i,j: i+j, x, x))
['HH', 'ee', 'll', 'll', 'oo', '  ', 'ww', 'oo', 'rr', 'll', 'dd', '..']
>>> len(x)                          # 字符串长度
12
>>> max(x)                          # 最大字符
'w'
>>> min(x)
' '
```

7.4.11 适用于字符串对象的内置函数

■内置函数eval()用来把任意字符串转化为Python表达式并进行求值

```
>>> eval("3+4")                          #计算表达式的值
7
>>> a = 3
>>> b = 5
>>> eval('a+b')                          #这时候要求变量a和b已存在
8
>>> import math
>>> eval('math.sqrt(3)')
1.7320508075688772

>>> eval('[1,2,3]')
[1, 2, 3]
```

7.4.12 字符串对象的切片操作

- 切片也适用于字符串，但仅限于读取其中的元素，不支持字符串修改。

```
>>> 'Explicit is better than implicit.'[:8]
'Explicit'

>>> path = 'C:\\Python35\\test.bmp'  #修改文件名
>>> path[:-4] + '_new' + path[-4:]
'C:\\Python35\\test_new.bmp'
```

第3章　文本处理(一)：字符串

- 7.1字符串编码格式简介
- 7.2转义字符与原始字符串
- 7.3字符串格式化
- 7.4字符串常用操作
- **7.5字符串常量**
- **7.6中英文分词**
- 7.8精彩案例赏析

7.5 字符串常量（了解）

- Python标准库string中定义数字字符、标点符号、英文字母、大写字母、小写字母等常量。

```
>>> import string
>>> string.digits
'0123456789'
>>> string.punctuation
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
>>> string.ascii_letters
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>> string.ascii_lowercase
'abcdefghijklmnopqrstuvwxyz'
>>> string.ascii_uppercase
'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
```

7.5 字符串常量

- **问题解决：**生成指定长度的随机密码。

```
>>> import string
>>> characters = string.digits + string.ascii_letters
>>> import random
>>> ''.join([random.choice(characters) for i in range(16)])
#''.join(random.choices(characters, k=16))
'zSabpGltJ0X4CCjh'
```

7.6 中英文分词(了解)

```
>>> import jieba                                #导入jieba模块
>>> x = '浙江工业大学真漂亮'
>>> jieba.cut(x)                                #使用默认词库进行分词
<generator object Tokenizer.cut at 0x000001353B6CA140>
>>> list(_)
['浙江工业大学', '真', '漂亮']

>>> list(jieba.cut('花纸杯'))
['花', '纸杯']
>>> jieba.add_word('花纸杯')                    #增加词条
>>> list(jieba.cut('花纸杯'))                   #使用新词库进行分词
['花纸杯']

>>> import snownlp    #导入snownlp模块,Natural Language Processing自然语言处理
>>> snownlp.SnowNLP(x).words
['浙江', '工业', '大学', '真', '漂亮']
```

7.6 中英文分词

```
s = '一个是阆苑仙葩，一个是美玉无瑕。若说没奇缘，今生偏又遇着他；\n若说有奇缘，如何心事终虚话?一个枉自嗟呀，一个空劳牵挂。一个是水中月，一\n个是镜中花。想眼中能有多少泪珠儿，怎禁得秋流到冬，春流到夏!\n'

import jieba
list(jieba.cut(s.replace('\n', '')))

['一个', '是', '阆', '苑仙葩', '，', '一个', '是', '美玉无瑕', '。', '若', '说', '没', '奇缘', '，', '今生', '偏', '又', '遇', '着', '他', '；', '若', '说', '有', '奇缘', '，', '如何', '心事', '终', '虚话', '?', '一个', '枉自', '嗟', '呀', '，', '一个', '空劳', '牵挂', '，', '一个', '是', '水中', '月', '，', '一个', '是', '镜中花', '。', '想', '眼中', '能', '有', '多少', '泪珠儿', '，', '怎禁', '得秋流', '到', '冬', '，', '春流', '到', '夏', '!']

import snownlp
snownlp.SnowNLP(s.replace('\n', '')).words
['一', '个', '是', '阆苑', '仙', '葩', '，', '一个', '是', '美', '玉', '无瑕', '。', '若', '说', '没奇', '缘', '，', '今生', '偏', '又遇', '着', '他', '；', '若', '说', '有奇', '缘', '，', '如何', '心事', '终', '虚话', '?', '一个', '枉', '自', '嗟', '呀', '，', '一个', '空', '劳', '牵挂', '。', '一个', '是', '水中', '月', '，', '一个', '是镜', '中', '花', '。', '想', '眼中', '能', '有', '多少', '泪珠', '儿', '，', '怎', '禁得', '秋流', '到', '冬', '，', '春流', '到', '夏', '!']
```

7.6 中英文分词

特性	jieba	snownlp
主要用途	中文分词	中文**自然语言**处理
核心功能	分词、词性标注、关键词提取	分词、**情感分析**、文本分类、摘要
算法基础	基于词典 + HMM	基于贝叶斯等传统机器学习
分词精度	⭐⭐⭐⭐⭐	⭐⭐⭐⭐
情感分析	**无内置**	⭐⭐⭐⭐⭐
速度	⭐⭐⭐⭐⭐	⭐⭐⭐
自定义性	⭐⭐⭐⭐⭐	⭐⭐⭐
学习成本	低	低

第3章　文本处理(一)：字符串

- 7.1字符串编码格式简介
- 7.2转义字符与原始字符串
- 7.3字符串格式化
- 7.4字符串常用操作
- 7.5字符串常量
- 7.6中英文分词
- **7.8精彩案例赏析**

7.8 精彩案例赏析

- **例7-1** 编写函数实现字符串加密和解密，循环使用指定密钥，采用简单的异或算法。（了解，此法过于简单，一般不用）

- 异或加密的特点：

1. 可逆性：加密和解密使用相同的算法: **明文 ^ 密钥 = 密文, 密文 ^ 密钥 = 明文**
2. 自反性：0^0 = 1^1 = 0, 0^1 = 1^0 = 1, **(A^B)^B = A (A和B为任意整数)**
3. 相同密钥：加密和解密使用相同的密钥: **(明文^密钥)^密钥 = 明文**

```
from itertools import cycle
cycle(iterable) --> cycle object(迭代器)
 |  Return elements from the iterable until it is exhausted.
 |  Then repeat the sequence indefinitely.
```

```
>>> temp = cycle('abc')
>>> for i in range(30):
        print(next(temp), end='')

abcabcabcabcabcabcabcabcabcabc
```

7.8 精彩案例赏析

实现1：

```
def crypt(source, key):
    from itertools import cycle
    result = ''

    temp = cycle(key)  # 创建循环迭代器，使密钥可重复使用

    for ch in source:     #谨慎：循环会降低效率，不适合大量字符处理
        result = result + chr(ord(ch) ^ ord(next(temp)))
                          # 对每个字符进行异或运算
    return result
```

7.8 精彩案例赏析

```
source = 'Hello123'
key = 'key'

encrypted = crypt(source, key)
decrypted = crypt(encrypted, key)
print(source[-100:],repr(encrypted[-100:]),decrypted[-100:], sep='\n')
```

对称加密!

7.8 精彩案例赏析

第一步：准备阶段

•source = "Hello123" (长度：8个字符)
•key = "key" (长度：3个字符)
•cycle(key) 创建无限循环：'k' → 'e' → 'y' → 'k' → 'e' → 'y' → ...

第二步：逐字符处理

步骤	明文字符	ASCII	密钥字符	ASCII	异或运算	结果字符
1	'H'	72	'k'	107	72^107=35	'#'
2	'e'	101	'e'	101	101^101=0	'\x00'
3	'l'	108	'y'	121	108^121=21	'\x15'
4	'l'	108	'k'	107	108^107=7	'\x07'
5	'o'	111	'e'	101	111^101=10	'\n'
6	'1'	49	'y'	121	49^121=72	'H'
7	'2'	50	'k'	107	50^107=89	'Y'
8	'3'	51	'e'	101	51^101=86	'V'

第三步：加密结果

•加密前："Hello123"
•加密后：'#\x00\x15\x07\nHYV'

7.8 精彩案例赏析

第四步：验证解密

使用相同的密钥解密：

步骤	密文字符	ASCII	密钥字符	ASCII	异或运算	结果字符
1	'#'	35	'k'	107	35^107=72	'H'
2	'\x00'	0	'e'	101	0^101=101	'e'
3	'\x15'	21	'y'	121	21^121=108	'l'
4	'\x07'	7	'k'	107	7^107=108	'l'
5	'\n'	10	'e'	101	10^101=111	'o'
6	'H'	72	'y'	121	72^121=49	'1'
7	'Y'	89	'k'	107	89^107=50	'2'
8	'V'	86	'e'	101	86^101=51	'3'

解密成功！
得到原始明文："Hello123"

7.8 精彩案例赏析

实现2：(了解)#用字节串处理，无for循环，效率提高

```
def new_crypt(source, key):
    #参数source为字节串，key为字符串

    key = key.encode() # 将字符串密钥转换为字节串

    #使key的字节串与source一样长
    key = key * (len(source)//len(key)) + key[:len(source)%len(key)]
     # 示例：source长8字节，key长3字节 → key扩展为 key*2 + key[0:2]

    # 将source和key转换为16进制字符串，再转为整数进行异或
    result = int(source.hex(), base=16) ^ int(key.hex(), base=16)
    result = hex(result)[2:] # 将结果转换回16进制字符串，去掉'0x'前缀

    #保证16进制数字字符串的长度为偶数，这样才能作为fromhex的参数
    if len(result)%2 == 1:
        result = '0' + result
    return bytes.fromhex(result)  # 将16进制字符串转换回字节串
```

7.8 精彩案例赏析

=== 加密过程 ===
1. 原始source: b'Hello123'
2. 原始key: 'key'
3. key编码为字节: b'key'
4. 扩展后的key: b'keykeyke'
5. source的16进制: 48656c6c6f313233
6. key的16进制: 6b65796b65796b65
7. 异或结果(十进制): 94530093669029328
8. 异或结果(16进制): d20c05070b045a46
9. 16进制长度正常: d20c05070b045a46
10. 最终字节结果:
b'\xd2\x0c\x05\x07\x0b\x04ZF'

加密结果: b'\xd2\x0c\x05\x07\x0b\x04ZF'

=== 解密过程 ===
1. 原始source:
b'\xd2\x0c\x05\x07\x0b\x04ZF'
...
解密结果: b'Hello123'
解密文本: Hello123

7.8 精彩案例赏析

```
#实现2，无循环，可以高效地运行大量字符

source = '浙江工业大学是一所非常漂亮的大学'
key = 'abcdefg'

encrypted = new_crypt(source.encode(), key)
decrypted = new_crypt(encrypted, key).decode()
print(source[-100:],encrypted[-100:],decrypted[-100:],
sep='\n')
```

7.8 精彩案例赏析

- **例7-2** 编写程序，生成大量随机信息，这在需要获取大量数据来测试或演示软件功能的时候非常有用，不仅能真实展示软件功能或算法，还可以避免泄露真实数据或者引起不必要的争议。

●汉字unicode范围：19968 =< 汉字 <= 40959

#三个随机汉字所组成的名字

```
>>> ''.join(chr(random.randint(19968, 40959)) for i in range(3))
```

#11位随机数字组成的电话号码

```
>>> ''.join(str(random.randint(0,9)) for i in range(11))
```

7.8 精彩案例赏析

■ **例7-3** 检查并判断密码字符串的安全强度。

```
import string


def check(pwd):
    #密码必须至少包含6个字符
    if not isinstance(pwd, str) or len(pwd)<6:
        return 'not suitable for password'
    #密码强度等级与包含字符种类的对应关系
    d = {1:'weak', 2:'below middle', 3:'above middle', 4:'strong'}
    #分别用来标记pwd是否含有数字、小写字母、大写字母和指定的标点符号
    r = [False] * 4
```

7.8 精彩案例赏析

```
    for ch in pwd:
        #是否包含数字
        if not r[0] and ch in string.digits:
            r[0] = True
        #是否包含小写字母
        elif not r[1] and ch in string.ascii_lowercase :
            r[1] = True
        #是否包含大写字母
        elif not r[2] and ch in string.ascii_uppercase:
            r[2] = True
        #是否包含指定的标点符号
        elif not r[3] and ch in ',.!;?<>':
            r[3] = True
    #统计包含的字符种类，返回密码强度
    return d.get(r.count(True), 'error')

print(check('a2Cd3e,'))
```

7.8 精彩案例赏析

测试密码分析

密码：'a2Cd3e,'

包含的字符类型：

数字：'2', '3' → r[0] = True

小写字母：'a', 'e' → r[1] = True

大写字母：'C', 'D' → r[2] = True

标点符号：',' → r[3] = True

r.count(True) = 4 → 返回 'strong'

7.8 精彩案例赏析

■使用**集合运算**实现更简洁：

```
r = [bool(set(pwd) & set(string.digits)),     #交集不为空，则bool为True
bool(set(pwd) & set(string.ascii_lowercase)),
bool(set(pwd) & set(string.ascii_uppercase)),
bool(set(pwd) & set(',.!;?<>'))]

#列表推导式，更符合python风格
r = [bool(set(pwd) & set(s)) for s in (string.digits,
string.ascii_lowercase, string.ascii_uppercase, ',.!;?<>')]
```

作业

```
>>> s = """STRAY birds of summer come to my window
to sing and fly away.
And yellow leaves of autumn,
which have no songs,
flutter and fall there with a sigh.

O TROUPE of little vagrants of the world,
leave your footprints in my words.

THE world puts off its mask of vastness to its lover.
It becomes small as one song,
as one kiss of the eternal.

IT is the tears of the earth
that keep her smiles in bloom.

THE mighty desert is burning
for the love of a blade of grass
who shakes her head and laughs
and flies
away.

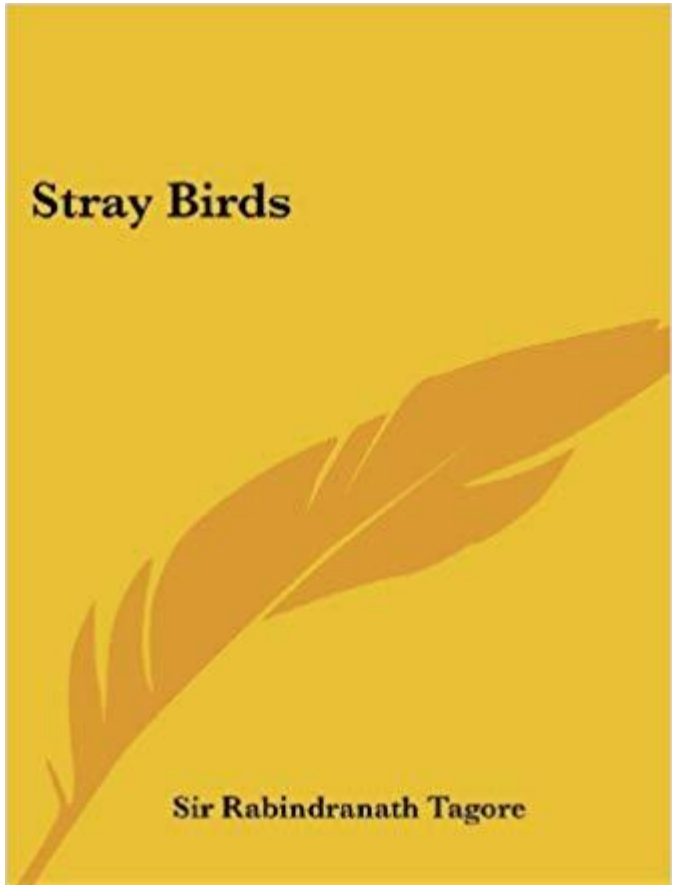
IF you shed tears when you miss the sun,
you also miss the stars.

THE sands in your way beg for your song
and your movement,
dancing water.
Will you carry the burden of their lameness?

HER wistful face haunts my dreams
like the rain at night.

ONCE we dreamt that we were strangers.
We wake up to find that we were dear to each other.

SORROW is hushed into peace in my heart
like the evening among the silent trees."""
```



作业

1. 设字符串s为上一页的诗句，写下程序语句和运行结果：
(1)计算s中所有单词的数目，以变量n_word表示
(2)计算s中所有非空的行的数目，以变量n_line表示
(3)使用字符串的format()方法输出n_word和n_line：格式要求每个数字占6个格

2. 设字符串s为上一页的诗句，写下程序语句和运行结果：
(1)将s中所有的标点符号(,.?)替换为空字符，然后以/将s中所有非空的行连起来，并输出所生成字符串的前50个字符
(2)将s中所有的标点符号(,.?)替换为空字符，然后以,将s中所有的单词连起来，并输出所生成字符串的前50个字符

第9章　文件内容操作

第9章 文件内容操作

- 9.1文件操作基本知识
  - 9.1.1内置函数open()
  - 9.1.2上下文管理语句with
  - 9.1.3文件对象属性与常用方法
  - 9.1.4文件基本操作
- 9.2文本文件内容操作案例精选
- 9.3CSV文件操作
- 9.4json库的使用
- 9.5二进制文件操作案例精选
  - 9.5.1使用pickle模块读写二进制文件



第9章　文件内容操作

- 为了长期保存数据以便重复使用、反复修改和共享，必须将数据以文件的形式存储到外部(相对内存而言)存储介质(如磁盘、U盘、光盘或网盘)中



第9章　文件内容操作

- 按文件中数据的组织形式把文件分为文本文件和二进制文件两类
- **文本文件：**
  - ✓ 文本文件存储的是Unicode字符（**字母、数字、标点、换行符**等）构成的常规字符串，由若干文本行组成，通常每行以换行符**'\n'结尾**，可用**记事本**等编辑并**人类可读**。
  - ✓ 使用正确的编码格式(如utf-8，cp936)可读取和查看文件存储的所有字符
- **二进制文件：**
  - ✓ 二进制文件把对象内容以**字节串**(bytes)进行存储，无法用记事本以文本形式直接阅读和理解，需要使用**专门的软件或工具**进行处理或反序列化。
  - ✓ 图像文件、音视频文件、可执行文件、各类office文档等都属于二进制文件

第9章　文件内容操作

一、常见的文本文件

源代码文件

.txt - 纯文本文件
.py - Python脚本
.java - Java源代码
.cpp / .h - C++源代码
.html - 网页文件
.css - 样式表文件
.js - JavaScript文件
.json - 数据交换格式
.xml - 另一种标记语言

配置文件

.ini - 初始化配置文件
.conf / .config
.yaml / .yml
.properties

日志文件

.log - 系统或软件运行日志

标记语言文件

.md - Markdown文档
.csv - 逗号分隔值文件（本质是纯文本）
.svg - 矢量图形（本质是XML文本）

其他

.bat / .sh - Windows批处理 / Linux Shell脚本
.sql - 数据库查询语句文件

第9章　文件内容操作

二、常见的二进制文件示例
这些文件如果用记事本打开，你会看到大量乱码（如“秭轇罔…”），必须用特定软件。

可执行程序

.exe - Windows应用程序
.dll - Windows动态链接库
.app / .dmg - macOS应用程序
.so / .out - Linux库/可执行文件

文档类（包含复杂格式、字体、图片等）

.pdf - Adobe PDF文档
.docx / .xlsx / .pptx - Microsoft Office文档（新版，本质是ZIP压缩包）
.doc / .xls / .ppt - 旧版Office文档（OLE复合文档）

压缩归档文件

.zip / .rar / .7z - 压缩包
.tar.gz - Linux压缩包

媒体文件

.jpg / .png / .gif - 图片
.mp3 / .wav / .flac - 音频
.mp4 / .avi / .mkv - 视频

系统与专业文件

.db / .sqlite / .mdb - 数据库文件
.iso - 光盘镜像文件
.psd - Photoshop工程文件
.blend - Blender工程文件

虚拟化与序列化文件

.class - Java编译后的字节码文件
.pyc - Python编译后的字节码文件

第9章 文件内容操作

- 9.1文件操作基本知识
  - 9.1.1内置函数open()
  - 9.1.2上下文管理语句with
  - 9.1.3文件对象属性与常用方法
  - 9.1.4文件基本操作
- 9.2文本文件内容操作案例精选
- 9.3CSV文件操作
- 9.4json库的使用
- 9.5二进制文件操作案例精选
  - 9.5.1使用pickle模块读写二进制文件



9.1 文件操作基本知识

- 无论是文本文件还是二进制文件，其操作流程基本都是一致的
- 首先打开文件并创建文件**对象**，
- 然后通过该文件**对象**对文件内容进行读取、写入等操作
- 最后关闭并保存文件内容

9.1.1 内置函数open()

```
open(file, mode='r', encoding=None)
```

✓ file参数指定了被打开**文件的名称**，类型为字符串
- ✓ file参数可为文件的路径，如’C:\\Users\\XHN\\Desktop\\python**zjut.txt**’
- ✓ 路径中的\必须用转义字符’\\’表示，或在字符串前面加r，表示原始字符串

✓ mode参数指定了文件的**打开模式**，类型为字符串
- ✓ 默认为文本只读模式

✓ encoding参数指定对文本进行**编码和解码的方式**，类型为字符串
- ✓ 只适用于文本模式，可以使用’utf-8’等
- ✓ 默认值为系统的默认编码格式，如中文Windows系统的’cp936’。

cp936 是代码页936的简称

- 对应中国的国标编码标准：GBK (汉字内码扩展规范)
- 是GB2312的扩展，支持更多汉字
- 在中文Windows系统中默认使用

- **不同的IDE，默认编码格式也不同**

```
'Spyder': 'utf-8',   # 通常更倾向UTF-8
'IDLE': 'cp936',     # 可能跟随系统默认
'VS Code': 'utf-8',  # 现代编辑器偏好UTF-8
'PyCharm': 'utf-8', # 智能检测，偏好UTF-8
'Jupyter Notebook': 'utf-8', # 通常使用UTF-8
```

9.1.1 内置函数open()

- 文件打开模式

模式	说明
r	读模式（默认模式，可省略），如果文件不存在则抛出异常FileNotFoundError
w	写模式，如果文件已存在，先清空原有内容
x	写模式，创建新文件，如果文件已存在则抛出异常FileExistsError
a	追加模式，不覆盖文件中原有内容
b	二进制模式（可与其他模式组合使用）
t	文本模式（**默认模式**，可省略）
+	读、写模式（可与其他模式组合使用）
"rt"、"wt"、"xt"、"at"	文本模式，默认值
"rb"、"wb"、"xb"、"ab"	二进制文件的读写模式
"+"，例如"r+"、"rb+"等	与r/w/x/a/rb/wb/xb/ab一起使用，在原功能基础上增加读写功能

9.1.1 内置函数open()

- 如果执行正常，open()函数返回1个**文件对象**，通过该文件对象可以对文件进行读写操作。如果指定文件不存在、访问权限不够、磁盘空间不足或其他原因导致创建文件对象失败则抛出异常。

```
>>> f1 = open('C:\\Users\\XHN\\Desktop\\python\\zjut.txt', 'r')     # 以只读模式打开文本文件
>>> f2 = open('C:\\Users\\XHN\\Desktop\\python\\zjut.txt', 'w')   # 以只写模式打开文本文件
```

```
>>> type(f1)
<class '_io.TextIOWrapper'>
#Python的io模块，
#用于处理文本文件的读写操作
```

```
>>> f1.name
 'file1.txt'
>>> f1.mode
'r'
>>> f1.encoding
'cp936'  #IDLE
```

```
>>> f1.name
 'file1.txt'
>>> f1.mode
'r'
>>> f1.encoding
'utf-8'  #Spyder
```

9.1.1 内置函数open()

- 当对文件内容操作完以后，一定要关闭文件对象，这样才能保证所做的任何修改都确实被保存到文件中。

```
>>> f1.closed       #文件状态检查属性
False


>>> f1.close()      #关闭文件的方法


>>> f1.closed
True
```

Practice

- 如何以utf-8编码和只写模式打开文件D:\tmp.txt，并将文件对象赋值给变量fp

✓ fp = open('D:\\tmp.txt','w', encoding='utf-8') # 使用双反斜杠转义（推）

✓ fp = open(r'D:\tmp.txt', 'w', encoding='utf-8') # 使用原始字符串

✓ fp = open('D:/tmp.txt', 'w', encoding='utf-8') # 使用正斜杠（跨平台兼容）

- Windows系统IDLE打开文件默认的编码格式是什么？

✓ cp936

第9章 文件内容操作

- 9.1文件操作基本知识
  - 9.1.1内置函数open()
  - 9.1.2上下文管理语句with
  - 9.1.3文件对象属性与常用方法
  - 9.1.4文件基本操作
- 9.2文本文件内容操作案例精选
- 9.3CSV文件操作
- 9.4json库的使用
- 9.5二进制文件操作案例精选
  - 9.5.1使用pickle模块读写二进制文件



9.1.3 上下文管理语句with

- 在实际开发中，读写文件应优先考虑使用上下文管理语句with
- 关键字with可以**自动**管理资源，不论因为什么原因（哪怕是代码引发了异常）跳出with块，总能保证文件被正确关闭
- 常用于文件操作、数据库连接、网络连接等场合

```
with open('C:\\Users\\XHN\\Desktop\\python\\zjut.txt') as fp:  #注意缩进!
    for line in fp:
        print(repr(line))    #跳出这个with块，文件就被自动关闭
```

- 上下文管理语句with还支持同时打开多个文件

```
with open('test.txt', 'r') as src, open('test_new.txt', 'w') as dst:
      dst.write(src.read())   #实现了文本复制
```

第9章 文件内容操作

- 9.1文件操作基本知识
  - 9.1.1内置函数open()
  - 9.1.2上下文管理语句with
  - 9.1.3文件对象属性与常用方法
  - 9.1.4文件基本操作
- 9.2文本文件内容操作案例精选
- 9.3CSV文件操作
- 9.4json库的使用
- 9.5二进制文件操作案例精选
  - 9.5.1使用pickle模块读写二进制文件



9.1.2 文件对象属性与常用方法

- 文件对象常用方法：**读**、**写**文件等

  - 文件读写操作会自动改变**文件指针**的位置
  - 文件指针移动以**字节**为单位
  - 以只读’r’和只写’w’模式打开文件时，文件指针的初始位置为**文件头**
  - 以追加’a’模式打开文件时，文件指针的初始位置为**文件尾**

9.1.2 文件对象属性与常用方法

方法	功能说明
close()	把缓冲区的内容写入文件，同时关闭文件，并释放文件对象
flush()	把缓冲区的内容写入文件，但不关闭文件
read([size])	从文本文件中读取size个字符（Python 3.x）的内容作为结果返回，或从二进制文件中读取指定数量的字节并返回，如果省略size则表示读取所有内容
readline()	从文本文件中读取一行内容作为结果返回，包含行尾的’\n’
readlines()	把文本文件中的每行文本作为一个字符串存入列表中，返回该列表
seek(offset[, whence])	把文件指针移动到新的字节位置，offset表示相对于whence的位置(按字节计数，而不是字符数目)。whence为0表示从文件头开始计算，1表示从当前位置开始计算，2表示从文件尾开始计算，默认为0
tell()	返回文件指针的当前位置(按字节计数，而不是字符数目)
write(s)	把字符串s写入文件，返回写入字符的个数，不会自动写入换行符
writelines(s)	把字符串列表写入文本文件，不添加换行符

第9章 文件内容操作

- 9.1文件操作基本知识
  - 9.1.1内置函数open()
  - 9.1.2上下文管理语句with
  - 9.1.3文件对象属性与常用方法
  - 9.1.4文件基本操作
- 9.2文本文件内容操作案例精选
- 9.3CSV文件操作
- 9.4json库的使用
- 9.5二进制文件操作案例精选
  - 9.5.1使用pickle模块读写二进制文件



9. 4文件操作基本

1. 读取文件

打开文件后返回一个文件对象，可以通过对此对象操作实现文件读取。需要注意的是：

- 当文件**以文本文件方式打开**时，会按照**字符串**进行读取，字符编码会采用计算机系统编码或者指定编码；
- 当文件**以二进制文件方式打开**时，会按照**字节流**进行读取。

读取文件可以使用：

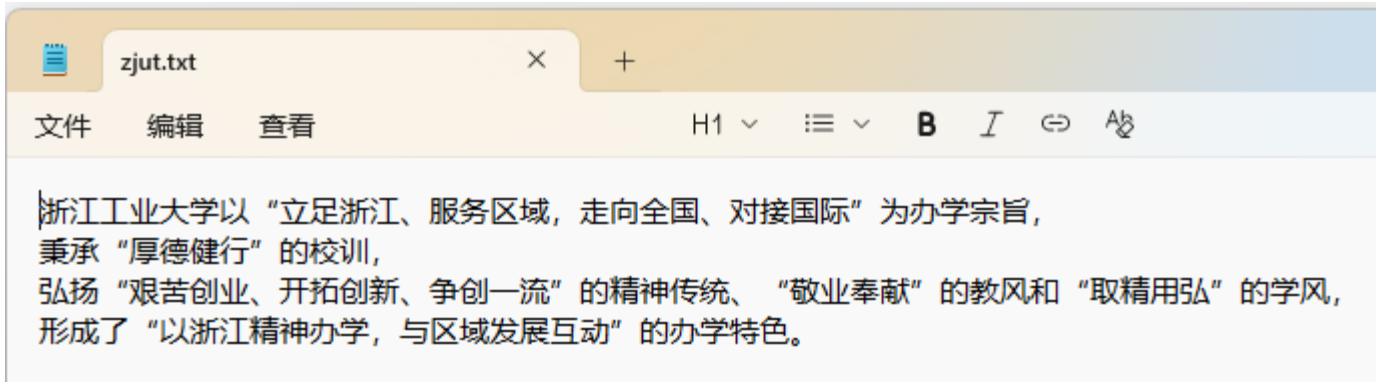
- **read()方法**
- **readlines()方法**
- **readline()方法**
- **for循环遍历的方式**

9.4文件操作基本

1. 读取文件-read()方法

方法	功能说明
read([size])	从文本文件中读取size个字符（Python 3.x）的内容作为结果返回，或从二进制文件中读取指定数量的字节并返回，如果省略size则表示读取所有内容

创建一个zjut.txt文件



9. 4文件操作基本

1. 读取文件-read()方法

```
topdir = "C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\"
with open(topdir+"zjut.txt","r",encoding='utf-8') as fp:
    text = fp.read()
    print(text)
```

浙江工业大学以“立足浙江、服务区域，走向全国、对接国际”为办学宗旨，
秉承“厚德健行”的校训，
弘扬“艰苦创业、开拓创新、争创一流”的精神传统、“敬业奉献”的教风和“取精用弘”的学风，
形成了“以浙江精神办学，与区域发展互动”的办学特色。

9. 4文件操作基本

1. 读取文件-read()方法

read()方法读取test.txt文件的前20个字符

```
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\zjut.txt","r") as fp:
    text = fp.read(20)
    print(text)
```



浙江工业大学以“立足浙江、服务区域，走向

9.4文件操作基本

1. 读取文件-readlines()方法

方法	功能说明
readlines()	把文本文件中的每行文本作为一个字符串存入列表中，返回该列表

9. 4文件操作基本

1. 读取文件-readlines()方法

readlines()方法会一次性读取文件中的所有行，**如果文件非常大，此方法就会占用大量的内存空间，读取的过程也比较长**，对大文件的操作不建议使用此方法。

```
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\zjut.txt","r") as fp:
    text = fp.readlines()    #返回一个列表，列表中的每个元素是文件的一行内容（包括换行符）
    print(text)
```



```
['浙江工业大学以“立足浙江、服务区域，走向全国、对接国际”为办学宗旨，\n', '秉承“厚德健行”的校训，\n', '弘扬“艰苦创业、开拓创新、争创一流”的精神传统、“敬业奉献”的教风和“取精用弘”的学风，\n', '形成了“以浙江精神办学，与区域发展互动”的办学特色。\n']
```

9.4文件操作基本

1. 读取文件-readline()方法

方法	功能说明
readline()	从文本文件中读取**一行内容**作为结果返回，包含**行尾的’\n’**

9. 4文件操作基本

1. 读取文件-readline()方法

```
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\zjut.txt","r") as fp:
    while True:
        text = fp.readline()
        if not text:
            break
        print(text)
```

在test.txt文件中，每行的末尾都有个看不见的换行符，**print()函数默认自动换行，这样每行末尾就会有两个换行符。那么如何去除换行符呢?**

浙江工业大学以“立足浙江、服务区域，走向全国、对接国际”为办学宗旨，

秉承“厚德健行”的校训，

弘扬“艰苦创业、开拓创新、争创一流”的精神传统、 “敬业奉献”的教风和“取精用弘”的学风，

形成了“以浙江精神办学，与区域发展互动”的办学特色。

9. 4文件操作基本

1. 读取文件-readline()方法

使用rstrip()函数去除右端多余空白行

```
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\zjut.txt","r") as fp:
    while True:
        text = fp.readline()
        if not text:
            break
        print(text.rstrip())
```

浙江工业大学以“立足浙江、服务区域，走向全国、对接国际”为办学宗旨，
秉承“厚德健行”的校训，
弘扬“艰苦创业、开拓创新、争创一流”的精神传统、“敬业奉献”的教风和“取精用弘”的学风，
形成了“以浙江精神办学，与区域发展互动”的办学特色。

9. 4文件操作基本

1. 读取文件-for循环遍历文件

文件对象是可迭代对象，故可以通过for循环遍历文件

```
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\zjut.txt","r") as fp:
    for line in fp:  # 这里fp是可迭代对象，每次迭代返回一行
        print(line.rstrip())
```

浙江工业大学以“立足浙江、服务区域，走向全国、对接国际”为办学宗旨，
秉承“厚德健行”的校训，
弘扬“艰苦创业、开拓创新、争创一流”的精神传统、“敬业奉献”的教风和“取精用弘”的学风，
形成了“以浙江精神办学，与区域发展互动”的办学特色。

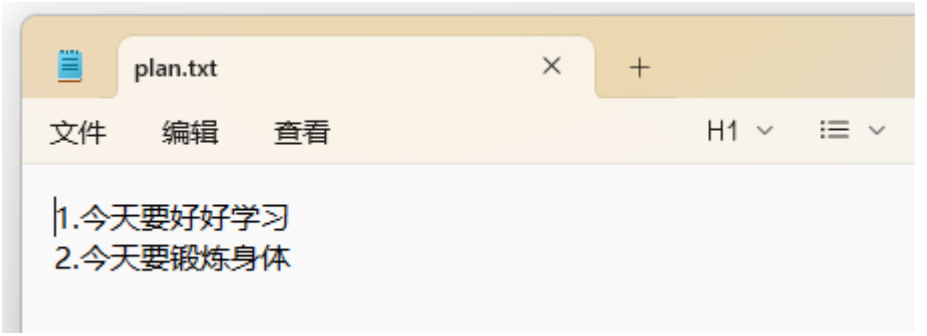
9.4文件操作基本

2. 写入文件-write()方法

方法	功能说明
write(s)	把字符串s写入文件，返回写入字符的个数，不会自动写入换行符

```
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云
\\2025秋冬_python程序设计\\PPT\\py\\today.txt","w") as fp:
    fp.write("1.今天要好好学习\n")
    fp.write("2.今天要锻炼身体\n")
```



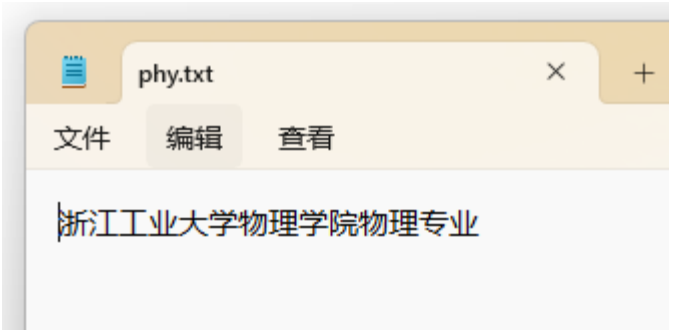


9.4文件操作基本

2. 写入文件-writelines()方法

方法	功能说明
writelines(s)	把**字符串列表**写入文本文件，不添加换行符

```
list01 = ["浙江工业大学","物理学院","物理专业"]
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\phy.txt","w") as fp:
    fp.writelines(list01)
```



9. 4文件操作基本

3. 定位读写位置

写入文件内容后读取该文件

为什么写入文件后，最后没有打印出来呢?

```
list01 = ["人生得意须尽欢\n","莫使金樽空对月\n","天生我材必有用\n","千金散尽还复来\n"]
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\poem.txt","w+") as fp:
    fp.writelines(list01)
    for line in fp:
        print(line.rstrip())
```



写入文件内容后，文件指针指向了文件的末尾位置，此时去读取文件，会从文件末尾开始读取，写入的文件内容在文件指针之前，那么就完全读取不到文件内容。seek()方法可以很好地解决此问题

9.4文件操作基本

3. 定位读写位置——seek()方法

方法	功能说明
seek(offset[, whence])	把文件指针移动到新的**字节位置**，offset表示相对于whence的位置(**按字节计数，而不是字符数目**)。whence为0表示从文件头开始计算，1表示从当前位置开始计算，2表示从文件尾开始计算，默认为0

9. 4文件操作基本

3. 定位读写位置——seek()方法

```
list01 = ["人生得意须尽欢\n","莫使金樽空对月\n","天生我材必有用\n","千金散尽还复来\n"]
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\poem.txt","w+") as fp:
    fp.writelines(list01)
    fp.seek(0)
    for line in fp:
        print(line.rstrip())
```

seek()方法从将文件指针移到文件开头（whence=0）的第0位（offset=0），使得写入文件的内容可以顺利被读取。

```
In [294]: %runcell
人生得意须尽欢
莫使金樽空对月
天生我材必有用
千金散尽还复来
```

9.4文件操作基本

3. 定位读写位置——tell()方法

方法	功能说明
tell()	返回文件指针的当前位置(按字节计数，而不是字符数目)

```
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\poem.txt","w") as fp:
    print("文件指针初始位置：",fp.tell())
    fp.write("李白《将进酒》")
    print("写入内容后文件指针位置：",fp.tell())
```

```
In [296]: %runcell -i 52 C:/
文件指针初始位置：  0
写入内容后文件指针位置：  21
```

Practice

- 如何读取文件对象fp的全部内容，并赋值给s？

✓ `s = fp.read()`

- 如何逐行遍历文件对象fp的内容，打印每一行的长度？

✓ `for line in fp:`

✓ `    print(len(line))`

第9章 文件内容操作

- 9.1文件操作基本知识
  - 9.1.1内置函数open()
  - 9.1.2上下文管理语句with
  - 9.1.3文件对象属性与常用方法
  - 9.1.4文件基本操作
- 9.2文本文件内容操作案例精选
- 9.3CSV文件操作
- 9.4json库的使用
- 9.5二进制文件操作案例精选
  - 9.5.1使用pickle模块读写二进制文件



9.2 文本文件内容操作案例精选

- **示例9-1** 向文本文件中写入内容，然后再读出。

```
s = 'Hello world\n文本文件的读取方法\n文本文件的写入方法\n'

topdir = "C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计
   \\PPT\\py\\"
with open(topdir+'sample.txt', 'w',encoding='utf-8') as fp:    #更推荐此方式
    fp.write(s)

with open(topdir+'sample.txt','r',encoding='utf-8') as fp:
    print(fp.read())
```

```
#或者
s = 'Hello world\n文本文件的读取方法\n文
本文件的写入方法\n'

with open(topdir+'sample.txt',
'w+',encoding='utf-8') as fp:    #改成读写
模式
    fp.write(s)
    fp.seek(0)    #指针回到文件头再开始读取
    print(fp.read())
```

9.2 文本文件内容操作案例精选

- **示例9-2** 将一个文本文件中的内容全部复制到另一个文本文件中。

```
with open(topdir+'sample.txt', 'r', encoding='utf-8') as p:
    with open(topdir+'sample_new', 'w', encoding='utf-8') as n:  #也可以同时打开两个文件
        n.write(p.read())
```

#poem_new后面没有加.txt

- 扩展名（如 .txt）只是约定俗成，不是强制要求 #但建议最好加上
- 操作系统通过文件头内容而不是扩展名来判断文件类型

9.2 文本文件内容操作案例精选

- **示例9-5** 假设文件data.txt中有若干行整数，每一行整数之间使用英文分隔，编写程序读取所有整数，将其按升序排序后再写入文本文件data_asc.txt中。

```
topdir = 'C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\'

with open(topdir+'data.txt', 'r') as fp:
    data = fp.read()

data = data.replace(',', ' ')

data = sorted(int(item) for item in data.split())

with open(topdir+'data_asc.txt', 'w') as fp:
    fp.write(' '.join(str(num) for num in data))  #write()只能接收字符串
```

9.2 文本文件内容操作案例精选

- **示例9-6** 统计文本文件中最长行的长度和该行的内容。

```
with open(topdir+'zjut.txt') as fp:
    lines = fp.readlines()

max(lines, key=len)
```

['浙江工业大以“立足浙江、服务区域，走向全国、对接国际”为办学宗旨，\n', '秉承“厚德健行”的校训，\n', '弘扬“艰苦创业、开拓创新、争创一流”的精神传统、“敬业奉献”的教风和“取精用弘”的学风，\n', '形成了“以浙江精神办学，与区域发展互动”的办学特色。']

'弘扬“艰苦创业、开拓创新、争创一流”的精神传统、“敬业奉献”的教风和“取精用弘”的学风，\n'

第9章 文件内容操作

- 9.1文件操作基本知识
  - 9.1.1内置函数open()
  - 9.1.2上下文管理语句with
  - 9.1.3文件对象属性与常用方法
  - 9.1.4文件基本操作
- 9.2文本文件内容操作案例精选
- 9.3CSV文件操作
- 9.4json库的使用
- 9.5二进制文件操作案例精选
  - 9.5.1使用pickle模块读写二进制文件



9.3 CSV文件操作

9.3.1 CSV文件概述

一维数据是最简单的数据组织类型，数据之间可以用一个或多个空格分隔，也可以通过英文半角逗号或者其它符号组合进行分隔。二维数据由多条一维数据构成，本节将介绍一种国际通用的一**二维**数据存储格式——**CSV格式。逗号分隔值（Common-Separated Values,CSV）格式是一种由逗号分隔数值的存储格式，也是一种文件格式**，常用于**转移表格数据**。

姓名,学号,年龄,年级
小千,202201,19,大二
小锋,202202,20,大三
小狮,202203,19,大二

9.3 CSV文件操作

9.3.1 CSV文件概述

CSV格式有以下规则:

- **纯文本格式**，字符需要时单一编码。
- 开头不留空行，以行为单位，一行数据不跨行，**行之间没有空行。**
- 包含或者不包含列名均可，**包含列名时需要放在文件第一行。**
- 每行表示一个一维数据，多行表示二维数据。
- 每列数据之间**以英文半角逗号分隔**，列数据**为空时也要保留逗号。**

9.3 CSV文件操作

9.3.1 CSV文件概述

CSV格式的**后缀名一般是“.csv”**，可以通过Windows系统中的**记事本**或者微软中的**Office Excel**工具打开，也可以通过其他文本编辑工具打开。

	A	B	C	D	E	F	G	H	I
1	姓名	学号	年龄	年级					
2	小千	202201	19	大二					
3	小锋	202202	20	大三					
4	小狮	202203	19	大二					
5									
6									
7									
8									
9									
10									

Aim

用Office Excel工具打开CSV格式的文件

9.3 CSV文件操作

9.3.2 写入CSV文件

写入CSV文件，可以通过：

- csv模块的**writer对象向CSV文件中写入序列**；
- csv模块中的**DictWriter类以字典的形式写入数据。**

导入csv模块的具体语法格式如下。

```
import csv
```

9.3 CSV文件操作

9.3.2 写入CSV文件-**以序列形式写入CSV文件**

以序列形式写入CSV文件，需要先用到csv模块中的**writer()方法**将文件对象转为**writer对象**。

```
csv.writer(csvfile)
```

csvfile是打开的csv文件

9.3 CSV文件操作

9.3.2 写入CSV文件-**以序列形式写入CSV文件**

writerow()和writerows()方法可以将序列内容写入csv文件

方法	说明
csvwriter.writerow(row)	单行写入，将序列的所有元素写入到CSV文件的一行中
csvwriter.writerows(rows)	多行写入，将序列中的每个元素逐行写到CSV文件中

9.3 CSV文件操作

9.3.2 写入CSV文件-**以序列形式写入CSV文件**

将列表[["名称","数量"],["苹果","18"],["西瓜","10"]]使用**writerow()方法**写入fruit.csv文件中。

```
import csv
list01 = [["名称","数量"],["苹果","18"],["西瓜","10"]]
topdir = 'C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\'
with open(topdir+"fruit.csv","w",encoding="utf-8") as fp:
    writer = csv.writer(fp)
    writer.writerow(list01)
```



```
"['名称', '数量']","['苹果', '18']","['西瓜', '10']"
```

9.3 CSV文件操作

9.3.2 写入CSV文件-**以序列形式写入CSV文件**

使用**writerows()方法**将列表写入fruit.csv文件

```
import csv
list01 = [["名称","数量"],["苹果","18"],["西瓜","10"]]
topdir = 'C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬
_python程序设计\\PPT\\py\\'
with open(topdir+"fruit.csv","w",encoding="utf-8") as fp:
    writer = csv.writer(fp)
    writer.writerows(list01)
```



```
名称,数量

苹果,18

西瓜,10
```

在fruit.csv文件中，**每一行数据下都有一行空行**，这是由于writerows()方法将字符串逐行写进文件时，**windows系统会将\n转换为系统默认的换行符\r\n**。为了解决这个问题，**可以在open()函数中设置参数newlines为""或者"\n"。**

9.3 CSV文件操作

9.3.2 写入CSV文件-**以序列形式写入CSV文件**

在open()函数中设置参数**newlines为""**

```
import csv
list01 = [["名称","数量"],["苹果","18"],["西瓜","10"]]
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计
\\PPT\\py\\fruit.csv","w",newline="") as fp:
    writer = csv.writer(fp)
    writer.writerows(list01)
```



```
名称,数量
苹果,18
西瓜,10
```

9.3 CSV文件操作

9.3.2 写入CSV文件-**以字典形式写入CSV文件**

以字典形式写入CSV文件需要用到csv模块中的DictWriter类

csv.**Dict**Writer(fp,fieldnames)

fp是指文件对象，**fieldnames是指要写入的字典的键**。创建DictWriter类的对象后，**需要调用writeheader()方法写入表头**，并结合writerow()或者writerows()方法写入表的内容

9.3 CSV文件操作

9.3.2 写入CSV文件-**以字典形式写入CSV文件**

以字典形式写入CSV文件

```
import csv
student_dict = [
    {"name":"小千","age":"19"},
    {"name":"小锋","age":"20"},
]
fileheader = ["name","age"]
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\student.csv","w",newline="") as fp:
    writer = csv.DictWriter(fp,fileheader)
    writer.writeheader()
    writer.writerows(student_dict)
```



```
name,age
小千,19
小锋,20
```

9.3 CSV文件操作

9.3.3 读取CSV文件-reader对象读取CSV文件

创建**reader对象**需要用到**reader()方法**

```
csv.reader(csvfile)
```

csvfile是打开的csv文件。**reader对象是可迭代对象，可以通过for循环遍历。**

9.3 CSV文件操作

9.3.3 读取CSV文件-reader对象读取CSV文件

student.csv

```
name,age
小千,19
小锋,20
```

```
import csv
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\student.csv","r") as fp:
    reader = csv.reader(fp)
    for item in reader:  #reader是可迭代对象
        print(item)
```

```
['name', 'age']
['小千', '19']
['小锋', '20']
```

9.3 CSV文件操作

9.3.3 读取CSV文件-reader对象读取CSV文件

创建DictReader类的对象

csv.DictReader(fp,fieldnames)

fp是指文件对象，fieldnames是一个序列，如果省略fieldnames，则文件f第一行中的值将用作fieldnames。**DictReader类的对象也是可迭代对象**，可以用for循环遍历。

```
import csv
with open("C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py\\student.csv","r") as fp:
    reader = csv.DictReader(fp)
    for item in reader:   #reader是可迭代对象
        print(item)
```



```
{'name': '小千', 'age': '19'}
{'name': '小锋', 'age': '20'}
```

9.3 CSV文件操作

- **例9-8** 使用csv模块读写文件内容。

```
import csv

with open('test.csv', 'w', newline='', encoding='utf-8') as fp:  #newline=''防止在 Windows 系统下出现空行
    test_writer = csv.writer(fp)                    #生成一个csv可写对象
    test_writer.writerow(['name', 'age', 'city'])  # 写入一行内容，内容用序列表示
    test_writer.writerow(['Alice',25,'Beijing'])
    test_writer.writerow(['Bob',30,'Shanghai'])
    test_writer.writerow(['Charlie',35,'Guangzhou'])

with open('test.csv','r', newline='', encoding='utf-8') as fp:      #默认mode='r'
    test_reader = csv.reader(fp)          #生成一个csv可读对象
    for row in test_reader:                    # 遍历所有行
        print(row)                            # 每行作为一个列表返回
```

```
['name', 'age', 'city']
['Alice', '25', 'Beijing']
['Bob', '30', 'Shanghai']
['Charlie', '35', 'Guangzhou']
```

9.3 CSV文件操作

```
import csv

data = [                                                    # 写入字典数据  #更常用的方式
    {'name': 'Alice', 'age': 25, 'city': 'Beijing'},
    {'name': 'Bob', 'age': 30, 'city': 'Shanghai'},
    {'name': 'Charlie', 'age': 35, 'city': 'Guangzhou'}
]

with open('dict_data.csv', 'w', newline='', encoding='utf-8') as fp:
    fieldnames = ['name', 'age', 'city']     #键  # 定义列的顺序
    writer = csv.DictWriter(fp, fieldnames=fieldnames)

    writer.writeheader()  # 写入标题行
    writer.writerows(data)  # 写入所有数据

with open('dict_data.csv', 'r', newline='', encoding='utf-8') as fp:
    reader = csv.DictReader(fp)
    for row in reader:
        print(row)
        # 可以直接用字段名访问
```

```
{'name': 'Alice', 'age': '25', 'city': 'Beijing'}
{'name': 'Bob', 'age': '30', 'city': 'Shanghai'}
{'name': 'Charlie', 'age': '35', 'city': 'Guangzhou'}
```

第9章 文件内容操作

- 9.1文件操作基本知识
  - 9.1.1内置函数open()
  - 9.1.2上下文管理语句with
  - 9.1.3文件对象属性与常用方法
  - 9.1.4文件基本操作
- 9.2文本文件内容操作案例精选
- 9.3CSV文件操作
- 9.4json库的使用
- 9.5二进制文件操作案例精选
  - 9.5.1使用pickle模块读写二进制文件



9.4 json库的使用

9.4.1 json库的基本介绍

在Python中处理JSON格式的是**json库**(内置标准库)，主要用于数据的序列化和反序列化，是现代 Web 开发和数据交换的核心工具。

```
import json
```

JSON（JavaScript Object Notation）是一个轻量级的**数据交换**格式，Python标准库json完美实现了对该格式的支持，使用dumps()函数把**对象序列化**为**字符串**，使用loads()函数把JSON格式**字符串还原**为Python**对象**，使用dump()函数把数据**序列化**并直接**写入文件**，使用load()函数读取JSON格式**文件**并直接**还原**为Python对象。

9.4 json库的使用

9.4.1 json库的基本介绍

JSON是**通用**数据交换格式，而Python对象是内存中的**特定**数据结构。

场景	需要JSON字符串	原因
网络传输	✅	网络协议传输的是字节流/字符串
文件存储	✅	文件内容必须是字符串/字节
跨系统通信	✅	**不同编程语言都能理解JSON**
人类可读	✅	JSON格式易于阅读和编辑
API接口	✅	Web标准数据格式
纯Python内部	❌	可以直接使用Python对象

JSON之于数据交换，类似于英语之于国际交流。

9.4 json库的使用

9.4.1 json库的基本介绍

JSON格式存在以下语法规则。

- 数据**保存在键值对中**。
- 数据之间**由英文半角逗号分隔**。
- 花括号**“{}”用于保存键值对数据组成的对象**。
- 方括号**“[]”用于保存数组**，数组可以包括多个键值对数据组成的对象。

```
{
  "岗位信息": [
    {"岗位名称":"程序员","薪资":"15k"},
    {"岗位名称":"产品经理","薪资":"12k"},
    {"岗位名称":"软件测试","薪资":"10k"}
  ]
}
```

9.4 json库的使用

9.4.2 json库的常用操作

dumps()和loads()函数 vs dump()和load()函数

- 常用参数（两者通用）

  - s 代表 string（字符串）

dumps = dump to string
loads = load from string

  - 没有 s 的代表 file（文件）

dump = dump to file
load = load from file

9.4 json库的使用

9.4.2 json库的常用操作 dumps()和loads()函数的使用

```
import json
# Python对象
data = {'users': [{'id': 1, 'name': '张三'}, {'id': 2, 'name': '李四'}]}

# 转换为美观的JSON字符串
json_str = json.dumps(data, ensure_ascii=False, indent=2)
print(json_str)

#还原为python
data_p = json.loads(json_str)
print(data_p)
```

```
{
  "users": [
    {
      "id": 1,
      "name": "张三"
    },
    {
      "id": 2,
      "name": "李四"
    }
  ]
}
```

```
{'users': [{'id': 1, 'name': '张三'}, {'id': 2, 'name': '李四'}]}
```

ensure_ascii=False：确保中文字符（如“张三”）正常显示，而不是被转义为\u5f20\u4e09。
这是现代Python开发的**推荐做法**。

indent=2： 生成格式化的、带缩进的字符串，便于人类阅读和调试。

9.4 json库的使用

```
# Python内部处理（对象形式）
order = {
    "order_id": "ORD123456",
    "customer": {"id": 789, "name": "张三"},
    "items": [
        {"product": "手机", "price": 2999, "quantity": 1},
        {"product": "耳机", "price": 299, "quantity": 2}
    ],
    "total": 3597,
    "status": "pending"
}
```

```
# 需要转换为JSON的场景：

# 1. 发送给支付系统
payment_data = json.dumps({
    "order_id": order["order_id"],
    "amount": order["total"]
})

# 2. 保存到日志文件
with open('orders.log', 'a') as f:
    f.write(json.dumps(order) + '\n')

# 3. 推送到前端页面
# 前端JavaScript可以直接解析：JSON.parse(orderJson)将接收到的JSON字符串转换回JavaScript对象，然后动态更新网页内容。

# 4. 发送到消息队列（Kafka/RabbitMQ）
#send_to_message_queue(json.dumps(order))
#库存系统、物流系统、营销系统等可以各自独立地从队列中获取消息，按自己的节奏处理数据。
```

9.4 json库的使用

9.4.2 json库的常用操作 dump()和load()函数的使用

- **例9-9** 使用标准库json读写JSON格式的文本文件。

```
import json
with open('test.txt', 'w') as fp:
    json.dump({'a':1, 'b':2, 'c':3}, fp) # 写入文件

with open('test.txt', 'r') as fp:
    print(json.load(fp))                 # 从文件中读取
```

```
{'a': 1, 'b': 2, 'c': 3}
```

Practice

- 分别使用标准库json的什么方法将数据写入文件和从文件读取数据？

✓ json.dump()和json.load()

- 使用文件对象fp的什么方法读取每一行到一个列表中？

✓ lines = fp.readlines()

第9章 文件内容操作

- 9.1文件操作基本知识
  - 9.1.1内置函数open()
  - 9.1.2上下文管理语句with
  - 9.1.3文件对象属性与常用方法
  - 9.1.4文件基本操作
- 9.2文本文件内容操作案例精选
- 9.3CSV文件操作
- 9.4json库的使用
- 9.5二进制文件操作案例精选
  - 9.5.1使用pickle模块读写二进制文件



9.3 二进制文件操作案例精选

- 对于二进制文件(如docx文档),必须正确理解其文件结构和序列化规则，然后设计正确的反序列化规则(如使用Word软件、python-docx模块)，才能准确地解析二进制文件内容
- 所谓序列化，简单地说就是把内存中的数据在不丢失其类型信息的情况下转成二进制形式的过程，对象序列化后的数据经过正确的反序列化过程应该能够准确无误地恢复为原来的对象
- Python中常用的序列化模块有struct、**pickle**、shelve、marshal
- pickle可以处理Python所有的内置类型

9.3 二进制文件操作案例精选

■pickle可以保存/恢复任意Python对象

特点：

- Python专属：只能Python使用
- 功能强大：几乎可序列化任何对象
- 二进制格式：紧凑高效
- 安全风险：不要加载不可信数据

适用场景：

- 机器学习模型保存（scikit-learn, PyTorch）
- 复杂Python对象存储
- Python进程间通信
- 临时缓存

9.3.1 使用pickle模块读写二进制文件

- **示例9-12** 使用pickle模块完整的保存python数据



```
import pickle

i = 13000000
a = 99.056
s = '中国人民 123abc'
lst = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
tu = (-5, 10, 8)
coll = {4, 5, 6}
dic = {'a':'apple', 'b':'banana', 'g':'grape', 'o':'orange'}
data = (i, a, s, lst, tu, coll, dic)

with open('sample_pickle.dat', 'wb') as fp:
    for item in data:
        pickle.dump(item, fp)          #序列化数据并写入文件

with open('sample_pickle.dat', 'rb') as fp:
    for _ in range(7):  #向文件中dump几次，就只能load几次
        v = pickle.load(fp)  #读取平反序列化每个数据
        print(type(v), v)
```

9.3.1 使用pickle模块读写二进制文件

- **示例9-13** 把文本文件test.txt中的所有信息使用pickle进行序列化并写入二进制文件test_pickle.dat。

```
import pickle

with open('test.txt') as t, open('test_pickle.dat', 'wb') as d:
    for line in t:
        pickle.dump(line, d)

with open('test_pickle.dat', 'rb') as fp:
    while True:
        try:
            print(pickle.load(fp))
        except:
            break
```

作业7.9

1.
```
student = {
    "id": "2024001",
    "name": "小明",
    "age": 19,
    "scores": [85, 92, 78],
    "is_active": True
}
```

(1).将学生数据保存到 student.json 文件中,要求使用 with open() 打开文件,使用 json.dump() 写入数据。

(2).读取 JSON 文件，要求使用 with open() 打开文件，使用 json.load() 读取数据，打印读取到的数据，计算学生的平均分并显示。

作业7.9

2.

```
a = [("张三", 18, 95.5), ("李四", 19,True),{1, 2, 3, 4, 5},{"apple",
"banana", "cherry"},["列表元素", 1, 2, 3],{"姓名": "王五","年龄":
20},12345,3.14159,"字符串元素"]
```

使用pickle模块将将列表a保存到二进制文件lista.dat文件中，并重新load到列表b中，打印b中类型为元组和集合的元素。

第10章　文件与文件夹操作



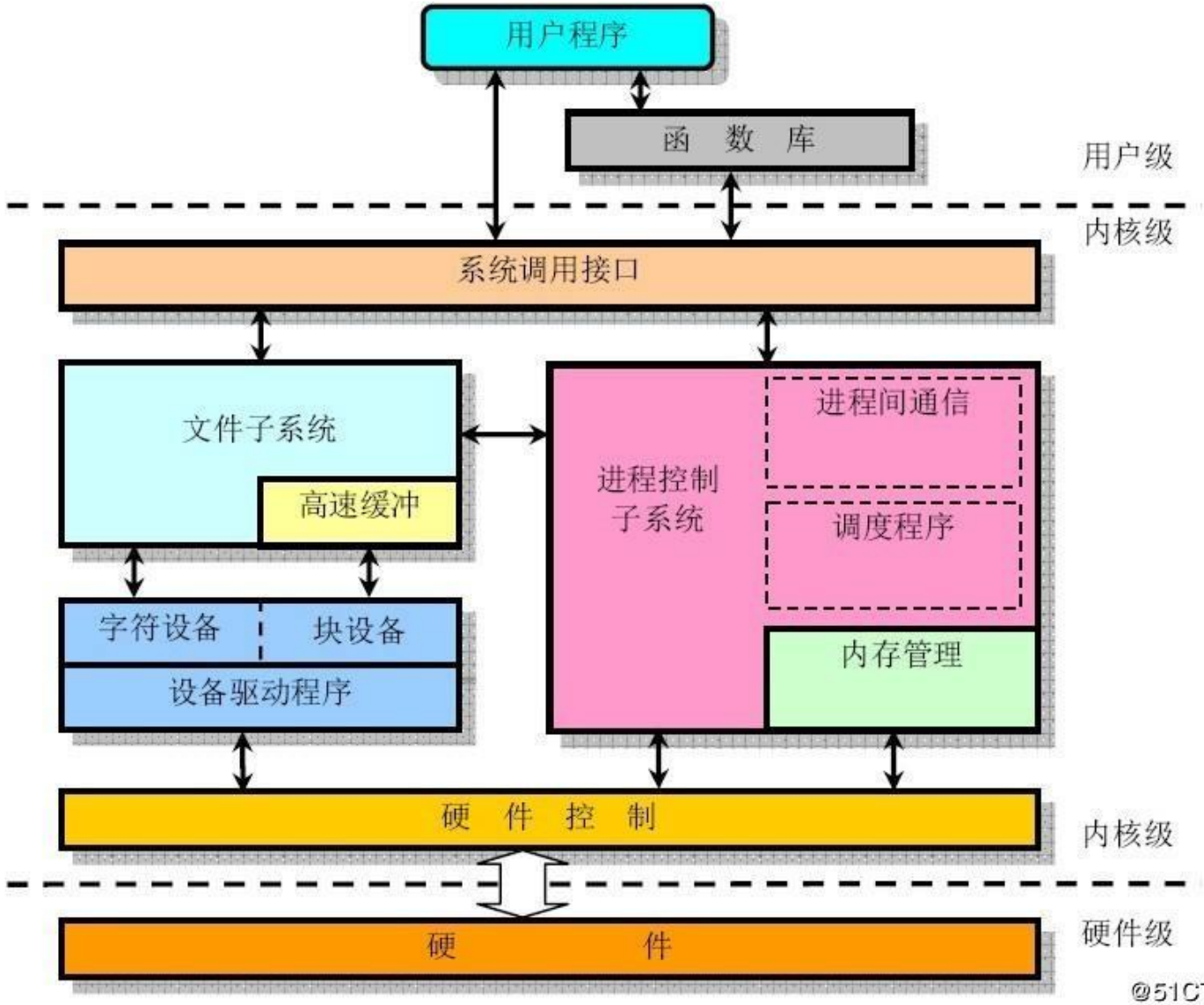






第10章 文件与文件夹操作

- 10.1 os模块
- 10.2 os.path模块
- 10.3 shutil模块
- 10.4 精彩案例赏析



@51CTO博客

10.1 os模块

- os模块提供处理NT或Posix系统的操作系统(OS)函数
  - NT: Windows系OS
  - Posix: Unix系OS

- os模块中的函数均以字符串表示路径

- 建议Windows路径中使用'\\'表示\，而不是原始字符串
  - 如D盘根目录可写作'D:\\'，而r'D:\'是有问题的

- Unix系OS的路径以/分隔

- 若要求程序可跨NT和Posix平台使用，应尽量使用os模块函数处理路径，而不是使用字符串操作

10.1 os模块

函数	功能说明
chdir(path)	把path设为当前工作目录
curdir	当前文件夹，即’.’
environ	包含系统环境变量和值的字典
extsep	当前操作系统所使用的文件扩展名分隔符
get_exec_path()	返回可执行文件的搜索路径
getcwd()	返回当前工作目录
listdir(path)	返回path目录下的文件和目录列表

10.1 os模块

函数	功能说明
remove(path)	删除指定的文件，要求用户拥有删除文件的权限，并且文件没有只读或其他特殊属性
rename(src, dst)	重命名文件或目录，可以实现文件的移动，若目标文件已存在则抛出异常，不能跨越磁盘或分区
replace(old, new)	重命名文件或目录，若目标文件已存在则直接覆盖，不能跨越磁盘或分区
scandir(path='.')	返回包含指定文件夹中所有DirEntry对象的迭代对象，遍历文件夹时比listdir()更加高效
sep	当前操作系统所使用的路径分隔符
startfile(filepath [, operation])	**使用关联的应用程序打开指定文件或启动指定应用程序**
system()	**启动外部程序**

10.1 os模块

特性	.py(Python Source File)	.pyc(Python Compiled File)	.pyw(Python Windowed File)
性质	源代码	编译后的字节码（缓存）	源代码（Windows专用）
内容	人类可读的纯文本	二进制字节码，不可读	人类可读的纯文本

```
>>> import os

>>> cdir = 'C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py'
>>> [fname for fname in os.listdir(cdir)  #列出目录内容(列表) #listdir 是 list directory 的缩写
    if fname.endswith(('.pyc', '.py', '.pyw'))]

>>> os.getcwd()                          #返回当前工作目录 #getcwd 是 get current working directory 的缩写
'C:\\Users\\XHN'

>>> os.chdir(os.getcwd()+'\\.spyder-py3')        #改变当前工作目录 #chdir 是 change directory 的缩写
>>> os.getcwd()
'C:\\Users\\XHN\\.spyder-py3'
>>> os.listdir('.')        # 一个点表示当前目录

>>> os.chdir('..')         #两个点表示上级目录
'C:\\Users\\XHN'
```

10.1 os模块

```
>>> os.startfile('notepad.exe')          #启动记事本程序
>>> os.getcwd()                          #还可以继续执行其他指令
'C:\\Users\\XHN'
>>> os.system('notepad.exe')             #程序处于等待状态，无法执行其他指令

0                                        #直到记事本被关闭后，该函数才执行完毕
>>> os.getcwd()
```

特性	os.startfile()	os.system()
平台	仅Windows	跨平台(Windows、Linux、macOS)
执行方式	异步（立即返回）	同步（阻塞当前进程，等待程序结束）
主要用途	打开文件/文档	执行系统命令
返回值	None	退出状态码(通常 0 表示成功，非零表示错误)

Practice

- os.listdir('.')返回值的类型是什么?内容是什么?

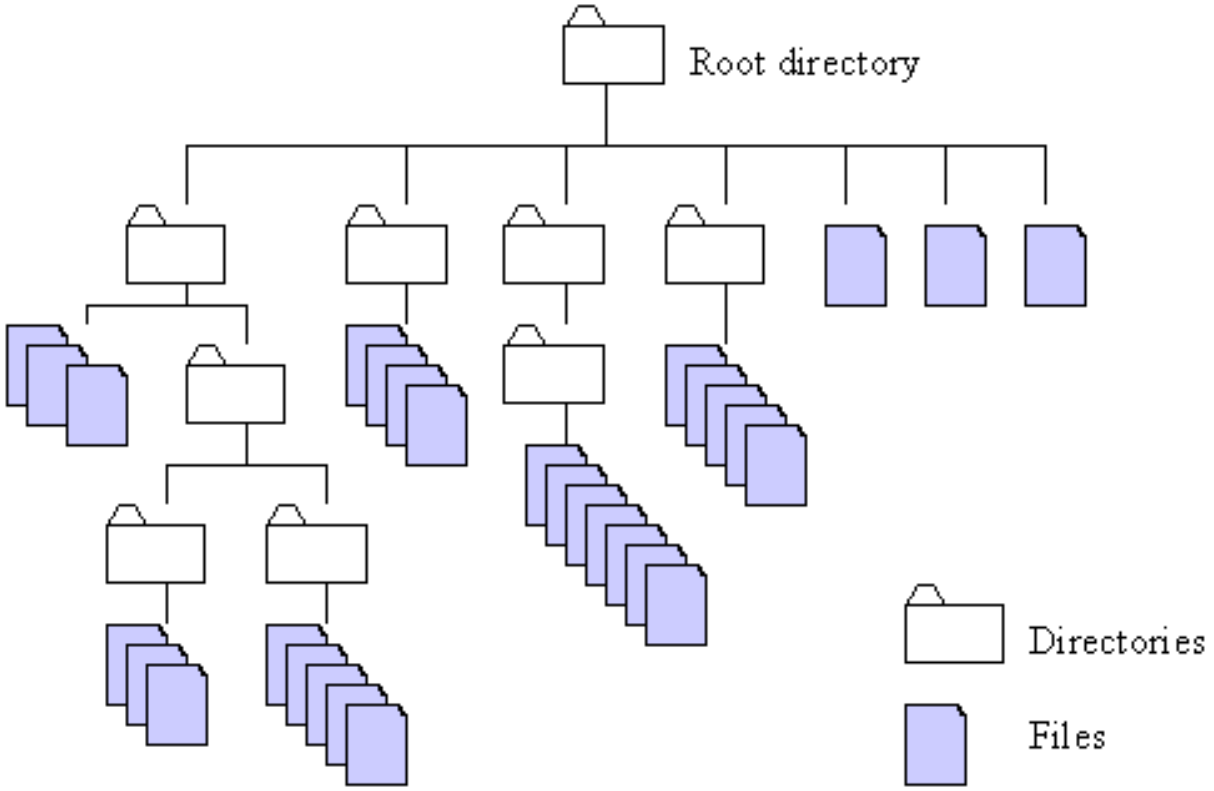
✓ 列表；当前目录下文件名和目录名列表

- 如何打开Windows系统的画图程序(mspaint.exe)?

✓ os.system('mspaint.exe')或os.startfile('mspaint.exe')

第10章 文件与文件夹操作

- 10.1 os模块
- 10.2 os.path模块
- 10.3 shutil模块
- 10.4 精彩案例赏析



10.2 os.path模块

函数	功能说明
abspath(path)	返回给定路径的绝对路径
basename(path)	返回指定路径的最后一个组成部分
commonpath(paths)	返回给定的多个路径的最长公共路径
commonprefix(paths)	返回给定的多个路径的最长公共前缀
dirname(p)	返回给定路径的文件夹部分
exists(path)	判断文件是否存在
getatime(filename)	返回文件的最后访问时间
getctime(filename)	返回文件的创建时间
getmtime(filename)	返回文件的最后修改时间，以Unix Epoch（1970年1月1日）以来以秒为单位的时间
getsize(filename)	返回文件的大小

10.2 os.path模块

函数	功能说明
isabs(path)	判断path是否为绝对路径
isdir(path)	**判断path是否为文件夹**
isfile(path)	**判断path是否为文件**
join(path, *paths)	**连接两个或多个path**
realpath(path)	返回给定路径的绝对路径
relpath(path)	返回给定路径的相对路径，不能跨越磁盘驱动器或分区
samefile(f1, f2)	测试f1和f2这两个路径是否引用的同一个文件
split(path)	以路径中的最后一个斜线为分隔符把路径分隔成两部分，以元组形式返回
splitext(path)	从路径中分隔文件的扩展名
splitdrive(path)	从路径中分隔驱动器的名称

10.2 os.path模块

```
>>> import os

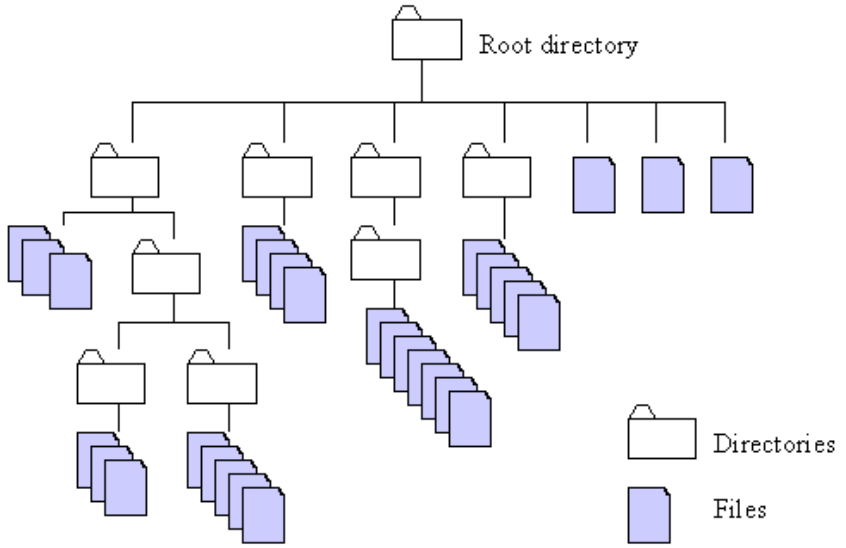
>>> path='C:\\Users\\XHN\\.spyder-py3'
>>> os.path.dirname(path)                          #返回路径的文件夹名
'C:\\Users\\XHN'
>>> os.path.basename(path)                         #返回路径的最后一个组成部分
'.spyder-py3'

>>> os.path.join('C:\\Users\\XHN', '.spyder-py3')
'D:\\mypython_exp\\new_test.txt‘

>>> [os.path.join('D:\\tmp', item) for item in os.listdir('D:\\tmp')]
['C:\\Users\\XHN\\.anaconda', 'C:\\Users\\XHN\\.android', 'C:\\Users\\XHN\\.conda']
```

10.2 os.path模块

■案例：遍历指定目录下所有子目录和文件




```
from os import listdir
from os.path import join, isfile, isdir

def listDirDepthFirst(directory):
    '''深度优先遍历文件夹'''
    #遍历文件夹，如果是文件就直接输出
    #如果是文件夹，就输出显示，然后递归遍历该文件夹
    for subPath in listdir(directory):
        path = join(directory, subPath)
        if isfile(path):
            print(path)
        elif isdir(path):
            print(path)
            listDirDepthFirst(path)   #递归调用,实现对内层文件夹内容的遍历
```

10.2 os.path模块

假设目录结构：

```
A/
├── B/
│   ├── D.txt
│   └── E/
├── C.txt
└── F/
```

```
第1层: listDirDepthFirst("A")
    |
    ├── 目录: B/
    |   |
    |   第2层: listDirDepthFirst("A/B")
    |       |
    |       ├── 文件: D.txt ✓
    |       |
    |       └── 目录: E/
    |           |
    |           第3层: listDirDepthFirst("A/B/E")
    |               |
    |               └── (空目录)
    |
    ├── 文件: C.txt ✓
    |
    └── 目录: F/
        |
        第2层: listDirDepthFirst("A/F")
            |
            └── (空目录)
```

深度优先遍历顺序（当前函数）：

```
A/B/      # 先深入B目录
A/B/D.txt # B目录下的文件
A/B/E/    # 再深入E目录
A/C.txt   # 返回后处理C文件
A/F/      # 最后处理F目录
```

10.2 os.path模块

>>>listDirDepthFirst('C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py')

```
In [88]: listDirDepthFirst('C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py')
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_1
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_1\02_1_1.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_1\02_1_2.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_2.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_2.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\data.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\data_asc.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\fruit.csv
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\helloworld.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\poem.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\ppt.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\t.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\temp.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\zjut.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\红楼梦.txt
```

10.2 os.path模块

■ 下面的代码使用了**广度**优先遍历方法(先遍历同一层的所有内容，再进入下一层)

```
from os import listdir
from os.path import join, isfile, isdir

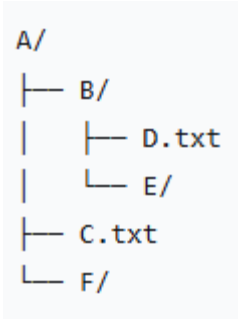
def listDirWidthFirst(directory):
    # 广度优先遍历文件夹，不使用递归函数

    dirs = [directory]    #创建一个列表dirs，用于存储待遍历的目录,初始时只包含起始目录directory
    while dirs:      #只要dirs列表不为空（即还有待遍历的目录），就继续循环
        current = dirs.pop(0)  #从列表开头取出并删除第一个元素,将取出的目录路径赋值给current变量

        for subPath in listdir(current):      #遍历该文件夹，
            path = join(current, subPath)   #将当前目录路径current和子项名称subPath合并成完整路径
            if isfile(path):
                print(path)       #如果是文件就直接输出显示；
            elif isdir(path):
                print(path)      #如果是文件夹，输出显示后，添加到dirs
                dirs.append(path)
```

10.2 os.path模块

假设目录结构：



执行过程：

1.dirs = ["A"] （初始状态）
2.取出current = "A"，遍历A目录发现： B, C.txt, F
3.处理文件C.txt：直接打印
4.处理目录B：打印并添加到队列 → dirs = ["B"]
5.处理目录F：打印并添加到队列 → dirs = ["B", "F"]
6.取出current = "B"，遍历B目录发现： D.txt, E
7.处理文件D.txt：直接打印
8.处理目录E：打印并添加到队列 → dirs = ["F", "E"]
9.取出current = "F"，遍历F目录（假设为空）
10.取出current = "E"，遍历E目录（假设为空）
11.dirs为空，循环结束

广度优先遍历顺序：

```
A/B/      # 先处理同一层
A/C.txt   # 同一层的文件
A/F/      # 同一层的目录
A/B/D.txt # 再处理下一层
A/B/E/    # 下一层的目录
```

10.2 os.path模块

>>> listDirWidthFirst('C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py')

```
In [90]: listDirWidthFirst('C:\\Users\\XHN\\Nutstore\\1\\我的坚果云\\2025秋冬_python程序设计\\PPT\\py')
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\data.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\data_asc.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\fruit.csv
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\helloworld.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\poem.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\ppt.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\t.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\temp.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\zjut.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\红楼梦.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_2.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_1
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_2.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_1\02_1_1.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_1\02_1_2.txt
```

10.2 os.path模块

■ 使用os.walk()自动遍历(了解)

```
# topdown参数控制遍历顺序
os.walk(directory, topdown=True)  # 默认，从上到下（深度优先）
os.walk(directory, topdown=False) # 从下到上
```

Practice

- 使用os.path中的哪个函数返回文件的最后修改时间？返回值类型是什么？

✓ os.path.getmtime()　　#getmtime 是 get modification time 的缩写，意为“获取修改时间”。

✓ float

```
>>> path='D:\\mypython_exp\\new_test.txt'
>>> os.path.dirname(path)的返回值是?
```

✓ 'D:\\mypython_exp'

- 使用os.path中的哪两个函数分别判断路径是否为文件或文件夹？

✓ os.path.isfile()和os.path.isdir()

第10章 文件与文件夹操作

- 10.1 os模块
- 10.2 os.path模块
- 10.3 shutil模块(了解)
- 10.4 精彩案例赏析





10.3 shutil模块(了解)

- shutil（Shell Utilities，Shell工具）是Python的标准库模块，提供了大量用于文件和目录操作的高级接口，比基本的 os 模块更加强大和方便。
- shutil模块的主要优势：

- 高级抽象：简化复杂的文件操作
- 跨平台：在不同操作系统上表现一致
- 功能丰富：覆盖文件、目录、归档等操作

操作	os模块	shutil模块	优势
复制文件	需要手动读写	shutil.copy()	更简单，保留元数据
复制目录	需要递归实现	shutil.copytree()	一行代码完成
删除目录	os.rmdir() (只能删空目录)	shutil.rmtree()	递归删除整个树
归档操作	需要多个步骤	shutil.make_archive()	内置压缩支持

10.3 shutil模块(shell utility,了解)

方法	功能说明
copy(src, dst)	复制文件，新文件具有同样的文件属性，如果目标文件已存在则抛出异常
copy2(src, dst)	复制文件，新文件具有原文件完全一样的属性，包括创建时间、修改时间和最后访问时间等等，如果目标文件已存在则抛出异常
copyfile(src, dst)	复制文件，不复制文件属性，如果目标文件已存在则直接覆盖
copyfileobj(fsrc, fdst)	在两个文件对象之间复制数据，例如copyfileobj(open('123.txt'), open('456.txt', 'a'))
copymode(src, dst)	把src的模式位（mode bit）复制到dst上，之后二者具有相同的模式
copystat(src, dst)	把src的模式位、访问时间等所有状态都复制到dst上
copytree(src, dst)	**递归复制文件夹**
disk_usage(path)	查看磁盘使用情况
move(src, dst)	移动文件或递归移动文件夹，也可以给文件和文件夹重命名
rmtree(path)	**递归删除文件夹**
make_archive(base_name, format, root_dir=None, base_dir=None)	创建tar或zip格式的压缩文件，**事实上仅打包，未压缩**
unpack_archive(filename extract_dir=None, format=None)	解压缩压缩文件

rsc: resource dst: destination

10.3 shutil模块

```
>>> import shutil                                          #导入shutil模块
>>> shutil.copyfile('C:\\dir.txt', 'C:\\dir1.txt')  #复制文件
```

■将C:\Python35\Dlls文件夹以及该文件夹中所有文件压缩至D:\a.zip文件：

```
>>> shutil.make_archive('D:\\a', 'zip', 'C:\\Python35', 'Dlls')
'D:\\a.zip‘
```

■将刚压缩得到的文件D:\a.zip解压缩至D:\a_unpack文件夹：

```
>>> shutil.unpack_archive('D:\\a.zip', 'D:\\a_unpack')
```

■使用shutil模块的方法删除刚刚解压缩得到的文件夹：

```
>>> shutil.rmtree('D:\\a_unpack')      #rmtree 是 remove tree 的缩写
```

10.3 shutil模块

■使用shutil的copytree()函数递归复制文件夹，并忽略扩展名为pyc的文件和以“新”字开头的文件和子文件夹：

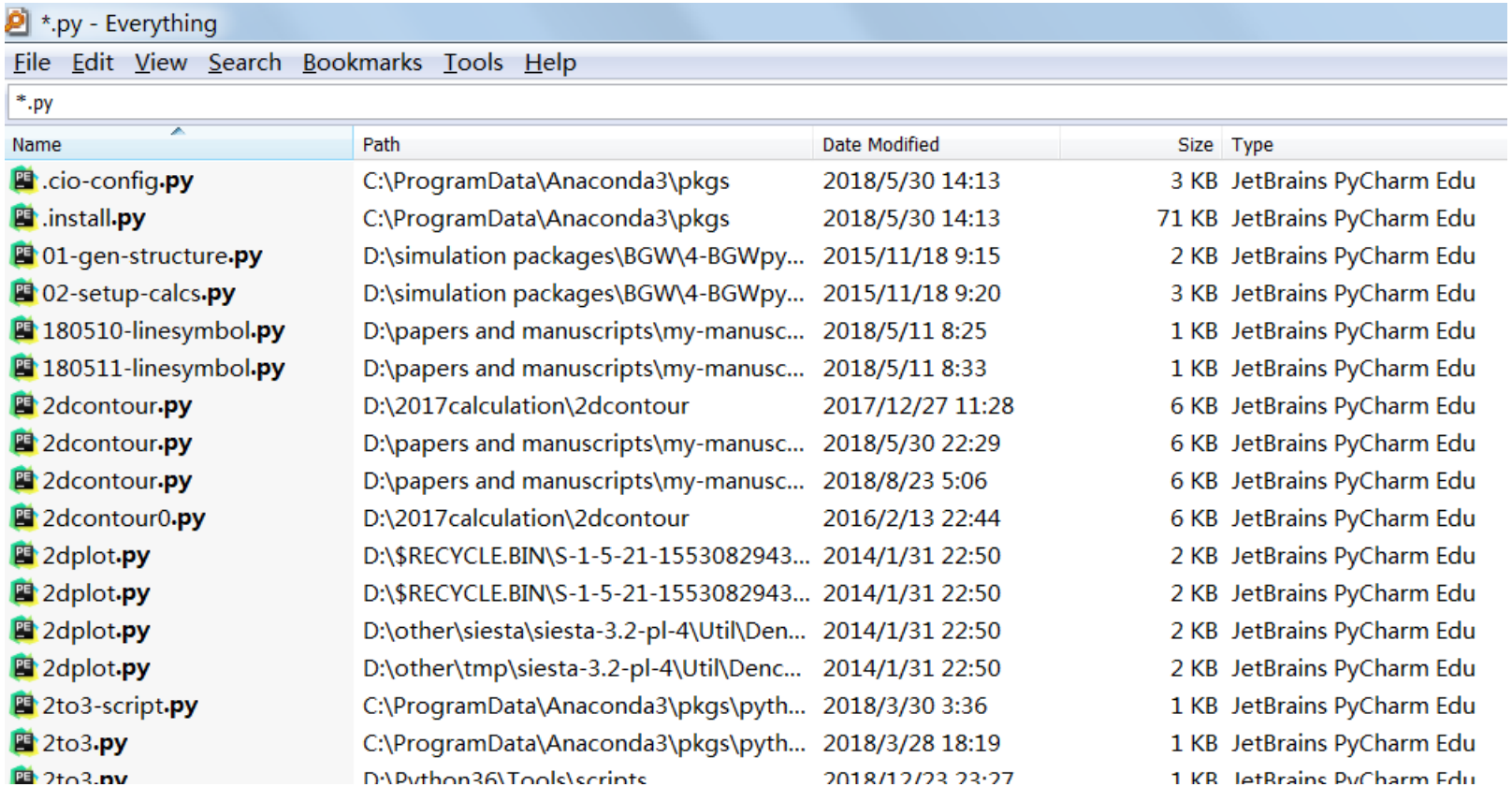
```
>>> from shutil import copytree, ignore_patterns
>>> copytree('C:\\python35\\test', 'D:\\des_test',
             ignore=ignore_patterns('*.pyc', '新*'))
```

‘*.pyc’，‘新*’为文件名通配符写法

Wildcard	Description	Example	Matches	Does not match
*	matches any number of any characters including none	Law*	Law, Laws, or Lawyer	GrokLaw, La, or aw
		Law	Law, GrokLaw, or Lawyer.	La, or aw
?	matches any single character	?at	Cat, cat, Bat or bat	at
[abc]	matches one character given in the bracket	[CB]at	Cat or Bat	cat or bat
[a-z]	matches one character from the (locale-dependent) range given in the bracket	Letter[0-9]	Letter0, Letter1, Letter2 up to Letter9	Letters, Letter or Letter10

第10章 文件与文件夹操作

- 10.1 os模块
- 10.2 os.path模块
- 10.3 shutil模块
- 10.4 精彩案例赏析



10.4 精彩案例赏析

■**示例10-2** 编写程序，比较两个目录中文件和子文件夹的差异

- 程序功能与用法：指定源文件夹与目标文件夹，自动检测自上次备份以来**源文件夹**中内容的改变，包括修改的文件、新建的文件、新建的文件夹等等；

  不处理被删除的文件和文件夹

```
import os
import filecmp  （了解）#filecmp模块，用于比较文件和目录
```

```
filecmp.cmp(f1, f2, shallow=True)
#若shallow=True，根据cmp的源代码，首先比较os.stat结果的st_mode，st_size和
st_mtime，若相同则直接返回True，否则就读取两个文件的二进制内容进行比较。
```

```
result_shallow = filecmp.cmp("a.txt", "b.txt", shallow=True)  # 浅层比较（默认）：只比较文件大小和修改时间
result_deep = filecmp.cmp("a.txt", "b.txt", shallow=False)    # 深层比较：比较文件的实际内容
```

10.4 精彩案例赏析

```
def autoBackup(scrDir, dstDir):

    for item in os.listdir(scrDir):
        scrItem = os.path.join(scrDir, item)  #构建完整源路径
        dstItem = scrItem.replace(scrDir, dstDir) #构建目标路径
                                  #通过字符串替换，将源路径转换为目标路径

        if os.path.isdir(scrItem):  #如果源文件是目录
            if not os.path.exists(dstItem):  #如果目标目录不存在
                print('{:70s}{:70s}'.format(scrItem, dstItem)) #打印备份信息
            autoBackup(scrItem, dstItem) #递归调用自己，处理子目录（深度优先遍历）
        else: #如果源文件不是目录
            if ((not os.path.exists(dstItem)) or
                    (not filecmp.cmp(scrItem, dstItem, shallow=True))):
                    #如果目标文件不存在 或 文件存在但内容不同
                print('{:70s}{:70s}'.format(scrItem, dstItem))
```

10.4 精彩案例赏析

假设目录结构：

```
source/
├── doc/
│   └── readme.txt
├── data.csv
└── config.ini
```



目标目录：backup/（初始为空）

执行过程： >>>autoBackup("source","backup")

第1次调用：scrDir="source", dstDir="backup"

•遍历 ["doc", "data.csv", "config.ini"]
•item="doc" → 是目录，目标不存在 → 打印 source/doc → backup/doc，递归
•item="data.csv" → 是文件，目标不存在 → 打印 source/data.csv → backup/data.csv
•item="config.ini" → 是文件，目标不存在 → 打印 source/config.ini → backup/config.ini

第2次调用：scrDir="source/doc", dstDir="backup/doc"

•遍历 ["readme.txt"]
•item="readme.txt" → 是文件，目标不存在 → 打印 source/doc/readme.txt → backup/doc/readme.txt

这是一个备份系统的"检测阶段"，告诉你哪些文件需要备份！

10.4 精彩案例赏析

```
>>> scrDir = r'C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01'
>>> dstDir = r'C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\02'
>>> autoBackup(scrDir,dstDir)
```

```
In [107]: autoBackup(scrDir,dstDir)
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\02\01_1
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_1
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\02\01_1\02_1
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计
\PPT\py\01\01_1\02_1\02_1_1.txtC:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python
程序设计\PPT\py\02\01_1\02_1\02_1_1.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计
\PPT\py\01\01_1\02_1\02_1_2.txtC:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python
程序设计\PPT\py\02\01_1\02_1\02_1_2.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计
\PPT\py\01\01_1\02_2.txtC:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计
\PPT\py\02\01_1\02_2.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_2.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\02\01_2.txt
```

10.4 精彩案例赏析

■**示例10-3** 编写程序，统计指定文件夹大小以及文件和子文件夹数量

```
import os
totalSize, fileNum, dirNum = 0, 0, 0
def visitDir(path):
    global totalSize, fileNum, dirNum
    for item in os.listdir(path):
        sub_path = os.path.join(path, item)
        if os.path.isfile(sub_path):
            fileNum += 1
            totalSize += os.path.getsize(sub_path) #以字节为单位
        elif os.path.isdir(sub_path):
            dirNum += 1
            visitDir(sub_path)

#下面调用visitDir()
visitDir(r'C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py')
print(totalSize, fileNum, dirNum)
```

```
2590931 14 4
```

10.4 精彩案例赏析

■实际案例：搜索一定日期范围内的文件

```
import os
import time
from os.path import join, isfile, isdir, getmtime


def search_file(directory, ts='', te=''):
    if ts == '':
        ts = time.localtime(0)
    else:
        ts = time.strptime(ts.strip(), '%Y-%m-%d')

    if te == '':
        te = time.localtime()
    else:
        te = time.strptime(te.strip(), '%Y-%m-%d')
```

10.4 精彩案例赏析

■实际案例：搜索一定日期范围内的文件

```
def search(directory):
    dirs = []
    for item in os.listdir(directory):
        path = join(directory, item)
        if isdir(path) and (not item.startswith('.')):  #不检索隐藏文件夹
            dirs.append(path)
        elif isfile(path):
            tm = time.localtime(getmtime(path))
            if ts <= tm <= te:
                print(path)
    for item in dirs:
        search(item)

search(directory)
```

10.4 精彩案例赏析

■实际案例：搜索一定日期范围内的文件

>>> search_file(r'C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py',ts='2025-9-29') #te默认为当前时间

```
In [112]: search_file(r'C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计
\PPT\py',ts='2025-9-29')
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\data.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\data_asc.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\fruit.csv
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\helloworld.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\poem.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\ppt.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\t.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\temp.py
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\zjut.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\红楼梦.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_2.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_2.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_1\02_1_1.txt
C:\Users\XHN\Nutstore\1\我的坚果云\2025秋冬_python程序设计\PPT\py\01\01_1\02_1\02_1_2.txt
```

Practice

- 设已导入模块os，使用什么表达式来判断文件D:\tmp.txt是否存在?

✓ os.path.exists('D:\\tmp.txt')

- 设已导入模块os，使用什么表达式来获得文件D:\tmp.txt的大小？

✓ os.path.getsize('D:\\tmp.txt')

作业7.10

1.编写函数search_file(directory, lb, ub）搜索文件夹directory所有层次中大小在lb到ub之间的文件。lb和ub以字节为单位，并调用search_file('D:\\Python36', 1024, 2048），写下输出结果的最后两行。(参考讲义示例10-3；第一个参数为大家自己安装的Python的目录)